



Python para Todos

Apostila do Curso Python para Todos

Versão 1.0
08/08/2019





Sumário	
Introdução.....	8
Sobre os instrutores.....	8
Evaldo do Rosario Wolkers	8
Louis Fernando Littig Wolkers.....	8
Currículo de Lançamento do Curso.....	9
Por que Python?.....	9
Instalação	10
Qual versão utilizar?	10
Instalando o Python no Windows.....	10
Instalando o Python no Linux.....	12
Executando o primeiro código.....	13
Usando o interpretador do Python.....	13
Criando o primeiro arquivo e executando pelo terminal ou prompt de comando	14
Utilizando o IDLE Shell	16
Escolhendo e utilizando uma IDE.....	18
Instalando o PyCharm no Windows.....	18
Instalando o PyCharm no Linux	25
Usando um interpretador online	28
Conceitos básicos.....	28
Blocos de Código.....	28
Objetos.....	29
Atributos	29
Métodos.....	29
Comentários de código	30
Operações Matemáticas	30
Variáveis e tipos de dados	31
Nomes de variáveis.....	32
Atribuição.....	32
Variáveis numéricas	33
Função print().....	34
Variáveis do tipo lógico e operadores relacionais	35
Operadores lógicos e expressões lógicas.....	36
Variáveis do tipo string	39
Tipagem estática e dinâmica.....	40
Entrada de dados e conversão de dados	43



Aprendendo mais sobre strings	45
Concatenação.....	45
Composição ou Interpolação	45
Fatiamento de string, index e split.....	48
Estruturas condicionais.....	51
SE => IF	52
ELSE => OUTRO, "SENÃO", CASO CONTRÁRIO.....	53
SENÃO SE => ELIF	53
Exemplo Tabela do INSS 2017.....	55
Exemplo Autorização de compra	55
Exemplo Multa	56
Estruturas de repetição (laços, loops)	56
For e função Range	56
While	60
Estruturas de dados	61
Listas	61
Tuplas.....	68
Set	71
Dicionários	73
Funções, módulos e pacotes.....	77
Introdução a Funções.....	77
Variáveis locais e globais.....	79
Parâmetros de funções	82
Funções sem parâmetros.....	82
Funções com parâmetros opcionais	82
Nomeando parâmetros.....	83
Funções como parâmetros.	83
Parâmetros empacotados em listas.....	83
Desempacotamento de parâmetros.....	84
Módulos e pacotes.....	84
Funções recursivas.....	87
Validação de entrada de dados.....	88
Expressões lambda.....	89
A função type	91
Mapeamento (Função map())	93
Aprendendo ainda mais sobre Strings	94

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Verificação de Strings e conversão para maiúsculas e minúsculas	94
Contando e pesquisando elementos em uma String.....	95
Posicionamento de Strings.....	97
Separação, Substituição de Strings e Remoção de Espaços em Branco	99
Validação por tipo de conteúdo.....	100
Orientação a Objetos	104
Conceitos básicos.....	104
Classes, objetos, namespaces, sombreado de atributos e self	110
Herança	113
Herança múltipla.....	116
Polimorfismo.....	118
Classes abertas.....	122
Encapsulamento, atributos públicos e privados	125
if <code>__name__ == '__main__'</code>	131
Propriedades.....	132
Descritores	134
Sobrecarga de operadores.....	139
Metaclasses.....	141
Algumas definições	141
Classes-base abstratas	147
Mensagens e tratamento de erros	149
Mensagens de erro	149
Tratamento de erros – Exceções	154
Documentação.....	161
Pydoc.....	161
Documentando seu código	166
Trabalhando com arquivos e diretórios.....	168
Arquivos e diretórios/pastas.....	168
Funções para trabalhar com arquivos	169
Abertura e fechamento de arquivo	169
Escrevendo no arquivo com a função <code>write()</code>	169
Fechando um arquivo com <code>close()</code>	170
Lendo o arquivo inteiro com a função <code>read()</code>	170
Lendo o arquivo linha a linha com a função <code>readline()</code>	171
Retornando uma lista com a função <code>readlines()</code>	171
Percorrendo todas as linhas com a função <code>readlines()</code>	171



Características de um arquivo de texto	172
Gravando várias linhas com a função <i>writelines()</i> usando uma lista.....	172
Usar o <i>with</i> é uma boa prática	173
Lendo e escrevendo em arquivos com <i>with</i>	173
Lendo um arquivo de texto contendo o Hino Nacional	174
Programa para procurar palavras existentes no Hino Nacional.	175
Arquivos e diretórios (pastas)	175
Identificando o diretório atual.....	175
Criando diretórios	176
Criando vários diretórios.....	177
Navegando em diretórios	178
Renomeando arquivos e diretórios	179
Movendo um arquivo com a função <i>rename()</i>	180
Listando arquivos e diretórios com a função <i>listdir()</i>	181
Identificando diretórios e arquivos com <i>isdir()</i> e <i>isfile()</i>	182
Verificando a existência de um arquivo ou diretório.....	183
Obtendo informações do arquivo.....	184
Manipulando caminhos	185
Acessando subdiretórios recursivamente.....	186
Excluindo arquivos e diretórios/pastas.....	186
Banco de dados MySQL.....	189
O que é um banco de dados relacional.....	189
O que é um banco de dados NoSQL (Not Only SQL).....	189
O que é um sistema gerenciador de banco de dados.....	190
Conceitos básicos de bancos relacionais	190
SQL	191
Baixando e instalando o MySQL	192
Instalando o MySQL no Linux Mint 18.3	201
Trabalhando com o MySQL Workbench	207
Sobre o MySQL Workbench	207
Acessando o MySQL Workbench	208
A interface do MySQL Workbench.....	208
Criando um schema	209
Visualizando os objetos do schema	212
Criando uma tabela.....	213
Tipos de dados no MySQL.....	215

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Inserindo dados em uma tabela	217
Consultando dados em uma tabela	218
Alterando dados em uma tabela	221
Excluindo dados em uma tabela	222
Conectando ao banco MySQL com MySQL Connector/Python	222
Sobre o MySQL Connector/Python	223
Baixando e instalando o MySQL Connector/Python.....	223
Usando o MySQL Connector/Python	224
Executando instruções SQL com Python	226
Inserindo registros	226
Alterando registros	227
Apagando registros	227
Consultando registros	227
Criando um cadastro simples sem interface gráfica	228
Introdução.....	228
O que vamos usar?.....	228
Execução do sistema	228
Implementando o exemplo.....	229
Referências Bibliográficas do curso	242



Introdução

Esta apostila é disponibilizada aos alunos do Curso Python para Todos como ferramenta para acompanhamento das aulas, podendo ser impressa e distribuída livremente.

Sobre os instrutores

Evaldo do Rosario Wolkers

- Meu primeiro contato com linguagens de programação foi quando tinha 15 anos de idade.
- Possuo 20 anos de experiência em desenvolvimento de sistemas, trabalhando no desenvolvimento de aplicações Desktop, Web, Mobile (Android e iOS) e aplicações para equipamentos POS (Point of Sale).
- Sou Gerente de Tecnologia da Informação, bacharel em Sistemas de Informação e pós-graduado em Segurança da Informação.
- Trabalhei como professor universitário ministrando aulas das disciplinas de Sistemas de Informação e Segurança da Informação.
- Sou autor do Livro Segurança da Informação em Aplicações Web com Grails.
- Sou amante de todas linguagens de programação, sem preconceito algum, mas confesso ser apaixonado por Python.

Linguagens que já tive contato:

Como tudo começou (Pré-história)	Estudo/Diversão	Uso profissional
dBase	Assembly	Delphi
Clipper	Visual Objects	C e C++
Pascal	Visual Basic	ASP.Net e C#
Basic	ASP / PHP	Java
	ObjectiveC	Groovy & Grails
	Lua	Python
	Vb.Net	 → Infinito
	Ruby	

Louis Fernando Littig Wolkers

- Estudante.
- Entusiasta de linguagens de programação.
- Minha principal participação no curso será nas seções de exercícios e solução de desafios.

Linguagens que já tive contato:

Estudo/Diversão
C
Java
Python



Currículo de Lançamento do Curso

Por que Python?	Documentação
Instalação do Python	Exceções
Executando o primeiro código	Bibliotecas (padrão e de terceiros)
Conceitos básicos	Testes automatizados
Variáveis e tipos de dados	Threads
Estruturas condicionais	Trabalhando com Arquivos
Laços	Trabalhando com bancos de dados
Estruturas de dados	Interface gráfica
Trabalhando com Strings	Plataformas portáteis
Funções	Aplicações Web
Classes e Objetos	Programação de Redes
Mensagens de erro	

Por que Python?

- Python é uma linguagem de programação que permite trabalhar mais rapidamente e integrar os seus sistemas de forma mais eficaz.
- Python é poderoso... e rápido;
- corre em todos lugares;
- é amigável e fácil de aprender;
- é Open Source.
- Estas são algumas das razões pelas quais as pessoas que usam Python preferem não usar outra coisa.
- Linguagem mais popular para ensino introdutório em universidades dos EUA.
- É a quarta linguagem mais popular atrás dos antigos clássicos Java, C, C++.
- Fácil de usar, poderoso, versátil, sendo uma ótima opção para iniciantes e especialistas em programação.
- A legibilidade do Python faz dele uma grande primeira linguagem de programação.
- Permite que você pense como um programador sem perder tempo com sintaxes misteriosas como em outras linguagens.

Veja a simplicidade do Python comparado ao Java para imprimir a frase “Olá Mundo!”.

Java <pre>public class Main { public static void main(String[] args) { System.out.println("Olá Mundo!"); } }</pre>	Python <pre>print('Olá Mundo!')</pre>
--	---

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Instalação

Qual versão utilizar?

Ao acessar o endereço <https://www.python.org/downloads/windows/> nos deparamos com duas versões para Windows. Atualmente são as versões 3.7.2 e 2.7.15.

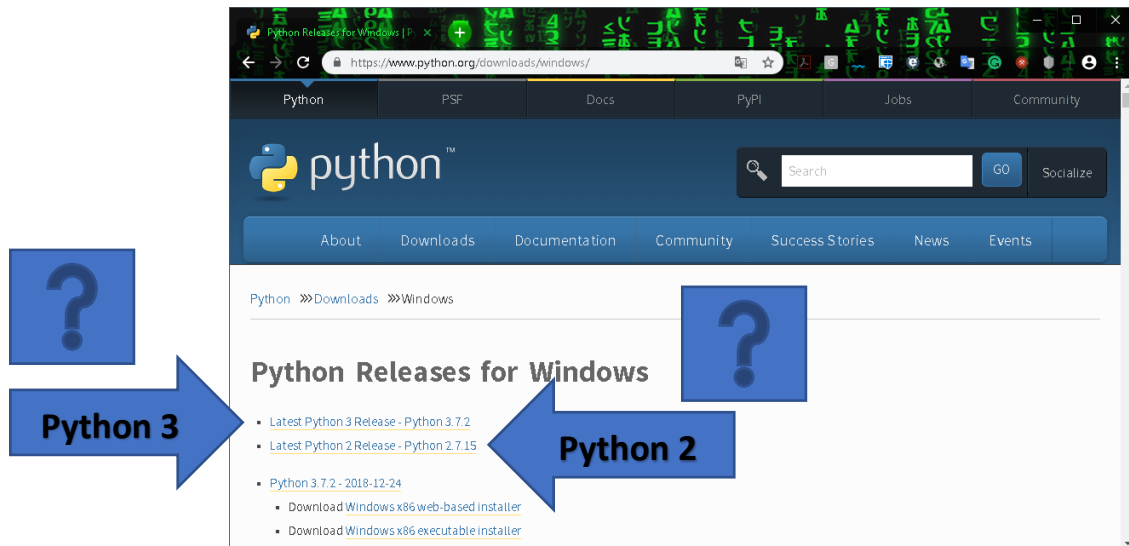


Figura 1 - Página de download do Python

Python 3.0 foi lançado em 2008. A versão final 2.x, que foi a versão 2.7 saiu em meados de 2010. A ramificação 2.x não verá novos lançamentos principais depois disso. 3.x está em desenvolvimento ativo e já viu mais de cinco anos de lançamentos estáveis, incluindo a versão 3.3 em 2012, 3.4 em 2014, 3.5 em 2015 e 3.6 em 2016. Isso significa que todas as melhorias recentes da biblioteca padrão, por exemplo, estão disponíveis somente por padrão no Python 3.x.

Basicamente a resposta é: Se você vai iniciar um projeto, use Python 3, porque é o presente e será o futuro.

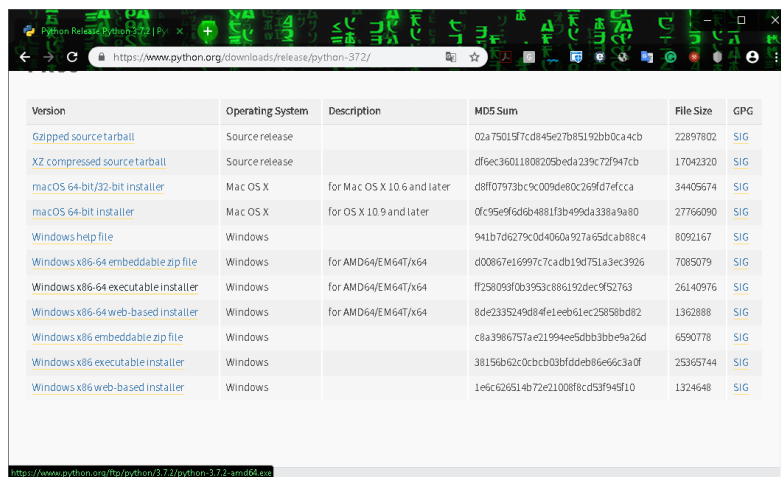
O Python 2 foi mantido devido a sistemas antigos que foram desenvolvidos nesta versão. Porém, estes sistemas devem ser migrados o quanto antes para o Python 3 porque o suporte ao Python 2 terá fim no ano de 2020.

Instalando o Python no Windows

Ao acessar o endereço <https://www.python.org/downloads/release/python-372/> veremos várias versões para download, eu vou baixar a versão 64 bits (Windows x86-64 executable installer).

O endereço completo para download é:

<https://www.python.org/ftp/python/3.7.2/python-3.7.2-amd64.exe>



Version	Operating System	Description	MD5 Sum	File Size	GPB
Gzipped source tarball	Source release		02a750197cd845e27b85192bb0ca4cb	22897802	SIG
XZ compressed source tarball	Source release		df6ec36011808205bda239c72f947cb	17042320	SIG
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	d8ff07973bc9c009de80c269d7efcca	34405674	SIG
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	0fc95e9f6d4b4881f3b499da338a9a80	27766090	SIG
Windows help file	Windows		941b7d6279c0d4060a927a65dca88c4	8092167	SIG
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64	d00867e16997c7cadb19d751a3ec3926	7085079	SIG
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64	ff258093f0b3953c886192dec9f52763	26140976	SIG
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64	8de2335249d84fe1eeb61ec2595bd82	1362888	SIG
Windows x86 embeddable zip file	Windows		c8a3986757ae2194ee5dbb3bbefa26d	6590778	SIG
Windows x86 executable installer	Windows		38156b62c0c0cb03bfdd8b6e66c3a0f	25365744	SIG
Windows x86 web-based installer	Windows		1e6c626514b72e21008f8cd53f949f10	1324648	SIG

Figura 2 - Página de download do Python 3.7.2

Após baixar o arquivo de instalação e executá-lo, será exibida uma janela de confirmação como a janela a seguir:

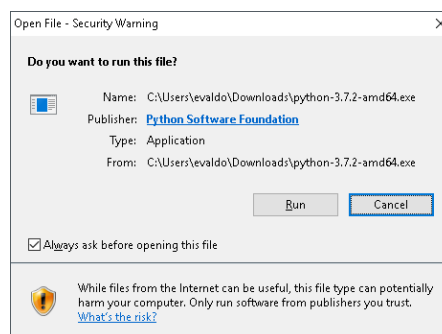


Figura 3 - Janela do Windows de confirmação de instalação

Escolha “Run / Executar”.

Na próxima janela exibida, marque a opção “Add Python 3.7 to PATH / Adicionar Python 3.7 ao PATH” para adicionar o Python ao path do Windows. Em seguida clique em “Install Now / Instalar Agora”. Se aparecer uma janela do Windows solicitando confirmação, clique em “Yes / Sim”.

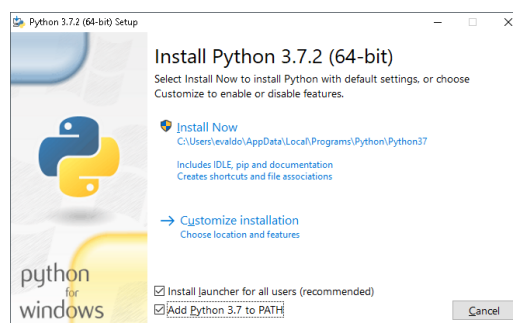


Figura 4 - Janela de instalação do Python

Ao término da instalação será exibida a janela a seguir:



Figura 5 - Python instalado com sucesso

Você pode fechar esta janela. Pronto, o Python está instalado em seu sistema.

Abra o prompt de comando do Windows e digite python, caso tudo tenha ocorrido corretamente, você verá uma tela como a seguir. Esta é a tela do interpretador interativo do Python.

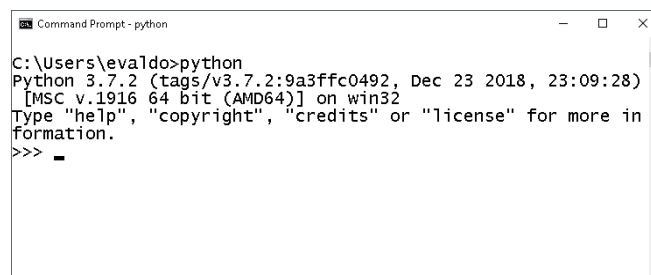


Figura 6 - Acessando o interpretador Python

Instalando o Python no Linux

O Linux Mint, usado no curso, em sua versão 19.1 vem com o Python 2.7.15 e o Python 3.6.7 instalados. Para executar o Python 2, basta digitar “python” (sem as aspas) no terminal, porém, para executar o Python 3, devemos executar “python3” (sem as aspas).

Vamos instalar o Python 3.7.2 no Linux Mint 19.1.

Primeiro vamos instalar os pré-requisitos:

```
$ sudo apt-get install build-essential checkinstall  
$ sudo apt-get install libreadline-gplv2-dev libncursesw5-dev libssl-dev \  
libsqlite3-dev tk-dev libgdbm-dev libc6-dev libbz2-dev
```

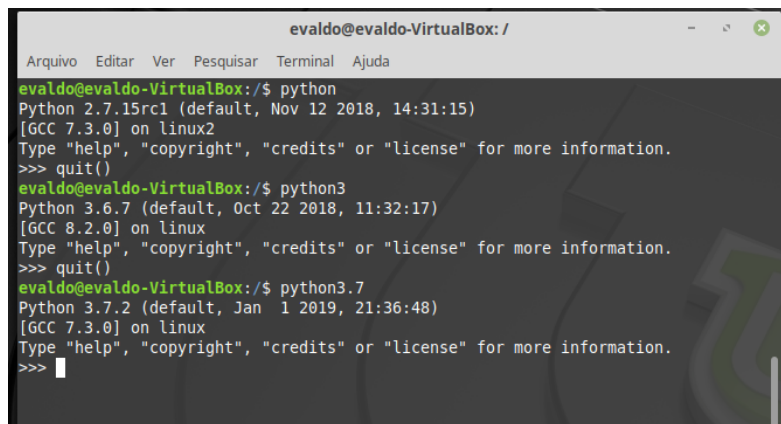
O “\$” representa o prompt do terminal Linux e a “\” no segundo comando indica que o comando continua na linha seguinte.

Após instalar os pré-requisitos, vamos fazer download do Python 3.7.2 e realizar sua instalação. Vamos acessar a pasta `/usr/src`, baixar o instalador do Python compactado para esta pasta, vamos realizar a descompactação usando o comando `tar`, em seguida vamos entrar na pasta criada “Python-3.7.2”, vamos executar o comando `configure` e por fim, vamos instalar com “`make altinstall`”. O “`make altinstall`” é usado para impedir a substituição do arquivo binário Python padrão `/usr/bin/python`.

```
$ cd /usr/src  
$ sudo wget https://www.python.org/ftp/python/3.7.2/Python-3.7.2.tgz  
$ sudo tar xzf Python-3.7.2.tgz  
$ cd Python-3.7.2  
$ sudo ./configure --enable-optimizations  
$ sudo make altinstall
```



Ao término da instalação, teremos 3 versões do Python instaladas no sistema. Podemos acessar o Python 2 com o comando “python”, o Python 3.6.7 com o comando “python3” e o Python 3.7.2 com o comando “python3.7”. Veja:



```
evaldo@evaldo-VirtualBox: /
Arquivo  Editar  Ver  Pesquisar  Terminal  Ajuda
evaldo@evaldo-VirtualBox:/$ python
Python 2.7.15rc1 (default, Nov 12 2018, 14:31:15)
[GCC 7.3.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> quit()
evaldo@evaldo-VirtualBox:/$ python3
Python 3.6.7 (default, Oct 22 2018, 11:32:17)
[GCC 8.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> quit()
evaldo@evaldo-VirtualBox:/$ python3.7
Python 3.7.2 (default, Jan 1 2019, 21:36:48)
[GCC 7.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Figura 7 - Acessando o interpretador Python

Executando o primeiro código

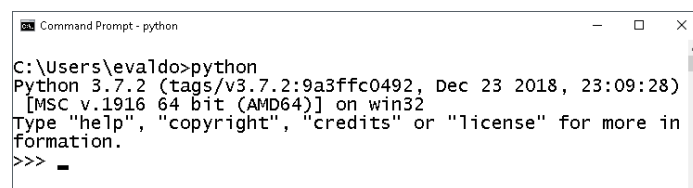
Usando o interpretador do Python

Python é uma linguagem interpretada.

O interpretador Python traduz em tempo de execução os comandos da linguagem intermediária em linguagem de máquina.

A vantagem é que um único sistema pode ser executado em inúmeras plataformas, basta para isso que existam máquinas virtuais para cada uma delas. Ou seja, o seu programa Python vai rodar em qualquer sistema operacional que tenha um interpretador Python.

Para executar o interpretador Python no Windows, basta abrir o prompt de comando do Windows e digitar “python”, sem as aspas. Ao entrar no interpretador interativo, vemos a versão do Python que está instalada no sistema.

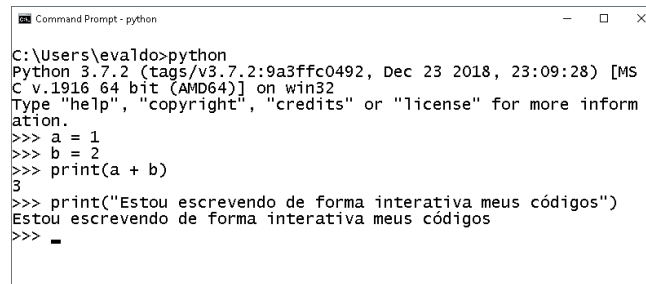


```
Command Prompt - python
C:\Users\evaldo>python
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 23:09:28)
[MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more in
formation.
>>>
```

Figura 8 - Acessando o interpretador Python

Ao entrar no interpretador você verá o prompt do interpretador Python representado por três sinais de maior “>>>”. Neste momento podemos começar a informar os comandos Python.

Veja um exemplo:

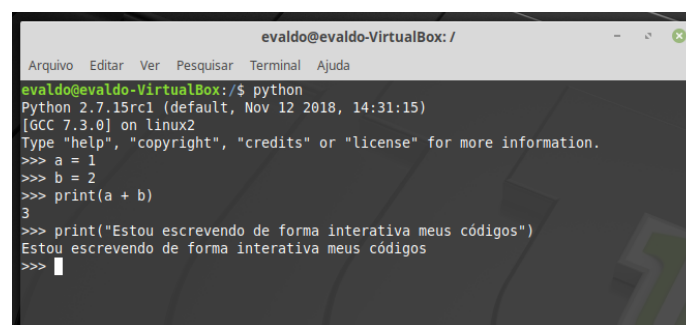


```
C:\Users\evaldo>python
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 23:09:28) [MS
C v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more inform
ation.
>>> a = 1
>>> b = 2
>>> print(a + b)
3
>>> print("Estou escrevendo de forma interativa meus códigos")
Estou escrevendo de forma interativa meus códigos
>>> _
```

Figura 9 - Informando comandos no interpretador

Mais à frente você entenderá o que foi feito nesta tela, a princípio, entenda que podemos usar o interpretador interativo do Python para digitar os nossos comandos e, após pressionar o ENTER, veremos o resultado automaticamente.

No Linux usamos o terminal e acessamos o modo interativo também digitando “python”, sem as aspas.



```
evaldo@evaldo-VirtualBox: /
Arquivo Editar Ver Pesquisar Terminal Ajuda
evaldo@evaldo-VirtualBox:/$ python
Python 2.7.15rc1 (default, Nov 12 2018, 14:31:15)
[GCC 7.3.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> a = 1
>>> b = 2
>>> print(a + b)
3
>>> print("Estou escrevendo de forma interativa meus códigos")
Estou escrevendo de forma interativa meus códigos
>>> _
```

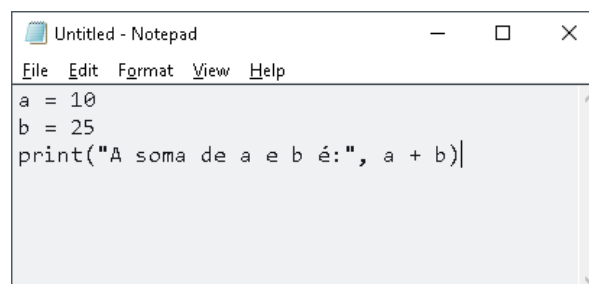
Figura 10 - Informando comandos no interpretador

Para sair do modo interativo podemos utilizar o comando `quit()` ou `exit()`, desta forma, voltamos ao prompt de comandos no Windows, ou ao terminal no Linux.

Criando o primeiro arquivo e executando pelo terminal ou prompt de comando

O modo interativo é útil para execução de códigos enquanto estamos estudando a linguagem. Nossos programas serão escritos em forma de arquivos e executados diretamente pelo interpretador.

Abra o bloco de notas do Windows e escreva o programa abaixo:



```
Untitled - Notepad
File Edit Format View Help
a = 10
b = 25
print("A soma de a e b é:", a + b)
```

Figura 11 - Criando um arquivo Python

Clique em “File / Arquivo”, depois em “Save / Salvar”, informe o nome do arquivo como “exemplo.py”, com as aspas, pois por padrão o bloco de notas salva o arquivo com a extensão txt. Em “Encoding / Codificação”, selecione “UTF-8”, defina um local de fácil localização para salvar o arquivo, pois precisaremos acessar o arquivo via prompt de comando ou terminal em seguida.

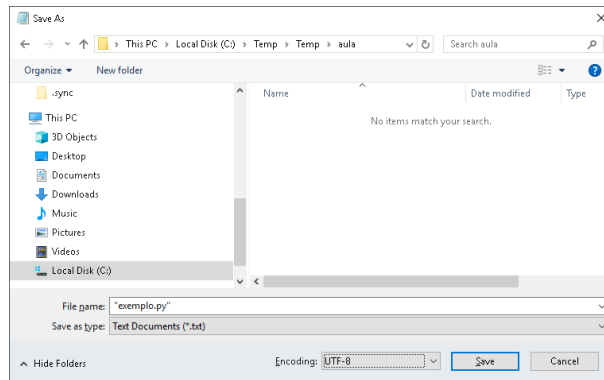


Figura 12 - Salvando o arquivo

Se você não tiver muita familiaridade com o prompt de comando do Windows ou o terminal Linux, sugiro que faça meu curso “Terminal Linux e Prompt de Comando do Windows”, que é um curso gratuito disponível em:

<https://www.udemy.com/terminal-linux-e-prompt-de-comando-do-windows>

Acesse o local onde o arquivo foi salvo e execute com o comando:

```
$ python exemplo.py
```

Veja o arquivo sendo executado:

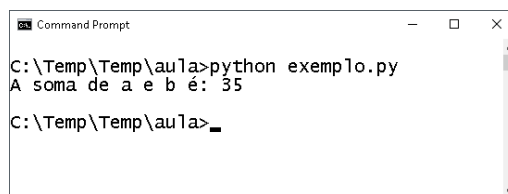


Figura 13 - Executando um arquivo Python

No Linux, da mesma forma, você escreve o programa em um editor de texto, salva e executa com o comando “python nomedoarquivo.py”. Como instalamos o Python 3.7.2, usaremos “python3.7 nomearquivo.py”.

Veja o arquivo sendo criado e salvo.

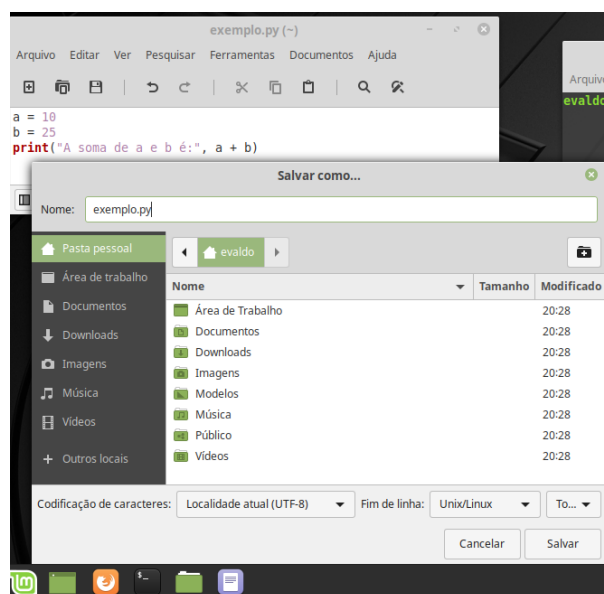


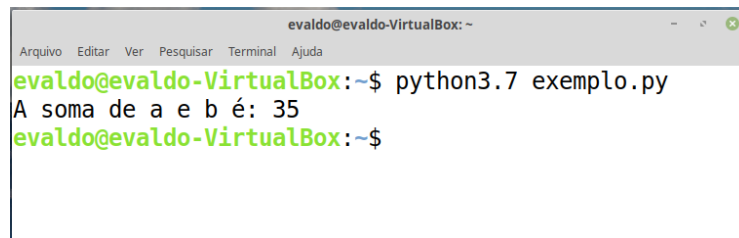
Figura 14 - Criando e salvando um arquivo Python

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



E agora sendo executado:



```
evaldo@evaldo-VirtualBox:~$ python3.7 exemplo.py
A soma de a e b é: 35
evaldo@evaldo-VirtualBox:~$
```

Figura 15 - Executando um arquivo Python

Utilizando o IDLE Shell

IDLE é um ambiente de desenvolvimento e aprendizado integrado do Python. O IDLE foi escrito usando o próprio Python.

Características:

- Codificado 100% em Python, usando tkinter GUI Toolkit;
- Multi-plataforma -> Linux, Windows e macOS;
- Janela de Shell Python (interpretador interativo) com cores para o código de entrada, saída e mensagens de erro;

Editor de texto com múltiplas janelas, cores para código Python, recuo inteligente, dicas de chamada, auto-complete, e outras funcionalidades como pesquisar em qualquer janela, substituir dentro das janelas do editor e pesquisar por múltiplos arquivos de debug com breakpoints persistentes, stepping e visualização de configurações e namespaces, navegadores e outros diálogos.

Você pode abrir o IDLE Shell através da opção "IDLE (Python 3.7 64-bit)" que fica dentro da pasta "Python 3.7" no menu iniciar do Windows:

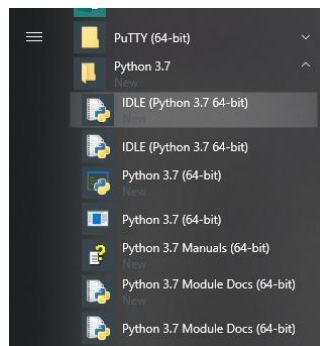


Figura 16 - Executando o IDLE Shell no Windows

Veja a janela do IDLE sendo executada:



Figura 17 - Janela do IDLE Shell

No Linux, como foi instalado o Python 3.7.2, o IDLE pode ser acessado através do seguinte comando:
\$ idle3.7

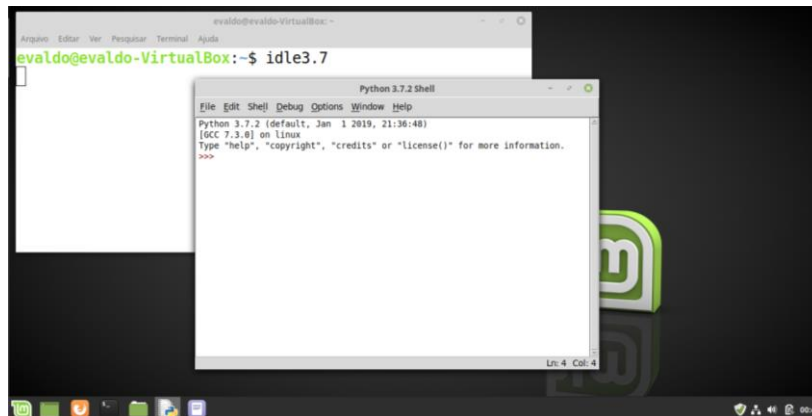


Figura 18 - Executando o IDLE Shell no Linux

Podemos usar o IDLE da mesma forma que utilizamos o interpretador Python, informando os comandos e obtendo as respostas ao pressionar ENTER:

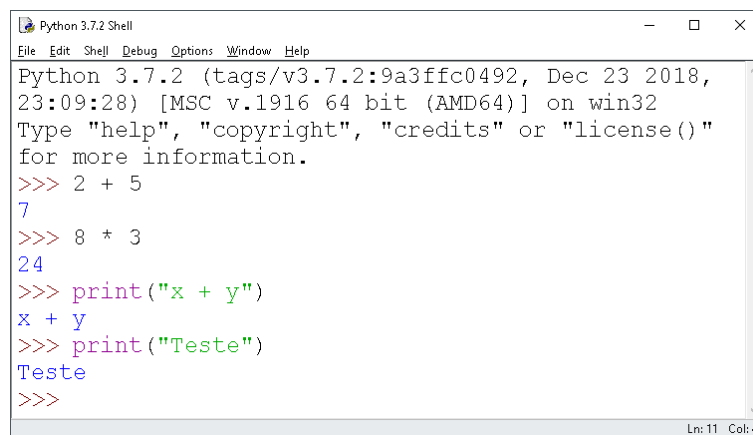


Figura 19 - Digitando comandos no IDLE Shell

Observe que o IDLE possui uma cor para diferenciar o resultado, a entrada de dados, os erros. Podemos alterar configurações do IDLE através do menu “Options”, opção “Configure IDLE”.

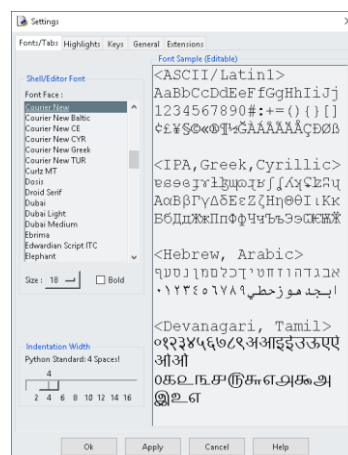


Figura 20 - Janela de configuração do IDLE Shell



Assim como o interpretador Python, o IDLE Shell não é muito indicado para desenvolvimento no dia a dia. Para desenvolver aplicações no dia a dia, você irá preferir utilizar uma IDE, ou seja, um ambiente de desenvolvimento integrado, o que veremos a seguir.

Escolhendo e utilizando uma IDE

Um ambiente de Desenvolvimento Integrado ou IDE facilita o desenvolvimento de sistemas. Muitas IDE são recheadas de recursos específicos para determinada linguagem de programação. Da mesma forma que existem “guerras” e “tribos” que defendem determinada linguagem de programação, em se tratando de IDE a coisa não muda muito.

Meu conselho é que você use a IDE ou um editor de texto que lhe agrade. Mas não fique preso à IDE “A” ou “B”, ou ao editor de textos “A” ou “B”, é bom sempre aprender coisas novas e conhecer mais de uma ferramenta para decidir qual irá usar.

Veja alguns editores e IDEs:

vi / vim, Nano, Sublime, Komodo, Wing, PyScripter, Eric, IEP, Notepad++, Visual Studio Code, Atom e **PyCharm** (o que vamos usar no curso).



Figura 21 - PyCharm

Para realizar download do PyCharm acesse <https://www.jetbrains.com/pycharm/download>, escolha o seu sistema operacional (Windows, macOS ou Linux) e escolha a versão Community, que é gratuita e open-source.

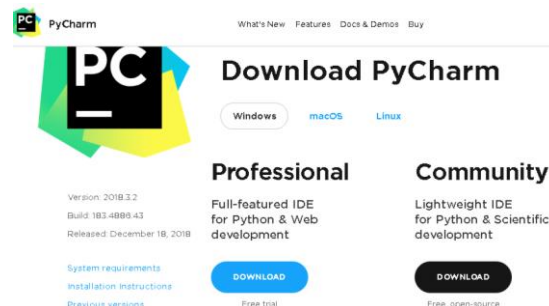


Figura 22 - Página de download

No momento da elaboração deste conteúdo, a versão para Windows era a 2018.3.2, disponível em: <https://download-cf.jetbrains.com/python/pycharm-community-2018.3.2.exe>

E a versão para Linux, disponível em:

<https://download-cf.jetbrains.com/python/pycharm-community-2018.3.2.tar.gz>

Instalando o PyCharm no Windows

Após realizar download do arquivo de instalação para Windows, execute-o. Ao executar o instalador, o Windows solicitará confirmação de execução.

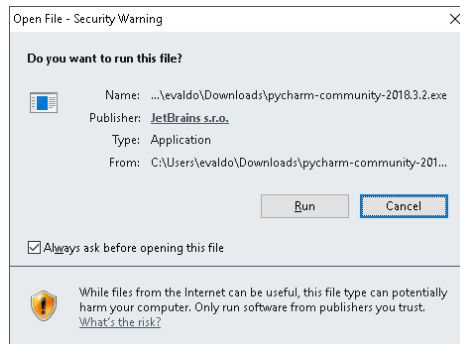


Figura 23 - Janela de confirmação do Windows

Caso o Windows mostre uma outra tela, confirme, clicando em “Yes / Sim”. Em seguida será exibida a tela de boas-vindas.

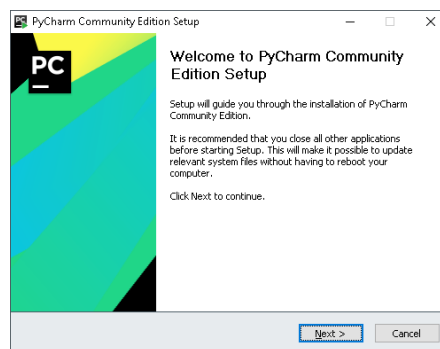


Figura 24 - Tela de boas-vindas do instalador

Ao clicar em “Next” na tela de boas-vindas, será exibida uma tela para escolha do local de instalação. Deixe o local padrão que foi sugerido e clique em “Next” novamente.

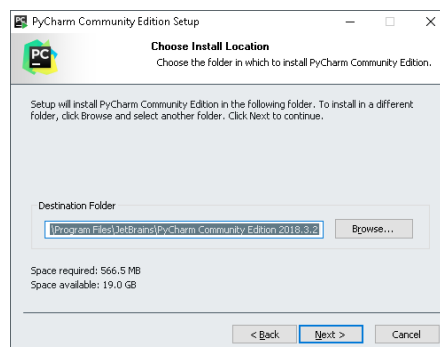


Figura 25 - Tela para escolha do local de instalação

Será exibida uma tela com opções de instalação.

Marque a opção para criar um atalho no desktop de acordo com sua plataforma, 32 ou 64 bits.

Marque a opção para adicionar os “lançadores” ao path do Windows, esta opção permite chamarmos o PyCharm diretamente do prompt de comando do Windows.

Marque a opção para adicionar o menu “Abrir pasta como projeto” ao menu de contexto do gerenciador de arquivos do Windows. Após a instalação, a opção a seguir estará disponível ao clicar em uma pasta com o botão direito do mouse no gerenciador de arquivos do Windows (Explorer).



Figura 26 - PyCharm no menu de contexto do Explorer

Marque a opção para vincular arquivos com a extensão “py”, assim quando dermos dois cliques em um arquivo py no gerenciador de arquivos do Windows, o mesmo será aberto com o PyCharm. Por fim, marque a opção para baixar e instalar o Java da JetBrains para que o PyCharm o utilize. O PyCharm foi desenvolvido em Java, sendo assim, precisa da máquina virtual Java para funcionar. Caso já tenha certa experiência e tenha o Java instalado, não é necessário escolher esta opção. Clique em Next.

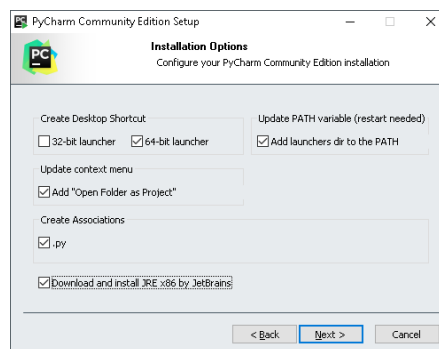


Figura 27 - Tela com opções de instalação

Na última tela é solicitada a seleção da pasta para criação dos atalhos. Deixe o padrão e clique em “Install”.

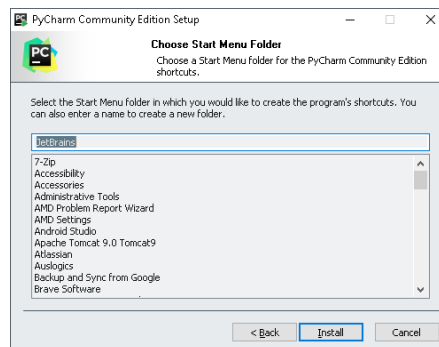


Figura 28 - Tela para escolha da pasta para criação dos atalhos

Ao término da instalação você pode reiniciar o computador, ou informar que irá reiniciar posteriormente, após esta escolha, clique em “Finish”.

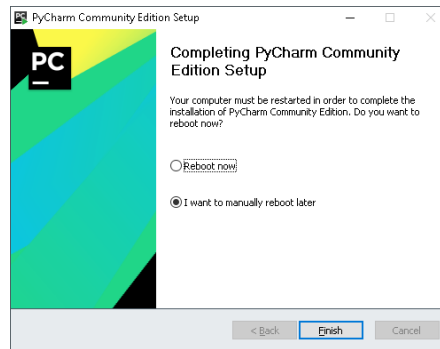


Figura 29 - Instalação completa

Você poderá executar o PyCharm através do menu iniciar do Windows, dentro da pasta JetBrains.

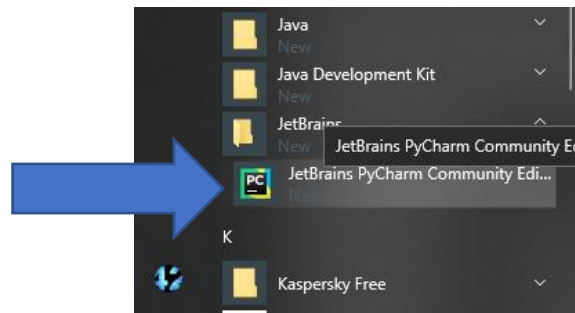


Figura 30 - Atalho do PyCharm no Menu Iniciar

Veja a tela de boas-vindas do PyCharm.

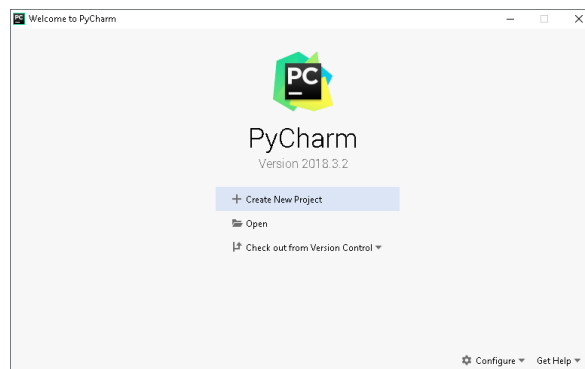


Figura 31 - Tela de boas-vindas do PyCharm

Para criar um projeto PyCharm clique em “Create New Project / Criar Novo Projeto”, será exibida a tela a seguir.

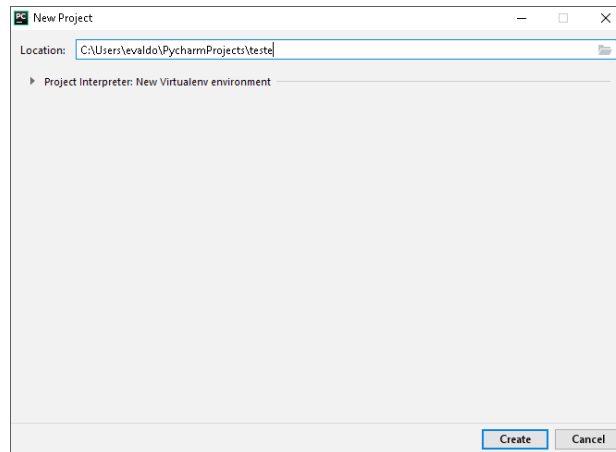


Figura 32 - Novo Projeto

Nesta tela temos que informar a localização e o nome do projeto que vamos criar. Eu mantive na pasta padrão e chamei o projeto de “teste” (conforme está na imagem).

Clique na opção “Project Interpreter / Interpretador do Projeto”, marque a opção “Existing interpreter / Interpretador existente”, pois vamos selecionar o interpretador do Python 3.7.2 para nosso projeto. Para selecionar o interpretador, clique no botão com reticências ao lado da caixa “Interpreter: / Interpretador:”.

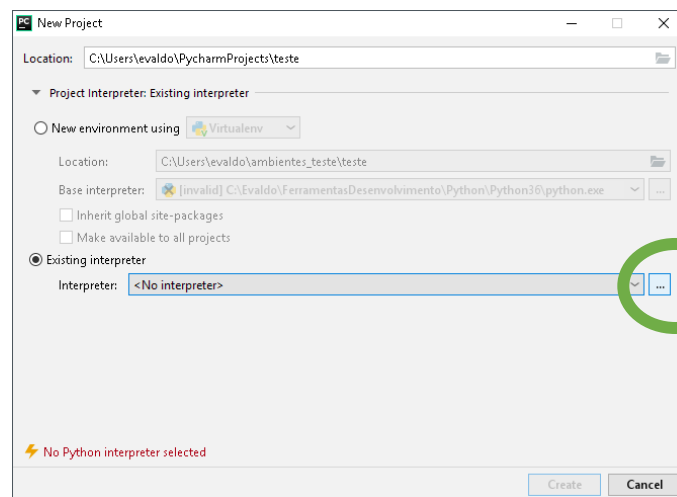


Figura 33 - Definindo o interpretador

Na janela seguinte, escolha “System Interpreter / Interpretador do Sistema”. À direita será exibida uma caixa com o interpretador Python 3.7.2 que foi instalado no sistema.

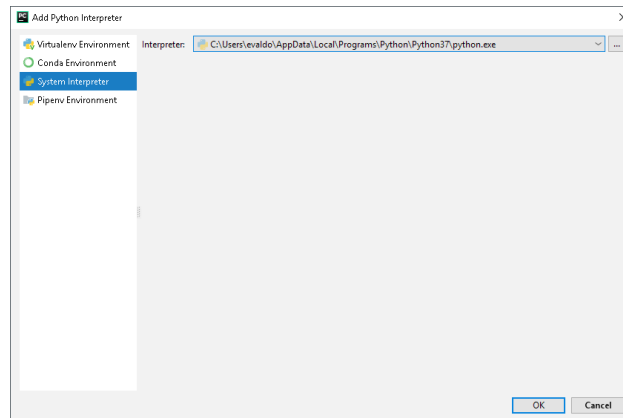


Figura 34 - Definindo o interpretador

Clique em OK para selecionar o interpretador.

O interpretador do Python 3.7.2 será exibido na caixa “Interpreter / Interpretador”. Para criar o projeto, clique no botão “Create / Criar”, exibido nesta janela.

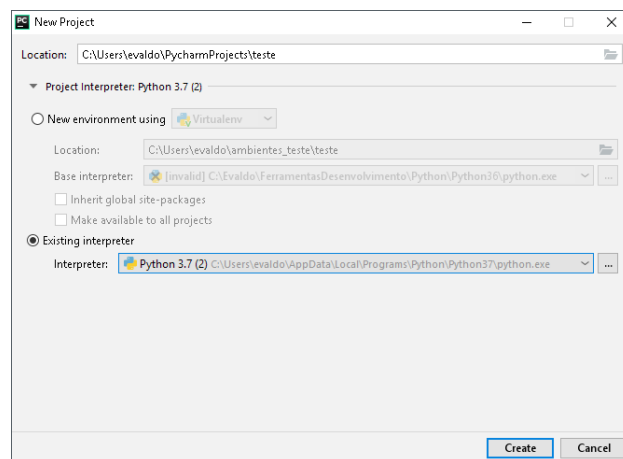


Figura 35 - Definindo o interpretador

Falaremos sobre ambientes virtuais no futuro, mas por enquanto, você irá configurar o interpretador do sistema.

Após criar o projeto você estará na janela principal do PyCharm.

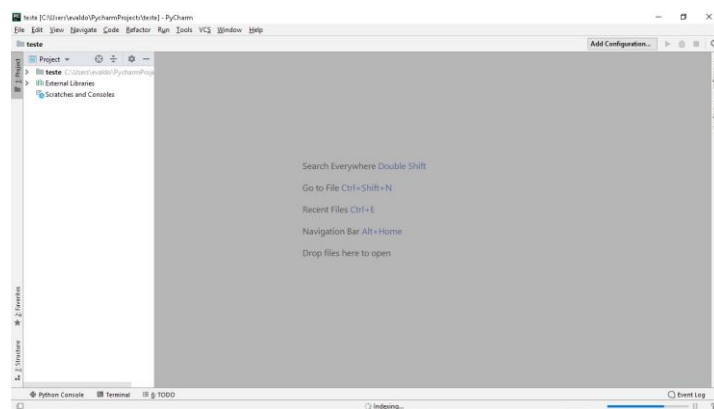


Figura 36 - Janela principal do PyCharm

Para criar nosso primeiro arquivo Python no PyCharm, basta clicar com o botão direito na pasta do projeto “teste”, escolher “New / Novo” e por fim “Python File / Arquivo Python”, conforme imagem a seguir.

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br

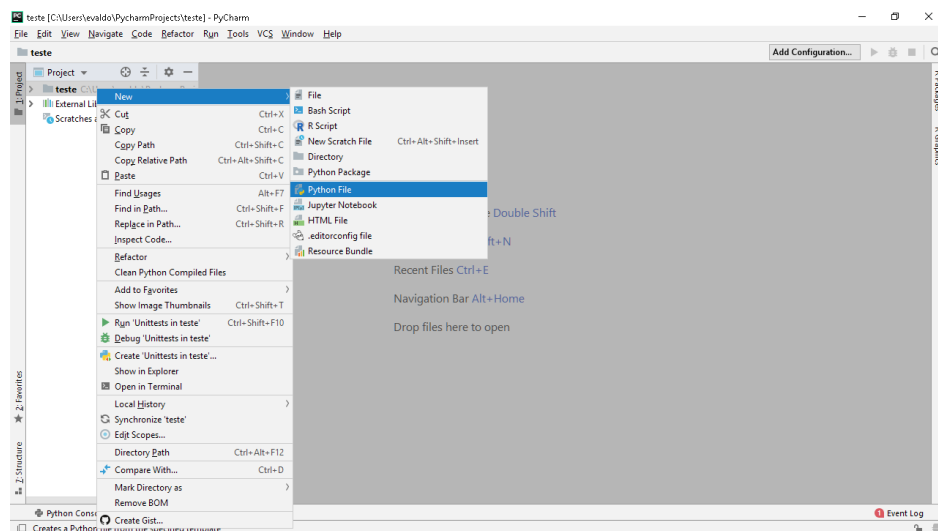


Figura 37 - Criando um arquivo Python

Ao clicar nesta opção, será solicitado o nome do arquivo. Informe teste conforme imagem a seguir. Pode informar também teste.py, mas o py não é necessário porque o PyCharm coloca o py quando criamos um arquivo Python.

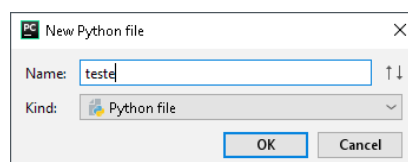


Figura 38 - Informando o nome do arquivo

Ao clicar em OK, o arquivo será exibido no PyCharm. Você já pode começar a escrever seu programa.

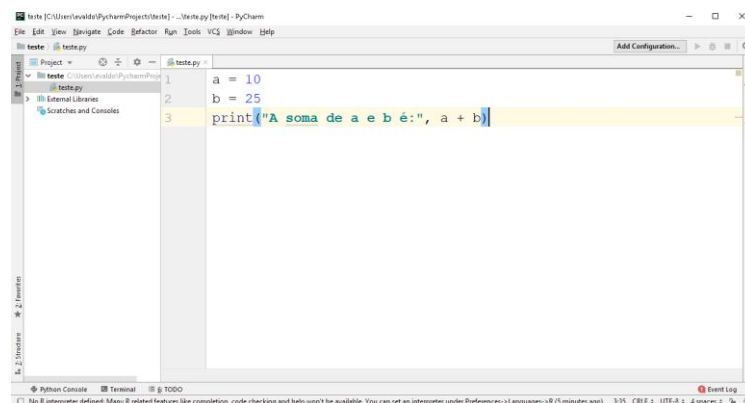


Figura 39 - Escrevendo o programa

Para executar o programa, pressione Ctrl + Shift + F10, ou clique com o botão direito do mouse na aba que exibe o nome do arquivo e escolha a opção "Run 'nome_arquivo'", que no caso é "Run 'teste'".

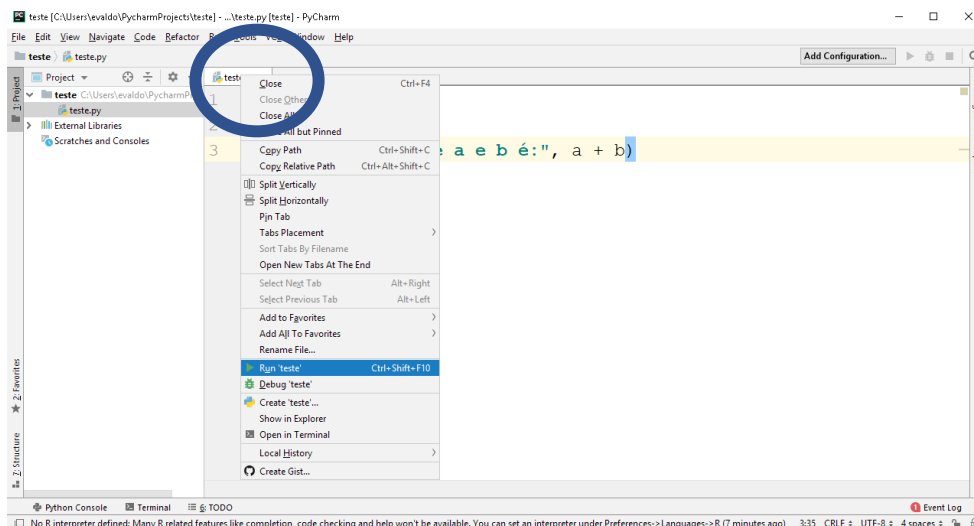


Figura 40 - Executando o programa

O resultado será exibido na parte inferior da janela do PyCharm conforme imagem a seguir.

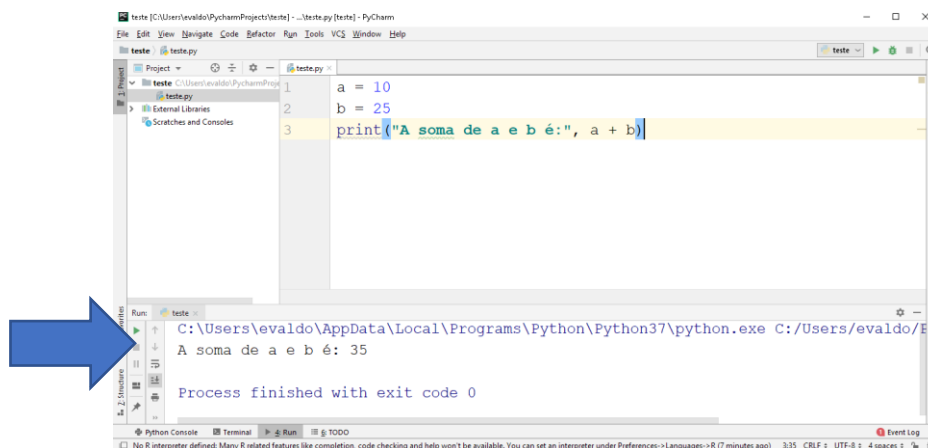


Figura 41 - Programa executado

Instalando o PyCharm no Linux

Após baixar o instalador para Linux, devemos acessar a pasta onde o arquivo foi salvo e descompactar o arquivo usando o comando a seguir.

```
$ tar xzf pycharm-community-2018.3.2.tar.gz
```

Após descompactar será criada a pasta pycharm-community-2018.3.2.

Devemos entrar nesta pasta com o comando “cd pycharm-community-2018.3.2”.

Em seguida devemos entrar na pasta bin com o comando “cd bin”.

Em seguida devemos executar o instalador com o seguinte comando:

```
$ sh pycharm.sh
```



Figura 42 - Executando a instalação

Será exibida uma tela perguntando se desejamos importar configurações existentes, deixe marcada a opção para não importar e clique em OK.

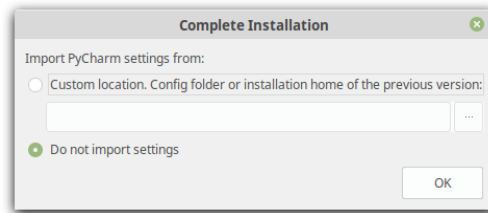


Figura 43 - Importar configurações

Marque a opção para aceitar os termos de uso, e clique em “Continue / Continuar”.

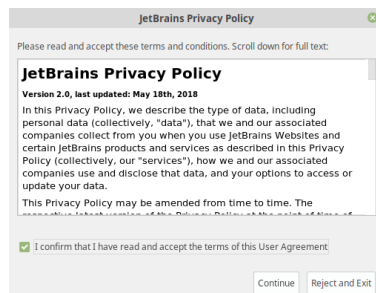


Figura 44 - Termos de uso

Em seguida será exibida uma solicitação de envio de estatísticas de uso. Você pode aceitar enviar estes dados ou não.

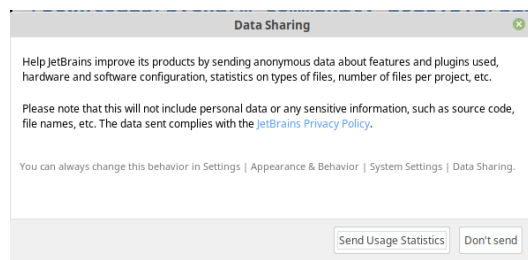


Figura 45 - Envio de estatísticas de uso

Na tela seguinte você poderá escolher o tema.

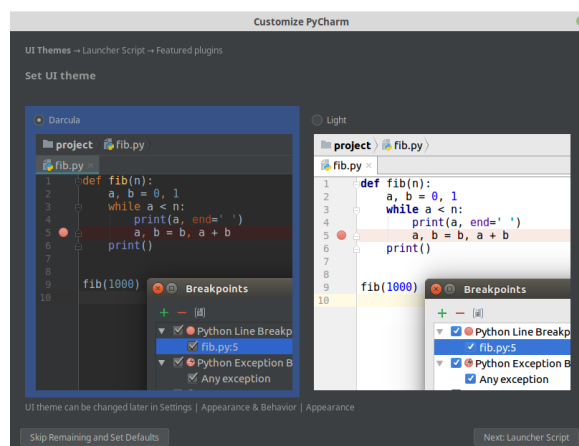


Figura 46 - Definindo o tema

Na janela seguinte, marque a opção para criar um script para abrir arquivos direto do terminal com o comando “charm arquivo.py” ou simplesmente “charm” (sem as aspas) para abrir o PyCharm.

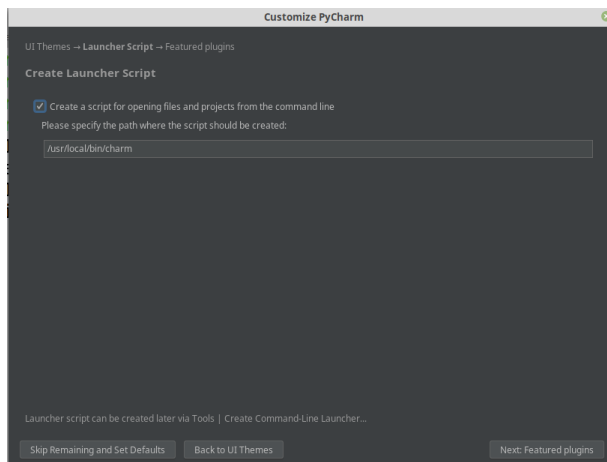


Figura 47 - Script charm

Na próxima tela é possível instalar alguns plug-ins, mas isso não é necessário, pode clicar em “Start using PyCharm / Comece a usar o PyCharm”.

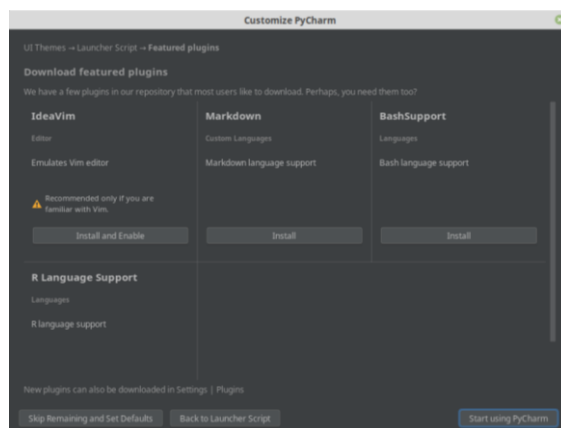


Figura 48 - Instalação de plugins

Se for solicitado, informe a senha de root para continuar.

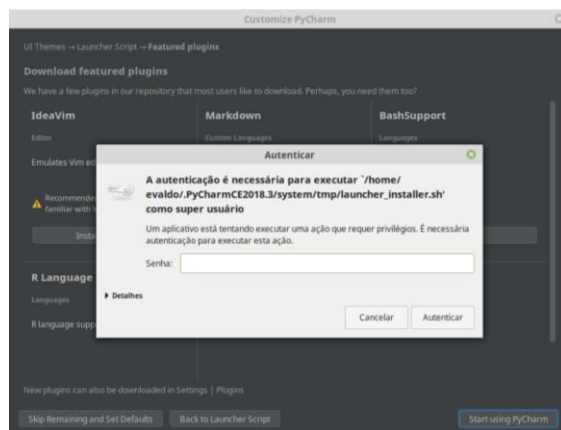


Figura 49 - Informando a senha de root



Em seguida será exibida a tela de boas-vindas do PyCharm.

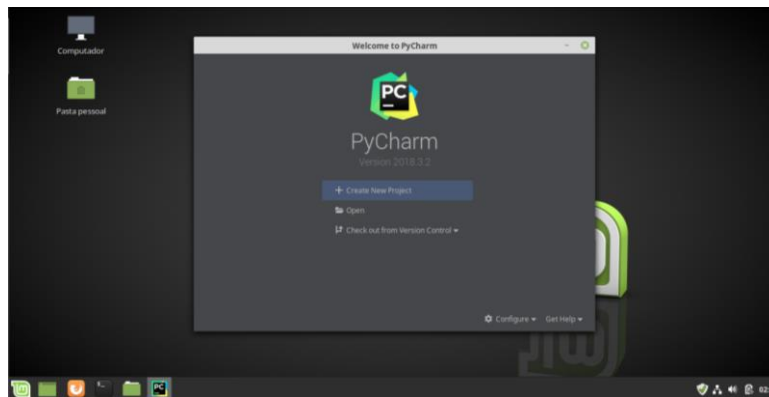


Figura 50 - Tela de boas-vindas do PyCharm

Usando um interpretador online

Caso você não queira instalar um ambiente em seu computador para começar a escrever seus primeiros códigos, saiba que existem alternativas online.

Basta fazer uma busca no Google por “Interpretador Python online” que serão exibidos vários resultados.

São alternativas para quem está começando no mundo da programação poder escrever pequenos códigos e executar no browser.

Veja alguns exemplos de interpretadores online:

- ✓ https://www.onlinegdb.com/online_python_interpreter
- ✓ <https://repl.it/languages/python3>
- ✓ <https://www.python.org/shell/>
- ✓ <https://www.pythonanywhere.com/try-ipython/>
- ✓ <https://py3.codeskulptor.org/>

Conceitos básicos

Blocos de Código

- Blocos de código são delimitados pelo uso de indentação.
- A indentação deve ser constante no bloco de código.
- É uma boa prática não misturar tabulação com espaços.
- Usar quatro espaços para indentação é uma convenção amplamente aceita, além de ser uma recomendação oficial (<http://python.org/dev/peps/pep-0008>).

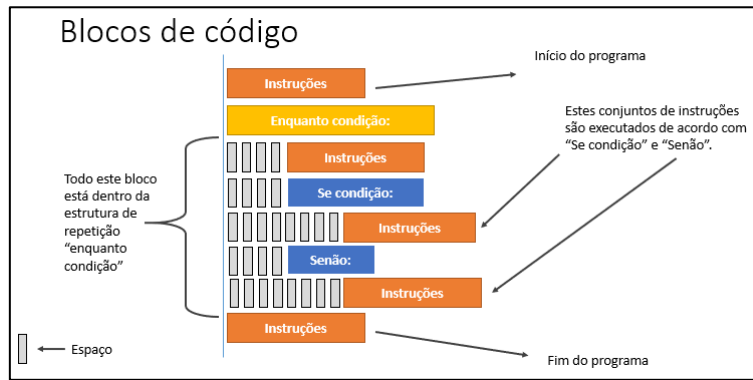


Figura 51 - Blocos de código

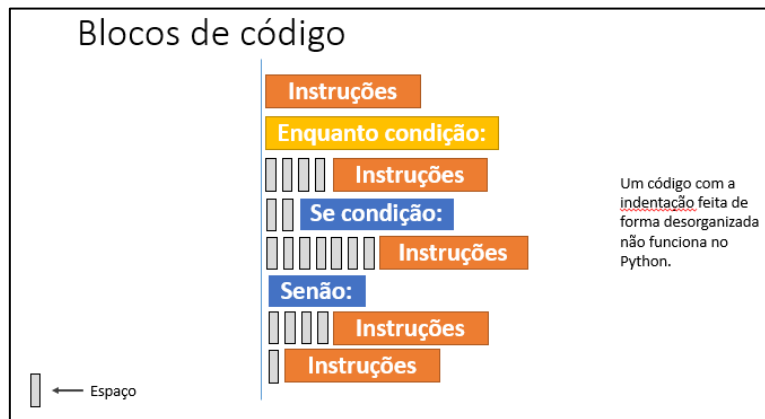


Figura 52 - Blocos de código

- O caractere de dois pontos ":" determina a linha anterior a um bloco. Representa uma estrutura de controle da linguagem ou a declaração de uma nova estrutura (uma função, por exemplo).
- Se você não indentar corretamente seu código o mesmo não será executado.

Objetos

Python é uma linguagem orientada a objeto.

As estruturas de dados possuem atributos e métodos.

Os atributos e métodos de um objeto são acessados utilizando o ponto ".".

Para mostrar um atributo de um objeto usamos "objeto.atributo".

Para executar um método usamos "objeto.método(argumentos)" ou "objeto.método()".

Mesmo métodos sem argumentos precisam dos parênteses.

Atributos

```
carro.combustivel = gasolina
```

```
carro.farol = aceso
```

Métodos



```
carro.andar(frente)
Carro.virar(direita)
```

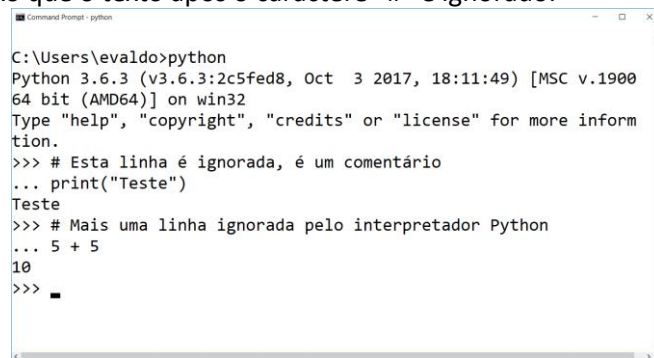
Tudo no Python é considerado objeto. No curso teremos uma seção completa somente sobre Orientação a Objetos.

Comentários de código

São textos ignorados pelos interpretadores ou compiladores.

Para definir uma linha inteira como comentário você deve usar o caractere “#”.

Veja no exemplo abaixo que o texto após o caractere “#” é ignorado.



```
C:\Users\evaldo>python
Python 3.6.3 (v3.6.3:2c5fed8, Oct 3 2017, 18:11:49) [MSC v.1900
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more inform
tion.
>>> # Esta linha é ignorada, é um comentário
... print("Teste")
Teste
>>> # Mais uma linha ignorada pelo interpretador Python
... 5 + 5
10
>>>
```

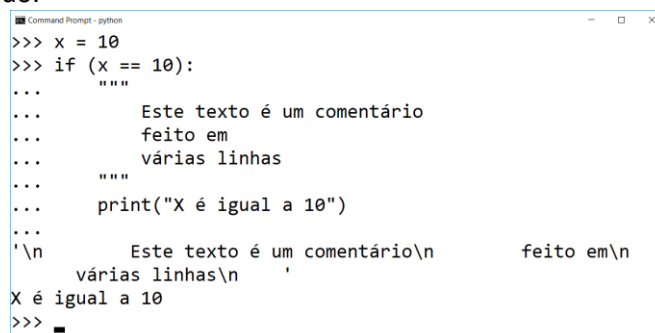
Figura 53 - Comentários de código

Para comentar várias linhas não é necessário colocar um “#” em cada uma das linhas. Para isso usamos três aspas como delimitador “""" ou ”””.

As três aspas são as docstrings que são usadas para documentar o código.

Este curso tem uma seção sobre documentação.

Veja na imagem abaixo que o código é executado sem erros, o Python apenas reproduz o texto do comentário no resultado.



```
>>> x = 10
>>> if (x == 10):
...     """
...     Este texto é um comentário
...     feito em
...     várias linhas
...     """
...     print("X é igual a 10")
...
'\n     Este texto é um comentário\n          feito em\nvárias linhas\n'
X é igual a 10
>>>
```

Figura 54 - Comentário de várias linhas

Operações Matemáticas

Podemos usar o interpretador do Python como se fosse uma calculadora para realizar operações matemáticas. Por exemplo, se você digitar 10 + 10, como no exemplo abaixo e der um ENTER, ele vai realizar a soma e exibirá 20.



```

C:\Users\evaldo>python
Python 3.6.3 (v3.6.3:2c5fed8, Oct 3 2017, 18:11:49) [MSC v
.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more i
nformation.
>>> 10 + 10
20
>>> 84 - 37
47
>>>

```

Figura 55 - Operações matemáticas

TABELA DE OPERAÇÕES MATEMÁTICAS

+	Soma
-	Subtração
*	Multiplicação
/	Divisão (Retorna sempre número real, mesmo que aplicado à dois inteiros)
//	Divisão inteira (Retorna sempre um inteiro truncado para o valor imediatamente inferior, mesmo que seja aplicado em números reais)
%	Resto da divisão
**	Exponenciação
**	Radiciação: Pode ser obtida através de expoentes fracionários.

Soma, subtração e multiplicação:

<pre> >>> 158 + 74 232 >>> 358.58 + 71 429.58 >>> 154 - 41 113 </pre>	<pre> >>> 123 - 74.5 48.5 >>> 25 * 8 200 >>> 25.6 * 4 102.4 </pre>
--	---

Divisão e divisão inteira

<pre> >>> 58 / 3 19.333333333333332 >>> 58 / 2.5 23.2 </pre>	<pre> >>> 58 // 3 19 >>> 58 // 2.5 23.0 </pre>
--	--

Módulo, potenciação e radiciação

<pre> >>> 100 % 3 1 >>> 9 % 1.5 0.0 >>> 9 % 1.4 0.60000000000000005 </pre>	<pre> >>> 2 ** 3 8 >>> 9 ** (1/2) 3.0 >>> 9 ** (1/3) 2.080083823051904 </pre>
---	--

Variáveis e tipos de dados



Nomes de variáveis

Em matemática usamos variáveis para representar incógnitas em equações.

Exemplo: $x + 6 = 8$

Devemos determinar o valor de “x” resolvendo a equação.

Em programação, usamos variáveis para armazenar valores na memória do computador.

Para poder trabalhar com estes valores, damos nomes à estas variáveis.

Desta forma, não precisamos acessar diretamente endereços da memória do computador para pegar seu conteúdo.

Em Python, nomes de variáveis devem iniciar obrigatoriamente com uma letra ou com o caractere underscore “_”, podendo ter números em seu nome.

A versão 3 da linguagem Python permite a utilização de acentos em nomes de variáveis, pois, por padrão, os programas são interpretados utilizando-se um conjunto de caracteres chamado UTF-8, capaz de representar praticamente todas as letras dos alfabetos conhecidos (<http://pt.wikipedia.org/wiki/Utf-8>).

Veja alguns exemplos permitidos de nomes de variáveis:

Nome	Permitido?	Comentários
a1	Sim	Pode conter número, contanto que não seja no início.
carro	Sim	Somente letras
carro_anos_70	Sim	O underscore é permitido e facilita a leitura.
Veículo rápido	Não	Não pode conter espaços.
_bicicleta	Sim	Underscore é aceito mesmo que no início do nome.
1aviao	Não	Não pode iniciar com um número.

Ao criar uma variável, é uma boa prática que a mesma tenha um nome que condiz com seu conteúdo, como por exemplo: salario, valor_aluguel, preco_produto. Assim, outras pessoas entenderão mais facilmente do que se trata a variável quando forem dar manutenção no código.

Apesar do Python permitir, não é uma boa prática criar variáveis acentuadas, pois, muitas outras linguagens não permitem, sendo assim, caso você for escrever um programa em outra linguagem, não terá o costume de acentuar, evitando assim, possíveis erros.

Além de nome e conteúdo uma variável possui outras propriedades, sendo uma delas o tipo que define a natureza dos dados que ela armazena.

Python possui vários tipos de dados.

Em capítulos futuros vamos detalhar estes tipos de dados que as variáveis podem armazenar.

Atribuição

Em Python usamos o caractere “=” para atribuição de valores às variáveis.

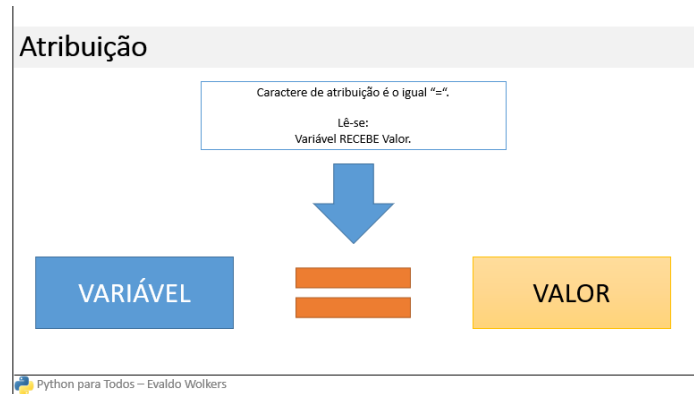


Figura 56 - Atribuição

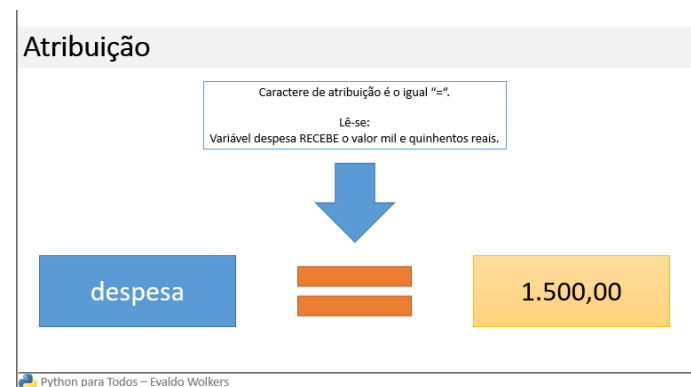


Figura 57 - Atribuição

Veja no exemplo a seguir onde o texto “Olá, eu sou um texto!!!” está sendo atribuído à variável denominada “fala” e o valor 55 à variável denominada “número”.

```
Terminal
File Edit View Search Terminal Help
Type "help", "copyright", "credits" or "license" for more
information.
>>>
>>> fala = "Olá, eu sou um texto!!!"
>>> número = 55
>>> print(fala)
Olá, eu sou um texto!!!
>>> print(número + 3)
58
>>> type(fala)
<class 'str'>
>>> type(número)
<class 'int'>
>>>
```

Figura 58 – Atribuição

Variáveis numéricas

Existem três tipos numéricos distintos:

Inteiros (*int*): $i = 1$

Números de ponto flutuante (*float*): $f = 1.28$

Números complexos (*complex*): $c = 3 + 4j$

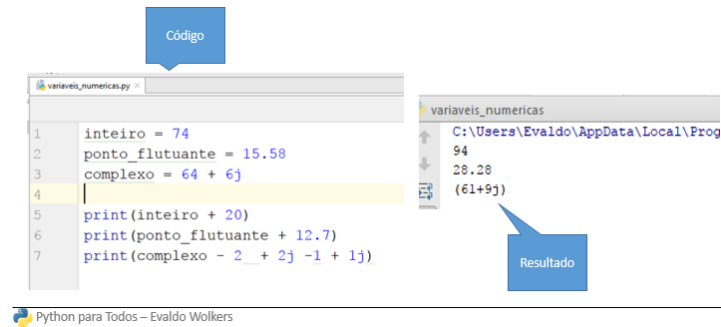


Figura 59 - Variáveis numéricas

Função print()

Até agora usamos a função print, mas não detalhamos seu funcionamento.

Usamos a função print para exibir uma informação ao usuário do nosso sistema.

Por exemplo, se quisermos exibir uma frase como esta: "Olá, tenha um bom dia", usamos desta forma:

```
>>> print("Olá, tenha um bom dia")
Olá, tenha um bom dia
>>>
```

Figura 60 - Imprimindo um texto

O texto entre as aspas é chamado de "string", uma string é simplesmente um texto (você verá mais detalhes quando falarmos de tipos de dados).

Se quisermos imprimir o conteúdo de uma variável, também podemos usar o print, por exemplo, se você tem uma variável chamada total que possui o valor 25, pode imprimir seu conteúdo assim:

```
>>> x = 10
>>> y = 15
>>> total = x + y
>>> print(total)
25
>>>
```

Figura 61 - Imprimindo o conteúdo de uma variável

E se você quiser juntar texto com variável, para imprimir por exemplo, "O valor da bicicleta é 500", onde este "500" não será digitado, mas estará dentro de uma variável. Pode fazer assim:

```
>>> valor = 500
>>> print("O valor da bicicleta é", valor)
O valor da bicicleta é 500
>>>
```

Figura 62 - Imprimindo texto e conteúdo de variável

O Python junta o texto e o conteúdo da variável já acrescentando espaços, por isso depois do "é" não coloquei espaço.



Como você pôde ver, podemos imprimir diversos valores, sejam textos, sejam variáveis, passando estas informações separadas por vírgula para o comando print.

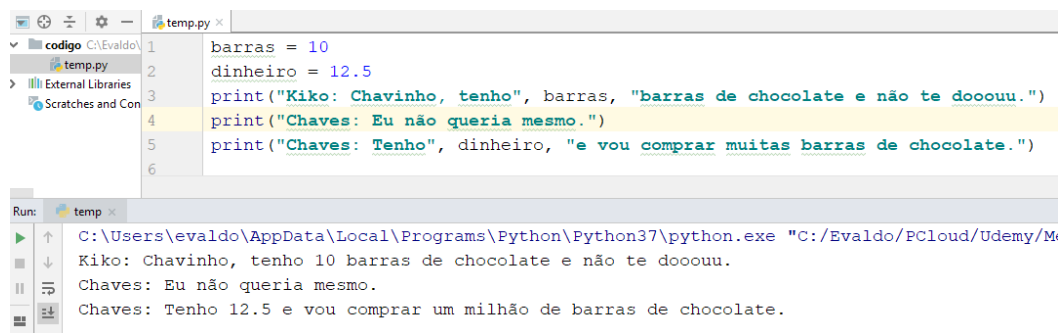


Figura 63 - Exemplo função print()

Se você não quer separar usando espaço, você pode modificar o caracter separador acrescentando o parâmetro "sep", passando o texto que quer usar como separador. Desta forma:

```
>>>
>>> print("Olá", 10, 25.8, "Teste", sep=";")
Olá;10;25.8;Teste
>>>
```

Figura 64 - Alterando o separador do print()

Neste exemplo eu coloquei um ponto e vírgula e um parênteses ";" como separador. Sendo assim, entre "Olá", 10, 25.8 e "Teste", haverá um ";" em vez de um espaço.

Variáveis do tipo lógico e operadores relacionais

Variáveis do tipo lógico são também conhecidas como variáveis booleanas. Elas armazenam apenas dois valores, que são:

- Verdadeiro = True
- Falso = False

Exemplo de código com seu resultado:

```
esta_frio = True
esta_chovendo = False

print(esta_frio)
print(esta_chovendo)
```

Figura 65 - Variáveis booleanas

```
C:\Users\Evaldo\AppData\Local\Programs\Python\Python36\python.exe C:/
True
False
```

Figura 66 - Resultado do código



Para realizarmos operações lógicas, utilizamos os operadores relacionais. O resultado destas operações será sempre True ou False.

Operador	Operação	Símbolo matemático
==	Igualdade	=
>	Maior que	>
<	Menor que	<
!=	Diferente	≠
>=	Maior ou igual	≥
<=	Menor ou igual	≤

Veja um exemplo de código onde estão sendo feitas algumas comparações e o resultado:

```
a = 3
b = 5

comparacao = (a == b)
print("1) a é igual a b:", comparacao)

comparacao = (a > b)
print("2) a é maior que b:", comparacao)

comparacao = (a <= b)
print("3) a é menor ou igual a b:", comparacao)
```

Figura 67 - Variáveis do tipo lógico e operadores relacionais

```
logico
C:\Users\Evaldo\AppData\Local\Progr
1) a é igual a b: False
2) a é maior que b: False
3) a é menor ou igual a b: True
```

Figura 68 - Resultado do código

Operadores lógicos e expressões lógicas

Operadores lógicos são operadores utilizados para agrupar operações com lógica booleana.

Python suporta três operadores básicos:

- Não (*not*) → Negação ¬
- E (*and*) → Conjunção ∧
- Ou (*or*) → Disjunção ∨

Cada operador obedece a um conjunto simples de regras, expresso pela tabela verdade deste operador.

A tabela verdade demonstra o resultado de uma operação com um ou dois valores lógicos ou operandos.

Quando o operador utiliza apenas um operando, dizemos que é um operador unário. Ao utilizar dois operandos, é chamado de operador binário.

O operador "*not*" (não/negação) é um operador unário, já os operadores *or* "ou" e *and* "e" são operadores binários, precisando, assim, de dois operandos.



O operador *not* (não) precisa de apenas um operando. A operação de negação também é chamada de inversão, porque um valor verdadeiro negado se torna falso e vice-versa.

Tabela verdade do operador <i>not</i> (não)	
V1	not V1
V	F
F	V

```

1 a = not True
2 b = not False
3
4 print("a =", a)
5 print("b =", b)

```

operadores_logicos

C:\Users\Evaldo\AppData\Local\Programs\Python\Python36\python.exe C:\Users\Evaldo\AppData\Local\Programs\Python\Python36\operadores_logicos.py

a = False
b = True

Figura 69 - Operador NOT

O operador *and* (e) resulta verdadeiro apenas quando seus dois operandos forem verdadeiros.

Tabela verdade do operador and (e)		
V1	V2	V1 and V2
V	V	V
V	F	F
F	V	F
F	F	F

```

1 a = True and True
2 b = True and False
3 c = False and True
4 d = False and False
5
6 print("a =", a)
7 print("b =", b)
8 print("c =", c)
9 print("d =", d)

```

operadores_logicos

C:\Users\Evaldo\AppData\Local\Programs\Python\Python36\python.exe C:\Users\Evaldo\AppData\Local\Programs\Python\Python36\operadores_logicos.py

a = True
b = False
c = False
d = False

Figura 70 - Operador E

```

1 """
2 Exemplo Operadores Lógicos
3 """
4 a = 10
5 proposicao1 = (a > 1) # True
6 proposicao2 = (a == 5) # False
7
8 resultado = proposicao1 and proposicao2
9 print(resultado)

```

operadores_logicos.py

exemplo_input exemplo_input exemplo_input operadores_logicos

C:\Users\Evaldo\AppData\Local\Programs\Python\Python36\python.exe C:\Users\Evaldo\AppData\Local\Programs\Python\Python36\operadores_logicos.py

False

Figura 71 - Operador E

O operador *or* (ou) resulta falso apenas quando seus dois operandos forem falsos.

Tabela verdade do operador or (ou)		
V1	V2	V1 or V2
V	V	V
V	F	V



F	V	V
F	F	F

```

1 a = True or True
2 b = True or False
3 c = False or True
4 d = False or False
5
6 print("a =", a)
7 print("b =", b)
8 print("c =", c)
9 print("d =", d)

```

operadores_logicos

```

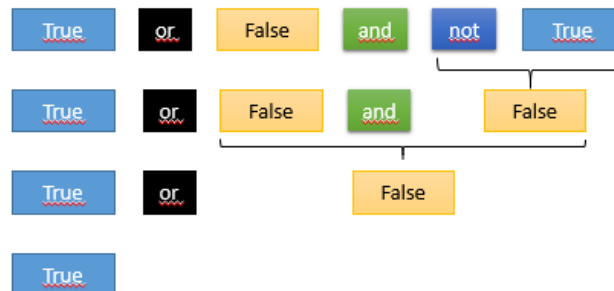
C:\Users\Evaldo\AppData\Local\Programs\Python\Py
a = True
b = True
c = True
d = False

```

Figura 72 - Operador OR

Os operadores lógicos podem ser combinados em expressões lógicas mais complexas. Quando uma expressão tiver mais de um operador lógico, avalia-se o operador *not* (não) primeiramente, seguido do operador *and* (e) e, finalmente o operador *or* (ou).

Ordem de avaliação de expressão lógica: not, and, or.



Operadores relacionais podem ser usados em expressões com operadores lógicos.

Exemplo: Salário > 1000 and idade > 18

Nesses casos, os operadores relacionais devem ser avaliados primeiramente.

Exemplo:

Salário = 937 e idade = 20

Salário > 1000 and idade > 18

937 > 1000 and 20 > 18

False and True

False

```

Command Prompt - python
>>> salario = 937
>>> idade = 18
>>> salario > 1000
False
>>> idade > 18
False
>>> salario > 1000 and idade > 18
False
>>>

```

Figura 73 - Expressão lógica



Variáveis do tipo string

Variáveis do tipo string são utilizadas para armazenar sequências de caracteres.

Usamos muito variáveis do tipo string, seja para mensagens exibidas para o usuário ou geração de informações em arquivos.

Para retornar o tamanho de uma string usamos a função `len`.

Esta função retorna um resultado do tipo inteiro contendo o número de caracteres da string.

```

1 frase = "Aula de Python sobre strings!"
2 tamanho_frase = len(frase)
3
4 print(tamanho_frase)

```

variáveis_texto

29

Figura 74 - Imprimindo o tamanho da string

Podemos acessar o conteúdo de uma string caractere a caractere. Sabendo o tamanho da string, podemos acessar seus caracteres usando um número inteiro para representar a posição de um determinado caractere.

O índice dos caracteres de uma string inicia com zero.

P	Y	T	H	O	N		P	A	R	A		T	O	D	O	S	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	

Conteúdo

Índice

Para acessarmos determinado caractere usamos o nome da variável e o índice desejado entre colchetes.

```
curso = "PYTHON PARA TODOS"
```

Para retornar a letra "Y" da variável `curso`, utilizamos:

```
curso[1]
```

Figura 75 - Usando índices de string

Veja agora como imprimir um caractere da string através do índice.

```

1 curso = "PYTHON PARA TODOS"
2
3 print(curso[1])

```

variáveis_texto

C:\Users\Evaldo\AppData\Local\Programs\Python\Python36\pyt

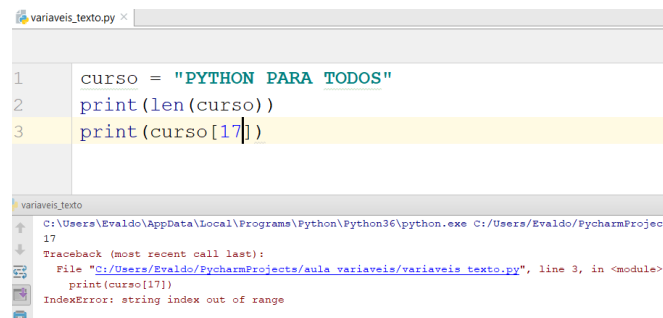
Y

Process finished with exit code 0

Figura 76 - Usando índices de string

Veja o que acontece quando usamos um índice inexistente. A variável possui o tamanho 17, ou seja, possui 17 caracteres. Então o índice vai de zero à 16. O índice 17 não existe, por isso a mensagem de erro:

“Índice da string fora da faixa”.



```
1 curso = "PYTHON PARA TODOS"
2 print(len(curso))
3 print(curso[17])

Traceback (most recent call last):
  File "C:/Users/Evaldo/PycharmProjects/aula_variaveis/variaveis_texto.py", line 3, in <module>
    print(curso[17])
IndexError: string index out of range
```

Figura 77 - Índice fora da faixa

Tipagem estática e dinâmica

Um sistema de tipos é um conjunto de regras que atribuem uma propriedade chamada de **tipo** para várias construções que um programa de computador é composto (variáveis, expressões, funções ou módulos).

Uma linguagem de programação pode ser classificada de acordo com o seu sistema de tipos. Vamos entender o que é linguagem de tipagem estática e linguagem de tipagem dinâmica.

Em uma linguagem de tipagem estática, verificações de tipo de dados são feitas em tempo de compilação.

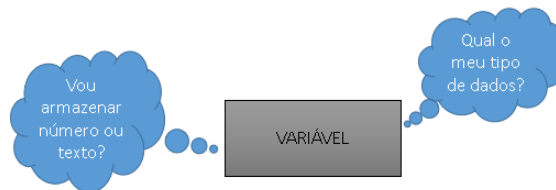


Figura 78 - Tipagem estática

Já em linguagem de tipagem dinâmica, as verificações são feitas em tempo de execução.

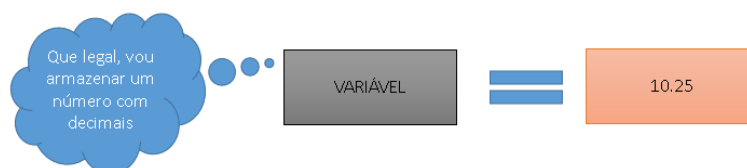


Figura 79 - Tipagem dinâmica

Em uma linguagem de tipagem estática é feita uma verificação dos tipos usados em dados e variáveis para garantir que se está usando um tipo de dados que é esperado em todas as situações. O compilador oferece algumas garantias que alguns problemas não irão ocorrer após o programa passar por esta verificação. Os erros são verificados antes do sistema ser executado. Mas isso não te salva de todos os erros que podem acontecer.

O compilador deve conhecer o tipo de uma variável ou método antes da execução do programa. O tipo de uma variável é explicitamente declarado. Uma variável não pode mudar seu tipo em tempo de execução.

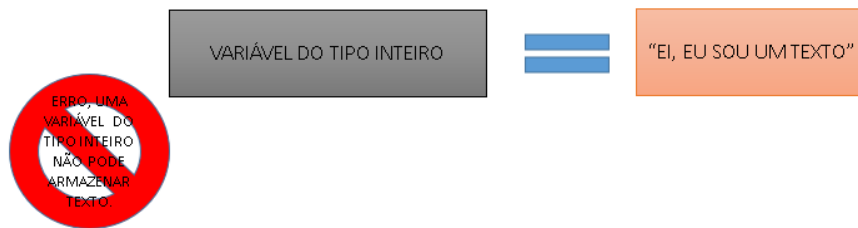


Figura 80 - Erro de tipo de dados

Exemplo de linguagens de tipagem estática: C, C++, Java, C#, Cobol.

Exemplo de código Java:

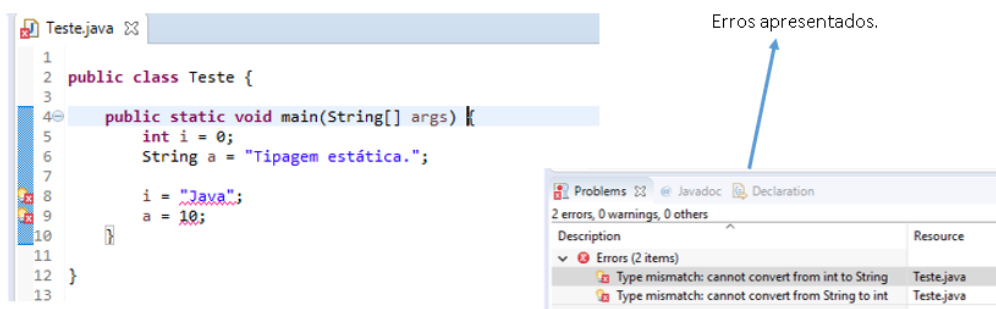


Figura 81 - Exemplo de código Java

Em uma linguagem de tipagem dinâmica, também há uma verificação, mas somente em cima dos dados, já que uma variável pode conter qualquer tipo de dado.

É dada ao programador a responsabilidade de realizar testes para garantir que todos os tipos estão corretos nos momentos certos.

O tipo de uma variável ou método é desconhecido até a execução do programa.

Exemplo de linguagens de tipagem dinâmica: Python, Perl, Ruby, PHP, Lisp.

Exemplo de código Python:

```
i = 0
a = "Tipagem dinâmica."

i = "Python"
a = 10
```

Figura 82 - Tipagem dinâmica

Python é uma linguagem fortemente tipada. Isso quer dizer que o interpretador do Python avalia as expressões e não faz coerções automáticas entre tipos de dados não compatíveis.

O Python não converte automaticamente os tipos de dados, fazendo com que o resultado seja mais consistente.

Exemplo de código Python:

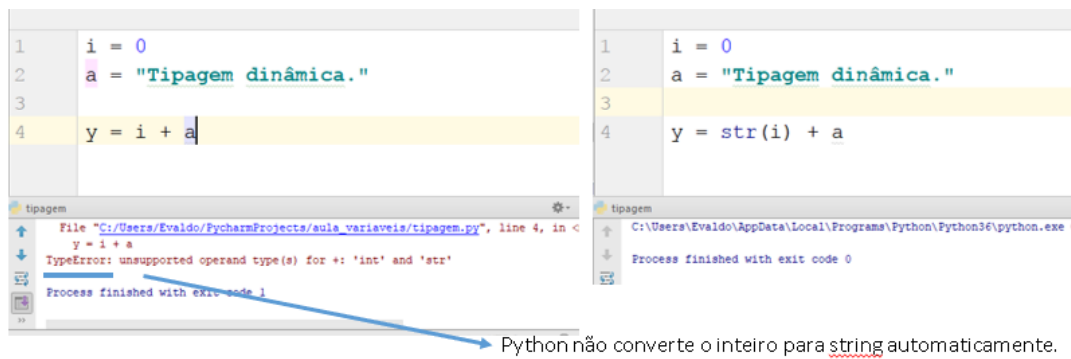


Figura 83 - Exemplo código Python

Mais um exemplo em Python:

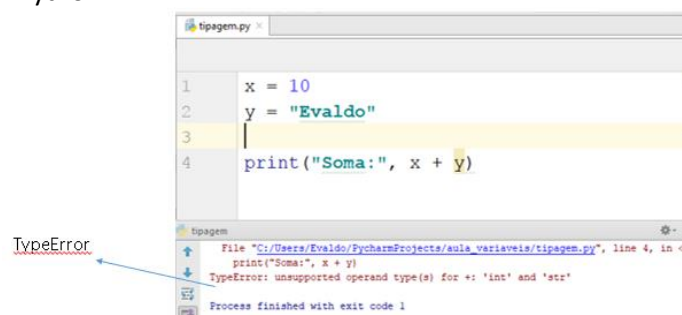


Figura 84 - Gerando um TypeError

Java também é uma linguagem de tipagem forte:

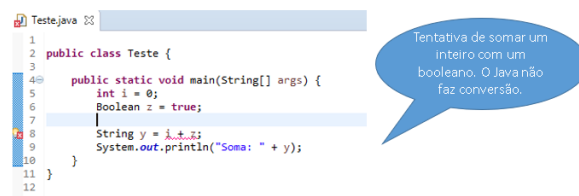


Figura 85 - Tipagem forte

Exemplo de código Java:

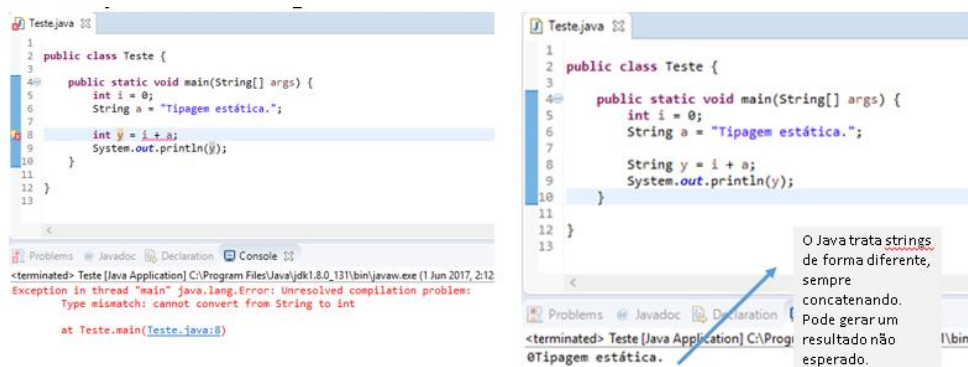


Figura 86 - Exemplo Tipagem (Java)



JavaScript é uma linguagem de tipagem fraca. Isso quer dizer que o JavaScript faz coerção de tipos ao executar operações de forma automática.

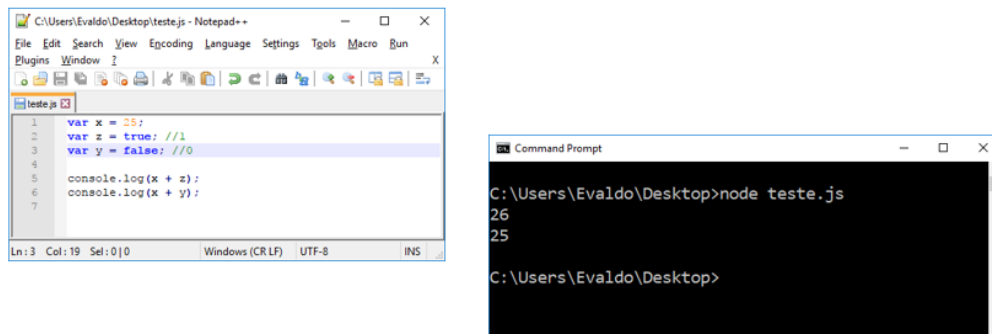


Figura 87 – Exemplo tipagem (JavaScript)

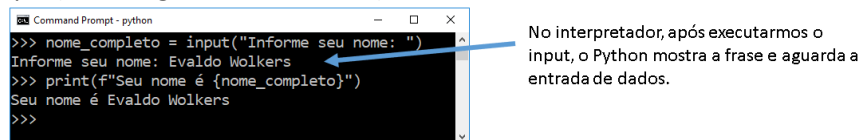
Entrada de dados e conversão de dados

Até esta aula em nossos exemplos, as variáveis receberam valores fornecidos diretamente no código-fonte. Vou mostrar agora como capturar dados informados pelo usuário. Desta forma, o mesmo programa poderá ser usado diversas vezes com valores fornecidos a cada execução. Entrada de dados é o momento em que o programa recebe dados ou valores através de dispositivos de entrada como teclado ou através de um arquivo em disco.

Para solicitar informações ao usuário usamos a função **input**.

A função **input** possui um parâmetro do tipo *string* que é a mensagem a ser exibida ao usuário e retorna também uma *string* que é a resposta fornecida pelo usuário.

`resposta = input("Mensagem a ser exibida")`



No PyCharm, usamos a janela de execução para a entrada de dados.

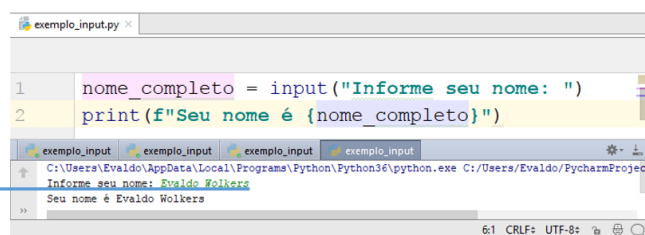


Figura 88 - Utilizando input



```
1 resposta = input("Informe a resposta correta: \n")
2     "Qual linguagem você está aprendendo agora?\n"
3     "a) Python\n"
4     "b) Java\n"
5     "resposta: ")
6
7 print("Sua resposta foi: ", resposta)
```

The screenshot shows a terminal window with the following output:

```
Informe a resposta correta:
Qual linguagem você está aprendendo agora?
a) Python
b) Java
resposta: a
Sua resposta foi: a
```

Figura 89 - Utilizando input

Ao tentar somar uma variável do tipo inteiro com uma string obtemos o erro *TypeError* (Erro de tipo).

```
1 valor_fixo = 45
2 resposta = input("Informe um número de 1 a 10: ")
3 soma = valor_fixo + resposta
4 print(resposta)
```

The screenshot shows a terminal window with the following output:

```
Informe um número de 1 a 10: 5
Traceback (most recent call last):
  File "C:/Users/Evaldo/PycharmProjects/aula_variaveis/exemplo_input.py", line 4, in <module>
    soma = valor_fixo + resposta
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Figura 90 - TypeError

A mesma coisa ocorre quando tentamos somar um float e uma string.

```
1 valor_fixo = 45.8
2 resposta = input("Informe um número de 1 a 10: ")
3 soma = valor_fixo + resposta
4 print(resposta)
```

The screenshot shows a terminal window with the following output:

```
Informe um número de 1 a 10: 7
Traceback (most recent call last):
  File "C:/Users/Evaldo/PycharmProjects/aula_variaveis/exemplo_input.py", line 3, in <module>
    soma = valor_fixo + resposta
TypeError: unsupported operand type(s) for +: 'float' and 'str'
```

Figura 91 - TypeError

Para converter o valor informado pelo usuário em um inteiro usamos a função *int*.
Para converter em um valor com decimais usamos a função *float*.



```

1 valor_unitario = float(input("Informe o valor unitário do produto: "))
2 quantidade = int(input("Informe a quantidade de produtos: "))
3 print(f"O total da compra é: {valor_unitario * quantidade}")

C:\Users\Evaldo\AppData\Local\Programs\Python\Python36\python.exe C:/Users/Evaldo/PycharmProjects/aula_variaveis/exemplo_input.py
Informe o valor unitário do produto: 25.75
Informe a quantidade de produtos: 17
O total da compra é: 437.75

```

Figura 92 - Conversão para float

Aprendendo mais sobre strings

Concatenação

Concatenar é unir strings. Como se estivéssemos “somando” os textos. Para concatenar usamos o operador de adição “+”.

```

>>> texto1 = "Olá, "
>>> texto2 = "tudo bem?"
>>> texto_concatenado = texto1 + texto2
>>> print(texto_concatenado)
Olá, tudo bem?
>>>

```

Figura 93 - Concatenando string

Você pode usar o operador de multiplicação “*” sobre uma string e informar quantas vezes ela vai se repetir.

```

>>> alimento = "pão "
>>> multiplica = alimento * 10
>>> print(multiplica)
pão pão pão pão pão pão pão pão pão pão
>>>

```



Figura 94 - Multiplicando string

Composição é a forma de juntar vários conteúdos em uma única string. Estes conteúdos podem ser outras strings ou até de outros tipos de dados.

A interpolação é mais eficiente no uso de memória que a concatenação convencional.

O operador “%”, denominado “marcador de posição”, é usado para definir o local onde será inserido o conteúdo na string.

Composição ou Interpolação

Veja nesta tabela alguns símbolos usados na composição.

%s	String
%d	Inteiro
%f	Real
%%	Sinal de porcentagem



Onde estes símbolos forem colocados dentro de uma string, serão substituídos pelos valores informados.

Usando %s para compor strings:

```
Command Prompt - python
>>>
>>> nome = "Evaldo"
>>> frase = "O instrutor %s está explicando composição de strings" % nome
>>> print(frase)
O instrutor Evaldo está explicando composição de strings
>>>
```

Quero colocar a string aqui.

Esta é a string que quero colocar no lugar do marcador (%s)

Figura 95 - Composição de strings

Usando %d para números inteiros:

```
Command Prompt - python
>>> nome = "João da Silva"
>>> idade = 38
>>> filhos = 2
>>> frase = "O jogador %s tem %d anos de idade e tem %d filhos." % (nome, idade, filhos)
>>> print(frase)
O jogador João da Silva tem 38 anos de idade e tem 2 filhos.
>>>
```

Figura 96 - Composição de inteiros

```
Command Prompt - python
>>>
>>> print("Esta é a aula de número %03d" % aula)
Esta é a aula de número 009
>>> print("Esta é a aula de número %3d" % aula)
Esta é a aula de número 9
>>> print("Esta é a aula de número [%3d]" % aula)
Esta é a aula de número [ 9]
>>> print("Esta é a aula de número [%03d]" % aula)
Esta é a aula de número [009]
>>> print("Esta é a aula de número [%-3d]" % aula)
Esta é a aula de número [9 ]
>>>
```

Figura 97 - Composição de inteiros

Usando %f para números reais:

```
Command Prompt - python
>>>
>>>
>>> valor_unitario = 165.78
>>> quantidade = 5
>>> valor_total = valor_unitario * quantidade
>>> frase = "O produto custa %f. Eu comprei %d. Paguei ao todo %f" % (valor_unitario, quantidade, valor_total)
>>> print(frase)
O produto custa 165.780000. Eu comprei 5. Paguei ao todo 828.900000
>>>
```

Figura 98 - Composição de números reais

Arredondando números reais.



```
Command Prompt - python
>>> valor_unitario = 165.78
>>> print("%f" % valor_unitario)
165.780000
>>> print("%.f" % valor_unitario)
166
>>> print("%.1f" % valor_unitario)
165.8
>>> print("%.2f" % valor_unitario)
165.78
>>> print("%.3f" % valor_unitario)
165.780
>>>
```

Figura 99 - Arredondamento

Usando %% para exibir sinal de porcentagem

```
Command Prompt - python
>>>
>>>
>>> percent = 17.5
>>> frase = "A taxa de juros é de %.2f%%" % percent
>>> print(frase)
A taxa de juros é de 17.50%
>>> _
```

Figura 100 - Exibindo sinal de porcentagem

Usando format em vez do marcador de posição “%”

```
Command Prompt - python
>>>
>>> nome = "José da Silva"
>>> idade = 50
>>> filhos = 3
>>> frase = "{} tem {} anos de idade e tem {} filhos".format(nome, idade, filhos)
>>> print(frase)
José da Silva tem 50 anos de idade e tem 3 filhos
>>>
```

Figura 101 - String.format

Outra forma de usar format

```
Command Prompt - python
>>> frase = "Olá, eu vou viajar de {3}".format('bicicleta', 'moto', 'trem', 'carro')
>>> print(frase)
Olá, eu vou viajar de carro
>>>
```

Figura 102 - String.format

Pode ficar melhor? Sim, no Python 3.6 foi criado o recurso de f-strings ou Literais de strings formatadas.

```
Command Prompt - python
>>> nome = "Messi"
>>> f"O nome dele é {nome}"
'O nome dele é Messi'
>>> valor = 12.58
>>> f"O valor é {valor}"
'O valor é 12.58'
>>>
```

Figura 103 - Utilizando f-Strings



Podemos usar formatação numérica também em `string.format()` e em f-strings.

A notação da formatação é `{valor:tamanho.precisão}`, sendo que o tamanho pode ser omitido. Exemplo: `{valor:,.2f}` -> Foi omitido o tamanho, foi colocada uma “,” para milhar e um “.” para decimais. O 2f indica que serão duas casas decimais.

No caso do uso do `format`, podemos colocar a formatação dentro das chaves que identificam a variável dentro do texto. No caso das f-strings, colocamos a formatação junto à variável.

Veja alguns exemplos:

```

Command Prompt - python
>>>
>>> a = 15.658745
>>> print(f"O valor da variável 'a' é: {a:.2f}")
O valor da variável 'a' é: 15.66
>>>
>>> b = 65844598.8475552
>>> print(f"O valor da variável 'b' é: {b:.2f}")
O valor da variável 'b' é: 65844598.85
>>>
>>> print(f"O valor da variável 'b' é: {b:,.2f}")
O valor da variável 'b' é: 65,844,598.85
>>>
>>> print("O valor da variável b é: {:.2f}".format(b))
O valor da variável b é: 65,844,598.85
>>>
>>> _

```

Figura 104 - Formatando número com f-strings

Fatiamento de string, index e split

Fatiamento (slicing) é o recurso de extrair partes de uma string utilizando o índice de seus caracteres.

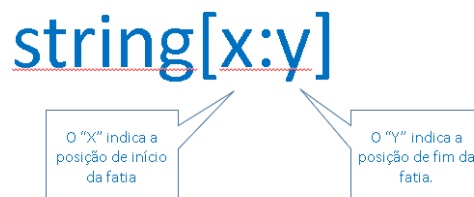


Figura 105 - Fatiamento

A posição inicial (x) é incluída no resultado, porém, a posição final (y) não é incluída.

```

endereco = "www.python.org"
# www . python . org
# 012 3 456789 0 123
fatia = endereco[4:10]
print(fatia)

```

O resultado obtido com o `print` foi o texto “python”. Observe os índices fornecidos:
 4 = posição da letra “p”
 10 = posição do “”, como o caractere da décima posição não é incluído, o valor retornado é do “p” até o “n”, que é o nono caractere.

Figura 106 – Fatiamento

Se os índices de limite inicial e final não forem informados, é realizada apenas uma cópia da string.



```
endereco = "www.python.org"
# www . python . org
# 012 3 456789 0 123
fatia = endereco[:]
print(fatia)
```

O conteúdo da variável *fatia* exibido com o print é www.python.org.

Figura 107 – Fatiamento

Se for omitido o índice inicial será considerado à partir do primeiro caractere.

```
endereco = "www.python.org"
# www . python . org
# 012 3 456789 0 123
fatia = endereco[:3]
print(fatia)
```

O conteúdo da variável *fatia* exibido com o print é "www", ou seja, do índice zero até o 3 (retornando até o índice 2, não considerando o índice 3).

Figura 108 - Fatiamento

Se for omitido o índice final será considerado até o último caractere.

```
endereco = "www.python.org"
# www . python . org
# 012 3 456789 0 123
fatia = endereco[11:]
print(fatia)
```

O conteúdo da variável *fatia* exibido com o print é "org", ou seja, do índice 11 até o último caractere (retornando o 11, 12 e 13).

Figura 109 - Fatiamento

É possível utilizar um índice negativo para iniciar no último caractere da string. O último caractere é o -1.

```
# Usando indice negativo para o final da string
# w w w . p y t h o n . o r g
# -14 -13 -12 -11 -10 -9 -8 -7 -6 -5 -4 -3 -2 -1
endereco = "www.python.org"
fatia = endereco[-10:-4]
print(fatia)
```

O conteúdo da variável *fatia* exibido com o print é "python".

Figura 110 - Fatiamento

Exemplo prático 1: Obtendo o usuário de um endereço de e-mail.

Como todo endereço de e-mail possui um arroba após o nome do usuário, podemos identificar a posição do arroba dentro do endereço de e-mail usando um método da string denominado "index".

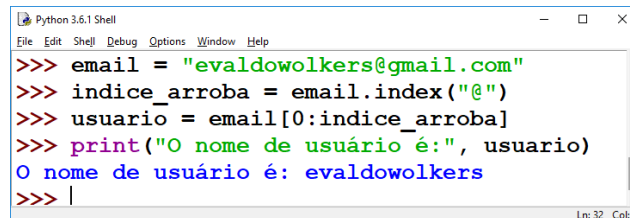
Para encontrar a primeira ocorrência de um caractere ou texto dentro da string usamos:

`string.index("caractere")`

`string.index("texto")`



```
email = "evaldowolkers@gmail.com"
indice = email.index(".com")
print(indice) # impresso 19
indice = email.index("@gmail")
print(indice) # impresso 13
indice = email.index("@")
print(indice) # impresso 13
```



```
>>> email = "evaldowolkers@gmail.com"
>>> indice_arroba = email.index("@")
>>> usuario = email[0:indice_arroba]
>>> print("O nome de usuário é:", usuario)
O nome de usuário é: evaldowolkers
>>> |
```

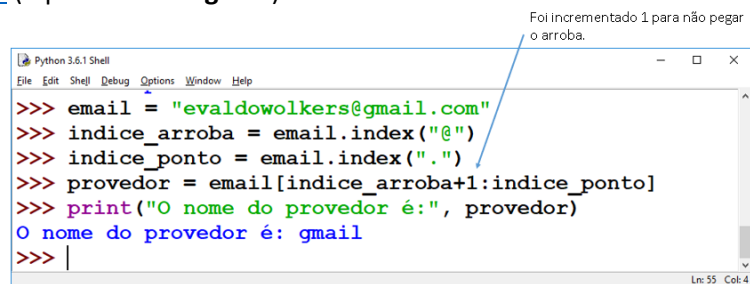
Figura 111 - Utilizando index

Exemplo prático 2: Obtendo o provedor de um endereço de e-mail.

O provedor é a informação que fica entre o arroba e um ponto.

Exemplo:

[fulano@gmail.com](#) (o provedor é o gmail)



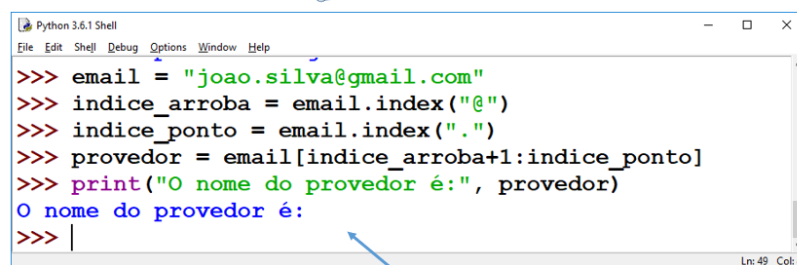
```
>>> email = "evaldowolkers@gmail.com"
>>> indice_arroba = email.index("@")
>>> indice_ponto = email.index(".")
>>> provedor = email[indice_arroba+1:indice_ponto]
>>> print("O nome do provedor é:", provedor)
O nome do provedor é: gmail
>>> |
```

Figura 112 - Utilizando index

E se o e-mail for assim:

[joao.silva@gmail.com](#)

?



```
>>> email = "joao.silva@gmail.com"
>>> indice_arroba = email.index("@")
>>> indice_ponto = email.index(".")
>>> provedor = email[indice_arroba+1:indice_ponto]
>>> print("O nome do provedor é:", provedor)
O nome do provedor é:
>>> |
```

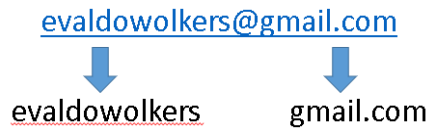
Nada foi impresso porque o índice final ficou menor que o índice inicial. `email[10:4]`

Figura 113 - Utilizando index e fatiamento



Que tal aprendermos um novo método para “quebrar” uma string à partir de um determinado caractere.

Veja:



Para isso podemos usar o método `split`. O método `split` retorna uma lista de palavras usando um separador informado por parâmetro. Lista é um assunto de aulas futuras.

Veja: `string.split("separador")`

No nosso caso usaremos assim: `"joao.silva@gmail.com".split("@")`

Serão retornadas duas strings, uma contendo `"joao.silva"` e a outra contendo `"gmail.com"`.

```
Python 3.6.1 Shell
File Edit Shell Debug Options Window Help
>>> email = "joao.silva@gmail.com"
>>> usuario, dominio = email.split("@")
>>> print(usuario)
joao.silva
>>> print(dominio)
gmail.com
>>> |
```

Figura 114 - Utilizando `split`

Agora que temos o domínio separado podemos procurar o `"."` para indentificá-lo e separar o provedor.

```
Python 3.6.1 Shell
File Edit Shell Debug Options Window Help
>>> email = "joao.silva@gmail.com"
>>> usuario, dominio = email.split("@")
>>> indice_ponto = dominio.index(".")
>>> provedor = dominio[0:indice_ponto]
>>> print("O usuário é:", usuario)
O usuário é: joao.silva
>>> print("O provedor é:", provedor)
O provedor é: gmail
>>> |
```

Figura 115 - Utilizando `split`, `index` e fatiamento

Estruturas condicionais

Estruturas condicionais servem para que o sistema decida se vai ou não executar uma parte do código. Podem ser usadas sempre que você precisar desviar o fluxo de execução do sistema de acordo com uma condição.

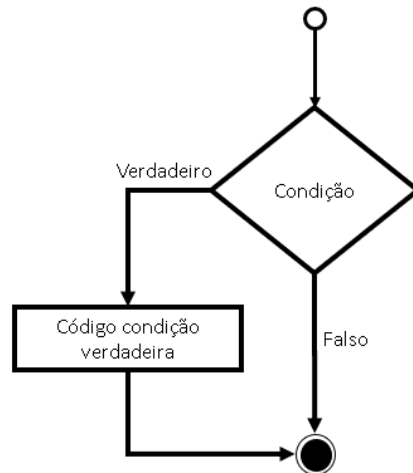


Figura 116 - Estrutura condicional

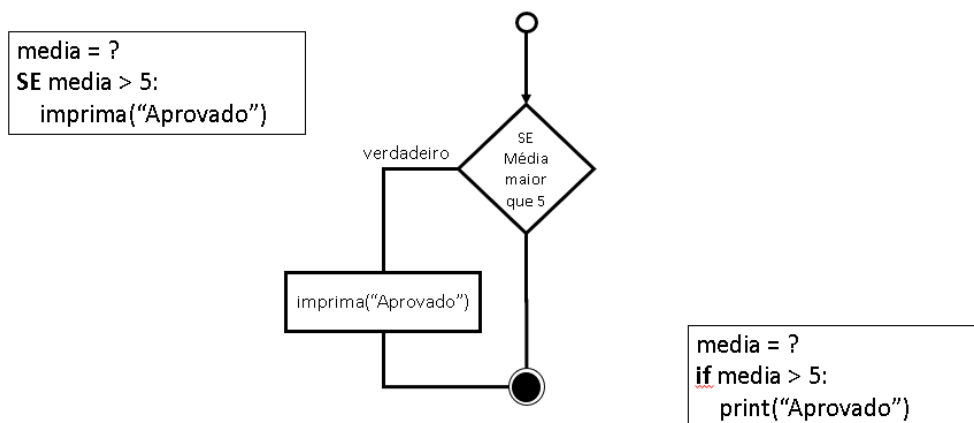


Figura 117 - Validando a média

SE => IF

```

media = float(input("Informe a média: "))
if media > 5:
    print("Aprovado")
  
```

adicional

C:\Users\Evaldo\AppData\Local\Programs\Python\Python39\python.exe

Informe a média: 4

Process finished with exit code 0

Figura 118 - Validando a média com if

```

media = float(input("Informe a média: "))
if media > 5:
    print("Aprovado")
  
```

adicional

C:\Users\Evaldo\AppData\Local\Programs\Python\Python39\python.exe

Informe a média: 6.5

Aprovado

Process finished with exit code 0

Figura 119 - Validando a média com if



ELSE => OUTRO, "SENÃO", CASO CONTRÁRIO.

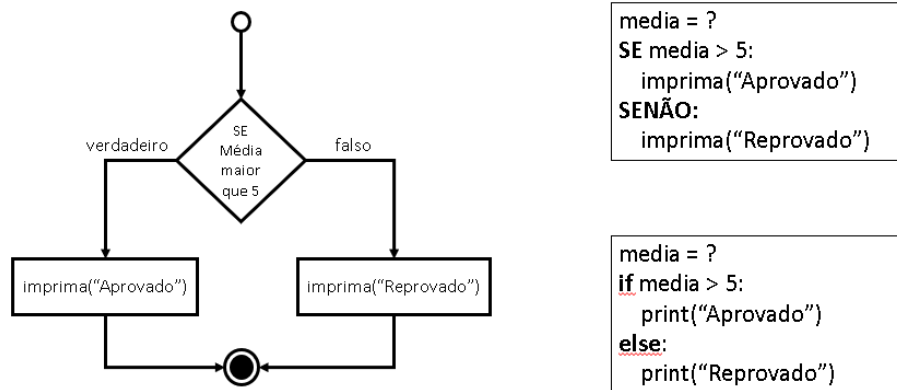


Figura 120 - Utilizando if e else

```

media = float(input("Informe a média: "))
if media > 5:
    print("Aprovado")
else:
    print("Reprovado")
  
```

ndicional

C:\Users\Evaldo\AppData\Local\Programs\Python
Informe a média: 5.5
Aprovado

Process finished with exit code 0

Figura 121 - Utilizando if e else

```

media = float(input("Informe a média: "))
if media > 5:
    print("Aprovado")
else:
    print("Reprovado")
  
```

ndicional

C:\Users\Evaldo\AppData\Local\Programs\Python
Informe a média: 3.5
Reprovado

Process finished with exit code 0

Figura 122 - Utilizando if e else

SENÃO SE => ELIF

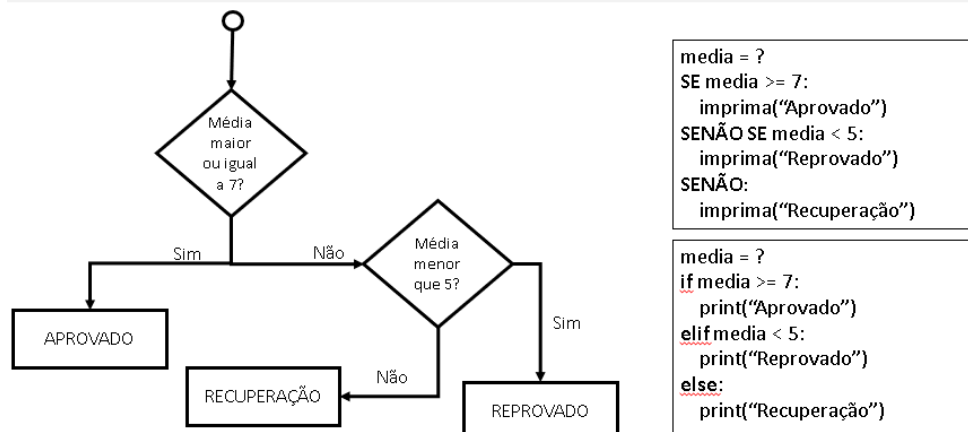


Figura 123 - Utilizando if, elif e else

```

1  media = float(input("Informe a média: "))
2  if media > 7:
3      print("Aprovado")
4  elif media < 5:
5      print("Reprovado")
6  else:
7      print("Recuperação")
  
```

condicional

C:\Users\Evaldo\AppData\Local\Programs\Python\Py...
 Informe a média: 8.5
 Aprovado

Figura 124 - Utilizando if, elif e else

```

media = float(input("Informe a média: "))
if media > 7:
    print("Aprovado")
elif media < 5:
    print("Reprovado")
else:
    print("Recuperação")
  
```

condicional

C:\Users\Evaldo\AppData\Local\Programs\Python\Py...
 Informe a média: 3
 Reprovado

Figura 125 - Utilizando if, elif e else

```

media = float(input("Informe a média: "))
if media > 7:
    print("Aprovado")
elif media < 5:
    print("Reprovado")
else:
    print("Recuperação")
  
```

condicional

C:\Users\Evaldo\AppData\Local\Programs\Python\Py...
 Informe a média: 6.5
 Recuperação

Figura 126 - Utilizando if, elif e else



Exemplo Tabela do INSS 2017

```
print("***** TABELA INSS 2017 *****")
print("* Salário de Contribuição (R$) - Alíquota do INSS *")
print("* até 1.659,38 - 8% *")
print("* de 1.659,39 até 2.765,66 - 9% *")
print("* de 2.765,67 até 5.531,31 (teto) - 11% *")
print("*****")

salario_informado = float(input("Informe o salário de contribuição: "))
salario_contribuicao = salario_informado
if (salario_contribuicao <= 1659.38):
    aliquota_inss = 8
elif (salario_contribuicao <= 2765.66):
    aliquota_inss = 9
elif (salario_contribuicao <= 5531.31):
    aliquota_inss = 11
else:
    salario_contribuicao = 5531.31
    aliquota_inss = 11

valor_inss = (salario_contribuicao * (aliquota_inss/100))
salario_liquido = salario_informado - valor_inss

print(f"O salário contribuição é: {salario_contribuicao:.2f}.\n"
      f"A alíquota de INSS é de: {aliquota_inss:.1f}%.\n"
      f"Será descontado R$ {valor_inss:.2f} de INSS.\n"
      f"Descontando o INSS você receberá R$ {salario_liquido:.2f}.")
```

Exemplo Autorização de compra

```
limite_credito = 900
total_compras = 325.56
saldo_disponivel = limite_credito - total_compras
valor_compra = float(input("Informe o valor da compra: "))
if valor_compra <= saldo_disponivel:
    saldo_disponivel = saldo_disponivel - valor_compra
    print("Sua compra foi autorizada.\n"
          f"Valor da compra: R$ {valor_compra:.2f}\n"
          f"Seu saldo atualizado é: {saldo_disponivel:.2f}")
else:
    print("Sua compra não foi autorizada.\n"
          "Saldo insuficiente.\n"
          f"Saldo disponível: R$ {saldo_disponivel:.2f}")
```



Exemplo Multa

```

velocidade_permitida = int(input("Informe a velocidade permitida "
                                "na via: "))
velocidade_veiculo = int(input("Informe a velocidade que você "
                                "passou no radar: "))

if velocidade_veiculo > velocidade_permitida:
    if velocidade_veiculo >= velocidade_permitida + (velocidade_permitida * (50/100)):
        valor_multa = 880.41
        pontos = 7
        suspensao = True
        descricao_multa = "Transitar em velocidade superior a " \
                           "50% da máxima permitida"
    elif velocidade_veiculo >= velocidade_permitida + (velocidade_permitida * (20 / 100)):
        valor_multa = 195.23
        pontos = 5
        suspensao = False
        descricao_multa = "Transitar em velocidade superior à " \
                           "máxima permitida em 20% até 50%"
    else:
        valor_multa = 130.16
        pontos = 4
        suspensao = False
        descricao_multa = "Transitar em velocidade superior à " \
                           "máxima permitida em até 20%"

    print(f"Você foi multado e perdeu {pontos} pontos.")
    if suspensao:
        print("Você teve a carteira suspensa.")
        print(f"O valor de sua multa é: R$ {valor_multa}")
        print(f"Motivo: {descricao_multa}")
else:
    print("Parabéns, você não foi multado.")

```

Estruturas de repetição (laços, loops)

For e função Range

Estruturas de repetição são utilizadas para executar a mesma parte de um programa várias vezes. Usamos estruturas de repetições para que o sistema fique “preso” em um loop até que uma determinada condição de parada seja alcançada.

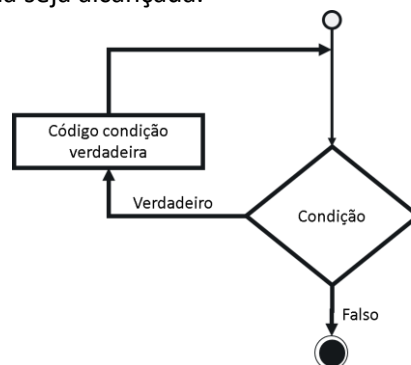


Figura 127 - Estruturas de repetição

O **for** normalmente é utilizado quando você quer repetir um bloco de código um número fixo de vezes.

Sintaxe:

for referência **in** sequência:
bloco de código

Referência:

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



É a variável que vai receber um valor da sequência a cada iteração. A cada iteração ela aponta para um valor da sequência. A referência normalmente é acessada dentro do bloco de código do **for** para que seu valor seja utilizado. A cada iteração no loop a referência é atualizada.

Sequência:

Pode ser uma sequência estática como uma lista ou uma string. Em capítulos futuros, falaremos sobre listas. Por enquanto entenda que uma lista é uma sequência de elementos dentro de um par de colchetes separados por vírgula. Veja a seguir uma lista de números: [5, 75, 110, 325, 400]

Pode ser também uma função que gere números, como a função `range`.

para referência em sequência:
bloco de código

```
for numero in [8, 7, 17, 25, 38]:  
    print(numero)
```

Usando o `for` com uma lista de números.

```
1 numeros = [0, 18, 56, 77, 95]  
2 for numero in numeros:  
3     print(numero)
```

0
18
56
77
95

Figura 128 - Comando `for` com uma lista de números

Usando o `for` com uma lista de strings.

```
1 frutas = ["Maçã", "Abacaxi", "Melão", "Laranja"]  
2 for fruta in frutas:  
3     print(f"Fruta: {fruta}")
```

Fruta: Maçã
Fruta: Abacaxi
Fruta: Melão
Fruta: Laranja

Figura 129 - Comando `for` com uma lista de strings

Usando o `for` com uma string.

```
for letra in "Python":  
    print(letra)
```

P
y
t
h
o
n

Figura 130 - Comando `for` com uma string

Podemos usar o `else` para executar algum código após o término do loop.



```
1 for numero in [0, 18, 56, 77, 95]:
2     print(numero)
3 else:
4     print("Acabou")
```

0
18
56
77
95
Acabou

Figura 131 - Comando for com else

Para interromper um loop podemos utilizar o **break** (quebrar o loop).

```
for numero in range(1000000):
    print(numero)
    if numero == 4:
        break
print("Até mais")
```

0
1
2
3
4
Até mais

Figura 132 - Comando for com break

Quando é usado o **break**, o conteúdo do **else** não é executado.

```
1 numeros = [1, 2, 3, 10, 12]
2 for numero in numeros:
3     if numero == 10:
4         break
5     print(f"Número: {numero}")
6 else:
7     print("Acabou")
```

Número: 1
Número: 2
Número: 3

Figura 133 - Comando for com break e else

Para passar para a próxima iteração podemos utilizar o **continue**.

```
1 for x in [1, 10, 20, 30, 40, 50]:
2     if x == 30:
3         continue
4     print(x)
```

1
10
20
40
50

Figura 134 - Comando for com continue

Range é uma função que gera uma lista de números.

Sintaxe: range(início, fim, salto)

Início e salto são opcionais, se não for informado o início, será considerado iniciando do zero, se não for informado o salto, será incrementado em 1.

Exemplo:

range(3) resulta em [0, 1, 2]

range(0, 10, 2) -> Inicia com zero, finaliza em 9 (e não em 10. É um intervalo aberto) e realiza um salto de 2 em 2. O resultado é 0, 2, 4, 6, 8.



```
for numero in range(3):
    print(numero)
```

mp

0

1

2

Figura 135 - Comando for com range

```
for numero in range(20, 30, 2):
    print(numero)
```

mp

20

22

24

26

28

Figura 136 - Comando for com range

Somando números do intervalo informado limitando o maior número

inicio = int(input("Informe o primeiro número: "))

fim = int(input("Informe o número final: "))

salto = int(input("Informe o salto: "))

texto = "Cálculo: "

soma = 0

for numero **in** range(inicio, fim, salto):

 soma = soma + numero

 texto = texto + str(numero)

if numero > 50:

 texto = texto + " Passou de 50"

break

if numero != fim-1:

 texto = texto + " + "

print(f"{texto}")

print(f"Soma: {soma}")

Execução 1:

Informe o primeiro número: 10

Informe o número final: 20

Informe o salto: 1

Cálculo: 10 + 11 + 12 + 13 + 14 + 15 + 16 + 17 + 18 + 19

Soma: 145

Execução 2:

Informe o primeiro número: 1

Informe o número final: 9999

Informe o salto: 2

Cálculo: 1 + 3 + 5 + 7 + 9 + 11 + 13 + 15 + 17 + 19 + 21 + 23 + 25 + 27 + 29 + 31 + 33 + 35 + 37 + 39 + 41
+ 43 + 45 + 47 + 49 + 51 Passou de 50

Soma: 676

Sintaxe completa:

for referência **in** sequência:

 bloco de código

 continue

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
break
else:
    bloco de código
```

While

O **while** normalmente é utilizado quando você quer repetir um bloco de código enquanto uma expressão for verdadeira.

O while usado indevidamente pode fazer com que o sistema entre em um loop infinito.

Sintaxe:

```
while condição:
    bloco de código
```

O **bloco de código** é executado enquanto a **condição** for verdadeira ou o loop for interrompido com um break.

```
enquanto condição:
    bloco de código
```

```
x = 1
while x < 10:
    print(x)
    x = x + 1
```

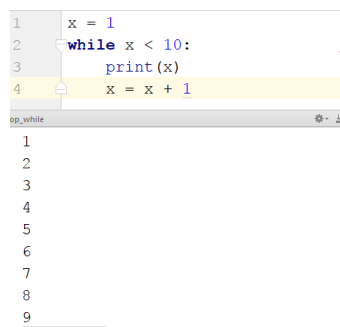


Figura 137 - Estrutura de repetição While

```
1 notas = 0
2 quant_notas_informadas = 0
3
4 while True:
5     nota = float(input("Informe a nota (-1 para finalizar): "))
6     if nota == -1:
7         break
8     notas = notas + nota
9     quant_notas_informadas = quant_notas_informadas + 1
10
11 if quant_notas_informadas > 0:
12     media = notas / quant_notas_informadas
13     print(f"Quantidade de notas informadas: {quant_notas_informadas}")
14     print(f"Média: {media}")
15 else:
16     print("Nenhuma nota informada.")
17
```

Figura 138 - Estrutura de repetição While - Exemplo

Execução:

Informe a nota (-1 para finalizar): 10



Informe a nota (-1 para finalizar): 9.5
 Informe a nota (-1 para finalizar): 7.5
 Informe a nota (-1 para finalizar): 8.5
 Informe a nota (-1 para finalizar): -1
 Quantidade de notas informadas: 4
 Média: 8.875

```

1  x = 0
2  soma = 0
3  quant_numeros = int(input("Quantos números deseja somar? "))
4  while x < quant_numeros:
5      soma = soma + int(input("Informe um número inteiro para a soma: "))
6      x = x + 1
7
8  print(f"Soma: {soma}")

```

op_while.2

```

Quantos números deseja somar? 4
Informe um número inteiro para a soma: 10
Informe um número inteiro para a soma: 25
Informe um número inteiro para a soma: 84
Informe um número inteiro para a soma: 65
Soma: 184

```

Figura 139 - While com condição informada pelo usuário

Usando else. O **else** é executado quando a condição se torna falsa.

<pre> x = 0 while x < 5: print(x) x = x + 1 else: print("Acabou") </pre>	Execução: <pre> 0 1 2 3 4 Acabou </pre>
---	--

Estruturas de dados

Listas

Listas são coleções de objetos que podem ser de qualquer tipo, inclusive outras listas.

As listas em Python são mutáveis, podendo ser alteradas a qualquer momento.

Listas podem ser fatiadas como strings.

Os valores de uma lista podem ser acessados pelo seu índice.

Uma lista pode conter zero ou mais elementos.

O tamanho de uma lista é igual a sua quantidade de elementos.

Criação de uma lista vazia:

```
lista = []
```

Uma lista com três elementos strings:

```
bancos = ["Banco do Brasil", "CEF", "Banestes"]
```

Cada item de uma lista possui um índice iniciando com zero.

```
bancos = ["Banco do Brasil", "CEF", "Banestes"]
```

0 = 'Banco do Brasil'

1 = 'CEF'

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



2 = 'Banestes'

Trabalhando com os índices dos elementos da lista.

Criando uma lista com 3 inteiros

```
lista_numeros = [25, 78, 55]
```

Será impresso 78, os elementos da lista iniciam com zero

0=25, 1=78, 2=55

Na linha abaixo será impresso 78

```
print(lista_numeros[1])
```

Alterando o segundo elemento da lista de 78 para 30

```
lista_numeros[1] = 30
```

Na linha abaixo será impresso 30

```
print(lista_numeros[1])
```

Acessando os elementos para calcular a média

```
notas = [7.5, 5.6, 9.5, 10.0]
```

```
media = (notas[0] + notas[1] + notas[2] + notas[3]) / 4
```

```
print(media)
```

Alterando itens da lista pelo seu índice, outro exemplo:

```
bancos = ["Banco do Brasil", "CEF", "Banestes"]
```

```
print(bancos)
```

Resultado: ['Banco do Brasil', 'CEF', 'Banestes']

```
bancos[1] = "Itaú"
```

```
print(bancos)
```

Resultado: ['Banco do Brasil', 'Itaú', 'Banestes']

Assim como no fatiamento de string, o último elemento de uma lista pode ser acessado pelo índice - 1.

```
bancos = ["Banco do Brasil", "CEF", "Banestes"]
```

```
bancos[-1] = "Itaú"
```

```
print(bancos)
```

Resultado: ['Banco do Brasil', 'CEF', 'Itaú']

Para incrementar o valor de uma lista podemos usar o operador de adição.

```
numeros = [1, 2, 3, 10, 12]
```

```
numeros = numeros + [8, 7, 15]
```

```
print(numeros)
```

#Resultado:[1, 2, 3, 10, 12, 8, 7, 15]

Mas também podemos fazer assim, usando o operador de atribuição com adição "+=":

```
numeros = [1, 2, 3, 10, 12]
```

```
numeros += [8, 7, 15]
```

```
print(numeros)
```

Resultado

```
[1, 2, 3, 10, 12, 8, 7, 15]
```



O operador de atribuição “+=” adiciona o operando direito ao operando esquerdo e atribui o resultado ao operando esquerdo.

Veja outro exemplo de adição de listas com “+=”.

```
bancos = ["Banco do Brasil", "CEF", "Banestes", "Itaú", "Sicoob", "Bradesco"]
bancos += ["Real", "Safra", "Santander"]
# Resultado: ['Banco do Brasil', 'CEF', 'Banestes', 'Itaú', 'Sicoob', 'Bradesco', 'Real', 'Safra', 'Santander']
```

Para adicionar um item à lista usamos o método **append**.

```
bancos = ["Banco do Brasil", "CEF", "Banestes"]
bancos.append("Bradesco")
print(bancos)
# Resultado: ['Banco do Brasil', 'CEF', 'Banestes', 'Bradesco']
```

Para adicionar um item à lista definindo um índice específico usamos o método **insert**.

```
bancos = ['Banco do Brasil', 'CEF', 'Banestes', 'Itaú', 'Santander', 'Bradesco']
bancos.insert(3, "Sicoob")
print(bancos)
# Resultado: ['Banco do Brasil', 'CEF', 'Banestes', 'Sicoob', 'Itaú', 'Santander', 'Bradesco']
```

Para remover um elemento da lista usamos o método **remove**.

```
bancos = ["Banco do Brasil", "CEF", "Banestes", "Itaú"]
bancos.remove("CEF")
print(bancos)
#Resultado: ['Banco do Brasil', 'Banestes', 'Itaú']
```

Para ordenar os itens de uma lista usamos o método **sort**.

```
bancos = ["Banco do Brasil", "CEF", "Banestes", "Itaú", "Sicoob", "Bradesco"]
bancos.sort()
print(bancos)
#Resultado:['Banco do Brasil', 'Banestes', 'Bradesco',
# 'CEF', 'Itaú', 'Sicoob']
```

Para inverter os itens de uma lista usamos o método **reverse**.

```
bancos = ["Banco do Brasil", "CEF", "Banestes", "Itaú", "Sicoob", "Bradesco"]
bancos.reverse()
print(bancos)
#Resultado:['Bradesco', 'Sicoob', 'Itaú', 'Banestes', 'CEF', 'Banco do Brasil']
```

Retornando a quantidade de ocorrências de um elemento em uma lista com **count**.

```
todos_bancos = ["Bradesco", "Banco do Brasil", "CEF", "Banestes", "Bradesco", "Bradesco"]
qtd_bradesco = todos_bancos.count("Bradesco")
print(f"Qtd. ocorrências Bradesco: {qtd_bradesco}")
numeros = [1, 10, 18, 45, 10, 7, 8, 9, 74, 10, 36, 10, 54, 65, 98, 10]
qtd_dez = numeros.count(10)
print(f"Qtd. Números dez: {qtd_dez}")
```

Para remover o último elemento de uma lista usamos o método **pop**.



```
bancos = ["Banco do Brasil", "CEF", "Banestes", "Itaú", "Sicoob", "Bradesco"]
bancos.pop()
print(bancos)
#Resultado: ['Banco do Brasil', 'CEF', 'Banestes', 'Itaú', 'Sicoob']
```

O método **pop** retorna o elemento removido.

```
bancos = ["Banco do Brasil", "CEF", "Banestes", "Itaú",
          "Sicoob", "Bradesco"]
banco = bancos.pop()
print(bancos)
#Resultado: ['Banco do Brasil', 'CEF', 'Banestes', 'Itaú', 'Sicoob']
print(banco)
#Resultado: Bradesco
```

A função **del** pode ser usada para remover elementos de uma lista.

```
linguagens = ["Python", "Cobol", "Clipper", "C", "C++", "Go", "JavaScript"]
print(linguagens)
del linguagens[6] # Removeu JavaScript
print(linguagens)
del linguagens[1:3] # Removeu Cobol e Clipper
print(linguagens)
```

Resultado:

```
['Python', 'Cobol', 'Clipper', 'C', 'C++', 'Go', 'JavaScript']
['Python', 'Cobol', 'Clipper', 'C', 'C++', 'Go']
['Python', 'C', 'C++', 'Go']
```

Limpando uma lista inteira.

```
lista_livros = ['livro 1', 'livro 2', 'livro 3']
lista_livros.clear()
print(lista_livros)
```

```
lista_livros = ['livro 1', 'livro 2', 'livro 3']
lista_livros = []
print(lista_livros)
```

Resultado:

```
[]
[]
```

Usando uma lista dentro de uma outra lista.

```
lista = [10.2, 13.75, 17.58, ["Maçã", "Banana", "Abacaxi"]]
print(lista)
#Resultado: [10.2, 13.75, 17.58, ['Maçã', 'Banana', 'Abacaxi']]
```

```
frutas = lista[3]
print(frutas)
#Resultado: ['Maçã', 'Banana', 'Abacaxi']
```

```
soma = lista[0] + lista[1] + lista[2]
print(soma)
#Resultado: 41.53
```



```
letras = ['a', 'b', 'c', 'd']
numeros = [1, 2, 3]
mistura = [letras, numeros]
print(mistura)
print(mistura[0])
print(mistura[1])
Resultado:
[['a', 'b', 'c', 'd'], [1, 2, 3]]
['a', 'b', 'c', 'd']
[1, 2, 3]
```

Obtendo o tamanho de uma lista.

```
lista = [100, 752, 845, 15, 74]
tamanho = len(lista)
print(tamanho)
#Resultado: 5
```

Obtendo o índice de um determinado elemento da lista.

```
linguagens = ["Java", "Python", "Go", "Pascal", "C#"]
print(linguagens.index("Python"))
#Resultado: 1
```

Verificando a existência de um item na lista

```
linguagens = ["JAVA", "PYTHON", "GO", "PASCAL", "C", "JAVASCRIPT"]
linguagem = input("Informe a linguagem: ")
if linguagem.upper() in linguagens:
    print(f"{linguagem.upper()} está na lista.")
else:
    print(f"{linguagem.upper()} não está na lista.")
```

Exemplos de execução:

Informe a linguagem: Cobol
COBOL não está na lista.

Informe a linguagem: Python
PYTHON está na lista.

Informe a linguagem: jAvA
JAVA está na lista.

Adicionando elementos fornecidos pelo usuário à lista.

```
lista = []
while True:
    numero = int(input("Digite um número inteiro (informe 0 para sair): "))
    if numero == 0:
        break
    lista.append(numero)

for num in lista:
    print(num)
```



Exemplo: Separando números pares e ímpares:

```
lista_pares = []
lista_impares = []
```

while True:

```
    numero = int(input("Digite um número inteiro (informe 0 para sair): "))
    if numero == 0:
        break
    if numero % 2 == 0:
        lista_pares.append(numero)
    else:
        lista_impares.append(numero)
```

```
lista_pares.sort()
lista_impares.sort()
```

```
print(f"Números pares: {lista_pares}")
print(f"Números ímpares: {lista_impares}")
```

O **enumerate** adiciona um contador a um objeto iterável. Uma lista é um objeto iterável.

Sintaxe:

```
enumerate(objeto_iterável, início)
```

Se o início for omitido, será iniciado com zero.

```
estacoes = ["Primavera", "Verão", "Outono", "Inverno"]
```

```
lista_estacoes = list(enumerate(estacoes))
```

```
print(lista_estacoes)
```

#Resultado:

```
[(0, 'Primavera'), (1, 'Verão'), (2, 'Outono'), (3, 'Inverno')]
```

```
lista_estacoes = list(enumerate(estacoes, start=5))
```

```
print(lista_estacoes)
```

#Resultado:

```
[(5, 'Primavera'), (6, 'Verão'), (7, 'Outono'), (8, 'Inverno')]
```

Duas formas de mudar o número inicial

```
itens = ["Carne Bovina", "Frango", "Salsicha", "Peixe", "Carne Suína"]
```

```
for indice, item in enumerate(itens, 1):
    print(indice, item)
```

Resultado:

```
1 Carne Bovina
2 Frango
3 Salsicha
4 Peixe
5 Carne Suína
```

```
itens.sort() # Usei sort para diferenciar as listas
```

```
for indice, item in enumerate(itens):
    print(indice+1, item)
```



Resultado:

```
1 Carne Bovina
2 Carne Suína
3 Frango
4 Peixe
5 Salsicha
```

Copiando listas.

```
lista_numeros = [1, 2, 3, 4, 5]
nova_lista = lista_numeros
```

```
print(lista_numeros) #Resultado: [1, 2, 3, 4, 5]
print(nova_lista) #Resultado: [1, 2, 3, 4, 5]
```

```
lista_numeros += [6]
print(lista_numeros) #Resultado: [1, 2, 3, 4, 5, 6]
print(nova_lista) #Resultado: [1, 2, 3, 4, 5, 6]
```

```
nova_lista += [7]
print(lista_numeros) #Resultado: [1, 2, 3, 4, 5, 6, 7]
print(nova_lista) #Resultado: [1, 2, 3, 4, 5, 6, 7]
```

Clonando listas.

```
lista_numeros = [1, 2, 3, 4, 5]
lista_clonada = lista_numeros[:]
lista_clonada += [6]
print(lista_numeros) #Resultado: [1, 2, 3, 4, 5]
print(lista_clonada) #Resultado: [1, 2, 3, 4, 5, 6]
lista_numeros += [7]
print(lista_numeros) #Resultado: [1, 2, 3, 4, 5, 7]
print(lista_clonada) #Resultado: [1, 2, 3, 4, 5, 6]
```

Fatiando listas.

```
linguagens = ["Python", "Cobol", "Clipper", "C", "C++", "Go", "JavaScript"]
linguagens2 = linguagens[3:5]
print(linguagens)
print(linguagens2)
#Resultado:
['Python', 'Cobol', 'Clipper', 'C', 'C++', 'Go', 'JavaScript']
['C', 'C++']
```

```
letras = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
print(letras)
letras[2:5] = ['C', 'D', 'E']
print(letras)
letras[2:5] = []
print(letras)
#Resultado:
['a', 'b', 'c', 'd', 'e', 'f', 'g']
['a', 'b', 'C', 'D', 'E', 'f', 'g']
```



```
['a', 'b', 'f', 'g']
```

Tuplas

Tuplas são semelhantes às listas, porém, são imutáveis. Não podemos acrescentar, apagar ou fazer atribuições aos seus itens.

As tuplas são criadas usando-se parênteses em vez de colchetes, porém, os parênteses não são obrigatórios.

Sintaxe:

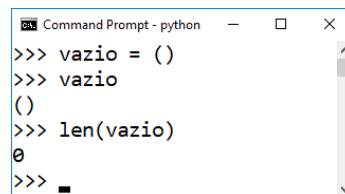
tupla = (a, b, ..., z) ou tupla = a, b, ..., z

Veja a criação de tuplas com e sem parênteses.

```
linguagens = ("Assembly", "Cobol", "C", "C++")
print(linguagens)
#Resultado: ('Assembly', 'Cobol', 'C', 'C++')
```

```
linguagens = "Python", "Java", "Go", "C#"
print(linguagens)
#Resultado: ('Python', 'Java', 'Go', 'C#')
```

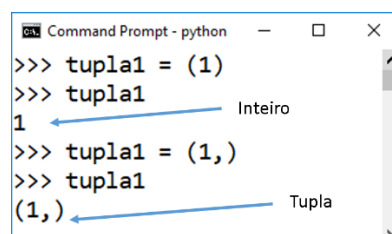
É possível criar uma tupla vazia.



```
Command Prompt - python
>>> vazio = ()
>>> vazio
()
>>> len(vazio)
0
>>> _
```

Figura 140 - Criando uma tupla vazia

Criando uma tupla com um único elemento.



```
Command Prompt - python
>>> tupla1 = (1)
>>> tupla1
1
>>> tupla1 = (1,)
>>> tupla1
(1,)
```

Diagrama de anotação: Uma seta aponta do texto 'Inteiro' para o valor '1' na segunda linha de código. Outra seta aponta do texto 'Tupla' para o valor '(1,)' na quarta linha de código.

Figura 141 - Criando uma tupla com um elemento



```

Command Prompt - python
>>> tupla1 = 1
>>> tupla1
1
>>> tupla1 = 1,
>>> tupla1
(1,)

```

Figura 142- Criando uma tupla com um elemento

Podemos acessar os elementos da tupla pelo índice e usar fatiamento.

```

países = "Brasil", "Paraguai", "Uruguai", "México"
país = países[0]
print(país) # Brasil

```

```

fatia = países[1:3]
print(fatia) # ('Paraguai', 'Uruguai')

```

Se tentarmos alterar um item da tupla, é gerado o erro “O objeto não suporta a atribuição de itens”.

```

países = "Brasil", "Paraguai", "Uruguai", "México"
países[1] = "Colômbia"

```

```

Traceback (most recent call last):
  File ".../tupla.py", line 2, in <module>
    países[1] = "Colômbia"
TypeError: 'tuple' object does not support item assignment

```

Podemos imprimir os elementos de uma tupla usando o comando **for**.

```

países = "Brasil", "Paraguai", "Uruguai", "México"
for país in países:
    print(país)

```

Como eu já disse, melhor não usar acentuação. Nem todas linguagens aceitam....

Resultado:

```

Brasil
Paraguai
Uruguai
México

```

Convertendo uma lista em uma tupla.

```

lista_carros = ["Gol", "Corolla", "Ranger", "Kadett",
               "Fusca", "Clio"]
tupla_carros = tuple(lista_carros)
print(f" Tupla carros: {tupla_carros}")

```

Resultado:

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>
contato@pythonparatodos.com.br



Tupla carros: ('Gol', 'Corolla', 'Ranger', 'Kadett', 'Fusca', 'Clio')

Desempacotando elementos da tupla.

```
tupla_carros = "Golf", "Corolla", "Civic"
carro1, carro2, carro3 = tupla_carros
print(f"Carro1: {carro1}")
print(f"Carro2: {carro2}")
print(f"Carro3: {carro3}")
```

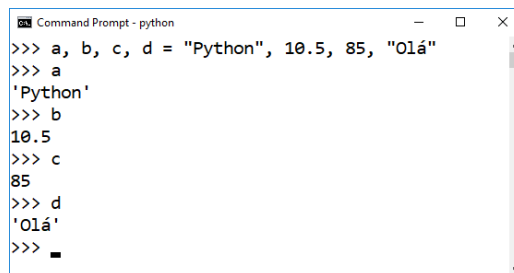
Resultado:

Carro1: Golf

Carro2: Corolla

Carro3: Civic

Desempacotando elementos da tupla.



```
Command Prompt - python
>>> a, b, c, d = "Python", 10.5, 85, "Olá"
>>> a
'Python'
>>> b
10.5
>>> c
85
>>> d
'Olá'
>>> _
```

Figura 143 - Desempacotando elementos da tupla

Desempacotando vários elementos para uma lista usando atribuição múltipla.

```
tupla_carros = "Golf", "Corolla", "Civic", "Opala", "Tucson", "Elantra"
carro1, *carros = tupla_carros
print(f"Carro1: {carro1}")
print(f"Carros: {carros}") # Uma lista com os itens restantes
```

Resultado:

Carro1: Golf

Carros: ['Corolla', 'Civic', 'Opala', 'Tucson', 'Elantra']

Atribuição múltipla não precisa estar no fim da sequência.

```
tupla_carros = "Golf", "Corolla", "Civic", "Opala", "Tucson", "Elantra"
*carros, tucson, elantra = tupla_carros
print(f"Carros: {carros}")
print(f"Tucson: {tucson}")
print(f"Elantra: {elantra}")
```



Resultado:

Carros: ['Golf', 'Corolla', 'Civic', 'Opala']

Tucson: Tucson

Elantra: Elantra

Uma lista como elemento de uma tupla.

```
Command Prompt - python
>>> tupla = ("Python", 10.25, 36, ["Ônibus", "Avião", "Navio", "Carro"])
>>> tupla[0]
'Python'
>>> tupla[1]
10.25
>>> tupla[2]
36
>>> tupla[3]
['Ônibus', 'Avião', 'Navio', 'Carro']
>>> lista = tupla[3]
>>> lista
['Ônibus', 'Avião', 'Navio', 'Carro']
```

Figura 144 - Lista como elemento de uma tupla

Set

Set (conjunto) é uma sequência mutável não ordenada e que não possui elementos duplicados.

Os sets suportam operações matemáticas como união, interseção e diferença.

Sintaxe:

***variavel* = {a, b, ...,z}**

set()

Criando o conjunto.

Criando um conjunto com valores

conjunto_numeros = {10, 20, 30, 40, 50, 60}

Para criar um conjunto vazio não podemos usar:

conjunto = {} # Chaves vazias criam um dicionário (assunto da próxima aula)

Forma correta de criar um conjunto vazio.

conjunto = set()

Os valores do **set** não se repetem.

conjunto_numeros = {10, 30, 40, 10, 42, 42, 56}

print("conjunto_numeros: ", conjunto_numeros)

Será impresso:

conjunto_numeros: {40, 10, 42, 56, 30}

Uma lista pode ser convertida em um conjunto.

lista_frutas = ["Abacaxi", "Abacaxi", "Laranja", "Maçã", "Abacaxi"]

conjunto_frutas = set(lista_frutas)

print("conjunto_frutas: ", conjunto_frutas)

Será impresso:

conjunto_frutas: {'Abacaxi', 'Maçã', 'Laranja'}



União

```
conjunto_numeros = {10, 30, 40, 10, 42, 42, 56}
conjunto_frutas = {"Abacaxi", "Abacaxi", "Laranja", "Maçã", "Abacaxi"}
conjunto_uniao = conjunto_frutas.union(conjunto_numeros)
print(conjunto_uniao)
```

Será impresso:

```
{'Laranja', 10, 'Maçã', 30, 'Abacaxi', 40, 42, 56}
```

Diferença

```
conjunto_numeros = {10, 30, 40, 42, 56}
conjunto_todos = {10, 30, 40, 42, 56, "Abacaxi", "Laranja", "Maçã"}
conjunto_diferenca = conjunto_todos.difference(conjunto_numeros)
print("conjunto_diferenca: ", conjunto_diferenca)
```

Será impresso:

```
conjunto_diferenca: {'Abacaxi', 'Laranja', 'Maçã'}
```

Interseção

```
conjunto1 = {10, 42, 50, "Python", "Programação"}
conjunto2 = {10, "Python", 42, "Java", "Bicicleta", "Carro"}
conjunto_intersecao = conjunto1.intersection(conjunto2)
print(f"conjunto_intersecao: {conjunto_intersecao}")
```

Será impresso:

```
conjunto_intersecao: {'Python', 10, 42}
```

Testando se um **set** inclui outro **set**.

```
linguagens = {"Python", "Java", "C++"}
linguagem = {"Python"}
if linguagens.issuperset(linguagem):
    print("O conjunto de linguagens inclui o conjunto linguagem.")
else:
    print("O conjunto de linguagens não inclui o conjunto linguagem.")
```

```
linguagem = {"Cobol"}
if linguagens.issuperset(linguagem):
    print("O conjunto de linguagens inclui o conjunto linguagem.")
else:
    print("O conjunto de linguagens não inclui o conjunto linguagem.")
```

Será impresso:

O conjunto de linguagens inclui o conjunto linguagem.

O conjunto de linguagens não inclui o conjunto linguagem.

Testando se um **set** está incluso em outro **set**.

```
linguagens = {"Python", "C++", "Assembly"}
linguagem = {"Python"}
if linguagem.issubset(linguagens):
    print("O conjunto linguagem está no conjunto de linguagens.")
else:
```



```
print("O conjunto linguagem não está no conjunto de linguagens.")

linguagem = {"Cobol"}
if linguagem.issubset(linguagens):
    print("O conjunto linguagem está no conjunto de linguagens.")
else:
    print("O conjunto linguagem não está no conjunto de linguagens.")
```

Será impresso:

O conjunto linguagem está no conjunto de linguagens.

O conjunto linguagem não está no conjunto de linguagens.

Verificando se um **set** não possui elementos comuns a outros **set**.

```
linguagens1 = {"Python", "C++", "Assembly"}
linguagens2 = {"Pascal", "Clipper", "Fortran"}
if linguagens1.isdisjoint(linguagens2):
    print("Os conjuntos não possuem elementos em comum.")
else:
    print("Os conjuntos possuem elementos em comum.")
```

```
linguagens1 = {"Python", "C++", "Assembly"}
linguagens2 = {"Pascal", "Clipper", "Fortran", "Python"}
if linguagens1.isdisjoint(linguagens2):
    print("Os conjuntos não possuem elementos em comum.")
else:
    print("Os conjuntos possuem elementos em comum.")
```

Será impresso:

Os conjuntos não possuem elementos em comum.

Os conjuntos possuem elementos em comum.

Dicionários

Dicionários são estruturas de dados similares às listas, porém, com propriedades de acesso diferentes. Um dicionário consiste em um conjunto de chaves e valores.

Sintaxe:

```
dicionario = {'a': a, 'b': b, ..., 'z': z}
```

```
dicionario = {chave: valor, chave: valor}
```

Assim como as listas, os dicionários são mutáveis.

Mas suas chaves devem ser de um tipo imutável.

Para criar listas usamos colchetes “[]”, para criar tuplas, usamos parênteses “()”, para criar dicionários usamos chaves “{ }”.

Chave	Valor
curso	Python para todos
aula	Dicionários
professor	Evaldo

```
dic = { 'curso': 'Python para todos',
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
'aula': 'Dicionários',  
'professor': 'Evaldo' }
```

Listas: Acessamos os elementos pelo número do índice.

Dicionários: Acessamos um elemento utilizando a chave. Exemplo: `dicionario["chave"]`

Criando o dicionário

```
carros = {'marca': 'VW', 'modelo': 'Gol', 'ano_modelo': 2016}
```

Acessando seus elementos

```
print(f'O ano do modelo do carro é: {carros["ano_modelo"]}')  
print(f'A marca do carro é: {carros["marca"]}')  
Resultado:  
O ano do modelo do carro é: 2016  
A marca do carro é: VW
```

Para adicionar/alterar elementos ao dicionário usamos:

```
dicionario["chave"] = valor
```

Criando o dicionário

```
carros = {'marca': 'VW', 'modelo': 'Gol', 'ano_modelo': 2016}
```

Adicionando elementos ao dicionário

```
carros['ano_fabricacao'] = 2017
```

```
print(f'O ano de fabricação do carro é: {carros["ano_fabricacao"]}')  
Resultado:  
O ano de fabricação do carro é: 2017
```

Para apagar um elemento de um dicionário usamos:

```
del dicionario["chave"]
```

Criando o dicionário

```
carros = {'marca': 'VW', 'modelo': 'Gol', 'ano_modelo': 2016}
```

Apagando um elemento

```
del carros['modelo']
```

```
print(f'Removido modelo: {carros}')
```

Resultado:

```
Removido modelo: {'marca': 'VW', 'ano_modelo': 2016, 'ano_fabricacao': 2017}
```

Para apagar todos elementos do dicionário usamos:

```
dicionario.clear()
```

O dicionário continua existindo, porém vazio. Para apagar todo o dicionário usamos: **del** dicionario

Neste caso o dicionário não existe mais.

```
Command Prompt - python  
>>> dicionario = {'teste1': 1, 'teste2': 2}  
>>> print(dicionario)  
{'teste1': 1, 'teste2': 2}  
>>> del dicionario  
>>> print(dicionario)  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'dicionario' is not defined  
>>> _
```

Figura 145 - Apagando elementos do dicionário



Obtendo itens, chaves e valores de um dicionário.

key(), items() e values() retornam views (visualizações), que são iteradores dos tipos dict_keys, dict_items e dict_values, que devolvem um elemento de cada vez.

Obtendo itens, chaves e valores de um dicionário.

```
carros = {'marca': 'VW', 'modelo': 'Gol', 'ano_modelo': 2016}
print(f'Itens do dicionário carros: {carros.items()}')
print(f'Chaves do dicionário carros: {carros.keys()}')
print(f'Valores do dicionário carros: {carros.values()}')
```

Resultado:

Itens do dicionário carros: dict_items([('marca', 'VW'), ('modelo', 'Gol'), ('ano_modelo', 2016)])

Chaves do dicionário carros: dict_keys(['marca', 'modelo', 'ano_modelo'])

Valores do dicionário carros: dict_values(['VW', 'Gol', 2016])

Acessando uma chave inexistente.

```
Command Prompt - python
>>> precos = {"mouse": 125.00, "teclado": 180.25, "monitor": 1350.00}
>>> x = precos["gabinete"]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'gabinete'
>>>
```

Figura 146 - Acessando uma chave inexistente

Verificando a existência de uma chave.

```
Command Prompt - python
>>> precos = {"mouse": 125.00, "teclado": 180.25, "monitor": 1350.00}
>>> "monitor" in precos
True
>>> "pendrive" in precos
False
>>>
```

Figura 147 - Verificando a existência de uma chave

Um programa exemplo:

```
produtos = {"Mouse": 98.75,
            "Teclado": 125.65,
            "Monitor": 134.78,
            "Gabinete": 170.00,
            "HD Externo": 510.50,
            "Headset": 125.45 }
```

while True:

 produto = input("Informe o produto a pesquisar o preço ou fim para sair: ")

 if produto == "fim":

 break

 if produto in produtos:

 print(f"Produto {produto} custa {produtos[produto]}")

 else:

 print(f"Produto {produto} não encontrado.")

Em Python podemos ter dicionários contendo listas ou até mesmo outros dicionários.



```
# Dicionário com lista
# "estoque": código, [nome_produto, qtd_estoque, valor]
estoque = { "1": ["Teclado", 300, 166.71],
            "2": ["Mouse", 125, 80.57],
            "3": ["Processador", 25, 875.64],
            "4": ["Cooler", 70, 35.14]}

produtos_comprados = {}
total_geral = 0
while True:
    codigo_produto = input("Informe o código ou zero (0) para sair: ")
    if codigo_produto == "0":
        break
    if codigo_produto in estoque:
        qtd = int(input("Informe a quantidade: "))
        confirma = input("Confirma? 'S' para sim ou 'N' para não: ")
        if confirma == 'S':
            total_produto = 0
            estoque[codigo_produto][1] -= qtd
            valor_unitario = estoque[codigo_produto][2]
            total_produto = valor_unitario * qtd
            total_geral += total_produto
            produtos_comprados[codigo_produto] = (codigo_produto,
                                                  estoque[codigo_produto][0],
                                                  qtd,
                                                  f"R$ {valor_unitario:.2f}",
                                                  f"R$ {total_produto:.2f}")
        else:
            print("Você desistiu do produto.")
    else:
        print("Produto não encontrado no estoque.")

print(f"Estoque: {estoque}")
print(f"Produtos comprados: {produtos_comprados}")
```

Copiando um dicionário para outro.

```
dic1 = {'nome': 'Fulano', 'sobrenome': 'de Tal'}
dic2 = dic1.copy()
print(f'dic1: {dic1}')
print(f'dic2: {dic2}')
```

Resultado:

```
dic1: {'nome': 'Fulano', 'sobrenome': 'de Tal'}
dic2: {'nome': 'Fulano', 'sobrenome': 'de Tal'}
```

Adicionando conteúdo de um dicionário em outro dicionário.

```
dic1 = {'nome': 'Fulano', 'sobrenome': 'de Tal'}
dic2 = {'idade': 40, 'UF': 'ES'}
print(f'dic1: {dic1}')
print(f'dic2: {dic2}')
```



```
dic1.update(dic2)
print(f'dic1: {dic1}')
```

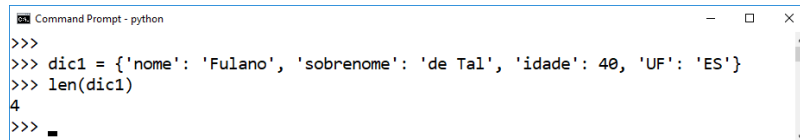
Resultado:

```
dic1: {'nome': 'Fulano', 'sobrenome': 'de Tal'}
```

```
dic2: {'idade': 40, 'UF': 'ES'}
```

```
dic1: {'nome': 'Fulano', 'sobrenome': 'de Tal', 'idade': 40, 'UF': 'ES'}
```

Para retornar quantos conjuntos chave/valor existem no dicionário usamos a função `len()`.



```
Command Prompt - python
>>>
>>> dic1 = {'nome': 'Fulano', 'sobrenome': 'de Tal', 'idade': 40, 'UF': 'ES'}
>>> len(dic1)
4
>>> _
```

Figura 148 - Tamanho do dicionário

Usando `dict()`

O construtor `dict()` constrói dicionários a partir de sequências de pares de chave-valor.

Exemplo:

```
>>> dicionario = dict([('curso', 'Python'), ('aula', 'Dicionários')])
>>> dicionario
{'curso': 'Python', 'aula': 'Dicionários'}
```

Outro exemplo:

```
>>> dicionario = dict(curso='Python', aula='Dicionários')
>>> dicionario
{'curso': 'Python', 'aula': 'Dicionários'}
```

Funções, módulos e pacotes

Introdução a Funções

Até agora usamos várias funções, lembre-se de algumas:

```
print()
```

```
len()
```

```
int()
```

```
input()
```

Agora vamos aprender a criar nossas próprias funções.

Funções são blocos de códigos que possuem um nome e podem ou não receber parâmetros.

Para criar uma função usamos a instrução ***def***.

Veja como criar uma função.

```
def nome(parâmetros):
    bloco de código
```

O bloco de código da função não é executado enquanto a mesma não for chamada.



Criando uma função.

```
# Definindo a função
def soma():
    numero1 = 10
    numero2 = 25
    total = numero1 + numero2
    print(f"Total: {total}")

# Chamando a função
soma()
```

Resultado da execução da função:

Total: 35

Passando parâmetros.

```
# Definindo a função
def soma(numero1, numero2):
    total = numero1 + numero2
    print(f"Total: {total}")

# Chamando a função
soma(10, 20)
```

Resultado:

Total: 30

Indicando um valor de retorno usando **return**.

```
# Definindo a função
def soma(numero1, numero2):
    total = numero1 + numero2
    return total

# Chamando a função
total = soma(10, 20)
print(f"Soma: {total}")
```

Resultado:

Soma: 30

```
# Chamando a função diretamente no print
print(f"Soma: {soma(10, 20)}")
```

Resultado:

Soma: 30

Uma função que chama outra função.

```
def maior_valor(valores):
    return max(valores)

def capturar_numeros():
    lista_numeros = []
    while True:
        numero = int(input("Informe um número inteiro ou zero para sair: "))
        if numero == 0:
            break
        lista_numeros.append(numero)
    return maior_valor(lista_numeros)

# Chamada da função
print(f"Maior valor: {capturar_numeros()}")
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Detalhando a função max()

A função max() é uma função interna do Python que retorna o maior valor, seja fornecido diretamente ou com base em um objeto iterável, como uma lista.

Por exemplo:

```
>>> max(10, 20, 55, 187, 1, 2)
187
>>> lista = [1, 2, 5, 8, 4, 1, 3, 9, 55]
>>> max(lista)
55
```

A função max() também retorna a string com o maior valor tendo como base a ordem alfabética, veja:

```
>>> max("Melancia", "Jaca", "Abacaxi", "Laranja")
'Melancia'
>>> max("a", "b", "c", "k", "m", "y")
'y'
```

Variáveis locais e globais

Ao usarmos funções, começamos a trabalhar com variáveis locais e globais.

Uma variável local, declarada dentro de uma função, existe apenas dentro desta função, sendo inicializada a cada chamada à função.

Desta forma, ela não é visível fora da função.

Uma variável global é definida fora de uma função, podendo ser vista por todas as funções do módulo e por todos os módulos que importam o módulo que a definiu.

```
def soma(num1, num2):
    total = num1 + num2
    print(total)
```

```
soma(10, 25)
```

```
print(total)
```

```
35
```

Traceback (most recent call last):

File "C:/Evaldo/PCloud/Udemy/MeusCursos/001-PythonParaTodos/Secao 8 - Estruturas de dados/Secao08_Aula04_Dicionarios_Desafios/codigo/temp.py", line 6, in <module>

```
    print(total)
```

NameError: name 'total' is not defined

```
salario_minimo = 937.00 # Variável global
```

```
def despesas_fixas():
    agua = 30.00 # Variável local
    luz = 100.00 # Variável local
    despesas = agua + luz
    saldo = salario_minimo - despesas
    print(f"Saldo despesas fixas: {saldo}")
    return despesas
```

```
def despesas_variaveis():
    supermercado = 500.00 # Variável local
    lanchonete = 100.00 # Variável local
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```

despesas = supermercado + lanchonete
saldo = salario_minimo - despesas
print(f"Saldo despesas variáveis: {saldo}")
return despesas

def despesas_mes():
    return despesas_fixas() + despesas_variaveis()

print(f"Salário mínimo: {salario_minimo}")
print(f"Despesas mês: {despesas_mes()}")

```

Resultado da execução:

Salário mínimo: 937.0
 Saldo despesas fixas: 807.0
 Saldo despesas variáveis: 337.0
 Despesas mês: 730.0

```

valor = 100.00 # Variável global

def calculo():
    valor = 50.00 # Variável local
    print(f"Valor dentro da função: {valor}")

print(f"Valor fora da função: {valor}")
calculo()
print(f"Valor fora da função: {valor}")

```

Resultado da execução:

Valor fora da função: 100.0
 Valor dentro da função: 50.0
 Valor fora da função: 100.0

```

valor = 100.00 # Variável global

def calculo():
    global valor
    valor = 50.00 # Variável global
    print(f"Valor dentro da função: {valor}")

print(f"Valor fora da função: {valor}")
calculo()
print(f"Valor fora da função: {valor}")

```

Resultado da execução:

Valor fora da função: 100.0
 Valor dentro da função: 50.0
 Valor fora da função: 50.0

O escopo de nomes em Python é mantido por meio de namespaces, que são dicionários que relacionam os nomes dos objetos (referências) e os objetos em si.

Os nomes ficam definidos em dois dicionários que podem ser consultados utilizando-se as funções **locals()** e **globals()**. Estes dicionários são atualizados em tempo de execução.

```

TAXA_JUROS = 1.2 / 100

def calculo1(num1, num2):
    total = num1 + num2
    total += total * TAXA_JUROS
    print(f"Variáveis locais calculo1: {locals()}")
    print(f"Calculo1: {total}")

```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
def calculo2(num1):
    total = 0
    for numero in num1:
        total += numero
    total += total * TAXA_JUROS
    print(f"Variáveis locais calculo2: {locals()}")
    print(f"Calculo2: {total}")

calculo1(25, 14)
numeros = [10, 50, 75.5, 96]
calculo2(numeros)
print(f"Variáveis globais: {globals()}")
```

Resultado da execução:

Variáveis locais calculo1: {'num1': 25, 'num2': 14, 'total': 39.468}

Calculo1: 39.468

Variáveis locais calculo2: {'num1': [10, 50, 75.5, 96], 'total': 234.278, 'numero': 96}

Calculo2: 234.278

Variáveis globais: {'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <_frozen_importlib_external.SourceFileLoader object at 0x00000265678152E8>, '__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>, '__file__': 'C:/Evaldo/PCloud/Udemy/MeusCursos/001-PythonParaTodos/Secao 9 - Funcoes Modulos e Pacotes/Secao09_Aula02_VariaveisGlobais/codigo/variaveis_loc_glo6.py', '__cached__': None, 'TAXA_JUROS': 0.012, 'calculo1': <function calculo1 at 0x0000026565CCC268>, 'calculo2': <function calculo2 at 0x0000026567A97B70>, 'numeros': [10, 50, 75.5, 96]}

Variáveis globais devem ser utilizadas o mínimo possível, pois dificultam a leitura e violam o encapsulamento da função (veremos encapsulamento na aula de orientação a objetos).

A dificuldade de leitura se dá pelo fato de que devemos ficar procurando as variáveis e conteúdos fora da função.

A variável global pode ter seu conteúdo alterado por qualquer função, desta forma fica difícil saber qual função alterou.

O encapsulamento é comprometido porque a função precisa de uma variável externa. Uma variável que não é declarada dentro da função e não é recebida por parâmetro.

Um bom uso de variáveis globais é utilizando as mesmas como constantes. Sempre que várias funções precisem de uma informação que é fixa, podemos criar constantes globais. Normalmente criamos constantes em maiúsculas.

```
TAXA_JUROS = 1.2 / 100
```

```
def calculo1(num1, num2):
    total = num1 + num2
    total += total * TAXA_JUROS
    print(f"Calculo1: {total}")

def calculo2(num1):
    total = 0
    for numero in num1:
        total += numero
    total += total * TAXA_JUROS
    print(f"Calculo2: {total}")

calculo1(10, 35)
valores = [10.0, 58.74, 65.41, 80.0]
calculo2(valores)
```

Resultado da execução:

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Calculo1: 45.54

Calculo2: 216.7198

Parâmetros de funções

Parâmetros ou argumentos de funções são essenciais para que possamos chamar funções informando valores. Desta forma, tornamos as funções flexíveis, recebendo valores diferentes a cada execução.

Funções sem parâmetros

Funções sem parâmetros não são flexíveis. Veja no exemplo a seguir a função soma, que toda vez que é chamada retorna como resultado, 35, ou seja, a soma de dois valores fixos no sistema.

```
def soma():
    valor1 = 10
    valor2 = 25
    return valor1 + valor2

total = soma()
print(f"O total é: {total}")
```

Funções com parâmetros opcionais

Veja no exemplo a seguir, que se você não quiser informar o valor do parâmetro "imprime", o mesmo terá o valor padrão False, que foi informado na declaração da função.

```
def soma(valor1, valor2, imprime = False):
    resultado = valor1 + valor2
    if imprime:
        print(f"Soma: {resultado}")
    return resultado

total = soma(10, 84)
```

Total recebe 94, porém, nada é impresso, porque o valor padrão para "imprime" é falso e não foi informado na chamada. Os parâmetros "valor1" e "valor2" são obrigatórios, já o parâmetro "imprime" é opcional porque já foi definido como falso.

Se informarmos True no parâmetro "imprime", a função print é executada.

```
total = soma(10, 84, True)
```

Parâmetros opcionais não podem estar no início caso seja combinado com parâmetros obrigatórios. Veja um exemplo, onde o parâmetro opcional está no início. Ao executarmos o programa teremos o erro a seguir.

```
def soma(imprime = False, valor1, valor2):
    resultado = valor1 + valor2
    if imprime:
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
print(f"Soma: {resultado}")
return valor1 + valor2
```

File ".../parametros_opcionais.py", line 2
def soma(imprime = False, valor1, valor2):

^

SyntaxError: non-default argument follows default argument

Esta mensagem está informando que argumentos sem valor padrão estão após argumentos que possuem valor padrão e isso é um erro.

Nomeando parâmetros.

Quando informamos o nome do parâmetro, não importa a ordem em que passamos os mesmos.

```
def retangulo(largura, altura, caractere="*"):
    linha = caractere * largura
    for i in range(altura):
        print(linha)
```

```
retangulo(caractere="$", altura=5, largura=15)
```

Funções como parâmetros.

Podemos passar funções como parâmetros de uma outra função.

```
def soma(num1, num2):
    return num1 + num2

def multiplicacao(num1, num2):
    return num1 * num2

def calcular(funcao, num1, num2):
    return funcao(num1, num2)

total_soma = calcular(soma, 10, 20)
total_multiplicacao = calcular(multiplicacao, 10, 20)

print(f"Total Soma: {total_soma}")
print(f"Total Multiplicação: {total_multiplicacao}")
```

Parâmetros empacotados em listas.

Podemos empacotar parâmetros passando uma lista.

```
lista = [10, 20, True]

def soma(valor1, valor2, imprime = False):
    resultado = valor1 + valor2
    if imprime:
        print(f"Soma: {resultado}")
    return resultado

soma(lista[0], lista[1], True)
# O empacotamento de lista evita termos que informar individualmente os valores da
# lista pelo índice.
soma(*lista)
# O asterisco indica que estamos desempacotando a lista utilizando seus valores
```



como parâmetros da função.

Desempacotamento de parâmetros

```
def soma(imprime, *valores):  
    total = 0  
    for valor in valores:  
        total += valor  
    if imprime:  
        print(f"Soma: {total}")  
    return total
```

```
soma(True, 10, 20, 30, 78)  
soma(False, 10, 50)
```

Resultado da execução:
Soma: 138

Módulos e pacotes

Nos capítulos anteriores aprendemos a criar funções em Python. À medida que um programa aumenta e são adicionadas novas funções, fica impossível mantê-las em um único arquivo. É neste momento que entra o conceito de módulos. Todo arquivo .py é um módulo. Um programa pode conter diversos módulos. Um módulo pode usar outros módulos, desta forma aproveitamos códigos existentes. Como exemplo criaremos um módulo denominado operacoes.py que conterá duas funções (**soma** e **media**) e um módulo denominado programa.py que não possuirá nenhuma função, mas importará o módulo operações para usar as funções **soma** e **media**.

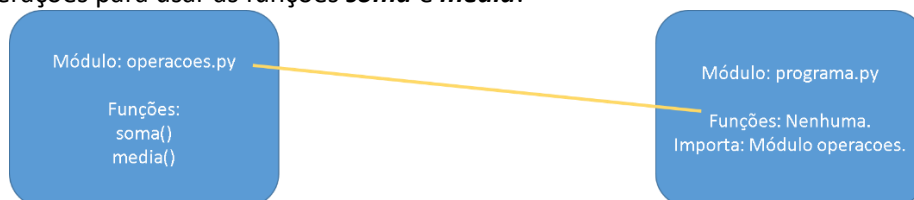


Figura 149 - Módulos

Conteúdo de operacoes.py

```
def soma(*valores):  
    total = 0  
    for numero in valores:  
        total += numero  
    return total  
  
def media(*valores):  
    total = soma(*valores)  
    return total / len(valores)
```

Conteúdo de programa.py

```
import operacoes
```

```
valor1 = operacoes.soma(10, 36, 87, 45, 50, 47, 21)
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
valor2 = operacoes.media(10, 8.5, 4, 7.5)

print(f"Valor1: {valor1}")
print(f"Valor2: {valor2}")
```

Conteúdo de programa2.py

Este programa irá solicitar os números ao usuário.

```
import operacoes

lista_numero = []

while True:
    numero = int(input("Informe um número inteiro ou zero para sair: "))
    if numero == 0:
        break
    lista_numero.append(numero)

soma = operacoes.soma(*lista_numero)
media = operacoes.media(*lista_numero)

print(f"Soma: {soma}")
print(f"Média: {media}")
```

Módulos são compilados quando importados pela primeira vez e armazenados em um arquivo com extensão .pyc ou .pyo.

O módulo é instanciado apenas uma vez em memória ficando disponível de forma global para o programa.

Quando um módulo é importado o Python procura sua localização utilizando a lista de pastas PYTHONPATH (sys.path) que, normalmente, inclui a pasta corrente em primeiro lugar.

Verificando o conteúdo de sys.path

O Python localiza os módulos através do sys.path.

```
import sys
print(sys.path)
```

Resultado da impressão:

```
['...PycharmProjects\\aula_modulos', '...PycharmProjects\\aula_modulos',
'...Evaldo\\AppData\\Local\\Programs\\Python\\Python36\\python36.zip',
'...Evaldo\\AppData\\Local\\Programs\\Python\\Python36\\DLLs',
'...Evaldo\\AppData\\Local\\Programs\\Python\\Python36\\lib',
'...Evaldo\\AppData\\Local\\Programs\\Python\\Python36',
'...Evaldo\\AppData\\Local\\Programs\\Python\\Python36\\lib\\site-packages']
```

Quando o módulo operacoes.py foi importado pelo módulo programa.py o Python criou o arquivo .pyc na pasta __pycache__.

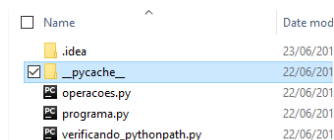


Figura 150 – Pasta __pycache__

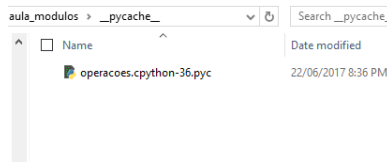


Figura 151 - Arquivo .pyc

Enquanto módulos são organizados em arquivos, pacotes ou packages são organizados em pastas identificadas com o arquivo `__init__.py`.

Os pacotes funcionam como coleções para organizar módulos de forma hierárquica.

É possível importar todos os módulos de um pacote, tirando os que começam com “_” (2 underscores) usando a declaração a seguir:

`from nome_do_pacote import *`

O arquivo `__init__.py` pode estar vazio, conter código de inicialização do pacote ou definir uma variável chamada `__all__`, que é uma lista de módulos que pertencem ao pacote e que serão importados quando usamos `*`.

Exemplo:

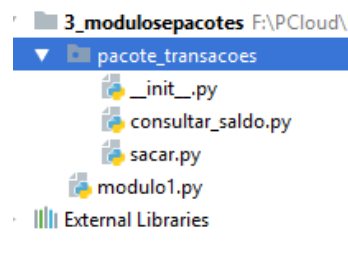


Figura 152 - Exemplo módulo

Código do arquivo `consultar_saldo.py`:

```
def retorna_saldo():
    return 500.00
```

Código do arquivo `sacar.py`:

```
from pacote_transacoes import consultar_saldo

def efetuar_saque(valor_saque):
    saldo_atual = consultar_saldo.retorna_saldo()
    if valor_saque < saldo_atual:
        print(f"Pode sacar. Resta: {saldo_atual - valor_saque}")
    else:
        print(f"Não pode sacar. Saldo da conta: {saldo_atual}")
```

Código do arquivo `modulo1.py`:

```
from pacote_transacoes import sacar

def realizar_saque(valor):
    sacar.efetuar_saque(valor)

realizar_saque(600.00)
```

Código do arquivo `__init__.py`:



```
__all__ = ['consultar_saldo', 'sacar']
```

Funções recursivas

Chamamos de função recursiva uma função que chama a si mesma.

Recursividade pode ser um problema se gerar chamadas infinitas.

O Python tem um limite de execuções recursivas por padrão igual a 1.000, ou seja, uma função recursiva não vai ficar rodando infinitamente.

```
def recursiva():
    recursiva()
```

```
recursiva()
```

Ao executar este código, obtemos o seguinte resultado:

```
recursiva()
```

[Previous line repeated 995 more times]

RecursionError: maximum recursion depth exceeded

Veja outro exemplo:

```
import sys

class Exemplo_01(object):
    def __init__(self):
        self.contador = 0
        print("Limite recursividade:", sys.getrecursionlimit())

        # Mude este valor para 3000 por exemplo para testar
        sys.setrecursionlimit(10)
        print("Limite recursividade:", sys.getrecursionlimit())

    def funcao_recursiva(self):
        self.contador += 1
        print("Contador: ", self.contador)

        # O Python identificará o limite vai ser atingido
        # e vai parar a execução antes de chegar em getrecursionlimit
        if self.contador < sys.getrecursionlimit():
            self.funcao_recursiva()

e = Exemplo_01()
e.funcao_recursiva()
```

sys.getrecursionlimit():

Retorna o limite de recursividade.

sys.setrecursionlimit():

Define o limite de recursividade.

Erro quando o limite é alcançado.

RecursionError: maximum recursion depth exceeded while calling a Python object

Um exemplo muito utilizado para explicar funções recursivas é desenvolver uma função que calcule o fatorial de um número inteiro.

O fatorial de um número inteiro m não negativo, é indicado por $m!$ (lê-se “ m fatorial”) e é definido pela relação:

$$m! = m \cdot (m-1) \cdot (m-2) \cdot (m-3) \dots 3 \cdot 2 \cdot 1, \text{ para } m \geq 2.$$



Algumas definições são:

- $1! = 1$
- $0! = 1$

Exemplos:

- $3! = 3 \cdot 2 \cdot 1 = 6$
- $4! = 4 \cdot 3 \cdot 2 \cdot 1 = 24$
- $6! = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 720$

Código do sistema de cálculo de fatorial:

```
def fatorial(x):
    if x <= 1:
        return 1
    else:
        return (x * fatorial(x-1))

num = int(input("Informe um número para encontrar o fatorial: "))

print("O fatorial de", num, "é", fatorial(num))
```

Exemplo de execução:

Informe um número para encontrar o fatorial: 4

O fatorial de 4 é 24

```
fatorial(4)      # Primeira chamada com 4
4 * fatorial(3)  # Segunda chamada com 3
4 * 3 * fatorial(2) # Terceira chamada com 2
4 * 3 * 2 * fatorial(1) # Quarta chamada com 1
4 * 3 * 2 * 1      # retorno da quarta chamada foi o número=1
4 * 3 * 2          # retorno da terceira chamada
4 * 6              # retorno da segunda chamada
24                # retorno da primeira chamada
```

Validação de entrada de dados

Podemos utilizar funções para criar uma infinidade de coisas, uma das coisas para a qual podemos criar funções é para validar a entrada de dados do usuário, reduzindo assim, possíveis erros no sistema.

Imagine que você tenha que validar uma entrada de dados específica, em vez de ficar sempre escrevendo o código de validação, você pode criar uma função e chamá-la sempre que necessário.

Vamos ao primeiro exemplo. Vamos supor que você precise solicitar que o usuário informe um número inteiro dentro de uma faixa. Você pode criar uma função que receberá os três números e fará esta validação.

```
def validar_faixa(numero, inicio, fim):
    if numero.isdigit():
        if int(numero) < int(inicio) or int(numero) > int(fim):
            print(f"Valor inválido! Informe um número inteiro entre {inicio} e {fim}.")
        else:
            return True
    else:
        print(f"Número inválido! Informe um número inteiro entre {inicio} e {fim}.")
        print(f"https://www.udemy.com/python-3-para-todos - https://www.pythonparatodos.com.br")
        print(f"contato@pythonparatodos.com.br")
```



```
{fim}.)

while True:
    resposta = input("Informe um número inteiro entre 1 e 10: ")
    if validar_faixa(resposta, 1, 10):
        break
```

No segundo exemplo vamos validar um limite máximo de caracteres de um texto informado pelo usuário.

```
def validar_tamanho(texto, maximo):
    if len(texto) > maximo:
        print(f"O texto deve conter no máximo {maximo} caracteres!")
    else:
        return True

while True:
    texto = input("Informe um texto de no máximo 20 caracteres: ")
    if validar_tamanho(texto, 20):
        break
```

No terceiro exemplo vamos validar a entrada do usuário com base em uma lista fornecida verificando se o texto informado está contido na lista.

```
def valida_elementos(texto, lista):
    if texto.upper() in lista:
        return True
    else:
        print("Opção inválida, veja as opções disponíveis:")
        for item in lista:
            print(item)

while True:
    lista = ["CARRO", "NAVIO", "ÔNIBUS", "AVIÃO"]
    resposta = input("Informe um meio de transporte: ")
    if valida_elementos(resposta, lista):
        break
```

Expressões lambda

Python permite a criação de funções anônimas (funções que não estão vinculadas a um nome) em tempo de execução usando o construtor "*lambda*".

Ao executar uma expressão *lambda* o *Python* retorna uma função, ao invés de atribuí-la a um nome como acontece quando usamos *def*, por isso são anônimas.

Estas funções são úteis quando precisamos de uma função para ser passada como parâmetro para outra função e que não será mais utilizada após isso, sendo descartada em seguida.

Uma expressão *lambda* permite criar uma função em uma única linha de código.

Por ser uma expressão, pode aparecer em lugar que um *def* não pode, como por exemplo, dentro de uma lista ou nos argumentos de chamada de uma função.

Sintaxe:

`lambda arg1, arg2, ...argN : expressão usando os argumentos`

Podemos passar vários argumentos separados por vírgula. E o retorno será uma função com o resultado do código informado.

Uso:

multiplica = *lambda* x: x * 2

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



valor = multiplica(10)
10 é o valor de "x", o resultado será $10 * 2$, ou seja, 20.

Exemplo 1:

```
def a(x):  
    return x**2  
print(a(4))  
  
b = lambda x: x**2  
print(b(4))
```

Exemplo 2:

```
# Função normal utilizando def e atribuindo um nome  
def dobrar(valor):  
    return valor * 2  
  
print("O dobro de quinze é:", dobrar(15))  
  
# Variável dobrar2 recebe o retorno da expressão lambda.  
# A expressão lambda retorna uma função  
# Onde x é o parâmetro recebido pela expressão e x * 2 é o código que será executado  
dobrar2 = lambda x: x * 2  
print("O dobro de vinte é:", dobrar2(20))  
  
# Podemos passar vários parâmetros em uma expressão lambda  
multiplicar = lambda x, y: x * y  
  
print("10 * 8 =", multiplicar(10, 8))
```

O dobro de quinze é: 30

O dobro de vinte é: 40

$10 * 8 = 80$

Exemplo 3:

```
def incrementar(n):  
    return lambda x: x + n  
  
somar2 = incrementar(2)  
somar8 = incrementar(8)  
  
a = 1  
b = somar2(a)  
print(b)  
c = somar8(a)  
print(c)  
d = somar2(10)  
print(d)
```

Exemplo 4:

```
soma = lambda lista: sum(lista)  
  
numeros = [1, 5, 10, 50]  
total = soma(numeros)  
print(total)  
  
numeros = [80, 74, 25]
```



```
total = soma(numeros)
print(total)
```

Exemplo 5:

```
curso = lambda a = "Curso", b = "Python", c = "para Todos!!!": a + " " + b + " " + c
print(curso())

def nomeCompleto():
    completo = lambda nome, sobrenome: nome + " " + sobrenome
    return completo

meuNome = nomeCompleto()
print(meuNome("Evaldo", "Wolkers"))
```

Exemplo 6:

```
tabuada = [lambda x: f"{x} * 1 = " + str(x * 1),
            lambda x: f"{x} * 2 = " + str(x * 2),
            lambda x: f"{x} * 3 = " + str(x * 3),
            lambda x: f"{x} * 4 = " + str(x * 4),
            lambda x: f"{x} * 5 = " + str(x * 5),
            lambda x: f"{x} * 6 = " + str(x * 6),
            lambda x: f"{x} * 7 = " + str(x * 7),
            lambda x: f"{x} * 8 = " + str(x * 8),
            lambda x: f"{x} * 9 = " + str(x * 9),
            lambda x: f"{x} * 10 = " + str(x * 10)]

try:
    numero = int(input("Informe o número para gerar a tabuada: "))
except:
    print("Número inválido!!!")
    exit()

for t in tabuada:
    print(t(numero))
```

Exercício:

Usando expressões lambda, crie funções para as quatro operações matemáticas básicas, adição, subtração, multiplicação e divisão.

Solicite que o usuário escolha uma das quatro operações, validando sua escolha, em seguida solicite que o usuário informe dois números inteiros que serão utilizados nas operações.

Esses dois números serão passados por parâmetro para que a operação seja realizada sobre os mesmos.

A função type

A função type retorna o tipo de uma variável, função ou objeto.



```
>>> a = 10
>>> type(a)
<class 'int'>
>>> b = "Olá mundo!"
>>> type(b)
<class 'str'>
>>> type(print)
<class 'builtin_function_or_method'>
>>> type(True)
<class 'bool'>
>>> lista = [1,2,3,4]
>>> type(lista)
<class 'list'>
>>>
```

Figura 153 - Usando a função type

```
>>> type(15.4)
<class 'float'>
>>> dict = {"a":1,"b":2,"c":3}
>>> type(dict)
<class 'dict'>
>>> def funcao():
...     return True
...
>>> type(funcao)
<class 'function'>
>>>
```

Figura 154 - Usando a função type

Vamos criar um programa onde vamos validar os itens de uma lista para tomar uma determinada decisão conforme o tipo do elemento.

```
def validar_itens(lista):
    soma_inteiros = 0
    qtd_inteiros = 0
    lista_str = []
    lista_list = []
    lista_dict = []
    for item in lista:
        if type(item) == int:
            soma_inteiros += item
            qtd_inteiros += 1
        elif type(item) == str:
            lista_str.append(item)
        elif type(item) == list:
            lista_list.append(item)
        elif type(item) == dict:
            lista_dict.append(item)
        else:
            print("Tipo não reconhecido!")

    if soma_inteiros > 0:
        print(f"Foram encontrados {qtd_inteiros} inteiros. A soma dos números inteiros é: {soma_inteiros}")

    if len(lista_str) > 0:
        print(f"Foram encontradas {len(lista_str)} strings. Veja as strings:")
        for i in lista_str:
            print(i)

    if len(lista_list) > 0:
        retorno = ""
        print(f"Foram encontradas {len(lista_list)} listas. Veja os itens das listas:")
        for lista in lista_list:
            for item in lista:
                retorno = retorno + str(item) + ", "
        print(retorno[0:-2])

    if len(lista_dict) > 0:
```



```
print(f"Foram encontrados {len(lista_dict)} dicionários. Veja os itens:")
for dic in lista_dict:
    print(f"Chaves: {dic.keys()}")
    print(f"Valores: {dic.values()}")

lista = [10, 43, 42, "Python", "Olá", [1,2,3], "Mundo", ["a","b","c"],
        {"Item 1":25, "Item 2": 65}, "Teste", {"key1":"Valor1","key2":"Valor2"}]

validar_itens(lista)
```

Mapeamento (Função map())

O mapeamento é feito através da função **map()**, que consiste em aplicar uma função a todos os itens de uma sequência, seja uma lista, tupla, etc., gerando uma outra sequência contendo os resultados e com o mesmo tamanho da sequência inicial. A função **map()** sempre retorna um gerador.

A sintaxe da função map é:

map(função, objeto_iterável, ...)

Onde:

função: **map()** passa cada item do objeto iterável para a função informada.

objeto_iterável: Objeto iterável que deve ser mapeado. Podemos passar mais de um objeto iterável.

Para exemplificar, vamos criar uma função que retorna um número informado ao quadrado:

```
def calculaQuadrado(numero):
    return f"O quadrado de {numero} é: {numero ** 2}"
```

Em seguida vamos criar uma lista de números. Depois criaremos uma variável chamada “calculo” que receberá o resultado da função **map()**, onde vamos passar a função calculaQuadrado e a lista de números como parâmetros.

```
lista_numeros = [1, 2, 3, 4, 5, 6]
calculo = map(calculaQuadrado, lista_numeros)
```

Por fim, vamos percorrer o objeto iterável “calculo”, que é um map, imprimindo cada item deste objeto.

```
for n in calculo:
    print(n)
```

Agora veja o programa em execução.

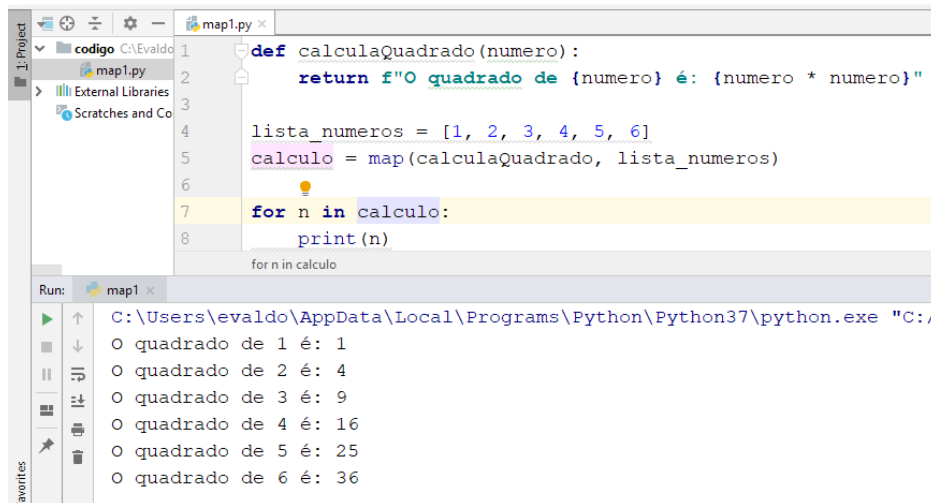


Figura 155 - Exemplo usando map()

Podemos usar também expressões **lambda** como função para o **map()**.

```
calculo = map(lambda x: f"O quadrado de {x} é: {x ** 2}", lista_numeros)
```

Podemos passar mais de um iterador para o **map()**.

```
lista1 = [10, 25, 37]
lista2 = [42, 18, 74]
```

```
soma = map(lambda x, y: x+y, lista1, lista2)
```

```
for n in soma:
    print(n)
```

Aprendendo ainda mais sobre Strings

Verificação de Strings e conversão para maiúsculas e minúsculas

Podemos utilizar os métodos *startswith* e *endswith* para validar se a string começa ou termina com uma determinada sequência de caracteres.

```
>>> nome = "Evaldo Wolkers"
>>> nome.startswith("Eval")
True
>>> nome.startswith("Cle")
False
>>> nome.endswith("lkers")
True
>>> nome.startswith("evaldo")
False
```

Figura 156 - Usando startswith e endswith

Podemos também verificar se uma palavra está contida na String usando o operador *in*.



```
>>> texto = "Aula de Python!"
>>> "Aula" in texto
True
>>> "Teste" in texto
False
>>> |
```

Figura 157 - Usando o operador in

Podemos também verificar se uma palavra não está contida na String usando *not in*.

```
>>> texto = "Aula de Python!"
>>> "Teste" not in texto
True
>>> "Aula" not in texto
False
>>> |
```

Figura 158 - Usando not in

Para converter uma String em maiúsculas ou minúsculas podemos utilizar os métodos *upper* e *lower*.

```
>>> texto = "Aula de Python!"
>>> texto.lower()
'aula de python!'
>>> texto.upper()
'AULA DE PYTHON!'
>>> texto.upper().startswith("Aula")
False
>>> texto.upper().startswith("AULA")
True
>>> |
```

Figura 159 - Usando upper e lower

Contando e pesquisando elementos em uma String

Podemos contar elementos em uma string usando o método *count*.

```
>>> texto = "Python é uma linguagem de programação. Python é simples. Python é organizado.
Python é uma excelente linguagem."
>>> texto.count("Python")
4
>>> texto.count("linguagem")
2
>>> texto.count("a")
8
>>> |
```



Figura 160 - Usando o método count

Para um exemplo prático, pesquisei no Google por filetype:txt e baixei um arquivo do link:

http://www.konnexion.biz/discos_estoque.txt

Vamos abrir este arquivo e contar as ocorrências de uma determinada palavra.

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>
contato@pythonparatodos.com.br



```

contador = 0
palavra = "volvo"
nome_arquivo = "discos_estoque.txt"
with open(nome_arquivo, "r") as arquivo:
    for linha in arquivo:
        contador = contador + linha.upper().count(palavra.upper())

print("Palavra pesquisada: " + palavra.upper())
print("Arquivo pesquisado: " + nome_arquivo)
print("Total de palavras encontradas: " + str(contador))

```

Para que o programa possa ser reutilizado vamos possibilitar que o usuário informe a palavra a ser pesquisada e o nome do arquivo.

```

contador = 0
palavra = input("Informe a palavra a ser pesquisada: ")
nome_arquivo = input("Informe o nome do arquivo: ")
with open(nome_arquivo, "r") as arquivo:
    for linha in arquivo:
        contador = contador + linha.upper().count(palavra.upper())

print("Palavra pesquisada: " + palavra.upper())
print("Arquivo pesquisado: " + nome_arquivo)
print("Total de palavras encontradas: " + str(contador))

```

Podemos utilizar o método *find* para obter a posição da primeira ocorrência de uma String. O Python retorna a posição do primeiro caractere da String. Caso não encontre a ocorrência, o Python retorna -1.

```

>>> texto = "Python é uma linguagem de programação. Python é simples. Python é organizado.
Python é uma excelente linguagem."
>>> texto.find("língua")
13
>>> texto.find("Olá")
-1

```

Figura 161 - Usando find

Podemos utilizar o método *rfind* para realizar a pesquisa da direita para a esquerda.

```

>>> texto = "Python é uma linguagem de programação. Python é simples. Python é organizado.
Python é uma excelente linguagem."
>>> texto.rfind("língua")
101
>>> |

```

Figura 162 - Usando rfind

Tanto *find* quanto *rfind* possuem dois parâmetros para que sejam delimitados o início e o fim para realização da pesquisa.



```
>>> texto = "Python é uma linguagem de programação. Python é simples. Python é organizado. Python é uma excelente linguagem."
>>> texto.find("Python", 25, 50)
39
>>> texto.rfind("Python", 25, 80)
57
>>> |
```

Figura 163 - Usando find e rfind

Para localizar o índice de ocorrência de uma string, podemos usar também o **index** (já apresentado na seção “Variáveis e tipos de dados”, na aula “Aprendendo mais sobre strings – aula 2”) e **rindex**, porém, caso a substring não seja localizada, esses métodos geram uma exceção **ValueError**.

```
>>> texto = "Python é uma linguagem de programação. Python é simples. Python é organizado. Python é uma excelente linguagem."
>>> texto.index("é")
7
>>> texto.rindex("é")
85
>>> texto.index("Teste")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: substring not found
>>> texto.rindex("Teste")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: substring not found
```

Figura 164 - Erro de substring não encontrada

Posicionamento de Strings

Provavelmente você algum dia terá que apresentar o resultado do seu programa com strings alinhadas, tornando o resultado mais fácil de ler. Seja em um relatório ou simplesmente na saída do programa.

O método **center** centraliza uma string em um número de posições passado como parâmetro, preenchendo com espaços à direita e à esquerda, até que a string esteja centralizada. Podemos substituir os espaços por qualquer caracter informando como parâmetro para a função.



```
>>> Texto1 = "Olá"
>>> Texto2 = "Mundo"
>>> Texto3 = Texto1.center(10)
>>> Texto4 = Texto2.center(10)
>>> Texto3
'  Olá  '
>>> Texto4
'  Mundo  '
>>> print(Texto3)
Olá
>>> print(Texto4)
Mundo
>>> print(Texto3 + "\n" + Texto4)
Olá
Mundo
>>> |
```

Figura 165 - Usando string.center()

```
>>> Texto1 = "Olá"
>>> Texto2 = "Mundo"
>>> Texto3 = Texto1.center(10, '-')
>>> Texto4 = Texto2.center(10, '-')
>>> print(Texto3 + "\n" + Texto4)
---Olá---
--Mundo--
>>> |
```

Figura 166 - Usando string.center()

O Python também possui métodos para alinhar o texto à esquerda e à direita. São os métodos ***ljust*** e ***rjust***. O método ***ljust*** alinha o texto à esquerda e preenche com espaços à direita, já o método ***rjust*** alinha o texto à direita e preenche com espaços à esquerda.

```
>>> Texto1 = "Olá"
>>> Texto2 = "Mundo"
>>> Texto3 = Texto1.ljust(5)
>>> print(f"{Texto3}*\n{Texto2}*")
*Olá *
*Mundo*
>>> |
```

Figura 167 - Usando string.ljust()

Colocou 2 espaços à direita e alinou o texto à esquerda.

```
>>> Texto1 = "Olá"
>>> Texto2 = "Mundo"
>>> Texto3 = Texto1.rjust(5)
>>> print(f"{Texto3}*\n{Texto2}*")
*  Olá*
*Mundo*
>>> |
```

Figura 168 - Usando string.rjust()

Colocou 2 espaços à esquerda e alinou o texto à direita.



Separação, Substituição de Strings e Remoção de Espaços em Branco

Para separar strings à partir de um determinado caractere podemos utilizar o método ***split***. O resultado é uma lista com os itens que eram delimitados pelo caractere informado na chamada ao método ***split***.

```
>>> soneto35ws = "Não chores mais o erro cometido; Na fonte, há lodo; a rosa tem espinho;  
O sol no eclipse é sol obscurecido; Na flor também o inseto faz seu ninho;"  
>>> lista = soneto35ws.split(";")  
>>> lista  
['Não chores mais o erro cometido', ' Na fonte, há lodo', ' a rosa tem espinho', ' O sol n  
o eclipse é sol obscurecido', ' Na flor também o inseto faz seu ninho', '']  
>>>
```

Figura 169 - Usando o método *split*

Para separar strings à partir do caractere de nova linha “\n” podemos usar o método ***splitlines***.

```
>>> soneto30ws = "Quando à corte silente do pensar\nEu convoco as lembranças do passado,\nSuspiro pelo que ontem fui buscar,\nChorando o tempo já desperdiçado,"  
>>> lista = soneto30ws.splitlines()  
>>> lista  
['Quando à corte silente do pensar', 'Eu convoco as lembranças do passado,', 'Suspiro pelo  
que ontem fui buscar,', 'Chorando o tempo já desperdiçado,']  
>>>
```

Figura 170 - Usando o método *splitlines*

Para substituir trechos de uma string podemos usar o método ***replace***. Este método recebe por parâmetro a string que será substituída, a nova string, e por fim um número que limitará a quantidade de substituições.

```
>>> texto = "Python é uma linguagem de programação. Python é simples. Python é organizado.  
Python é uma excelente linguagem."  
>>> texto2 = texto.replace("Python", "Bicicleta")  
>>> texto2  
'Bicicleta é uma linguagem de programação. Bicicleta é simples. Bicicleta é organizado. Bi  
cicleta é uma excelente linguagem.'  
>>> texto2 = texto.replace("Python", "Chiclete", 2)  
>>> texto2  
'Chiclete é uma linguagem de programação. Chiclete é simples. Python é organizado. Python  
é uma excelente linguagem.'  
>>> |
```

Figura 171 - Usando o método *replace*

Se o primeiro parâmetro do ***replace*** for vazio, será inserido o caractere informado antes de cada caractere da string e se o segundo parâmetro for vazio, o trecho será apagado.



```
>>> texto = "Python é uma linguagem de programação. Python é simples. Python é organizado.
Python é uma excelente linguagem."
>>> texto2 = texto.replace(" ", "")
>>> texto2
'*P*y*t*h*o*n* *é* *u*m*a* *l*i*n*g*u*a*g*e*m* *d*e* *p*r*o*g*r*a*m*a*ç*ã*o*.* *P*y*t*h*o*
n* *é* *s*i*m*p*l*e*s*.* *P*y*t*h*o*n* *é* *o*r*g*a*n*i*z*a*d*o*.* *P*y*t*h*o*n* *é* *u*m*
a* *e*x*c*e*l*e*n*t*e* *l*i*n*g*u*a*g*e*m*.*'
>>> texto2 = texto.replace("Python", "")
>>> texto2
' é uma linguagem de programação. é simples. é organizado. é uma excelente linguagem.'
>>>
```

Figura 172 - Usando replace

Para remover espaços em branco de uma string temos três métodos:

Strip: Remove espaços no início e fim da string.

Lstrip: Remove espaços no início da string.

Rstrip: Remove espaços no fim da string.

Você pode informar um caractere a remover em vez de remover os espaços em branco.

```
>>> a = "    Essa string tem espaços no início e no fim.    "
>>> print("'" + a + "'")
*    Essa string tem espaços no início e no fim.    *
>>> b = a.strip()
>>> print("'" + b + "'")
*Essa string tem espaços no início e no fim.*
>>> b = a.lstrip()
>>> print("'" + b + "'")
*Essa string tem espaços no início e no fim.    *
>>> b = a.rstrip()
>>> print("'" + b + "'")
*    Essa string tem espaços no início e no fim.*
>>> |
```

Figura 173 - Usando lstrip e rstrip

```
>>> a = "-----Olá mundo!-----"
>>> b = a.strip("-")
>>> b
'Olá mundo!'
>>> b = a.lstrip("-")
>>> b
'Olá mundo!-----'
>>> b = a.rstrip("-")
>>> b
'-----Olá mundo!'
>>> |
```

Figura 174 - Usando strip, lstrip e rstrip

Validação por tipo de conteúdo

Nesta aula veremos como analisar e verificar uma string para identificar letras e números em seu conteúdo.



Para verificar se uma string tem conteúdo e se todos os seus caracteres são letras ou números podemos utilizar o método *isalnum* (é alfanumérico). Este método retorna verdadeiro caso a string não esteja vazia e se todos seus caracteres são letras e números.

```
1 texto = "EstaStringPossuiApenasLetrasENumeros1234567890"
2 print(texto.isalnum())
3
4 texto = "Esta String possui letras, números, espaços, vírgulas, ponto, 1, 2, 3, 4, 5."
5 print(texto.isalnum())
6
7 texto = "Esta string possui letras e espaços"
8 print(texto.isalnum())
9
```

Run: validacao_conteudo1 x

C:\Users\evaldo\AppData\Local\Programs\Python\Python37\python.exe C:/Evaldo/PCloud/Udemy/MeusCursos

True
False
False

Figura 175 - Método isalnum

O método *isalpha* retorna verdadeiro se a string não for vazia e se todos seus caracteres forem letras (incluindo vogais acentuadas).

```
1 texto = "EstaStringPossuiApenasLetras"
2 print(texto.isalpha())
3
4 texto = "Esta String possui letras, números, espaços, vírgulas, ponto, 1, 2, 3, 4, 5."
5 print(texto.isalpha())
6
7 teste = "texto string possui letras e espaços"
8 print(texto.isalpha())
9
```

Run: validacao_conteudo2 x

C:\Users\evaldo\AppData\Local\Programs\Python\Python37\python.exe C:/Evaldo/PCloud/Udemy/MeusCursos

True
False
False

Figura 176 - Método isalpha

O método *isdigit* retorna verdadeiro se a string não for vazia e se todos seus caracteres forem numéricos em Unicode. Além dos números de 0 a 9, *isdigit* reconhece outros caracteres numéricos, como por exemplo, o 9 tibetano que é *Unicode* (\u0f29).



```

1 texto = "EstaStringPossuiApenasLetras"
2 print(texto.isdigit())
3
4 texto = "12345678921251455452554565"
5 print(texto.isdigit())
6
7 texto = "\u0f292545667878"
8 print(texto.isdigit())
9

```

Run: validacao_conteudo3

```

C:\Users\evaldo\AppData\Local\Programs\Python\Python37\python.exe C:/Evaldo/Projeto/validacao_conteudo3.py
False
True
True

```

Figura 177 - Método isdigit

O método *isnumeric* retorna verdadeiro se a string não for vazia e se todos seus caracteres forem numéricos (Unicode também). Este método é mais abrangente que o *isdigit* e inclui dígitos e representações numéricas como frações, por exemplo, 1/3 (\u2153).

```

1 texto = "EstaStringPossuiApenasLetras"
2 print(texto.isnumeric())
3
4 texto = "12345678921251455452554565"
5 print(texto.isnumeric())
6
7 texto = "\u0f292545667878\u2153"
8 print(texto.isnumeric())
9

```

Run: validacao_conteudo4

```

C:\Users\evaldo\AppData\Local\Programs\Python\Python37\python.exe C:/Evaldo/Projeto/validacao_conteudo4.py
False
True
True

```

Figura 178 - Método isnumeric

O método *isupper* verifica se todos os caracteres da string são maiúsculos.

```

1 texto = "EstaStringPossuiLetrasMaiusculasMinusculas"
2 print(texto.isupper())
3
4 texto = "ESTA STRING POSSUI TODAS LETRAS MAIÚSCULAS"
5 print(texto.isupper())
6
7 texto = "ESTASTRINGPOSSUITODASLETRASEMMAIÚSCULAS"
8 print(texto.isupper())
9

```

Run: validacao_conteudo5

```

C:\Users\evaldo\AppData\Local\Programs\Python\Python37\python.exe C:/Evaldo/Projeto/validacao_conteudo5.py
False
True
True

```

Figura 179 - Método isupper

O método *islower* verifica se todos os caracteres da string são minúsculos.



```

1 texto = "EstaStringPossuiLetrasMaisculasMinusculas"
2 print(texto.islower())
3
4 texto = "esta string possui todas letras minúsculas"
5 print(texto.islower())
6
7 texto = "estaststringpossuitodasletrasminúsculas"
8 print(texto.islower())
9

```

Run: validacao_conteudo6

```

C:\Users\evaldo\AppData\Local\Programs\Python\Python37\python.exe C:/
False
True
True

```

Figura 180 - Método islower

O método *isspace* verifica se a string contém apenas caracteres em branco, como espaços, marcas de tabulação (TAB), quebras de linha (LF) ou retorno de carro (CR).

```

1 texto = " \n "
2 print(texto.isspace())
3
4 texto = " 1 "
5 print(texto.isspace())
6
7 texto = "xkjdfjs  ojl5456sf5sd7f8s sdfsd5458sd"
8 print(texto.isspace())
9

```

Run: validacao_conteudo7

```

C:\Users\evaldo\AppData\Local\Programs\Python\Python37\python.exe
True
False
False

```

Figura 181 - Método isspace

O método *isprintable* verifica se a string pode ser impressa na tela. Esse método retorna falso se algum caractere que não pode ser impresso for encontrado na string.

```

1 texto = " \n "
2 print(texto.isprintable())
3
4 texto = " 1\n "
5 print(texto.isprintable())
6
7 texto = "xkjdfjs \u0f292545667878 sdfsd5458sd"
8 print(texto.isprintable())
9

```

Run: validacao_conteudo8

```

C:\Users\evaldo\AppData\Local\Programs\Python\Python37\python.exe
True
False
False

```

Figura 182 - Método isprintable

Orientação a Objetos

Conceitos básicos

Orientação a Objetos é um assunto muito extenso.

Existem diversos livros (inteiros) sobre POO.

Neste curso veremos uma introdução à programação orientada a objetos, não sendo meu objetivo esgotar este assunto.

Objeto:

Todo mundo sabe o que é um objeto.

Objeto é uma coisa tangível, que podemos tocar, sentir e manipular.

Quando ainda éramos bebês interagíamos com objetos (brinquedos infantis).



Figura 183 - Objetos

Bebês aprendem que certos objetos fazem certas coisas:

Sinos tocam, botões são pressionados e alavancas são puxadas.

A definição de objeto em desenvolvimento de software não é muito diferente.



Figura 184 - Objetos

Objetos de software não são coisas tangíveis, não tocamos ou sentimos, mas são modelos de algo que pode realizar certas ações.

No desenvolvimento de software objeto é uma coleção de dados que possui comportamentos associados.

Um software é uma coleção de objetos que se comunicam entre si, dizendo uns aos outros o que devem fazer.

Objetos de software, assim como objetos do mundo real possuem características (propriedades, atributos) e ações (procedimentos, métodos).

Os atributos são estruturas de dados que armazenam informações sobre o objeto.

Os métodos são responsáveis pela manipulação dos atributos, são funções associadas ao objeto que descrevem como o objeto se comporta.

Resumindo, objetos são abstrações computacionais que representam entidades com características e ações.

Objeto: CARRO



Figura 185 - Objeto Carro

Atributos: Cor, Marca, Modelo, Ano, Capacidade do tanque.	Métodos: Acelerar, frear, virar à esquerda, virar à direita.
---	--

Paradigma:

Paradigma é uma forma de abordar um problema.

Paradigma de programação estruturada:

Os programadores abstraem o programa como uma sequência de funções executadas de modo empilhado.

Paradigma de programação orientada a objetos:

Programadores abstraem o programa como uma coleção de objetos que interagem entre si.

Classe:

É a estrutura básica do *paradigma* de orientação a objetos, que representa o tipo do objeto, um modelo a partir do qual os objetos serão criados.

Classes são abstrações utilizadas para representar um conjunto de objetos com características e comportamentos idênticos.

Uma classe pode ser vista como uma fábrica de objetos.

Classe: VEÍCULOS



Figura 186 - Objetos da classe veículos

Classe: CANINO

A classe CANINO representa qualquer canino. Descreve as características e ações dos caninos em geral.



Figura 187 - Cachorros fazem parte da classe CANINO.

Características/Atributos: Peso, altura, idade, raça.	Ações/Métodos: Comer, correr, latir e atacar.
---	---

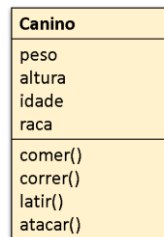


Figura 188 - Diagrama da classe Canino

Uma forma de representar visualmente uma classe é usando o Diagrama de classe UML (Linguagem de Modelagem Unificada).

Definição do diagrama de classes:

O Diagrama de Classes é utilizado para fazer a representação de estruturas de classes de negócio, interfaces e outros sistemas e classes de controle. Além disso, o diagrama de classes é considerado o mais importante para a UML, pois serve de apoio para a maioria dos demais diagramas.

Objeto Bulldog da Classe Canino:

O “objeto” BULLDOG representa um canino em particular.

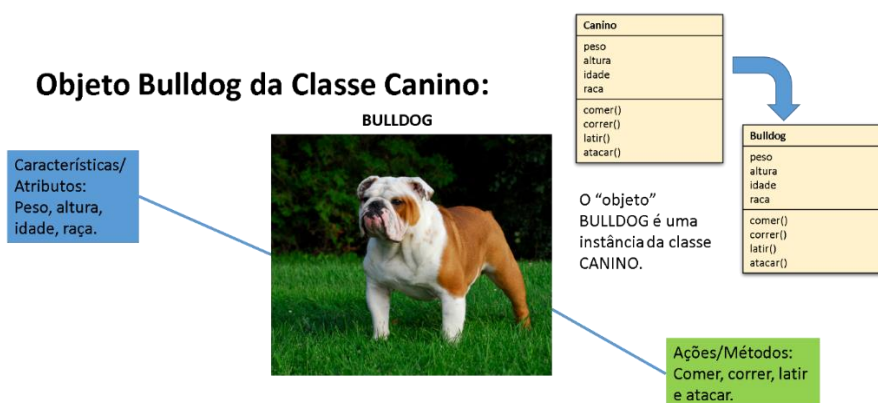


Figura 189 - Objeto Bulldog da classe Canino

Orientação a objetos:

Sabemos o que é um objeto, então, o que é orientação a objetos?

Orientado = direcionado para.

Sendo assim, orientado a objeto é praticamente direcionado para modelos de objetos.

Orientação a objetos é uma técnica para modelar sistemas complexos, descrevendo uma coleção de objetos interagindo através de seus dados e comportamentos.

Provavelmente você já ouviu falar em Análise Orientada a Objetos, Design Orientado a Objetos e Programação Orientada a Objetos.

Todos estes são conceitos relacionados à orientação a objetos, são estágios do desenvolvimento de software.

Chamá-los de “orientados a objetos” especifica o estilo de desenvolvimento que está sendo seguido.

Análise orientada a objetos: É o processo de olhar para um problema, sistema ou tarefa (que alguém quer transformar em um aplicativo) e identificar os objetos e as interações entre esses objetos.

A produto final do estágio de análise é um conjunto de requisitos.



Figura 190 - Análise orientada a objetos

Em desenvolvimento de softwares, o estágio da análise inclui entrevistar clientes, estudar seus processos e eliminar possibilidades.

Design orientado a objetos: O design orientado a objetos é o processo de converter os requisitos em uma especificação de implementação. O designer deve nomear os objetos, definir os comportamentos e especificar formalmente quais objetos podem ativar comportamentos específicos em outros objetos.

Esse estágio é responsável por definir como as coisas devem ser feitas.

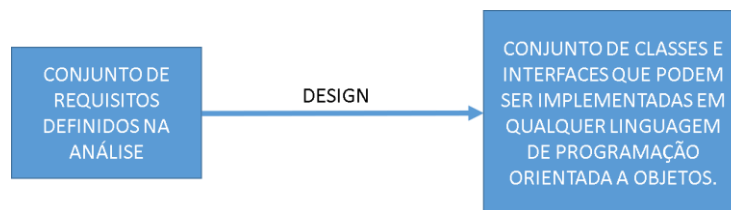


Figura 191 - Design orientado a objetos

Programação orientada a objetos: É o processo de converter o que foi definido pelo design em um programa que faz o que originalmente foi solicitado.

É uma técnica de programação que organiza nossos programas em classes e objetos em vez de apenas funções.

Orientação a objetos:

É comum dizer que Python é uma linguagem orientada a objetos.

Porém, ao contrário de Java, Python não impõe a programação orientada a objetos como o principal paradigma de programação. É perfeitamente viável um projeto em Python não ser orientado a objetos, ou seja, não utilizar mecanismos que sejam específicos da programação orientada a objetos. Pode-se utilizar Python para construir sistemas utilizando o paradigma de programação estruturada. Em Python, tudo é um objeto e pode ser manipulado como tal. Funções, classes, strings e até mesmo tipos são objetos. Todos têm um tipo, podem ser passados como argumento de função e podem ter métodos e propriedades. Nesse aspecto, Python é uma linguagem orientada a objetos.

Entender os conceitos essenciais da programação orientada a objetos envolve compreender os conceitos de objetos, classes e métodos.

Classe

No Python, novos objetos são criados a partir das classes por meio de atribuição. O objeto é uma instância da classe que possui características próprias. As classes são derivadas da classe base denominada *object*.

Veja como criamos um objeto:

```
Objeto = Classe()
```

No nosso exemplo seria:

```
Bulldog = Canino()
```

É como se o objeto fosse uma variável e o tipo fosse a classe.

Toda instância de classe ou variável tem seu próprio endereço de memória ou sua identidade.

Os objetos, que são instâncias de classes interagem entre si para servir ao propósito de uma aplicação em desenvolvimento.



Para criar uma classe em Python usamos a palavra reservada *class*.

```
class nome_classe:
    var = valor
    ...
    var = valor
    def metodo(self, ...arg):
    ...
    def metodo(self, ...arg):
    ...
```

O primeiro argumento de um método é o *self*. Esta variável representa o próprio objeto.

O nome *self* é uma convenção, podendo ser trocado por outro nome qualquer, porém é considerado como boa prática manter este nome.

Arquivo heroi.py

```
class Heroi:
    """
    Classe de heróis
    """
    voa = False
    possui_armas = False
    lanca_teia = False
    frase_comum = ""

    def falar(self):
        print(self.frase_comum)

    def detalhar(self):
        if self.voa:
            print("O herói voa.")
        if self.possui_armas:
            print("O herói possui armas.")
        if self.lanca_teia:
            print("O herói lança teia.")
```

Arquivo sistema.py

```
from heroi import Heroi

homem_aranha = Heroi()
homem_aranha.lanca_teia = True
print(homem_aranha.voa)
print(homem_aranha.lanca_teia)

he_man = Heroi()
he_man.possui_armas = True
he_man.lanca_teia = False
he_man.voa = False
he_man.frase_comum = "Eu tenho a força"
he_man.falar()

homem_aranha.detalhar()
he_man.detalhar()
```

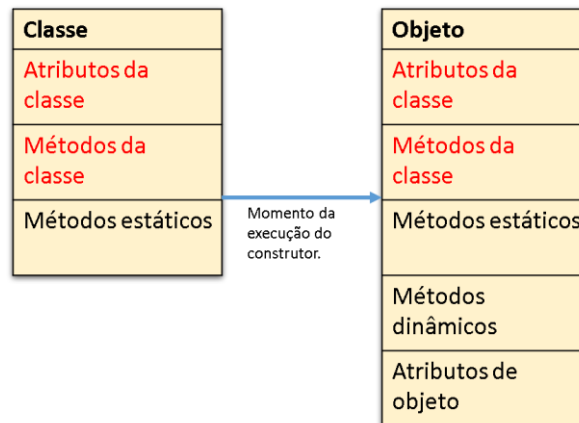


Figura 192 - Classe e objeto

Objeto = Classe()

No momento da criação do objeto, é executado o que chamamos de *construtor da classe*. O construtor é um método especial, chamado `__new__()`.

Após a chamada ao construtor, o método `__init__()` é chamado para inicializar a nova instância. O método `__init__()` pode ser usado para passar argumentos, assim podemos passar valores para os atributos do novo objeto.

Os métodos especiais em Python são identificados por nomes no padrão `__nome__()`. São utilizados dois underscores no início e no fim do nome.

Arquivo heroi2.py

```
class Heroi:
    """
    Classe de heróis
    """
    def __init__(self, voa, possui_arma,
                  lanca_teia, frase_comum):
        print("Executando init...")
        self.voa = voa
        self.possui_arma = possui_arma
        self.lanca_teia = lanca_teia
        self.frase_comum = frase_comum

    def falar(self):
        print(self.frase_comum)

    def detalhar(self):
        if self.voa:
            print("O herói voa.")
        if self.possui_arma:
            print("O herói possui arma.")
        if self.lanca_teia:
            print("O herói lança teia.")
```

Arquivo sistema2.py

```
from heroi2 import Heroi

# Heroi(voa, possui_arma, lanca_teia, frase_comum)
homem_aranha = Heroi(False, False, True, "")
print(homem_aranha.voa)
print(homem_aranha.lanca_teia)

he_man = Heroi(False, True, False, "Eu tenho a força!")
he_man.frase_comum = "Eu tenho a força"
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
he_man.falar()

homem_aranha.detalhar()
he_man.detalhar()
```

Classes, objetos, namespaces, sombreado de atributos e self

Um pouco mais sobre classes e objetos

```
class Classe_simples(): # Quando vazios, as chaves são opcionais, class
    Classe_simples:
        pass

print(type(Classe_simples)) # Qual o tipo do objeto? => <class 'type'>

obj = Classe_simples() # obj é uma instância da classe simples
print(type(obj)) # <class '__main__.Classe_simples'>

print(type(obj) == Classe_simples) # True, obj é do tipo Classe_simples
```

Outro exemplo de classe.

```
class Pessoa():
    especies = 'Humano'

print(Pessoa.especies) # Humano
Pessoa.vivo = True # Adicionado dinamicamente
print(Pessoa.vivo) # True
homem = Pessoa()
print(homem.especies) # Humano (herdado)
print(homem.vivo) # True (herdado)
Pessoa.vivo = False
print(homem.vivo) # False (herdado)
homem.nome = 'Evaldo'
homem.sobrenome = 'Wolkers'
print(homem.nome, homem.sobrenome) # Evaldo Wolkers
```

Escopo e espaço de nome (Namespace):

Namespace é um espaço ou região dentro do programa, onde um nome (seja uma variável, uma função, etc.) é considerado válido.

Depois que o objeto de classe é criado, ele basicamente representa um namespace. Podemos chamar essa classe para criar suas instâncias. Cada instância herda os atributos e métodos da classe e recebe seu próprio namespace.

No Python temos basicamente 3 escopos:

- 1 – Escopo local: que contém nomes locais (função atual).
- 2 – Escopo global: escopo do módulo que contém nomes globais, acessado por todas funções do módulo.
- 3 – Built-in names (nomes embutidos): que é o namespace que contém as funções built-in do Python (funções padrões como abs(), cmp(), etc.) e built-in exception names (usados para tratar erros específicos nas cláusulas *except* de blocos *try*).

Exemplificando escopo e namespace.

```
def funcao_externa():
    b = 20
    a = 80
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```

print(f"Imprimindo 'b' em funcao_externa: {b}")
print(f"Imprimindo 'a' em funcao_externa: {a}")
def funcao_interna():
    c = 30
    b = 25
    a = 70
    print(f"Imprimindo 'c' em funcao_interna: {c}")
    print(f"Imprimindo 'b' em funcao_interna: {b}")
    print(f"Imprimindo 'a' em funcao_interna: {a}")

funcao_interna()
print(f"Imprimindo 'b' de novo em funcao_externa: {b}")

a = 10

print(f"Imprimindo 'a' no escopo global: {a}")
funcao_externa()
print(f"Imprimindo 'a' de novo no escopo global: {a}")

```

A variável “a” está no namespace global.

A variável “b” está no namespace local da função “funcao_externa”.

A variável “c” está no namespace local da função “funcao_interna”.

Quando estamos na funcao_interna, “c” é local, “a” é global e “b” é nonlocal. Se tentarmos atribuir um valor a “b”, será criada uma variável local para funcao_interna diferente da b nonlocal.

A mesma coisa acontece ao atribuirmos um valor a variável global “a”.

```

Imprimindo 'a' no escopo global: 10
Imprimindo 'b' em funcao_externa: 20
Imprimindo 'a' em funcao_externa: 80
Imprimindo 'c' em funcao_interna: 30
Imprimindo 'b' em funcao_interna: 25
Imprimindo 'a' em funcao_interna: 70
Imprimindo 'b' de novo em funcao_externa: 20
Imprimindo 'a' de novo no escopo global: 10

```

Exemplificando escopo e namespace novamente.

```

def funcao_externa():
    b = 20
    global a
    a = 80
    print(f"Imprimindo 'b' em funcao_externa: {b}")
    print(f"Imprimindo 'a' em funcao_externa: {a}")
    def funcao_interna():
        c = 30
        b = 25
        global a
        a = 70
        print(f"Imprimindo 'c' em funcao_interna: {c}")
        print(f"Imprimindo 'b' em funcao_interna: {b}")
        print(f"Imprimindo 'a' em funcao_interna: {a}")

    funcao_interna()
    print(f"Imprimindo 'b' de novo em funcao_externa: {b}")

a = 10

print(f"Imprimindo 'a' no escopo global: {a}")
funcao_externa()
print(f"Imprimindo 'a' de novo no escopo global: {a}")

```



Ao definir a variável “a” dentro das funções especificando “global” será referenciada a variável “a” do escopo global, ao alterar seu valor, será alterado o valor da variável “a” global.

```
Imprimindo 'a' no escopo global: 10
Imprimindo 'b' em funcao_externa: 20
Imprimindo 'a' em funcao_externa: 80
Imprimindo 'c' em funcao_interna: 30
Imprimindo 'b' em funcao_interna: 25
Imprimindo 'a' em funcao_interna: 70
Imprimindo 'b' de novo em funcao_externa: 20
Imprimindo 'a' de novo no escopo global: 70
```

Sombreamento de atributos:

Quando um atributo em um objeto não é encontrado, o Python continua buscando na classe que foi usada para criar esse objeto (e continua pesquisando até que seja encontrado ou o fim da cadeia de herança seja alcançado). Isso leva a um comportamento de sombreamento.

```
class Ponto():
    x = 10
    y = 7

p = Ponto()

print(p.x) # 10 (do atributo da classe)
print(p.y) # 7 (do atributo da classe)
p.x = 12 # p obtém seu próprio atributo "x"
print(p.x) # 12 (encontrado na instância)
print(Ponto.x) # 10 (O atributo da classe ainda é o mesmo)
del p.x # Apagando o atributo da instância
print(p.x) # 10 (Agora que não existe "x" na instância, será retornado da classe)
p.z = 3
print(p.z) # 3
print(Ponto.z) # O objeto Ponto não tem o atributo "z"
# AttributeError: type object 'Ponto' has no attribute 'z'
```

O que é o self?

Dentro de um método de classe, podemos nos referir a uma instância por meio de um argumento especial, chamado *self* por convenção. *Self* é sempre o primeiro atributo de um método de instância.

Primeiro exemplo

```
class Quadrado():
    lados = 8
    def area(self): # self é uma referência a uma instância
        return self.lados ** 2

quadrado = Quadrado()
print(quadrado.area()) # 64 ('lados' foi encontrado na classe)
print(Quadrado.area(quadrado)) # 64 (equivalente a quadrado.area())
# Aqui foi passada a instância para a chamada do
método
quadrado.lados = 10
print(quadrado.area()) # 100 ('lados' foi encontrado na instância)
```

Segundo exemplo

```
class Calculo():
    def calcular_total(self, quantidade, desconto):
        return (self.preco * quantidade - desconto)

calc = Calculo()
calc.preco = 15
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
print(calc.calcular_total(15, 10))
print(Calculo.calcular_total(calc, 15, 10))
```

Herança

Herança nos permite criar uma versão modificada de uma classe existente, adicionando novos atributos e métodos.

Com herança podemos adaptar o comportamento de classes existentes sem termos que modificá-las. A nova classe herda todos os métodos da classe existente.

A classe existente pode ser chamada de classe mãe, classe base ou superclasse e a nova classe, pode ser chamada de classe filha, classe derivada, ou subclasse.

Herança facilita o reuso de código pois podemos adaptar o comportamento de classes existentes, sem ter que modificá-las.

Para localizar os métodos e atributos, o Python procura na classe derivada, retornando pela cadeia de classes base até encontrá-los, similar ao que acontece nos namespaces local e global.

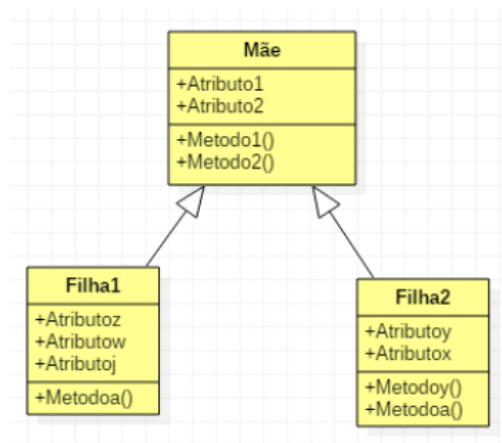


Figura 193 - Herança

Arquivo classe_base.py

```
class Classe_base():
    def __init__(self, valor1, valor2):
        print("Método construtor da classe base")
        self.valor1 = valor1
        self.valor2 = valor2

    def somar(self):
        return self.valor1 + self.valor2

    def subtrair(self):
        return self.valor1 - self.valor2
```

Arquivo classe_derivada1.py

```
from classe_base import Classe_base

class Classe_derivada1(Classe_base):
    def __init__(self, v1, v2):
        print("Método construtor da classe derivada1.")
        super().__init__(v1, v2)
```



```
def imprimir(self, texto):
    print(texto)
```

Arquivo classe_derivada2.py

```
from classe_base import Classe_base
```

```
class Classe_derivada2(Classe_base):
    def multiplicar(self, num1, num2):
        return num1 * num2
```

Arquivo programa.py

```
from classe_derivada1 import Classe_derivada1
from classe_derivada2 import Classe_derivada2
```

```
class Classe_teste():
    calculo = Classe_derivada1(10, 25)
    resultado = calculo.somar()
    print(resultado)
    resultado = calculo.subtrair()
    print(resultado)
    calculo.imprimir("Olá mundo!")

    calc = Classe_derivada2(70, 85)
    resultado = calc.multiplicar(20, 10)
    print(resultado)
    resultado = calc.somar()
    print(resultado)
```

Outro exemplo de herança.

```
# Criando a classe base
class Pai():
    def __init__(self):
        print('Construindo a classe Pai')

# Classe filha herda da classe pai
class Filha(Pai):
    def __init__(self):
        Pai.__init__(self) # Chamando o construtor da classe pai direto
# Criada classe mãe
class Mae():
    def __init__(self):
        print('Construindo a classe Mãe')

# Mudar a classe filha para herdar de Mae
class Filha(Mae):
    def __init__(self):
        Pai.__init__(self) # Chamando o construtor
                           # da classe pai direto. E agora?

# Em vez de fixar a classe Pai, melhor seria usar super() para definir
# que o método __init__ chamado é o da classe base.
class Filha(Pai):
    def __init__(self):
        super().__init__(self) # Chamando o construtor da classe pai direto
```

Outro exemplo de herança

Arquivo veiculo.py

```
class Veiculo:
    """
    Classe de veículos
    """
    def __init__(self, possui_motor, quantidade_rodas):
        self.possui_motor = possui_motor
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```

        self.quantidade_rodas = quantidade_rodas

    def ligar(self):
        if self.possui_motor:
            print("Ligou")
        else:
            print("Não tem motor.")

    def desligar(self):
        if self.possui_motor:
            print("Desligou")
        else:
            print("Não tem motor.")
    def andar(self):
        print("O veículo está andando.")
    def parar(self):
        print("O veículo foi parado.")

```

Arquivo carro.py

```

from veiculo import Veiculo

class Carro(Veiculo):
    """Classe carro herdando da classe Veiculo"""
    def __init__(self, quantidade_rodas):
        Veiculo.__init__(self, True, quantidade_rodas)

```

Arquivo bicicleta.py

```

from veiculo import Veiculo

class Bicicleta(Veiculo):
    def __init__(self, quantidade_rodas):
        Veiculo.__init__(self, False, quantidade_rodas)
        self.possui_guidao = True

    def empinar(self):
        print("A bicicleta empinou.")

```

Arquivo programa.py

```

from carro import Carro
from bicicleta import Bicicleta

bike = Bicicleta(2)
print(bike.possui_motor)
print(bike.possui_guidao)
print(bike.qtd_rodas)
bike.ligar()
bike.andar()
bike.empinar()
bike.parar()
bike.desligar()

carro = Carro(4)
print(carro.qtd_rodas)
carro.desligar()
carro.ligar()
carro.andar()
carro.parar()
carro.desligar()

```

Outro exemplo de herança

Arquivo pessoa.py

```

class Pessoa():
    def __init__(self, nome, idade):

```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
self.nome = nome
self.idade = idade
```

Arquivo pessoa_fisica.py

```
from pessoa import Pessoa
```

```
class Pessoa_Fisica(Pessoa):
    def __init__(self, cpf, nome, idade):
        super().__init__(nome, idade)
        self.cpf = cpf
```

Arquivo pessoa_juridica.py

```
from pessoa import Pessoa
```

```
class Pessoa_Juridica(Pessoa):
    def __init__(self, cnpj, nome, idade):
        super().__init__(nome, idade)
        self.cnpj = cnpj
```

Arquivo pessoa_programa.py

```
from pessoa_fisica import Pessoa_Fisica
from pessoa_juridica import Pessoa_Juridica

pf = Pessoa_Fisica("012.345.789-01", "Evaldo", 40)
pj = Pessoa_Juridica("01.000.000/0001-80", "Empresa teste", 5)

print(pf.nome)
print(pf.cpf)
print(pf.idade)
print(pj.nome)
print(pj.cnpj)
print(pj.idade)
```

Herança múltipla

Herança múltipla é quando uma classe deriva de duas ou mais classes existentes.

Cuidado ao usar herança múltipla porque sua utilização pode deixar o código confuso em alguns casos.

Primeiro exemplo

```
class Terrestre(object):
    anda_na_terra = True
    def __init__(self, velocidade):
        self.velocidade = velocidade

from terrestre import Terrestre

class Carro(Terrestre):
    rodas = 4
    def __init__(self, velocidade_em_terra, qtd_portas):
        self.qtd_portas = qtd_portas
        super().__init__(velocidade_em_terra)

class Aquatico(object):
    anda_na_agua = True
```



```

def __init__(self, velocidade):
    self.velocidade = velocidade

from aquatico import Aquatico

class Barco(Aquatico):
    def __init__(self, velocidade_na_agua, helices):
        self.helices = helices
        super().__init__(velocidade_na_agua)

from carro import Carro
from barco import Barco

class Anfibio(Carro, Barco):
    def __init__(self, velocidade_em_terra, velocidade_na_agua,
                  qtd_portas, helices):
        self.velocidade_em_terra = velocidade_em_terra
        self.velocidade_na_agua = velocidade_na_agua
        self.qtd_portas = qtd_portas
        self.helices = helices

        Carro.__init__(self, velocidade_em_terra, qtd_portas)
        Barco.__init__(self, velocidade_na_agua, helices)

from anfibio import Anfibio

meu_anfibio = Anfibio(120, 25, 2, 4)
print(meu_anfibio.velocidade_na_agua)
print(meu_anfibio.velocidade_em_terra)
print(meu_anfibio.qtd_portas)
print(meu_anfibio.helices)
print(meu_anfibio.anda_na_agua)
print(meu_anfibio.anda_na_terra)
print(meu_anfibio.rodas)

```

Segundo exemplo

```

class Terrestre(object):
    anda_na_terra = True
    def __init__(self, velocidade):
        self.velocidade = velocidade

from terrestre import Terrestre

class Carro(Terrestre):
    rodas = 4
    def __init__(self, velocidade_em_terra, qtd_portas, **kwargs):
        self.qtd_portas = qtd_portas
        super().__init__(velocidade_em_terra)

class Aquatico(object):
    anda_na_agua = True
    def __init__(self, velocidade):
        self.velocidade = velocidade

from aquatico import Aquatico

class Barco(Aquatico):
    def __init__(self, velocidade_na_agua, qtd_helices):
        self.qtd_helices = qtd_helices
        super().__init__(velocidade_na_agua)

```



```

from carrol import Carrol
from barcol import Barcol

class Anfibio(Carrol, Barcol):
    def __init__(self, velocidade_em_terra, velocidade_na_agua,
                  qtd_portas, qtd_helices):
        self.velocidade_em_terra = velocidade_em_terra
        self.velocidade_na_agua = velocidade_na_agua
        self.qtd_portas = qtd_portas
        self.qtd_helices = qtd_helices

    super().__init__(velocidade_em_terra=velocidade_em_terra,
                    velocidade_na_agua=velocidade_na_agua,
                    qtd_portas=qtd_portas,
                    qtd_helices=qtd_helices)

from anfibio import Anfibio

meu_anfibio = Anfibio(120, 20, 4, 2)
print(meu_anfibio.velocidade_na_agua)
print(meu_anfibio.velocidade_em_terra)
print(meu_anfibio.qtd_portas)
print(meu_anfibio.qtd_helices)
print(meu_anfibio.anda_na_agua)
print(meu_anfibio.anda_na_terra)
print(meu_anfibio.rodas)

```

*args = lista de argumentos de comprimento variável SEM palavras-chave.

**kwargs = lista de argumentos de comprimento variável COM palavras-chave.

Polimorfismo

O polimorfismo é baseado nas palavras gregas Poli (muitas) e morphos (formas). É a capacidade de um objeto adaptar o código ao tipo de dados que está processando.

Objetos podem ser de muitos tipos ou formas. Por exemplo, botões possuem formas diferentes, existem botões redondos, quadrados, botões com imagem, mas todos compartilham a mesma lógica, executam uma ação chamada “clique”. A capacidade de acessarmos o método “clique” independente do tipo de botão é chamada de polimorfismo.



Figura 194 - Método clique

O polimorfismo pode ser de dois tipos:

- Um objeto oferece diferentes implementações do método de acordo com os parâmetros de entrada;
- A mesma interface pode ser usada por objetos de tipos diferentes.

Em Python, o polimorfismo é um recurso embutido na linguagem.

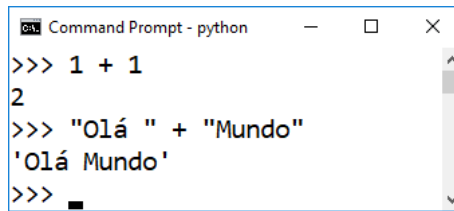
Por exemplo, o operador “+” pode ser usado para adicionar dois números mas também pode ser usado para concatenar duas strings.

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



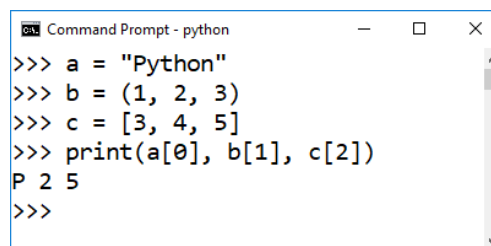
Devido ao polimorfismo o “+” possui uma função adicional para concatenar texto, além da função de somar dois números.



```
Command Prompt - python
>>> 1 + 1
2
>>> "Olá " + "Mundo"
'Olá Mundo'
>>>
```

Figura 195 - Exemplo polimorfismo

Strings, tuplas ou listas podem ser acessadas com um índice inteiro. Isto mostra como Python demonstra o polimorfismo em tipos embutidos.



```
Command Prompt - python
>>> a = "Python"
>>> b = (1, 2, 3)
>>> c = [3, 4, 5]
>>> print(a[0], b[1], c[2])
P 2 5
>>>
```

Figura 196 - Polimorfismo em tipos embutidos

A seguir vemos uma forma de utilizarmos polimorfismo com função.

Criamos duas classes: Cachorro e Gato, ambos podem fazer um som distinto. Em seguida, criamos duas instâncias e chamamos sua ação usando o mesmo método denominado “som”.

```
class Cachorro(object):
    def som(self):
        print("au au")

class Gato(object):
    def som(self):
        print("Miau")

def emitir_som(animal):
    animal.som()

cachorrinho = Cachorro()
gatinho = Gato()

emitir_som(cachorrinho)
emitir_som(gatinho)
```

A classe Cachorro possui um método chamado “som” que imprime o texto “Au au”

A classe Gato possui um método chamado “som” que imprime o texto “Miau”

O método emitir_som recebe um objeto chamado “animal” e executa o método “som” desse objeto.

Foram instanciados dois objetos, um da classe Cachorro e outro da classe Gato.

O método emitir_som foi executado duas vezes, na primeira passando um objeto do tipo Cachorro e na segunda, um objeto do tipo Gato.

O resultado foi:

Au au

Miau

Outro exemplo:

```
class Porta(object):
    def fechar(self):
        print("A porta foi fechada!")

    def abrir(self):
        print("A porta foi aberta!")
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
class Janela(object):
    def fechar(self):
        print("A janela foi fechada!")

    def abrir(self):
        print("A janela foi aberta!")

def realizar_abertura(o_que_abrir):
    o_que_abrir.abrir()

def realizar_fechamento(o_que_fechar):
    o_que_fechar.fechar()

porta = Porta()
janela = Janela()

realizar_abertura(porta)
realizar_abertura(janela)
realizar_fechamento(porta)
realizar_fechamento(janela)
```

O mais comum é utilizarmos polimorfismo com classes abstratas.

Uma classe abstrata é uma classe que não possui implementação, possuindo apenas uma estrutura com os métodos que devem ser implementados nas subclasses. Desta forma, os métodos devem ser obrigatoriamente implementados na subclasse.

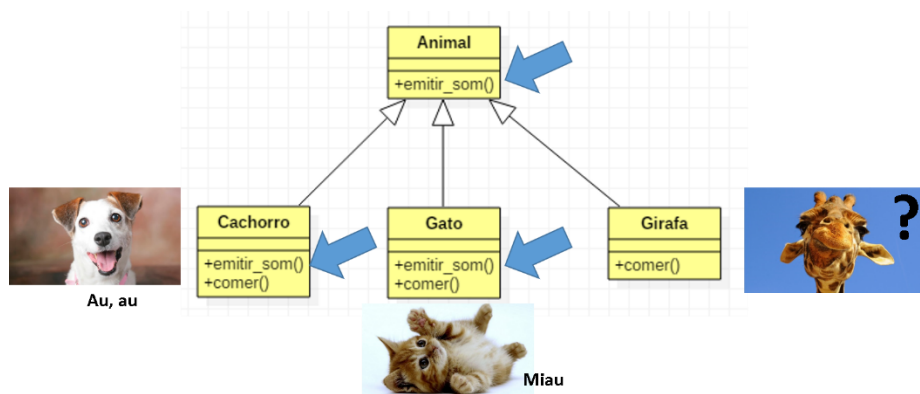


Figura 197 - Polimorfismo

```
class Animal(object):
    def emitir_som(self):
        raise NotImplementedError("Não foi implementado o método emitir_som.")

class Cachorro(Animal):
    def emitir_som(self):
        print("Au au")

class Gato(Animal):
    def emitir_som(self):
        print("Miau")

class Girafa(Animal):
    def comer(self):
        print("A girafa está comendo.")

def barulho(animal):
    animal.emitir_som()

cachorrinho = Cachorro()
gatinho = Gato()
```



```
girafinha = Girafa()

barulho(cachorrinho)
barulho(gatinho)
barulho(girafinha)
```

O **raise** é uma forma de forçar um tipo de exceção.

Neste caso se o método “emitir_som” não for implementado na classe derivada, irá emitir a mensagem “Não foi implementado o método emitir_som.” O método emitir_som não foi implementado na classe Animal, apenas criarmos o “esqueleto” e terá que ser implementado nas subclasses.

Resultado da execução do programa:

Au au

Miau

NotImplementedError: Não foi implementado o método emitir_som.

Quando eu implemento um método na classe base (não será mais uma classe abstrata) e o mesmo não é implementado na subclasse, será executado o método da classe base. Veja:

```
class Animal(object):
    def emitir_som(self):
        print("Um som qualquer")

class Girafa(Animal):
    def comer(self):
        print("A girafa está comendo.")

def barulho(animal):
    animal.emitir_som()

girafinha = Girafa()
barulho(girafinha)
```

Será impresso: “Um som qualquer”.

```
class Animal(object):
    def emitir_som(self):
        pass
```

Se eu fizer assim, não será obrigatório implementar na subclasse, da mesma forma. Porque foi implementado e não dará erro, porém, não irá fazer nada.

Quando o método é implementado na classe base e também implementado na subclasse, temos algo similar a sobrescrita de método que ocorre no Java (ou outras linguagens). O método executado é o da subclasse.

```
class Animal(object):
    def emitir_som(self):
        print("Um som qualquer")

class Cachorro(Animal):
    def emitir_som(self):
        print("Au au")

def barulho(animal):
    animal.emitir_som()

cao = Cachorro()
barulho(cao)
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Será impresso: "Au au".

```
class Documento(object):
    def __init__(self, nome):
        self.nome = nome

    def abrir(self):
        raise NotImplementedError("O método abrir deve ser implementado na
subclasse.")

class Pdf(Documento):
    def abrir(self):
        return 'Foi aberto um arquivo PDF!'

class Doc(Documento):
    def abrir(self):
        return 'Foi aberto um arquivo DOC!'

class Xls(Documento):
    def abrir(self):
        return 'Foi aberto um arquivo DOC!'

documentos = [Pdf('Arquivo.pdf'),
               Pdf('Lista.pdf'),
               Doc('Documento.doc'),
               Xls('Planilha.xls')]

for documento in documentos:
    print(documento.abrir(), "de nome", documento.nome)
```

Resultado da execução:

```
C:\Evaldo\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 14 -
a06_OrientacaoAObjetosIntroducao\codigo>python polimorfismo6.py
Foi aberto um arquivo PDF! de nome Arquivo.pdf
Foi aberto um arquivo PDF! de nome Lista.pdf
Foi aberto um arquivo DOC! de nome Documento.doc
Foi aberto um arquivo DOC! de nome Planilha.xls
```

Agora modifique o método Xls como abaixo e execute novamente:

```
class Xls(Documento):
    pass
```

Resultado da execução:

```
a06_OrientacaoAObjetosIntroducao\codigo>python polimorfismo6.py
Foi aberto um arquivo PDF! de nome Arquivo.pdf
Foi aberto um arquivo PDF! de nome Lista.pdf
Foi aberto um arquivo DOC! de nome Documento.doc
Traceback (most recent call last):
  File "polimorfismo6.py", line 26, in <module>
    print(documento.abrir(), "de nome", documento.nome)
  File "polimorfismo6.py", line 6, in abrir
    raise NotImplementedError("O método abrir deve ser implementado na subclasse.")
NotImplementedError: O método abrir deve ser implementado na subclasse.
```

Classes abertas



No Python podemos alterar nossas classes e objetos em tempo de execução adicionando novos métodos e atributos.

Isso é possível porque o Python é uma linguagem dinâmica.

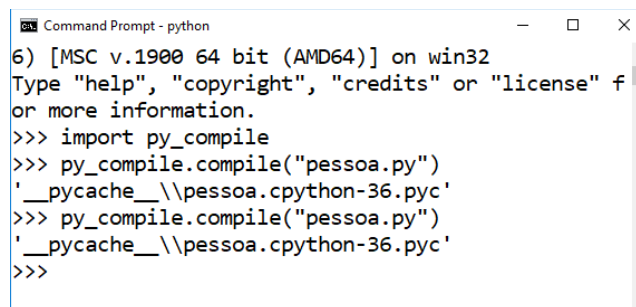
Somente não podemos modificar classes **built-ins** (ou seja, as classes embutidas do Python).

Podemos adicionar novos atributos e métodos de forma dinâmica quando não quisermos modificar uma classe existente ou quando não possuímos o código-fonte da classe, tendo apenas seu **bytecode**, por exemplo.

Vamos criar uma classe e gerar o **bytecode** para simular um caso real.

Primeiro vamos criar uma classe e compilar seu código.

```
class Pessoa(object):  
    def __init__(self, nome):  
        self.nome = nome
```



```
Command Prompt - python  
6) [MSC v.1900 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license" f  
or more information.  
>>> import py_compile  
>>> py_compile.compile("pessoa.py")  
'__pycache__\\pessoa.cpython-36.pyc'  
>>> py_compile.compile("pessoa.py")  
'__pycache__\\pessoa.cpython-36.pyc'  
>>>
```

Figura 198 - Importar py_compile

Temos que importar o módulo `py_compile` para gerar o bytecode, arquivo `.pyc`.

O comando para gerar o bytecode é:

```
py_compile.compile("arquivo.py")
```

Será gerado o arquivo `pessoa.cpython-36.pyc` na pasta `__pycache__`

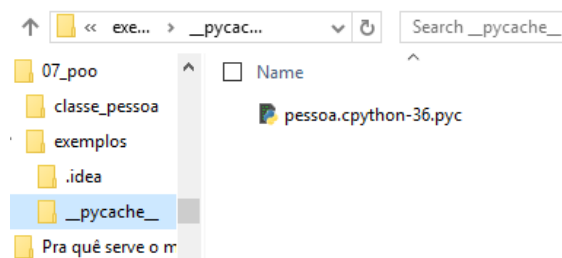


Figura 199 - Arquivo pyc

Vou excluir o `pessoa.py`, copiar o **`pessoa.cpython-36.pyc`** para a pasta do meu projeto e renomear para **`pessoa.pyc`**.

Em seguida vou apagar a pasta `__pycache__`.

A pasta do projeto ficará assim. O arquivo `exemplo1.py` será nosso programa que usará a classe **`Pessoa`** de **`pessoa.pyc`**.

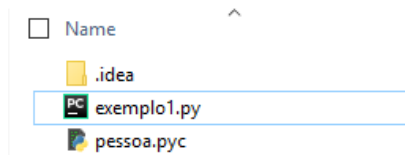


Figura 200 - Arquivo pyc

Se tentar editar o `pessoa.pyc` verá que ele é um binário (compilado). Veja o mesmo aberto nos editores Sublime e PyCharm.

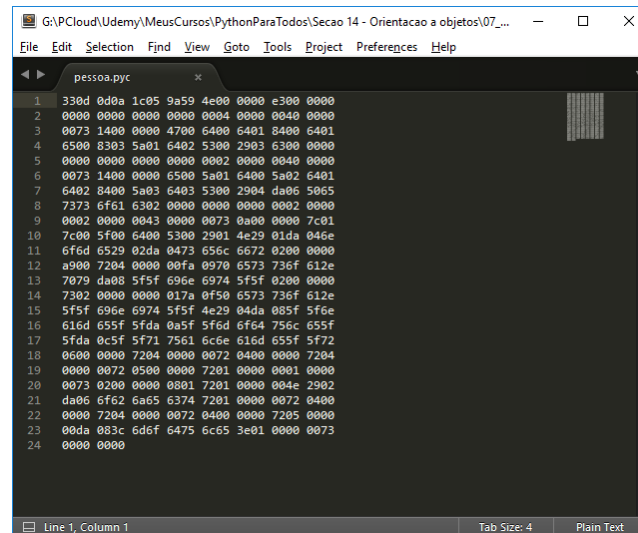


Figura 201 - Arquivo pyc aberto no Sublime

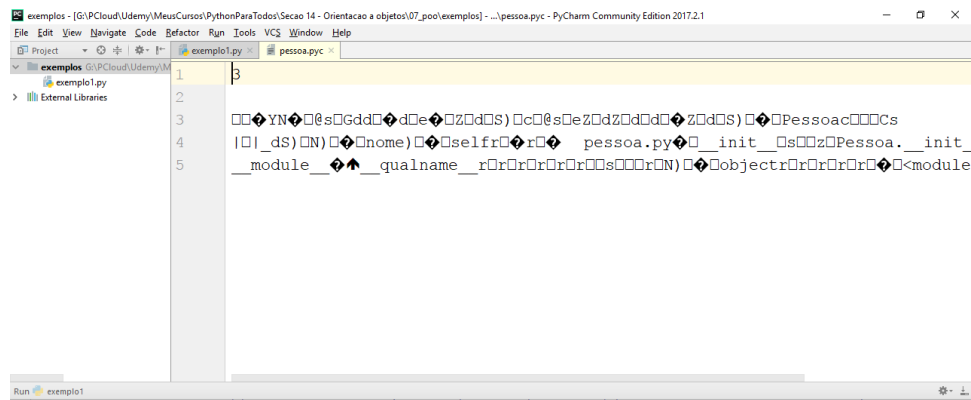


Figura 202 - Arquivo pyc aberto no PyCharm

Vamos ao programa que vai usar a classe Pessoa.

```
from pessoa import Pessoa
```

```
# Criado objeto p da classe pessoa passando 'José' para nome
p = Pessoa('José')
print(p.nome) # imprimindo o valor do atributo nome do objeto p
```

```
# Criando um método chamado retornar_nome que retorna a propriedade nome (de self)
def pegar_nome(self):
    return self.nome
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>
contato@pythonparatodos.com.br



```
# Criando o método pegar_nome na classe Pessoa com base no método pegar_nome que
criei acima
Pessoa.pegar_nome = pegar_nome

# Executando o método pegar_nome do objeto p
print(p.pegar_nome())

# Criando o método mudar_nome
def mudar_nome(self, novo_nome):
    self.nome = novo_nome

# Criando o método mudar_nome na classe Pessoa
Pessoa.mudar_nome = mudar_nome

# Executando o método mudar_nome no objeto p
p.mudar_nome('Maria')

# Criando um método denominado imprimir_ola para imprimir "Olá x", onde x é o valor
da propriedade nome
def imprimir_ola(self):
    print("Olá", self.nome)

# Criando o método ola na classe Pessoa com base no método imprimir_ola
Pessoa.ola = imprimir_ola

# Executando o método ola do objeto p
p.ola()
```

Encapsulamento, atributos públicos e privados

Encapsulamento é uma forma de fazer com que os objetos mantenham suas informações de estado privadas, tornando seu comportamento oculto para o mundo externo.

Com encapsulamento não é possível alterar o estado interno dos objetos atuando diretamente neles. Para manipular as informações dos objetos, enviamos “mensagens” aos mesmos, utilizando funções como *get* e *set*.

Nas linguagens Java e C++, por exemplo, é possível definir propriedades como públicas ou privadas. As propriedades privadas não podem ser acessadas diretamente, necessitando da implementação dos métodos *get* e *set*.

No Python o conceito de encapsulamento (ocultar dados e métodos) não é implícito, pois não existem palavras reservadas como no Java ou C++ (*public*, *private* e *protected*).

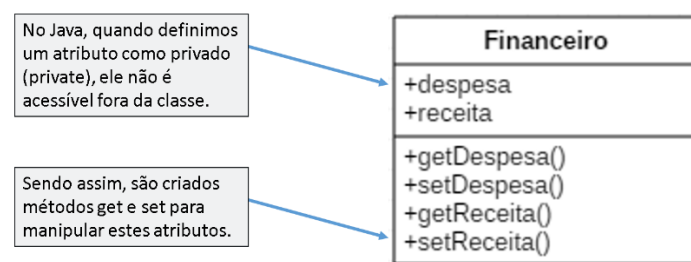


Figura 203 – Encapsulamento no Java



```

1 package teste_encapsulamento;
2
3 public class ContaCorrente {
4     private Double despesa;
5     private Double receita;
6     public Double variavel_publica;
7
8     public Double getDespesa() {
9         return despesa;
10    }
11
12    public void setDespesa(Double despesa) {
13        this.despesa = despesa;
14    }
15
16    public Double getReceita() {
17        return receita;
18    }
19
20    public void setReceita(Double receita) {
21        this.receita = receita;
22    }
23
24 }

```

Figura 204 - Exemplo Java

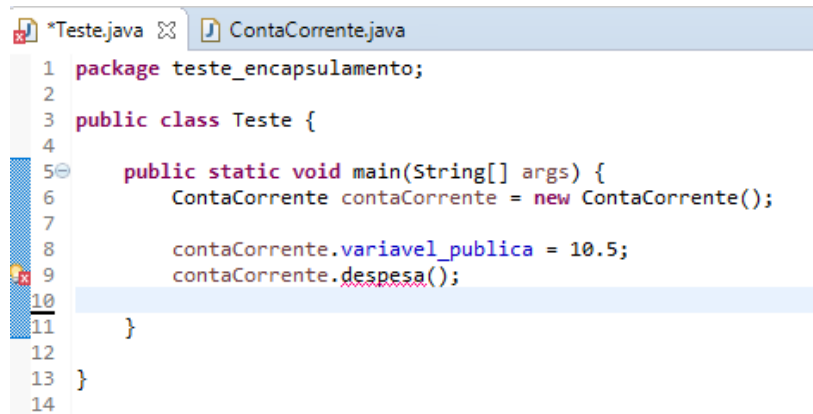
```

1 package teste_encapsulamento;
2
3 public class Teste {
4
5     public static void main(String[] args) {
6         ContaCorrente contaCorrente = new ContaCorrente();
7
8         contaCorrente.
9
10    }
11
12 }
13
14

```

variavel_publica : Double - ContaCorrente
 equals(Object obj) : boolean - Object
 getClass() : Class<?> - Object
 getDespesa() : Double - ContaCorrente
 getReceita() : Double - ContaCorrente
 hashCode() : int - Object
 notify() : void - Object
 notifyAll() : void - Object
 setDespesa(Double despesa) : void - ContaCorrente
 setReceita(Double receita) : void - ContaCorrente
 toString() : String - Object
 wait() : void - Object
 Press 'Ctrl+Space' to show Template Proposals

Figura 205 - Exemplo Java



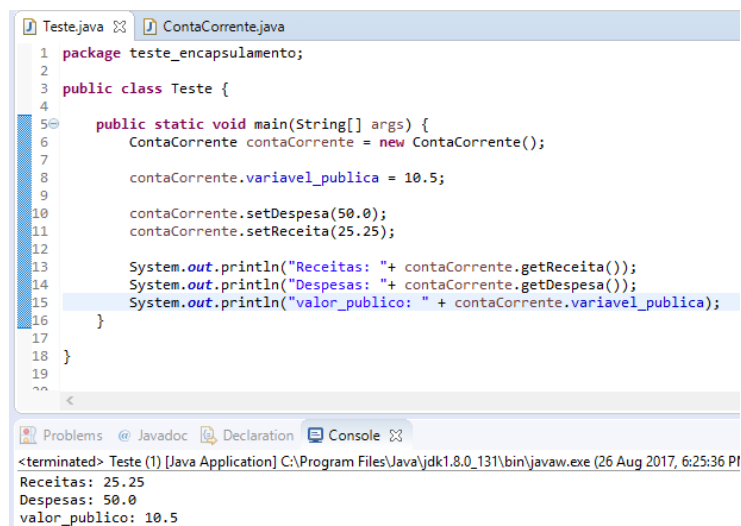
```

1 package teste_encapsulamento;
2
3 public class Teste {
4
5     public static void main(String[] args) {
6         ContaCorrente contaCorrente = new ContaCorrente();
7
8         contaCorrente.variavel_publica = 10.5;
9         contaCorrente.despesa();
10
11     }
12
13 }
14

```

Figura 206 - Exemplo Java

The method despesa() is undefined for the type ContaCorrente



```

1 package teste_encapsulamento;
2
3 public class Teste {
4
5     public static void main(String[] args) {
6         ContaCorrente contaCorrente = new ContaCorrente();
7
8         contaCorrente.variavel_publica = 10.5;
9
10        contaCorrente.setDespesa(50.0);
11        contaCorrente.setReceita(25.25);
12
13        System.out.println("Receitas: " + contaCorrente.getReceita());
14        System.out.println("Despesas: " + contaCorrente.getDespesa());
15        System.out.println("valor_publico: " + contaCorrente.variavel_publica);
16    }
17
18 }
19
20

```

Problems Javadoc Declaration Console

<terminated> Teste (1) [Java Application] C:\Program Files\Java\jdk1.8.0_131\bin\javaw.exe (26 Aug 2017, 6:25:36 PM)

Receitas: 25.25
Despesas: 50.0
valor_publico: 10.5

Figura 207 - Exemplo Java

Variáveis de instância privadas, que são acessadas somente do interior do objeto não existem no Python.

Porém, existe uma convenção seguida pelos programadores Python: Um nome prefixado com um underscore (por exemplo `_carro`) deve ser tratado como não público. Seja uma função, método ou membro de dados.

```

class Carro(object):

    def __init__(self, marca, modelo):
        self._marca = marca
        self._modelo = modelo

    def get_modelo(self):
        return self._modelo

    def set_modelo(self, modelo):
        self._modelo = modelo

carro = Carro("Ford", "Ranger")
print(carro._marca)

```



```
# Não é correto acessar os atributos que iniciam com um
# underscore diretamente, o correto é usar os métodos get/set
carro._marca = "Fiat"
print(Carro._marca)
carro.set_modelo("Uno")
print(carro.get_modelo())
print(carro.get_modelo())
```

Apesar de ser possível acessar o “_marca” diretamente, seu nome indica que é um método “não público”, sendo assim, é melhor não usá-lo diretamente.

Para criar um atributo “privado” no Python, temos que nomear o atributo iniciando com dois underscores. Exemplo:

Quando definimos com dois underscores, o atributo não é acessível externamente pelo seu nome “__despesa”.

O Python substitui seu nome por _nomedaclasse_variável: _Financeiro_despesa.

Na verdade, os atributos não se tornam realmente privados, porque mesmo assim, ainda é possível acessá-los externamente, mesmo que com outro nome.

Financeiro
+ __despesa + __receita
+get_despesa() +set_despesa() +get_receita() +set_receita()

Figura 208 - Classe Financeiro

Criando um atributo com dois sublinhados:

```
class Carro(object):
    marca = "Ford"
    __modelo = "Focus"
```

```
carro = Carro()
print(dir(carro))
```

Resultado:

```
['_Carro__modelo', '__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', 'marca']
```

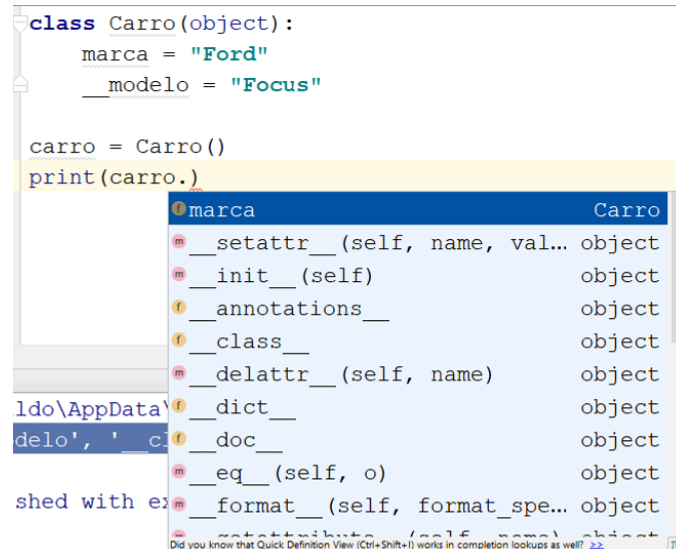


Figura 209 - Autocomplete não sugere atributos não públicos (__modelo)

```
class Carro(object):
    marca = "Ford"
    __modelo = "Focus"

    def get_modelo(self):
        return self.__modelo

    def set_modelo(self, modelo):
        self.__modelo = modelo

carro = Carro()
print(carro.marca)
carro.marca = "Fiat"
print(carro.marca)
carro.set_modelo("Uno")
print(carro.get_modelo())
carro.__Carro__modelo = "Palio"
print(carro.get_modelo())

print(carro.__modelo)
AttributeError: 'Carro' object has no attribute '__modelo'
print(carro.modelo)
AttributeError: 'Carro' object has no attribute 'modelo'
```

```
class Carro(object):
    def __init__(self, fabricante):
        self.__fabricante = fabricante

    def set_fabricante(self, fabricante):
        self.__fabricante = fabricante

    def get_fabricante(self):
        return self.__fabricante

carro = Carro("Toyota")
# Lista os atributos e métodos da classe
print(dir(carro))

# Forma correta de acessar o atributo privado __fabricante
print(carro.get_fabricante())

carro.set_fabricante("Fiat")
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
print(carro.get_fabricante())

# Forma incorreta de acessar o atributo privado __fabricante
print(carro.__Carro__fabricante)
```

Resultado:

```
['_Carro__fabricante', '__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__getattr__', '__gt__', '__hash__', '__init__', '__init_subclass__',
 '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',
 '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', 'get_fabricante',
 'set_fabricante']
```

Toyota

Fiat

Fiat

Campos “privados” não podem ser acessados por uma subclasse.

```
class Mae(object):
    def __init__(self):
        self.__atributo_privado = 10

class Filha(Mae):
    def pegar_atributo_privado(self):
        return self.__atributo_privado # Atributo não existe na classe Filha

filha = Filha()
print(filha.__dict__)
x = filha.pegar_atributo_privado()
print(x)
```

Resultado da execução:

```
{'_Mae__atributo_privado': 10}
```

```
return self.__atributo_privado
```

```
AttributeError: 'Filha' object has no attribute '_Filha__atributo_privado'
```

```
class Mae(object):
    def __init__(self):
        self.__atributo_privado = 10

class Filha(Mae):
    def pegar_atributo_privado(self):
        # return self.__atributo_privado
        return self._Mae__atributo_privado

filha = Filha()
print(filha.__dict__)
x = filha.pegar_atributo_privado()
print(x)
```

Resultado da execução:

```
{'_Mae__atributo_privado': 10}
```

```
10
```

“Por que então a sintaxe para atributos privativos não assegura as restrições de visibilidade como deveria? A resposta mais simples é um dos lemas mais citados do Python: ‘Somos todos adultos aqui e consentimos em dar liberdade uns aos outros’. Os programadores de Python acreditam que os benefícios da liberdade são maiores que as desvantagens de ter a cabeça fechada.”

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



SLATKIN, B. *Python Eficaz*. São Paulo: Novatec, 2016. 121 p.

```
if __name__ == '__main__':
```

Um módulo pode descobrir se ele está ou não sendo executado no escopo principal, verificando seu próprio `__name__`, que é um método especial do Python que retorna o nome do escopo em que o código está sendo executado.

A validação “`If __name__ == '__main__':`” é utilizada quando queremos que parte do código seja executada apenas se o mesmo for executado à partir do escopo principal, não à partir de outro módulo que tenha importado a classe.

arq1.py

```
class Arquivo1(object):
    def __init__(self):
        print("__name__:", __name__)

    def somar(self, x, y):
        return x + y

a = Arquivo1()
print("Soma em arq1:", a.somar(10, 25))
```

Resultado da execução:

```
__name__: __main__
Soma em arq1: 35
```

Ao executar o `arq1.py`, quando o objeto “a” foi criado, foi impresso o código do `init` e, em seguida foi feita a soma e impresso o resultado.

arq2.py

```
from arq1 import Arquivo1

a = Arquivo1()
print("Soma em arq2", a.somar(10, 10))
```

Resultado da execução:

```
__name__: arq1
Soma em arq1: 35
__name__: arq1
Soma em arq2 20
```

Ao executar o `arq2.py`, ao criar o objeto “a” o código de `arq1` é executado, sendo assim, temos na ordem de execução:

- 1 – Execução do `init` do objeto “a” de `arq1`.
- 2 – Execução do `print` da soma de `arq1`.
- 3 – Execução do `init` do objeto “a” de `arq2`.
- 4 – Execução do `print` da soma de `arq2`.

arq1.py (modificado)

```
class Arquivo1(object):
    def __init__(self):
        print("__name__:", __name__)

    def somar(self, x, y):
        https://www.udemy.com/python-3-para-todos - https://www.pythonparatodos.com.br
        contato@pythonparatodos.com.br
```



```

        return x + y

if __name__ == '__main__':
    a = Arquivo1()
    print("Soma em arq1:", a.somar(10 , 25))

```

Resultado da execução:

```

__name__: __main__
Soma em arq1: 35

```

Ao executar o arq1.py, quando o objeto “a” foi criado, foi impresso o código do init e, em seguida foi feita a soma e impresso o resultado. Executado igual anteriormente sem a validação do __main__.

Resultado da execução do arq2.py após a alteração no arq1.py:

```

__name__: arq1
Soma em arq2 20

```

O código que está dentro da validação de __name__ não é executado quando importamos a classe Arquivo1 do módulo arq1.

Propriedades

Python oferece um mecanismo embutido (***builtin***) para construção de propriedades de uma classe. Propriedades são como atributos, ou seja, são elementos acessados externamente, mas, internamente, são manipulados por funções. São calculados em tempo de execução. As propriedades são criadas pelo decorador *property*.

Usando propriedades podemos validar a entrada do atributo, criar atributos somente para leitura, simplificar o uso de classe disfarçando as funções ***get()*** e ***set()*** dos atributos e podemos mudar de um atributo convencional para uma propriedade sem a necessidade de alterar as aplicações que utilizam a classe.

Propriedades são interessantes quando desenvolvemos bibliotecas para serem usadas por outras pessoas.

Veja agora um exemplo em que não usamos propriedades.

```

class Arrancada(object):
    def __init__(self, metros, segundos):
        self._metros_percorridos = metros
        self._tempo_gasto = segundos
        self._velocidade = self._metros_percorridos / self._tempo_gasto

    def get_velocidade(self):
        return f"A média de velocidade foi: {self._velocidade:.2f} metros por segundo."

a = Arrancada(10, 30)
print(a.get_velocidade())

```

Resultado: A média de velocidade foi: 0.33 metros por segundo.

Veja agora o mesmo exemplo usando propriedade por meio do decorador @property.

```

class Arrancada(object):
    def __init__(self, metros, segundos):
        self._metros_percorridos = metros
        self._tempo_gasto = segundos

```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>
contato@pythonparatodos.com.br



```

        self._velocidade = self._metros_percorridos / self._tempo_gasto

    @property
    def velocidade(self):
        return f"A média de velocidade foi: {self._velocidade:.2f} metros por segundo."

a = Arrancada(10, 30)
print(a.velocidade)

```

Resultado: A média de velocidade foi: 0.33 metros por segundo.

Veja um exemplo de propriedade por meio de chamada de função.

```

class Arrancada(object):
    def __init__(self, metros, segundos):
        self._metros_percorridos = metros
        self._tempo_gasto = segundos
        self._velocidade = self._metros_percorridos / self._tempo_gasto

    def get_velocidade(self):
        print("Executou get_velocidade.")
        return f"A média de velocidade foi: {self._velocidade:.2f} metros por segundo."

    def set_velocidade(self, velocidade):
        print("Executou set_velocidade.")
        self._velocidade = velocidade

    def del_velocidade(self):
        print("Executou del_velocidade.")
        del self._velocidade

    velocidade = property(get_velocidade,
                          set_velocidade, del_velocidade,
                          "Propriedade velocidade do veículo.")

a = Arrancada(10, 30)
print(a.velocidade)
a.velocidade = 40
print(a.velocidade)
del a.velocidade

```

#Assinatura da função que cria a propriedade é:
#class property(fget=None, fset=None, fdel=None, doc=None)
#fget -> Função para obter um valor de atributo
#fset -> Função para definir um valor de atributo
#fdel -> Função para excluir um valor de atributo
#doc -> Cria uma docstring para o atributo

```

class Arrancada(object):
    def __init__(self, metros, segundos):
        self._metros_percorridos = metros
        self._tempo_gasto = segundos
        self._velocidade = self._metros_percorridos / self._tempo_gasto

    @property
    def velocidade(self):
        print("Retornando o valor de velocidade.")
        return f"A média de velocidade foi: {self._velocidade:.2f} metros por segundo."

    @velocidade.setter
    def velocidade(self, velocidade):
        print("Atribuindo valor à velocidade.")
        self._velocidade = velocidade

```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
@velocidade.deleter
def velocidade(self):
    print("Excluindo velocidade.")
    del self._velocidade

a = Arrancada(10, 30)
print(a.velocidade)

a.velocidade = 40
print(a.velocidade)

del a.velocidade
```

Retornando o valor de velocidade.

A média de velocidade foi: 0.33 metros por segundo.

Atribuindo valor à velocidade.

Retornando o valor de velocidade.

A média de velocidade foi: 40.00 metros por segundo.

Excluindo velocidade.

Descritores

Descritor (*descriptor*) é uma forma mais geral de definir como os atributos são calculados por meio de um protocolo. Com isso é possível obter um código mais fácil de reaproveitar do que com propriedades (*properties*) e os mesmos podem ser compartilhados entre as classes.

Um descritor permite que você personalize o que deve ser feito quando você se refere a um atributo em um objeto.

Descritores fornecem ao desenvolvedor a capacidade de adicionar atributos gerenciados aos objetos. Atributos gerenciados são usados para proteger um atributo contra alterações ou para atualizar automaticamente os valores de um atributo dependente.

Necessitamos de descritores quando queremos, por exemplo, validar a informação antes de designar a mesma ao atributo. Imagine que queira obrigar a informação de um número inteiro para idade, texto para nome e avaliar o formato de uma *string* de e-mail.

O protocolo para criar um descritor é realmente muito simples. Você somente precisa definir um ou mais dos métodos a seguir: `__get__`, `__set__` e `__delete__`.

Onde:

`__get__(self, obj, type = None)` => Retorna o valor do atributo.

`__set__(self, obj, value)` => Informar um valor ao atributo (sem retorno).

`__delete__(self, obj)` => Controla operação de exclusão (sem retorno).

Ao definir pelo menos um destes métodos, você está criando um descritor.

Um descritor que implementa `__get__` () e `__set__` () é chamado de descritor de dados.

Se ele apenas implementar `__get__` (), então ele é chamado de um descritor “sem dados” (*non-data descriptor*).

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Em que momento o get, set e delete é chamado?

get -> print(usuario.nome)

set -> usuario.nome = "Fulano de tal"

delete -> del(usuario.nome)

Veja um exemplo da documentação oficial do Python:

```
class RevelarAcesso(object):
    """Um descritor de dados que define e retorna valores
    normalmente e imprime uma mensagem registrando seu acesso.
    """
    def __init__(self, initval=None, name='var'):
        self.val = initval
        self.name = name

    def __get__(self, obj, objtype):
        print('Recuperando', self.name)
        return self.val

    def __set__(self, obj, val):
        print('Atualizando', self.name)
        self.val = val

class MinhaClasse(object):
    x = RevelarAcesso(10, 'var "x"')
    y = 5

m = MinhaClasse()

print("x: ", m.x)
m.x = 20
print("x: ", m.x)

print("y: ", m.y)
m.y = 8
print("y: ", m.y)
```

Resultado:

```
Recuperando var "x"
x: 10
Atualizando var "x"
Recuperando var "x"
x: 20
y: 5
y: 8
```



O exemplo anterior mostra claramente que, se uma classe tiver o descritor de dados para o atributo dado, o método `__get__()` do descritor é chamado para retornar o valor sempre que o atributo da instância é recuperado e `__set__()` é chamado sempre que um valor é atribuído a esse atributo.

No caso do método `__del__`, ele é chamado sempre que um atributo de instância é excluído com a declaração `del instance.atributo` ou a chamada `delattr(instância, 'atributo')`.

Veja outro exemplo:

```
class MeuDescritor(object):
    """
    Um exemplo simples de descritor
    """
    def __init__(self, valor_inicial=None, nome='my_var'):
        self.valor = valor_inicial
        self.nome = nome

    def __get__(self, instance, owner):
        print("Obtendo: ", self.nome)
        return self.valor

    def __set__(self, instance, valor):
        print(f"Atribuindo {valor} a {self.nome}");
        self.valor = valor

class MinhaClasse(object):
    descritor = MeuDescritor(valor_inicial='10', nome='dinheiro')
    normal = 20

classe = MinhaClasse()
print(classe.descritor) # Executa o get para imprimir o valor, então imprime a
                        # linha do "Obtendo"
print(classe.normal) # Não é um descritor
classe.descritor = 200 # Executa o __set__
print(classe.descritor) # Imprime agora 200
```

Resultado:

```
Obtendo: dinheiro
10
20
Atribuindo 200 a dinheiro
Obtendo: dinheiro
200
```

Observe o código a seguir. Veja como estou tratando para não permitir veículos com valor negativo.

```
class Carro(object):
    def __init__(self, marca, modelo, valor):
        self.marca = marca
        self.modelo = modelo
        if valor < 0:
            raise ValueError("O valor do carro não pode ser negativo.")
        else:
            self.valor = valor
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
def __str__(self):
    return f"Marca: {self.marca}, Modelo: {self.modelo}, Valor: R$ {self.valor:.2f}"

fusquinha = Carro("VW", "Fusca", 8500)
print(fusquinha)
fusquinha = Carro("VW", "Fusca", -1)
print(fusquinha)
```

Podemos resolver criando um descritor para validar valores negativos que pode ser reaproveitado.

```
class DescritorValor(object):
    def __init__(self):
        self.valor = 0

    def __get__(self, instance, owner):
        return self.valor

    def __set__(self, instance, value):
        if value < 0:
            raise ValueError("O valor do carro não pode ser negativo.")
        else:
            self.valor = value

    def __delete__(self, instance):
        del self.valor

class Carro(object):
    valor = DescritorValor()
    def __init__(self, marca, modelo, valor):
        self.marca = marca
        self.modelo = modelo
        self.valor = valor

    def __str__(self):
        return f"Marca: {self.marca}, Modelo: {self.modelo}, " \
            f"Valor: R$ {self.valor:.2f}"

fusquinha = Carro("VW", "Fusca", 8500)
print(fusquinha)

fusquinha = Carro("VW", "Fusca", -1)
```

Resultado:

Traceback (most recent call last):

Marca: VW, Modelo: Fusca, Valor: R\$ 8500.00

File ".../carro_descritores.py", line 31, in <module>

fusquinha = Carro("VW", "Fusca", -1)

File ".../carro_descritores.py", line 22, in __init__

self.valor = valor

File ".../carro_descritores.py", line 10, in __set__

raise ValueError("O valor do carro não pode ser negativo.")

ValueError: O valor do carro não pode ser negativo.

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Neste exemplo ainda temos um problema devido ao valor ser uma propriedade da classe Carro. Se executarmos assim:

```
fusquinha = Carro("VW", "Fusca", 8500)
print(fusquinha)

gol = Carro("VW", "Gol", 25000)
print(gol)
print(fusquinha)
```

Teremos este resultado:

```
Marca: VW, Modelo: Fusca, Valor: R$ 8500.00

Marca: VW, Modelo: Gol, Valor: R$ 25000.00

Marca: VW, Modelo: Fusca, Valor: R$ 25000.00 # O valor do Fusca foi alterado quando foi atribuído 2500 ao Gol.
```

Podemos resolver utilizando um dicionário na propriedade “valor” e passar a instância do objeto no *get*, *set* e *delete*.

```
class DescritorValor(object):
    def __init__(self):
        self.valor = {}

    def __get__(self, instance, owner):
        return self.valor[instance]

    def __set__(self, instance, value):
        if value < 0:
            raise ValueError("O valor do carro não "
                              "pode ser negativo.")
        else:
            self.valor[instance] = value

    def __delete__(self, instance):
        del self.valor[instance]

class Carro(object):
    valor = DescritorValor()
    def __init__(self, marca, modelo, valor):
        self.marca = marca
        self.modelo = modelo
        self.valor = valor

    def __str__(self):
        return f"Marca: {self.marca}, Modelo: {self.modelo}, " \
               f"Valor: R$ {self.valor:.2f}"

fusquinha = Carro("VW", "Fusca", 8500)
gol = Carro("VW", "GOL", 20000)
print(gol)
print(fusquinha)
fusquinha2 = Carro("VW", "Fusca 2", 700)
print(fusquinha2)
print(gol)
print(fusquinha)
```

Resultado:

```
Marca: VW, Modelo: GOL, Valor: R$ 20000.00
```



```

Marca: VW, Modelo: Fusca, Valor: R$ 8500.00
Marca: VW, Modelo: Fusca 2, Valor: R$ 700.00
Marca: VW, Modelo: GOL, Valor: R$ 20000.00
Marca: VW, Modelo: Fusca, Valor: R$ 8500.00

```

Sobrecarga de operadores

Sobrecarga de operador é a capacidade de estender a funcionalidade dos operadores nativos (+, -, *, >, <, etc.) de forma que os mesmos funcionem também com tipos definidos pelo usuário.

O *Python* permite que as classes definidas pelo usuário implementem métodos para os operadores definidos na linguagem.

Por exemplo, para personalizar o operador de soma "+", podemos implementar um método chamado `__add__`.

Operador	Método	Operação	Operador	Método	Operação
+	<code>__add__</code>	Adição	<=	<code>__le__</code>	Menor ou igual
-	<code>__sub__</code>	Subtração	>=	<code>__ge__</code>	Maior ou igual
*	<code>__mul__</code>	Multiplicação	==	<code>__eq__</code>	Igual a
/	<code>__div__</code>	Divisão	!=	<code>__ne__</code>	Diferente de
//	<code>__floordiv__</code>	Divisão inteira	<<	<code>__lshift__</code>	Deslocamento para esquerda
%	<code>__mod__</code>	Módulo	>>	<code>__rshift__</code>	Deslocamento para direita
**	<code>__pow__</code>	Potência	&	<code>__and__</code>	E bit-a-bit
+	<code>__pos__</code>	Positivo		<code>__or__</code>	Ou bit-a-bit
-	<code>__neg__</code>	Negativo	^	<code>__xor__</code>	Ou exclusivo bit-a-bit
<	<code>__lt__</code>	Menor que	~	<code>__inv__</code>	Inversão
>	<code>__gt__</code>	Maior que			



Se você reescrever o método “__add__” por exemplo, em sua classe, ao executar uma operação de adição utilizando o operador “+” com um objeto desta classe, será executado o conteúdo do método “__add__” que você implementou. Vamos exemplificar isto de uma forma bem simples com dois exemplos.

Primeiro exemplo:

```
class MinhaString(str):
    def __sub__(self, other):
        print("self:", self)
        print("other:", other)
        subtracao = self.replace(other, '')
        return f"Resultado de '{self}' - '{other}': {subtracao}"

    def __add__(self, other):
        print("self:", self)
        print("other:", other)
        return f"{self} + {other}"

s1 = MinhaString("Olá mundo cruel")
s2 = "cruel"

s3 = s1 - s2
print(s3)
```

Resultado:

```
self: Olá mundo cruel
other: cruel
Resultado de 'Olá mundo cruel' - 'cruel': Olá mundo
```

Segundo exemplo:

```
class MinhaString(str):
    def __sub__(self, other):
        print("self:", self)
        print("other:", other)
        subtracao = self.replace(other, '')
        return f"Resultado de '{self}' - '{other}': {subtracao}"

    def __add__(self, other):
        print("self:", self)
        print("other:", other)
        return f"{self} + {other}"

s1 = MinhaString("Curso Python para")
s2 = "Todos"

s3 = s1 + s2
print(s3)
```

Resultado:

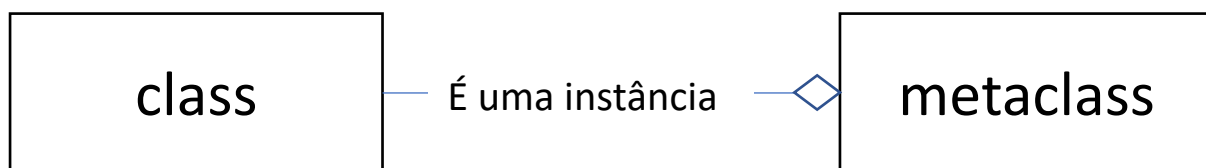
```
self: Curso Python para
other: Todos
Curso Python para + Todos
```



Metaclasses

Metaclasses é um recurso do Python que é considerado por muitos como uma das coisas mais difíceis da linguagem, sendo desta forma, evitada por um grande número de desenvolvedores. Na realidade, não é tão complicado quanto parece, uma vez que você compreenda alguns conceitos básicos. Metaclasses permite fazer coisas que não são possíveis usando outras abordagens.

Uma metaclasses é uma classe de outra classe, o que significa que a classe é uma instância de sua metaclasses.



Algumas definições

- Metaclasses é um tipo (classe) que define outros tipos (classes). A coisa mais importante a saber para entender como uma metaclasses funciona é que as classes que definem instâncias de objetos também são objetos. Então, se eles são objetos, eles possuem uma classe associada.
- Uma classe é um objeto, e assim como qualquer outro objeto, é uma instância de algo. Este algo é uma metaclasses.
- Metaclasses é uma classe que pode ser instanciada por outras classes.
- Uma metaclasses é uma fábrica de classes, sendo utilizada para criar instâncias de classes.
- A metaclasses define o comportamento das classes que derivam dela.
- Como uma classe comum define o comportamento de suas instâncias, uma metaclasses define o comportamento das classes e suas instâncias.

Em *Python*, a classe *type* é uma metaclasses e pode ser usada para criar novas metaclasses, ou seja, quando criamos uma classe sem definir sua metaclasses, será utilizada a metaclasses *type*.

Como em *Python* tudo é um objeto. Se dissermos que `a=5`, então `type(a)` devolve `<type 'int'>`, que significa que "a" é do tipo *int*. No entanto `type(int)` devolve `<type 'type'>`, que sugere a presença de uma metaclasses, pois *int* é uma classe do tipo *type*.

A definição de classe é determinada por sua metaclasses, portanto, quando criamos uma classe com **class A, Python** a cria com **A = type(name, bases, dict)**, onde:

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



name é nome da classe.

base é a classe-base.

dict é a variável de atributos.

Se definirmos uma metaclasses na criação da classe como **class A(metaclass=MinhaMetaClasse)**, Python criará a classe com **A = MinhaMetaClasse(name, bases, dict)**.

Todos objetos, por padrão, são do tipo *type*.

```
class MinhaClasse(object):  
    pass  
  
print(type(MinhaClasse))  
# type é a metaclasses da classe MinhaClasse  
  
# Todos objetos são do tipo type  
print(type(int))  
print(type(str))
```

Resultado:

<class 'type'>

<class 'type'>

<class 'type'>

A classe criada MinhaClasse é do tipo *type*.

int e *str* também são do tipo *type*.

Conferindo se MinhaClasse é uma instância de *type*.

```
class MinhaClasse(object):  
    pass  
  
print(isinstance(MinhaClasse, type))
```

Resultado: True

Diagrama de instâncias

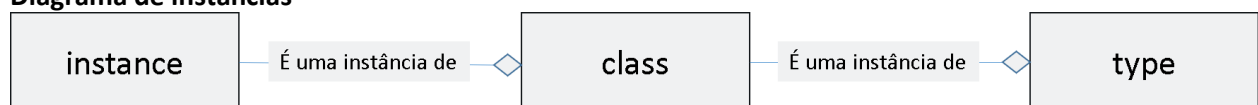


Figura 210 - Diagrama de instâncias



Em Python é possível substituir a metaclasses por um objeto de classe com nosso próprio tipo. Normalmente, a nova metaclasses ainda é uma subclasse da classe `type` porque não fazê-lo tornaria as classes resultantes altamente incompatíveis com outras classes em termos de herança.

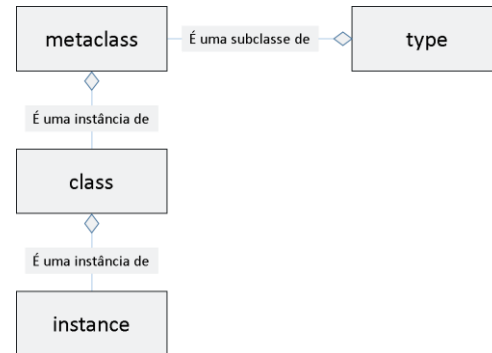


Figura 211 - Metaclasses

Utilizando `type` para criar uma classe

O Python também tem a capacidade de gerar uma classe “programaticamente” usando a função `type()`.

Sintaxe:

`type(name, bases, namespace) => type(nome, classes_base, atributos)`

`name` = Nome da classe que se tornará o atributo `__name__`

`bases` = Uma tupla que detalha a classe base, torna-se o atributo `__bases__`

`namespace` = Um dicionário que é o `namespace` contendo definições para o corpo da classe, torna-se o atributo `__dict__`

Então, a função `type()` serve para dois propósitos:

- 1) Determinar qual o tipo de um objeto.
- 2) Criar um novo tipo.

Utilizando `type` para criar uma classe. Exemplo1:

```
def meu_metodo(self):
    return 1

MinhaClasse = type('MinhaClasse', (object,), {'metodo': meu_metodo})

instancia = MinhaClasse()
x = instancia.metodo()
print(x)
```

O código anterior, equivale a:

```
class MinhaClasse:
    def metodo(self):
        return 1
```

```
instancia = MinhaClasse()
x = instancia.metodo()
print(x)
```

Utilizando `type` para criar uma classe. Exemplo2:

```
def inicializador_pessoa(self, nome, sobrenome):
    self.nome = nome
    self.sobrenome = sobrenome
```

```
Pessoa = type('Pessoa', (object,), {'__init__':
inicializador_pessoa, 'a': 'Teste', 'b': 10})
```

```
print("type (Pessoa) :", type(Pessoa))
```



```
print("vars (Pessoa) :", vars (Pessoa))

pes = Pessoa('Evaldo', 'Wolkers')

print("type (pes) :", type (pes))

print(pes.nome)
print(pes.sobrenome)
print(pes.a)
print(pes.b)
```

Criando uma metaclasses e usando como se fosse `type()`.

```
# Para criar nossa metaclasses, vamos declarar a classe herdando da classe type
class MinhaMetaClasse(type):
    pass

# A metaclasses (MinhaMetaClasse) pode ser usada como se fosse type
MinhaClasse = MinhaMetaClasse('MinhaClasse', (), {})

minhaInstancia = MinhaClasse()
print(type(minhaInstancia))
```

Resultado:

```
<class '__main__.MinhaClasse'>
```

Todas as classes criadas usam `type` como metaclasses. Podemos modificar este comportamento padrão informando a palavra-chave `metaclass` como argumento da declaração da classe.

```
# Para criar nossa metaclasses, vamos declarar a classe herdando da classe type
class MinhaMetaClasse(type):
    pass

# Definindo a metaclasses na declaração da classe
class MinhaClasse(metaclass=MinhaMetaClasse):
    pass

minhaInstancia = MinhaClasse()
print(type(minhaInstancia))
```

Agora, quando a classe `MinhaClasse` é construída, ela usará `MinhaMetaClasse` para construí-la. Isso não tem muita utilidade, a menos que a metaclasses estenda a classe `type` de alguma forma. O valor fornecido como argumento da metaclasses geralmente é outro objeto de classe, mas pode ser qualquer outro objeto chamado que aceita os mesmos argumentos que a classe `type` e espera retornar outro objeto de classe. Como já vimos, a assinatura de chamada de `type` é: `type(name, bases, namespace)`.

Estendendo a classe `type`

Se for declarado um método `__new__` em uma metaclasses, ele controla como a classe é construída.

A assinatura de `__new__` é:

`__new__(mcs, nome, bases, atributos)`, onde `mcs` é a instância da metaclasses e os três parâmetros restantes são os mesmos da função `type`.

Um uso para substituir o método `__new__` é gerar dinamicamente métodos para a classe.

Criando métodos dinamicamente substituindo o método `__new__`

```
def fabrica_de_funcoes(nome_metodo, retorno):
    def metodo_criado(self):
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```

        return retorno

    metodo_criado.__name__ = nome_metodo
    return metodo_criado

class MinhaMetaClasse(type):
    def __new__(mcs, nome, bases, atributos):
        for nome, retorno in (('metodo1', 'Olá'), ('metodo2', 'Como vai você?'),
                              ('metodo3', 30 + 40), ('metodo4', str(70 * 7))):
            atributos[nome] = fabrica_de_funcoes(nome, retorno)

        return super().__new__(mcs, nome, bases, atributos)

class MinhaClasse(metaclass=MinhaMetaClasse):
    pass

print("vars(MinhaClasse):", vars(MinhaClasse))
mc = MinhaClasse()
print(mc.metodo1())
print(mc.metodo2())
print(mc.metodo3())
print(mc.metodo4())

```

As metaclasses podem ser evitadas a favor de soluções mais simples como propriedades, descritores ou decoradores de classe.

Também é verdade que muitas vezes as metaclasses podem ser substituídas por outras abordagens mais simples, mas existem situações em que as coisas não podem ser feitas facilmente sem elas. Por exemplo, é difícil imaginar a implementação de ORM (Object Relational Mapping – Mapeamento objeto-relacional) do *Django* construída sem uso extensivo de metaclasses.

Exemplo de Uso com o padrão Singleton

O Singleton proporciona uma forma de ter um e somente um objeto de determinado tipo, além de disponibilizar um ponto de acesso global. Por isso, os Singletons são geralmente usados em casos como logging ou operações de banco de dados, spoolers de impressão e muitos outros cenários em que seja necessário que haja apenas uma instância disponível para toda a aplicação. Evitando assim, requisições conflitantes para o mesmo recurso.

Com o padrão Singleton, as intenções são:

- Garantir que um e somente um objeto da classe seja criado.
- Oferecer um ponto de acesso para um objeto que seja global no programa.
- Controlar o acesso concorrente a recursos compartilhados.

```

class MetaSingleton(type):
    _instancias = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instancias:
            cls._instancias[cls] = super(MetaSingleton, cls).__call__(*args,
**kwargs)
        return cls._instancias[cls]

class Logger(metaclass=MetaSingleton):
    pass

logger1 = Logger()
logger2 = Logger()
print(logger1)
print(logger2)

```

Método __call__

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



O método especial `__call__` de Python é chamado quando um objeto precisa ser criado para uma classe já existente. Nesse código, quando instanciamos a classe `Logger` com `Logger()`, o método `__call__` da metaclasses `MetaSingleton` é chamado, o que significa que a metaclasses agora controla a instânciação do objeto.

Resultado:

```
<__main__.Logger object at 0x000001B3A63A0C50>
```

```
<__main__.Logger object at 0x000001B3A63A0C50>
```

```
class PessoaMeta(type):
    # Inicializando a classe com o método __init__
    def __init__(cls, name, bases, dict):
        print("PessoaMeta: __init__")
        # Chamando o método init da classe base (type)
        super().__init__(name, bases, dict)
        # Criando uma variável de nível de classe que é um
        # dicionário que mapeia os parâmetros do construtor para uma
        # instância que foi construída com esses parâmetros
        cls._instance_map = {}

    # Interceptando a chamada para o construtor da classe
    # para verificar se um objeto já foi instanciado com
    # os valores passados para o construtor.
    # Se existir, ele retorna a instância já existente
    def __call__(cls, first_name, last_name):
        print("PessoaMeta: __call__")
        # Criando uma tupla com o conjunto (nome, sobrenome)
        key = (first_name, last_name)

        # Verificando se o conjunto (nome, sobrenome) não existe em _instance_map
        if key not in cls._instance_map:
            # Se não existir, será criada uma variável new_instance
            # que vai receber uma nova instância da classe Pessoa.
            # Chamando o método __call__ da classe base
            new_instance = super().__call__(first_name, last_name)
            # Adicionando a nova instância ao dicionário que criamos
            # para armazenar as instâncias com base na chave (nome, sobrenome)
            cls._instance_map[key] = new_instance
            # Devolve a instância armazenada conforme a chave (nome, sobrenome)
            return cls._instance_map[key]

class Pessoa(metaclass=PessoaMeta):
    def __init__(self, first_name, last_name):
        print("Pessoa: __init__")
        self.first_name = first_name
        self.last_name = last_name

pessoa1 = Pessoa("Evaldo", "Wolkers")
pessoa2 = Pessoa("Evaldo", "Wolkers")
pessoa3 = Pessoa("Seu", "Madruga")

print(pessoa1)
print(pessoa2)
print(pessoa3)
print(pessoa1 is pessoa2)
print((pessoa3 is pessoa1) or (pessoa3 is pessoa2))
```

Alguns exemplos de onde usar metaclasses:

- Registro de log e criação de perfil.
- Verificação de interface.
- Registro de classes no momento da criação.
- Adicionar novos métodos automaticamente.
- Criação de propriedades automáticas.

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



- Bloqueio e sincronização automática de recursos.

Classes-base abstratas

Python suporta classes-base abstratas (*Abstract Base Classes*), que são metaclasses que permitem forçar a implementação de determinados métodos e atributos das classes e subclasses derivadas. O módulo *abc* define a metaclasses *ABCMeta* e o decorador *abstractmethod* que identifica o método que deve ser implementado.

Combinando *abstractmethod* com *property*, podemos ter propriedades abstratas.

Um método abstrato é um método que é declarado, mas não contém implementação. As classes abstratas não podem ser instanciadas e exigem subclasses para fornecer implementações para os métodos abstratos.

Uma subclasse de uma classe abstrata só pode ser instanciada se todos os métodos abstratos estiverem sido implementados.

Métodos abstratos podem ser implementados em uma classe abstrata, porém, estes serão sobrescritos nas classes derivadas.

Como em casos de herança normal, os métodos da classe abstrata podem ser invocados usando o *super()*.

Exemplo 1 – Método abstrato implementado:

```
from abc import ABCMeta, abstractmethod

class Carro(metaclass=ABCMeta):
    @abstractmethod
    def ligar(self):
        # Sem implementação
        pass

class fusca(Carro):
    def __init__(self):
        print("Vou implementar o ligar.")

    def ligar(self):
        print("Uhu, o fusca ligou!!!")

fusquinha = fusca()
fusquinha.ligar()
```

Resultado da execução:

Vou implementar o ligar.

Uhu, o fusca ligou!!!

Exemplo 2 – Método abstrato não implementado:

```
from abc import ABCMeta, abstractmethod

class Carro(metaclass=ABCMeta):
    @abstractmethod
    def ligar(self):
        # Sem implementação
        pass

class kombi(Carro):
    def __init__(self):
        print("Não vou implementar o ligar.")

furgao = kombi()
```



Deu erro porque kombi não tem o método ligar implementado

Traceback (most recent call last):

File "...exemplo1.py", line 19, in <module>

furgao = kombi()

TypeError: Can't instantiate abstract class kombi with abstract methods ligar

Na hora de instanciar o objeto da classe kombi vai dar erro porque na classe kombi não foi implementado o método "ligar".

Exemplo 3 – Propriedade abstrata implementada:

```
from abc import ABCMeta, abstractmethod
```

```
class Carro(metaclass=ABCMeta):
    @property
    @abstractmethod
    def valor(self):
        return "Retorno de valor"

    @abstractmethod
    def ligar(self):
        # Sem implementação
        pass

class fusca(Carro):
    @property
    def valor(self):
        return "Propriedade concreta."

    def ligar(self):
        print("Uhu, o fusca ligou!!!")

fusquinha = fusca()
fusquinha.ligar()
print(fusquinha.valor)
```

Resultado da execução:

Uhu, o fusca ligou!!!

Propriedade concreta.

Exemplo 4 – Propriedade abstrata não implementada:

```
from abc import ABCMeta, abstractmethod
```

```
class Carro(metaclass=ABCMeta):
    @property
    @abstractmethod
    def valor(self):
        return "Retorno de valor"

    @abstractmethod
    def ligar(self):
        # Sem implementação
        pass

class fusca(Carro):
    def ligar(self):
        print("Uhu, o fusca ligou!!!")

fusquinha = fusca()
fusquinha.ligar()
print(fusquinha.valor)
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Resultado da execução:

Traceback (most recent call last):

File ".../exemplo4.py", line 18, in <module>

 fusquinha = fusca()

TypeError: Can't instantiate abstract class fusca with abstract methods valor

Na hora de instanciar o objeto da classe fusca vai dar erro porque na classe fusca não foi implementada a propriedade "valor"

Mensagens e tratamento de erros

Mensagens de erro

Nesta aula será apresentado ao aluno alguns dos tipos de erros que podem ocorrer em um programa Python.

Além do tipo de erro, o Python mostra a linha onde o interpretador foi interrompido, dando uma grande dica do que precisa ser corrigido em seu código.

SyntaxError

Erros de sintaxe, também conhecidos como erros de análise, ocorrem quando o interpretador não consegue ler o que você escreveu, o que pode ser ocasionado por erros de digitação ou símbolos esquecidos.

Esse é um erro muito comum para pessoas que estão iniciando os estudos em Python.

```
numero = 10
texto = "Olá, mundo"

print("Número:", numero)
print("Texto:", texto)
```

File ".../syntax_error.py", line 2

```
texto = "Olá, mundo
```

^

SyntaxError: EOL while scanning string literal

```
numero = 10
texto = "Olá, mundo"

print("Número:", numero)
print("Texto:", texto)
```

File ".../syntax_error2.py", line 5



```
print("Texto:", texto)
```

```
^
```

SyntaxError: invalid syntax

IndentationError

Erro de indentação, este erro é comum entre programadores de outras linguagens quando iniciam o aprendizado com Python.

Também ocorre muito quando o programador utiliza tabulação e espaços ao mesmo tempo em seu código (o que não é aconselhado).

```
def soma(n1, n2):  
    return n1 + n2  
  
x = soma(10, 78)  
    print("O resultado da soma é:", x)
```

File "../Indentation_error.py", line 5

```
    print("O resultado da soma é:", x)
```

```
^
```

IndentationError: unexpected indent

Indentação inesperada.

```
class Calculo(object):  
    def __init__(self, valor1, valor2):  
        self.valor1 = valor1  
        self.valor2 = valor2  
  
    def somar(self):  
        return self.valor1 + self.valor2  
  
    def subtrair(self):  
        return self.valor1 - self.valor2
```

```
teste = Calculo(50, 74)
```

```
print(teste.somar())  
print(teste.subtrair())
```

../indentation_error2.py", line 4

```
    self.valor2 = valor2
```

```
^
```

IndentationError: unindent does not match any outer indentation level

Não corresponde a nenhum outro nível de indentação

KeyError

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Este erro ocorre quando tentamos acessar uma chave inexistente em um dicionário.

```
dic = {"codigo": 1, "nome": "Evaldo", "celular": "(99) 99999-9999"}

print(dic["codigo"])
print(dic["nome"])
print(dic["telefone"])
```

1

Traceback (most recent call last):

Evaldo

File "../key_error.py", line 5, in <module>

```
print(dic["telefone"])
```

KeyError: 'telefone'

NameError

Este erro ocorre quando o interpretador Python não consegue identificar um nome fornecido. Seja uma variável, função ou qualquer outra referência que não seja identificada.

```
x = 10
y = 55

print("O resultado de x + y é:", X + y)
.../name_error.py", line 4, in <module>

    print("O resultado de x + y é:", X + y)
```

NameError: name 'X' is not defined

```
def soma(x, y):
    return x + y

resultado = somar(10, 75)
.../name_error2.py", line 4, in <module>

    resultado = somar(10, 75)
```

NameError: name 'somar' is not defined

```
class Teste(object):
    def __init__(self, nome, telefone):
        self.nome = nome
        self.telefone = telefone

if __nome__ == "__main__":
    a = Teste("Evaldo", "(99) 99999-9999")
    print(a.nome, a.telefone)
.../name_error3.py", line 6, in <module>

    if __nome__ == "__main__":
```

NameError: name '__nome__' is not defined



ValueError

O `ValueError` ocorre quando existe um problema com o conteúdo do objeto ao qual você tentou atribuir o valor.

```
print(int(5.75))
print(int("78"))
print(int("Olá"))
```

5

78

../value_error.py", line 3, in <module>

```
print(int("Olá"))
```

`ValueError: invalid literal for int() with base 10: 'Olá'`

```
inteiro = 5
lista = []
lista.remove(inteiro)
```

.../value_error2.py", line 3, in <module>

```
lista.remove(inteiro)
```

`ValueError: list.remove(x): x not in list`

```
nome, idade, sexo, endereco = ["José Wolkers", 78, "Masculino"]
```

.../value_error3.py", line 1, in <module>

```
nome, idade, sexo, endereco = ["José Wolkers", 78, "Masculino"]
```

`ValueError: not enough values to unpack (expected 4, got 3)`

TypeError

Um tipo em Python pode ser considerado como a especificação para uma categoria de dados relacionados. Embora possamos definir nossos próprios tipos, eles tendem a cair em um conjunto de categorias pré-definidas.

Estes são os tipos mais comuns que estão incorporados na linguagem Python:

- `int` – Números inteiros
- `float` – Números decimais
- `str` – uma sequência de caracteres
- `list` – uma lista ordenada
- `tuple` – uma lista ordenada imutável
- `boolean` – verdadeiro ou falso

Quando chamamos uma função ou usamos um operador, normalmente o interpretador *Python* espera ser informado parâmetros de um tipo específico. Por exemplo, o operador de adição “+” espera receber dois parâmetros de tipo numérico, ou seja, *integer* ou *float*.



Quando tentamos somar um integer com uma string o interpretador retorna o `TypeError`.

```

Command Prompt - python
>>> 10 + 58.3
68.3
>>> 58 + 74
132
>>> 125 + "Olá"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
>>>

```

Figura 212 - `TypeError`

```

inteiro = 30
print(type(inteiro))
x = inteiro[1]
.../type_error.py", line 2, in <module>

    x = inteiro[1]

```

`TypeError: 'int' object is not subscriptable`

Não é possível acessar um inteiro pelo índice.

Em geral, se você encontrar um `TypeError`, verifique se os dados em que você está operando são realmente do tipo que você esperava. Podem fazer utilizando a função `type()` sobre os dados que estão gerando dúvida.

IndexError

Um índice em Python representa uma posição em uma sequência. Mais comumente, isso se refere a uma posição em uma string ou uma lista. A indexação em Python, como na maioria das linguagens de programação, começa com zero.

```
lista_cores = ['Branco', 'Azul', 'Verde', 'Amarelo']
```

```

print(lista_cores[0])
print(lista_cores[3])
print(lista_cores[4])
Branco

```

Amarelo

Traceback (most recent call last):

File "P:\PCloud/Udemy/MeusCursos/PythonParaTodos/Secao 15 - Mensagens de erro/codigo/index_error.py", line 5, in <module>

```
    print(lista_cores[4])
```

`IndexError: list index out of range`

Process finished with exit code 1

```
frase = "O rato roeu a roupa do rei de Roma"
```

```

letra = frase[34]
print(letra)

```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
.../index_error2.py", line 3, in <module>
```

```
letra = frase[34]
```

IndexError: string index out of range

AttributeError

Um atributo em Python é uma propriedade associada a um determinado tipo de objeto. Em outras palavras, os atributos de um determinado objeto são os dados e habilidades que cada tipo de objeto possui inerentemente.

```
class teste(object):
    def __init__(self, nome, telefone):
        self.nome = nome
        self.telefone = telefone

t = teste("João Silva", "(00)00000-0000")
print(t.nome)
print(t.telefone)
print(t.idade)
```

```
.../attribute_error.py", line 9, in <module>
```

```
print(t.idade)
```

AttributeError: 'teste' object has no attribute 'idade'

João Silva

(00)00000-0000

Process finished with exit code 1

```
inteiro = 10
inteiro.append(20)
.../attribute_error2.py", line 2, in <module>
```

```
inteiro.append(20)
```

AttributeError: 'int' object has no attribute 'append'

Tratamento de erros – Exceções

Quando ocorre uma falha no programa em tempo de execução, uma exceção é gerada.

Caso a exceção não seja tratada, ela se propaga por meio das chamadas de funções até o módulo principal, interrompendo a execução do programa.

No código a seguir, foi solicitado um número ao usuário, mas não está sendo feito nenhum tratamento caso o usuário informe uma *string* que não seja um número.

Veja que, ao informar a *string* "Olá", gerou uma exceção do tipo *ValueError*.

Execução:

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>
contato@pythonparatodos.com.br



```
valor = input("Informe um número: ")  
  
soma = int(valor) + 10  
  
resultado = soma / int(valor)
```

“.../excecao.py”

Informe um número: Olá

Traceback (most recent call last):

File “.../excecao.py”, line 3, in <module>

soma = int(valor) + 10

ValueError: invalid literal for int() with base 10: 'Olá'

Veja também que logo em seguida está uma divisão, e não estamos tratando caso o usuário informe zero e ocorra um erro de divisão por zero.

Execução:

“.../excecao.py”

Informe um número: 0

Traceback (most recent call last):

File “.../excecao.py”, line 5, in <module>

resultado = soma / int(valor)

ZeroDivisionError: division by zero

try, except

Para tratar exceções utilizamos a instrução *try*. Caso ocorra alguma exceção em um bloco tratado com *try*, podemos tratar esta exceção utilizando a instrução *except*.

Sintaxe:

try:

bloco_de_código

except tipo_de_erro:

bloco_de_código_executado_em_caso_de_excecao

```
valor = input("Informe um número: ")
```

```
try:
```

```
    soma = int(valor) + 10  
    resultado = soma / int(valor)
```

```
    print("Soma:", soma)
```

```
    print("Resultado:", resultado)
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
except:
    print("Ocorreu um erro.")
```

Veja que foi feito um tratamento de exceção geral, sem especificar o tipo de exceção, assim, para qualquer exceção, o trecho de *except* vai ser executado.

Execução:

“.../excecao.py”

Informe um número: Olá

Ocorreu um erro no sistema.

“.../excecao.py”

Informe um número: 0

Ocorreu um erro no sistema.

```
valor = input("Informe um número: ")
```

```
try:
    soma = int(valor) + 10
    resultado = soma / int(valor)

    print("Soma:", soma)
    print("Resultado:", resultado)
except ZeroDivisionError:
    print("Ocorreu um erro de divisão por zero [ZeroDivisionError].")
except ValueError:
    print("Ocorreu um erro de valor [ValueError].")
```

Agora estão sendo tratados dois tipos de exceção, sendo assim, podemos tomar uma decisão diferente para cada exceção levantada.

Execução:

“.../excecao.py”

Informe um número: Olá

Ocorreu um erro de valor [ValueError].

“.../excecao.py”

Informe um número: 0

Ocorreu um erro de divisão por zero [ZeroDivisionError].

try...else

O tratamento de exceções pode conter um bloco *else*, que é executado quando não ocorre nenhum erro.

```
valor = input("Informe um número: ")
```

```
try:
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>
contato@pythonparatodos.com.br



```
soma = int(valor) + 10
resultado = soma / int(valor)

print("Soma:", soma)
print("Resultado:", resultado)
except ZeroDivisionError:
    print("Ocorreu um erro de divisão por zero [ZeroDivisionError].")
except ValueError:
    print("Ocorreu um erro de valor [ValueError].")
else:
    print("Não deu erro!!!")
```

Execução:

Informe um número: 25

Não deu erro!!!

try...finally

O tratamento de exceções pode conter um bloco *finally*, que é executado sempre, ocorrendo erro ou não. É muito utilizado quando você precisa realizar alguma ação mesmo que dê erro no sistema. Pode ser utilizada para liberar recursos que foram usados no bloco *try*, tais como conexões com bancos de dados e arquivos abertos.

```
valor = input("Informe um número: ")

try:
    soma = int(valor) + 10
    resultado = soma / int(valor)

    print("Soma:", soma)
    print("Resultado:", resultado)
except ZeroDivisionError:
    print("Ocorreu um erro de divisão por zero [ZeroDivisionError].")
except ValueError:
    print("Ocorreu um erro de valor [ValueError].")
finally:
    print("Esta linha será executada ocorrendo erro ou não!!!")
```

Execução:

Informe um número: 10

Soma: 20

Resultado: 2.0

Esta linha será executada ocorrendo erro ou não!!!

Informe um número: Boa noite!!!

Ocorreu um erro de valor [ValueError].

Esta linha será executada ocorrendo erro ou não!!!

Quando tratamos a exceção o sistema não é interrompido, ele continua seu fluxo normalmente.



```

valor = input("Informe um número: ")

try:
    soma = int(valor) + 10
    resultado = soma / int(valor)

    print("Soma:", soma)
    print("Resultado:", resultado)
except ZeroDivisionError:
    print("Ocorreu um erro de divisão por zero [ZeroDivisionError].")
except ValueError:
    print("Ocorreu um erro de valor [ValueError].")
finally:
    print("Esta linha será executada ocorrendo erro ou não!!!")

x = 10
y = 25
print("X + Y:", x + y)

```

Execução:

Informe um número: Olá

Ocorreu um erro de valor [ValueError].

Esta linha será executada ocorrendo erro ou não!!!

X + Y: 35

Capturando a mensagem da exceção.

A cláusula *except* pode especificar uma variável após o nome da exceção. A variável é ligada a uma instância de exceção com os argumentos armazenados em *instance.args*. A instância de exceção define *__str__()* para que os argumentos possam ser impressos diretamente, sem ter que fazer referência a *.args*.

```

valor = input("Informe um número: ")

try:
    soma = int(valor) + 10
    resultado = soma / int(valor)

    print("Soma:", soma)
    print("Resultado:", resultado)
except Exception as e:
    print("Ocorreu um erro. Veja o erro:", e)
    print("Ocorreu um erro. Veja o erro:", e.args)

```

Antes do Python 3.6 era usado:

except TIPO_EXCECAO, VARIÁVEL:

except Exception, e:

À partir do Python 3.6 a sintaxe correta é:

except TIPO_EXCECAO as VARIÁVEL:

except Exception as e:



Execução

Informe um número: a

Ocorreu um erro. Veja o erro: invalid literal for int() with base 10: 'a'

Ocorreu um erro. Veja o erro: ("invalid literal for int() with base 10: 'a'",)

Levantando uma exceção.

Você pode criar/forçar uma exceção usando o *raise*.

```
def converte_inteiro(string):
    numeros_permitidos = ("0", "1", "2", "3", "4", "5", "6", "7", "8", "9")
    if string not in numeros_permitidos:
        raise ValueError("Informe um número de 0 a 9!!!")
    return int(string)
```

```
from modulo_teste import converte_inteiro
```

```
numero = input("Informe um número de 0 a 9: ")
print(10 + converte_inteiro(numero))
```

Execução:

Informe um número de 0 a 9: a

Traceback (most recent call last):

File ".../usar_modulo_teste.py", line 4, in <module>

print(10 + converte_inteiro(numero))

File ".../codigo/modulo_teste.py", line 3, in converte_inteiro

raise ValueError("Informe um número de 0 a 9!!!")

ValueError: Informe um número de 0 a 9!!!

Hierarquia de exceções

```
BaseException
+-- SystemExit
+-- KeyboardInterrupt
+-- GeneratorExit
+-- Exception
+-- StopIteration
+-- StopAsyncIteration
+-- ArithmeticError
+-- FloatingPointError
+-- OverflowError
+-- ZeroDivisionError
+-- AssertionError
+-- AttributeError
+-- BufferError
+-- EOFError
+-- ImportError
+-- ModuleNotFoundError
+-- LookupError
+-- IndexError
+-- KeyError
+-- MemoryError
+-- NameError
+-- UnboundLocalError
```

```
+-- OSError
+-- BlockingIOError
+-- ChildProcessError
+-- ConnectionError
+-- BrokenPipeError
+-- ConnectionAbortedError
+-- ConnectionRefusedError
+-- ConnectionResetError
+-- FileExistsError
+-- FileNotFoundError
+-- InterruptedError
+-- IsADirectoryError
+-- NotADirectoryError
+-- PermissionError
+-- ProcessLookupError
+-- TimeoutError
+-- ReferenceError
+-- RuntimeError
+-- NotImplementedError
+-- RecursionError
```

```
+-- SyntaxError
+-- IndentationError
+-- TabError
+-- SystemError
+-- TypeError
+-- ValueError
+-- UnicodeError
+-- UnicodeDecodeError
+-- UnicodeEncodeError
+-- UnicodeTranslateError
+-- Warning
+-- DeprecationWarning
+-- PendingDeprecationWarning
+-- RuntimeWarning
+-- SyntaxWarning
+-- UserWarning
+-- FutureWarning
+-- ImportWarning
+-- UnicodeWarning
+-- BytesWarning
+-- ResourceWarning
```

Traceback

O *traceback* é a informação que o interpretador *Python* fornece para nos ajudar a identificar o problema em nosso código. Ele contém algumas informações úteis:

- A sequência de chamadas de funções até o ponto onde ocorreu o problema;
- O número da linha onde o erro foi gerado;
- O tipo de erro que ocorreu, bem como uma pequena mensagem informativa sobre o ocorrido.

".../execcao.py"

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Informe um número: Olá

Traceback (most recent call last):

File ".../execcao.py", line 3, in <module>

soma = int(valor) + 10

ValueError: invalid literal for int() with base 10: 'Olá'

Podemos utilizar o módulo *tracebak* para capturar a informação da exceção e utilizá-la por exemplo, para gravar os erros em um arquivo de log.

```

1 Traceback (most recent call last):
2   File "P:/PCloud/Udemy/MeusCursos/001-PythonParaTodos/Secao 15 - Mensagens e tratamento de
  erros/02_TratamentoDeErros/codigo/usando_traceback.py", line 5, in <module>
3     arquivo = open(nome_arquivo)
4   FileNotFoundError: [Errno 2] No such file or directory: 'Arquivo_inexistente.txt'
5

```

Figura 213 - Traceback

```
import traceback
```

```
try:
```

```
    nome_arquivo = input("Informe o nome do arquivo: ").strip()
```

```
    arquivo = open(nome_arquivo)
```

```
except:
```

```
    trace = traceback.format_exc()
```

```
    print("Ocorreu um erro:\n", trace)
```

```
    open('trace.log', 'a').write(trace)
```

...\python.exe ".../usando_traceback.py"

Informe o nome do arquivo: Arquivo_inexistente.txt

Ocorreu um erro:

Traceback (most recent call last):

File ".../usando_traceback.py", line 5, in <module>

arquivo = open(nome_arquivo)

FileNotFoundError: [Errno 2] No such file or directory: 'Arquivo_inexistente.txt'

Durante a manipulação da exceção ocorreu outra exceção

Caso ocorra um erro durante a manipulação da exceção, o Python informa o ocorrido.

```
valor = input("Informe um número: ")
```

```
try:
```

```
    soma = int(valor) + 10
```

```
    resultado = soma / int(valor)
```

```
    print("Soma:", soma)
```

```
    print("Resultado:", resultado)
```

```
except:
```

```
    x = 10 / 0
```

```
    print("ERROOOOOO!!!!")
```

...\python.exe ".../erro_manipula_execcao.py"

Informe um número: Olá

Traceback (most recent call last):

File ".../erro_manipula_execcao.py", line 4, in <module>

soma = int(valor) + 10

ValueError: invalid literal for int() with base 10: 'Olá'

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



During handling of the above exception, another exception occurred:

Traceback (most recent call last):

File ".../erro_manipula_excecao.py", line 10, in <module>

x = 10 / 0

ZeroDivisionError: division by zero

Process finished with exit code 1

Exceções definidas pelo usuário

Nós também podemos criar as nossas próprias exceções. Para isso temos que herdar de Exception.

```
class Error(Exception):
    """Classe base para outras exceções"""
    pass

class numeroMenorError(Error):
    """Exceção levantada quando o valor informado é menor do que o esperado"""
    pass

class numeroMaiorError(Error):
    """Exceção levantada quando o valor informado é maior do que o esperado"""
    pass

numero_da Sorte = 20

while True:
    try:
        numero = int(input("Informe um número: "))

        if numero < numero_da Sorte:
            raise numeroMenorError
        elif numero > numero_da Sorte:
            raise numeroMaiorError
        break
    except numeroMenorError:
        print("Você informou um número menor, tente novamente!")
    except numeroMaiorError:
        print("Você informou um número maior, tente novamente!")

print("Acertô miserávi!!! Quem ti insinô? kkkkk")
```

Documentação

Pydoc

O *pydoc* é uma ferramenta de documentação que já é instalada junto com o *Python*.

O módulo *pydoc* gera automaticamente documentação para módulos *Python*.

Esta documentação pode ser apresentada como páginas de texto no *console*, servidas em um navegador da *Web* ou salvas em arquivos *HTML*.

Para módulos, classes, funções e métodos, a documentação exibida é derivada do *docstring* (ou seja, o atributo `__doc__`) do objeto e, recursivamente de seus membros documentáveis.



Se não houver nenhuma *docstring*, o *pydoc* tenta obter uma descrição do bloco de linhas de comentários acima da definição da classe, função ou método no arquivo de origem ou na parte superior do módulo.

A função *help()* invoca o sistema de ajuda on-line no interpretador interativo, que usa o *pydoc* para gerar sua documentação como texto no *console*.

A mesma documentação de texto também pode ser visualizada de fora do interpretador *Python* executando *pydoc* como um script no *prompt* de comando (terminal) do sistema operacional.

Veja como usar o *pydoc* no Windows:

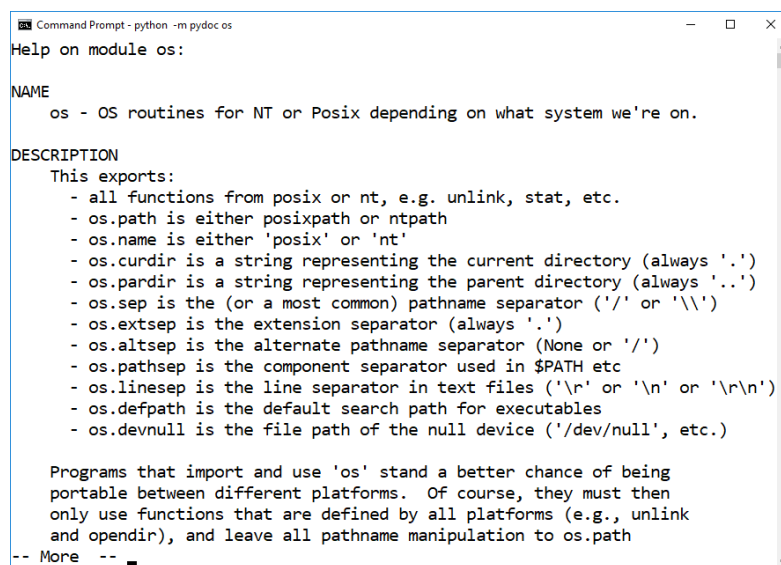
C:\>python -m pydoc argumento

Onde argumento é o nome de uma função, módulo ou pacote que você queira ver a documentação.

No *Linux* você pode usar similar ao uso no Windows ou usar o *pydoc* diretamente assim:

\$ pydoc argumento

Veja o resultado do comando “*python -m pydoc os*” no Windows:



```
Command Prompt - python -m pydoc os
Help on module os:

NAME
  os - OS routines for NT or Posix depending on what system we're on.

DESCRIPTION
  This exports:
  - all functions from posix or nt, e.g. unlink, stat, etc.
  - os.path is either posixpath or ntpath
  - os.name is either 'posix' or 'nt'
  - os.curdir is a string representing the current directory (always '.')
  - os.pardir is a string representing the parent directory (always '..')
  - os.sep is the (or a most common) pathname separator ('/' or '\\')
  - os.extsep is the extension separator (always '.')
  - os.altsep is the alternate pathname separator (None or '/')
  - os.pathsep is the component separator used in $PATH etc
  - os.linesep is the line separator in text files ('\r' or '\n' or '\r\n')
  - os.defpath is the default search path for executables
  - os.devnull is the file path of the null device ('/dev/null', etc.)

  Programs that import and use 'os' stand a better chance of being
  portable between different platforms. Of course, they must then
  only use functions that are defined by all platforms (e.g., unlink
  and opendir), and leave all pathname manipulation to os.path
  -- More --
```

Figura 214 - “python -m pydoc os” no Windows

Você pode pressionar *enter* para ir passando o conteúdo ou pressionar “q” para sair.

Veja o resultado do comando “*pydoc sys*” no *Linux*:

```

Help on built-in module sys:

NAME
    sys

FILE
    (built-in)

MODULE DOCS
    https://docs.python.org/library/sys

DESCRIPTION
    This module provides access to some objects used or maintained by the
    interpreter and to functions that interact strongly with the interpreter.

    Dynamic objects:

    argv -- command line arguments; argv[0] is the script pathname if known
    path -- module search path; path[0] is the script directory, else ''
    modules -- dictionary of loaded modules

    displayhook -- called to show results in an interactive session
    excepthook -- called to handle any uncaught exception other than SystemEx
    it

    To customize printing in an interactive session or to install a custom
    :
  
```

Figura 215 - “pydoc sys” no Linux

A saída do *pydoc* é similar às páginas de manual mostradas pelo comando *man* do *Unix*. Veja o resultado do comando “*man cp*” no *Linux*. O comando “*cp*” é utilizado para cópia de arquivos.

```

Terminal
Arquivo Editar Ver Pesquisar Terminal Ajuda
CP(1) General Commands Manual CP(1)

NOME
    cp - copia arquivos e diretórios

SINOPSE
    cp [opções] arquivo caminho
    cp [opções] arquivo... diretório

    Opções POSIX: [-fipRr]

    Opções GNU (forma reduzida): [-abdfilprsuvxPR] [-S SUFFIX] [-V
    {numbered,existing,simple}] [--sparse=WHEN] [--help] [--version]
    [--]

DESCRIÇÃO
    cp copia arquivos (ou, opcionalmente diretórios). Você pode copiar
    um arquivo para um destino fornecido, ou copiar arbitrariamente
    muitos arquivos para um diretório destino.

    Se o último argumento de nome é um diretório existente, cp copia
    cada arquivo origem para aquele diretório (com o mesmo nome). De
    outra forma, se somente dois arquivos são fornecidos, ele copia o
    primeiro para o segundo. É um erro se o último argumento não for um
    diretório e mais que dois argumentos serem fornecidos.

Manual page cp(1) line 1 (press h for help or q to quit)
  
```

Figura 216 - Exemplo comando man no Linux

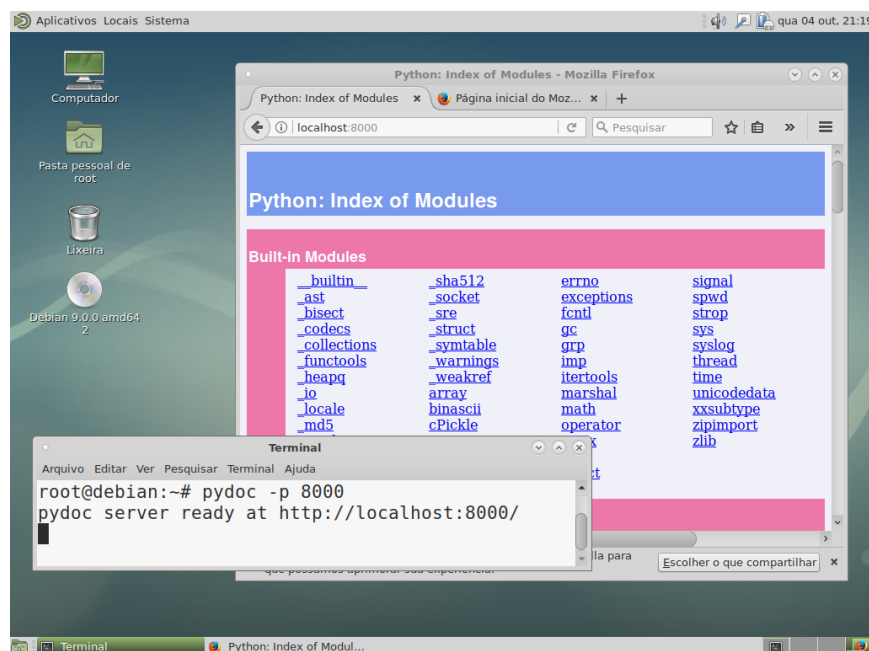
Você pode iniciar um servidor web com o *pydoc* para ler a documentação do *Python*. No *Windows* use: “*python -m pydoc -p 8000*”, onde 8000 é uma porta de sua escolha. Veja o resultado:

```

Command Prompt - python -m pydoc -p 8000
C:\>python -m pydoc -p 8000
Server ready at http://localhost:8000/
Server commands: [b]rowser, [q]uit
server>
  
```

Figura 217 - Iniciando servidor Web com pydoc

No Linux use: "pydoc -p 8000".



É possível também consultar a documentação de dentro do interpretador com o comando `help(argumento)`.

```
>>> help(int)
```

```
Command Prompt - python
>>> help(int)
Help on class int in module builtins:

class int(object)
|   int(x=0) -> integer
|   int(x, base=10) -> integer
|
|   Convert a number or string to an integer, or return 0 if no arguments
|   are given. If x is a number, return x.__int__(). For floating point
|   numbers, this truncates towards zero.
|
|   If x is not a number or if base is given, then x must be a string,
|   bytes, or bytearray instance representing an integer literal in the
|   given base. The literal can be preceded by '+' or '-' and be surrounded
```

Figura 220 - Consultando a documentação de dentro do interpretador Python

No *Linux*, o *pydoc* também tem uma interface gráfica, mas que depende da biblioteca Tkinter. Use o comando abaixo para instalar esta dependência:

```
$ sudo apt-get install python-tk
```

Após a instalação, execute o comando abaixo:

```
$ pydoc -g
```

Aparece uma interface com um campo de pesquisa “*Search for*”, onde você pode digitar um texto para pesquisar. Eu digite “*float*” nesta caixa e pressionei *enter*, na segunda imagem, estão sendo exibidos 3 resultados para “*float*”, veja que na terceira imagem temos o resultado ao escolher o primeiro item exibido e clicar em “*go to selected*”

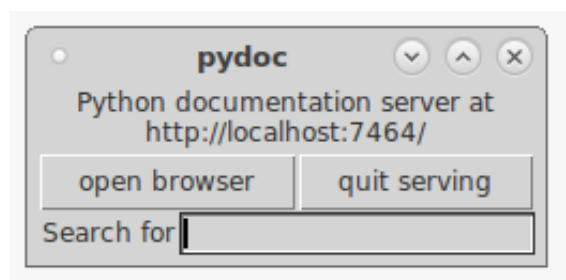


Figura 221 - Interface Gráfica pydoc 1

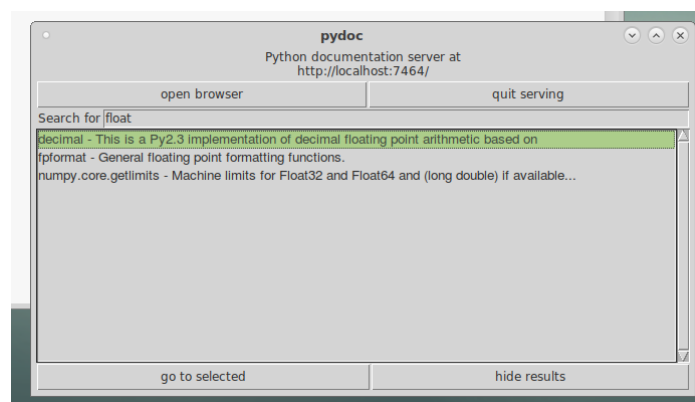


Figura 222 - Interface Gráfica pydoc 2

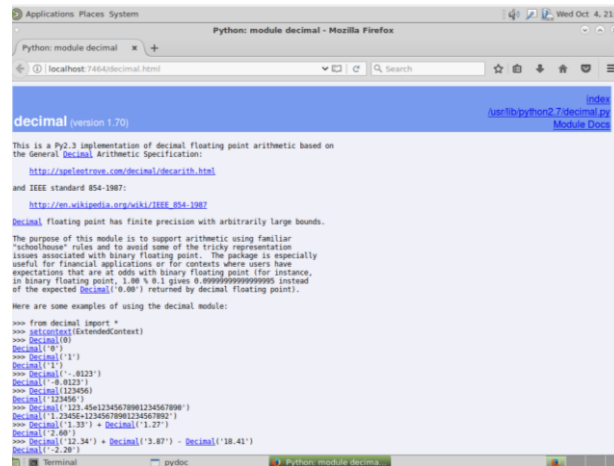


Figura 223 - Interface Gráfica pydoc 3

Documentando seu código

Vamos agora ver como usar *docstrings* para gerar documentos em *HTML* para nossos programas.

```
"""
```

Este módulo possui uma classe para a realização de diversos cálculos utilizando dois números

Sintaxe:

Calculo(valor1, valor2)

Métodos implementados:

somar, subtrair, dividir.

```
"""
```

```
class Calculo(object):
```

```
    """
```

Classe para realização dos cálculos

```
    """
```

```
    def __init__(self, numero1, numero2):
```

```
        """
```

Construindo um novo objeto Calculo

:param numero1: Variável que armazena o primeiro número

:param numero2: Variável que armazena o segundo número

:return: Sem retorno

```
        """
```

```
        self.numero1 = numero1
```

```
        self.numero2 = numero2
```

```
    def somar(self):
```

```
        """
```

Método que realiza a soma de dois números

:return: Retorna o resultado da soma de dois números

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
"""
    return self.numero1 + self.numero2

def subtrair(self):
    """
    Método que realiza a subtração de dois números
    :return: Retorna o resultado da subtração de dois números
    """
    return self.numero1 - self.numero2

def subtrair_do_maior(self):
    """
    Método que realiza a subtração de dois números, utilizando sempre maior - menor
    :return: Retorna o resultado da subtração de dois números
    """

    """ Validando qual número é maior para realizar o subtração -> Não será utilizado """
    if self.numero2 < self.numero1:
        resultado = self.numero2 - self.numero1
    else:
        resultado = self.numero1 - self.numero2

    return resultado

def dividir(self):
    """
    Método que realiza a divisão de dois números
    :return: Retorna a divisão de dois números
    """
    return self.numero1 / self.numero2
```

```
Command Prompt
P:\testepydoc>python -m pydoc -w P:\testepydoc
wrote calculo.html
P:\testepydoc>
```

Figura 224 - Visualizar documentação do código

No Linux pode-se usar:
pydoc -w diretório/pasta



Figura 225 - Visualizando documentação do código

Trabalhando com arquivos e diretórios

Arquivos e diretórios/pastas

Um arquivo é uma área em disco onde podemos ler e gravar informações. Essa área é gerenciada pelo sistema operacional, ou seja, não precisamos nos preocupar em como esse espaço é organizado em disco. Diretórios ou pastas são locais onde armazenamos nossos arquivos. Para trabalhar com arquivos em Python você não precisa importar nenhuma Biblioteca. Os recursos para ler e gravar arquivos são nativos no Python.



Para acessar um arquivo, precisamos abri-lo. Durante a abertura, informamos o nome do arquivo, com o nome do diretório onde ele se encontra (se necessário) e que operações queremos realizar (leitura e/ou escrita).



Podemos abrir arquivos em *Python* usando a função *open*, que utiliza os parâmetros **nome** e **modo**, desta forma *open("arquivo.txt", "r")*. O nome é o nome do arquivo, por exemplo, arquivo.txt. O modo indica as operações que vamos realizar. Os modos podem ser combinados ("r+", "w+", "a+", "r+b", "w+b", "a+b").

Modo	Operações
r	Leitura (padrão).
w	Escrita. Se não existir, cria, caso exista o arquivo, apaga seu conteúdo.
x	Exclusivo para criação, se já existir ocorre uma falha.

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



a	Escrita. Mas preserva o conteúdo se já existir.
b	Modo binário.
t	Modo texto (padrão).
+	Atualização (leitura e escrita).

Figure 1 - Tabela de modos de abertura de arquivo

Funções para trabalhar com arquivos

A função *open* retorna um objeto do tipo *file* (arquivo), que utilizamos para ler e escrever os dados no arquivo.

Para gravar os dados no arquivo utilizamos o método *write*, para ler utilizamos *read* e, para fechar o arquivo, usamos *close*.

Ao trabalharmos com arquivos, devemos sempre realizar o seguinte ciclo: abertura, leitura e/ou escrita, fechamento.

Abertura e fechamento de arquivo

A abertura realiza a ligação entre o programa e o espaço em disco, gerenciado pelo sistema operacional. As etapas de leitura e/ou escrita são as operações que desejamos realizar no programa, e o fechamento informa ao sistema operacional que não vamos mais trabalhar com o arquivo.

O fechamento do arquivo é muito importante, pois cada arquivo aberto consome recursos do computador. Só o fechamento do arquivo garante a liberação desses recursos e preserva a integridade dos dados do arquivo. O *close()* fecha completamente o arquivo, encerrando os recursos em uso, liberando-os para o sistema operacional. Após fechar o arquivo, qualquer tentativa de uso irá gerar uma falha.

Escrevendo no arquivo com a função *write()*.

```
# objeto_file = open("nome_do_arquivo", "modo")
# Usando o modo="w", caso o arquivo não exista, o mesmo será criado,
# caso já exista, terá seu conteúdo apagado
arquivo = open("arquivo_teste.txt", "w")

# Vou escrever uma linha no arquivo. O "\n" no final é para
# realizar uma quebra de linha (enter)
arquivo.write("Este é meu primeiro arquivo.\n")

# Escrevendo mais linhas
arquivo.write("Vou gravar nele um poema:\n")
arquivo.write("Poeminha do Contra - Mario Quintana\n")
arquivo.write("Todos estes que aí estão\n")
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
arquivo.write("Atravancando o meu caminho,\n")
arquivo.write("Eles passarão.\n")
arquivo.write("Eu passarinho!\n")

# Temos que fechar o arquivo no final!!! Não esqueça!!!
arquivo.close()
```

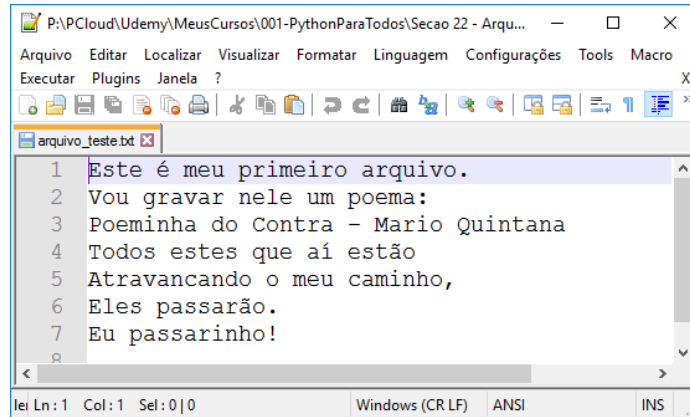


Figure 2 - Veja o arquivo aberto no editor Notepad++

Fechando um arquivo com `close()`

Fechar o arquivo é importante porque o sistema operacional realiza diversas funções para otimizar a velocidade de suas operações e uma dessas funções é não gravar os dados diretamente no disco, mantendo o conteúdo na memória auxiliar. Quando não fechamos o arquivo, corremos o risco de essa memória auxiliar não ser transferida para o disco e, assim, perdermos o que foi escrito pelo programa.

A sintaxe é: `arquivo.close()`.

Lendo o arquivo inteiro com a função `read()`.

```
# Abrindo o arquivo para leitura (modo="r")
arquivo = open("arquivo_teste.txt", "r")

# Imprimindo o conteúdo do arquivo
# A função read do objeto file retorna o conteúdo do arquivo
print(arquivo.read())

# Fechando o arquivo
arquivo.close()
```

```
Este é meu primeiro arquivo.
Vou gravar nele um poema:
Poeminha do Contra - Mario Quintana
Todos estes que aí estão
Atravancando o meu caminho,
Eles passarão.
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Eu passarinho!

Lendo o arquivo linha a linha com a função `readline()`.

```
# Abrindo o arquivo para leitura (modo="r")
arquivo = open("arquivo_teste.txt", "r")

# Imprimindo o conteúdo da primeira linha do arquivo
print(arquivo.readline())

# Imprimindo o conteúdo da próxima linha
print(arquivo.readline())

# Imprimindo 5 caracteres da próxima linha
print(arquivo.readline(5))

# Imprimindo o restante da próxima linha
print(arquivo.readline())

# Fechando o arquivo
arquivo.close()
```

Este é meu primeiro arquivo.
Vou gravar nele um poema:
Poemi
nha do Contra – Mario Quintana

Retornando uma lista com a função `readlines()`.

```
# Abrindo o arquivo para leitura (modo="r")
arquivo = open("arquivo_teste.txt", "r")

# O readlines percorre o arquivo e retorna seu conteúdo em uma lista,
# cada linha é um item desta lista
linhas = arquivo.readlines()

# Imprimindo a lista
print(linhas)

# Imprimindo o item 6 da lista
print(linhas[6])
```

['Este é meu primeiro arquivo.\n', 'Vou gravar nele um poema:\n', 'Poeminha do Contra – Mario Quintana\n', 'Todos estes que aí estão\n', 'Atravancando o meu caminho,\n', 'Eles passarão.\n', 'Eu passarinho!\n']
Eu passarinho!

Percorrendo todas as linhas com a função `readlines()`.

```
# Abrindo o arquivo para leitura (modo="r")
arquivo = open("arquivo_teste.txt", "r")
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>
contato@pythonparatodos.com.br
Página 171



```
# Imprimindo o conteúdo do arquivo uma linha por vez
for linha in arquivo.readlines():
    print(linha)

# Fechando o arquivo
arquivo.close()
```

Este é meu primeiro arquivo.
Vou gravar nele um poema:
Poeminha do Contra – Mario Quintana
Todos estes que aí estão
Atravancando o meu caminho,
Eles passarão.
Eu passarinho!

Características de um arquivo de texto

Trabalhamos com leitura e gravação de arquivos texto. A característica principal de um arquivo texto é que seu conteúdo é apenas texto simples, com alguns caracteres especiais de controle. O caractere de controle mais importante em um arquivo-texto é o que marca o fim de uma linha. No *Windows*, o fim de linha é marcado por uma sequência de dois caracteres, cujas posições na tabela *ASCII* são 10 (LF – *Line Feed*, avanço de linha) e 13 (CR – *Carriage Return*, retorno de cursor). No *Linux*, temos apenas um caractere marcando o fim de linha, o *Line Feed* (10). Já no Mac OS X, temos apenas o *Carriage Return* (13) marcando o fim de uma linha.

Mais sobre a tabela *ASCII*:

<https://www.wikiwand.com/pt/ASCII>

Gravando várias linhas com a função *writelines()* usando uma lista.

```
arquivo = open("arquivo_teste_dois.txt", "w")

linhas = ["A Centopeia – Marina Colasanti\n", "Quem foi que primeiro\n", "teve a\n", "ideia\n", "de contar um por um\n", "os pés da centopeia?\n", "Se uma pata você arranca\n", "será que a\n", "bichinha manca?\n", "E responda antes que eu esqueça\n", "se existe o bicho de cem pés\n", "será que existe algum de cem cabeças?\n"]

arquivo.writelines(linhas)
arquivo.close()
```

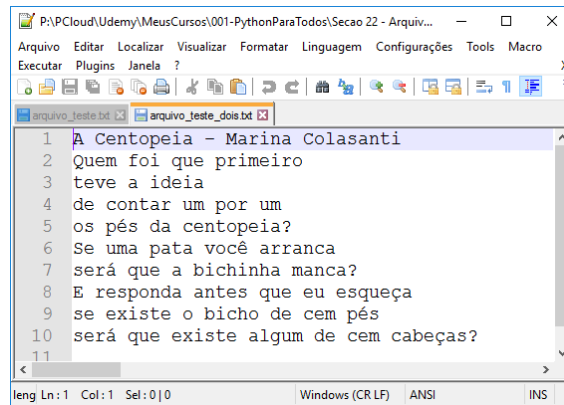


Figure 3 - Arquivo arquivo_teste_dois.txt

Usar o *with* é uma boa prática

O *with* é projetado para fornecer uma sintaxe muito limpa e gerenciamento de exceções, sendo assim, uma boa prática utilizá-lo ao trabalhar com arquivos.

O *with* fecha os arquivos abertos automaticamente após o término do uso, retirando do programador a preocupação em fechar o arquivo.

Lendo e escrevendo em arquivos com *with*.

```
# Abrindo o arquivo para escrita, mantendo o conteúdo
with open("arquivo_teste_dois.txt", "a") as arquivo:
    arquivo.write("Linha adicionada")

# como o modo "r" (leitura) é o padrão, caso eu
# não informe, será aberto como somente leitura
with open("arquivo_teste_dois.txt") as arquivo:
    for linha in arquivo:
        print(linha)
```

```
A Centopeia – Marina Colasanti
Quem foi que primeiro
teve a ideia
de contar um por um
os pés da centopeia?
Se uma pata você arranca
será que a bichinha manca?
E responda antes que eu esqueça
se existe o bicho de cem pés
será que existe algum de cem cabeças?
Linha adicionadaLinha adicionada
```



Lendo um arquivo de texto contendo o Hino Nacional.

```
# No open() podemos informar a codificação do arquivo através do encoding
# Para que os caracteres acentuados sejam mostrados corretamente, vamos
# usar a codificação utf-8.
with open("hino_nacional.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        palavras = linha.split()
        print(palavras)
```

```
['Ouviram', 'do', 'Ipiranga', 'às', 'margens', 'plácidas']
['De', 'um', 'povo', 'heroico', 'o', 'brado', 'retumbante']
['E', 'o', 'sol', 'da', 'Liberdade', 'em', 'raios', 'fúlgidos']
... Linhas removidas
['Dos', 'filhos', 'deste', 'solo', 'és', 'mãe', 'gentil']
['Pátria', 'amada']
['Brasil!']
```

```
# Veja agora sem o UTF-8
with open("hino_nacional.txt", "r") as arquivo:
    for linha in arquivo:
        palavras = linha.split()
        print(palavras)
```

```
['Ouviram', 'do', 'Ipiranga', 'Ã', 's', 'margens', 'plÃ¡cidas']
['De', 'um', 'povo', 'heroico', 'o', 'brado', 'retumbante']
['E', 'o', 'sol', 'da', 'Liberdade', 'em', 'raios', 'fÃ¶lgidos']
['Brilhou', 'no', 'cÃ©u', 'da', 'PÃ¡tria', 'nesse', 'instante']
[]
... Linhas removidas
['Terra', 'adorada']
['Entre', 'outras', 'mil']
['Ã¶s', 'tu', 'Brasil']
['Ã¶', 'PÃ¡tria', 'amada!']
[]
['Dos', 'filhos', 'deste', 'solo', 'Ã©s', 'mÃ£e', 'gentil']
['PÃ¡tria', 'amada']
['Brasil!']
```

```
palavras = []
with open("hino_nacional.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        palavras += linha.split()

print(palavras)
```

```
['Ouviram', 'do', 'Ipiranga', 'às', 'margens', 'plácidas', 'De', 'um', 'povo', 'heroico', 'o', 'brado', 'retumbante', 'E', 'o', 'sol', 'da', 'Liberdade', 'em', 'raios', 'fúlgidos', 'Brilhou', 'no', 'céu', 'da', 'Pátria', 'nesse', 'instante', 'Se', 'o', 'penhor', 'dessa', 'igualdade', 'Conseguimos', 'conquistar', 'com', 'braço', 'forte', 'Em', 'teu', 'seio', 'ó', 'Liberdade', 'Desafia', 'o', 'nosso', 'peito', 'a', 'própria', 'morte!', 'Ó', 'Pátria', 'amada', 'Idolatrada', 'Salve!', 'Salve!', 'Brasil', 'um', 'sonho', 'intenso', 'um', 'raio', 'vívido', 'De', 'amor', 'e', 'de', 'esperança', 'à', 'terra', 'desce', 'Se', 'em', 'teu', 'formoso', 'céu', 'risonho', 'e', 'límpido', 'A', 'imagem', 'do', 'Cruzeiro', 'resplandece', 'Gigante', 'pela', 'própria', 'natureza', 'És', 'belo', 'és', 'forte', 'impávido', 'colosso', 'E', 'o', 'teu', 'futuro', 'espelha', 'essa', 'grandeza', 'Terra', 'adorada', 'Entre', 'outras', 'mil', 'És', 'tu', 'Brasil', 'Ó', 'Pátria', 'amada!', 'Dos', 'filhos', 'deste', 'solo', 'és', 'mãe', 'gentil', 'Pátria', 'amada', 'Brasil!', 'Deitado', 'eternamente', 'em', 'berço', 'esplêndido', 'Ao', 'som', 'do', 'mar', 'e', 'à', 'luz', 'do', 'céu', 'profundo', 'Fulguras', 'ó', 'Brasil', 'florão', 'da', 'América', 'Iluminado', 'ao', 'sol', 'do', 'Novo', 'Mundo!', 'Do', 'que', 'a', 'terra', 'mais', 'garrida', 'Teus', 'risonhos', 'lindos', 'campos', 'têm', 'mais', 'flores', 'Nossos', 'bosques', 'têm', 'mais', 'vida', 'Nossa', 'vida', 'no', 'teu', 'seio', 'mais', 'amores', 'Ó', 'Pátria', 'amada', 'Idolatrada', 'Salve!', 'Salve!', 'Brasil', 'de', 'amor', 'eterno', 'seja', 'símbolo', 'O', 'lábaro', 'que', 'ostentas', 'estrelado', 'E', 'diga', 'o', 'verde-louro', 'desta', 'flâmula', 'Paz', 'no', 'futuro', 'e', 'glória', 'no', 'passado', 'Mas', 'se', 'ergues', 'da', 'justiça', 'a', 'clava', 'forte', 'Verás', 'que', 'um', 'filho', 'teu', 'não', 'foge', 'à', 'luta', 'Nem', 'teme', 'quem',
```



```
'te', 'adora', 'a', 'própria', 'morte', 'Terra', 'adorada', 'Entre', 'outras', 'mil', 'És', 'tu', 'Brasil', 'Ó', 'Pátria', 'amada!', 'Dos', 'filhos', 'deste', 'solo', 'és', 'mãe', 'gentil', 'Pátria', 'amada', 'Brasil!']
```

Programa para procurar palavras existentes no Hino Nacional.

```
palavras = []
with open("hino_nacional.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        linha = linha.replace(",","").replace("!","")
        palavras += linha.upper().split()

print(palavras)

while True:
    palavra = input("Informe uma palavra para ver se faz parte do Hino Nacional (ou sair para finalizar): ")

    if palavra.upper() == "SAIR":
        print("\nObrigado por usar o sistema, volte sempre.")
        break
    elif palavra.upper() in palavras:
        print("A palavra", palavra, "foi encontrada no Hino Nacional.")
        print("Quantidade de ocorrências:", palavras.count(palavra.upper()))
    else:
        print("A palavra", palavra, "não foi encontrada no Hino Nacional.")
```

Arquivos e diretórios (pastas)

Aprendemos a ler, criar e alterar arquivos, vamos aprender agora como listar arquivos, verificar o tamanho e a data de criação dos mesmos. Vamos ainda aprender a manipular diretórios.

Identificando o diretório atual

Para identificar o diretório onde nosso programa está sendo executado usamos o método `getcwd()` do módulo `os`.

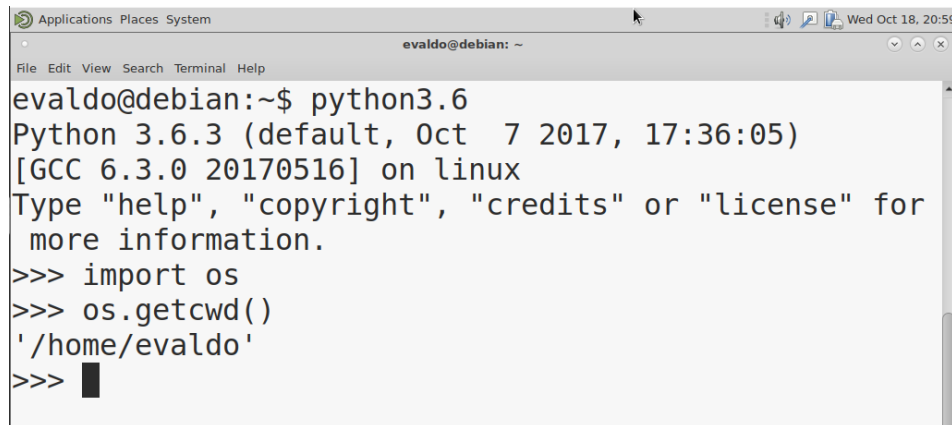
```
import os

print(os.getcwd())
```

```
...\\python.exe "...\\12_caminho_atual.py"
```

```
P:\\PCloud\\Udemy\\MeusCursos\\001-PythonParaTodos\\Secao 22 - Arquivos\\01_Introducao\\codigo
```

```
Process finished with exit code 0
```



```
evaldo@debian:~$ python3.6
Python 3.6.3 (default, Oct 7 2017, 17:36:05)
[GCC 6.3.0 20170516] on linux
Type "help", "copyright", "credits" or "license" for
more information.
>>> import os
>>> os.getcwd()
'/home/evaldo'
>>>
```

Figure 4 - Executando o `getcwd()` no Linux

Criando diretórios

Para criar diretórios usamos o método `mkdir()` do módulo `os`.

```
import os

os.mkdir("Curso Python")
os.mkdir("Curso Linux")
os.mkdir("Curso Missangas")
os.mkdir("Curso Python/Aula Arquivos")
os.mkdir("Curso Python/Aula Chatbot")
```

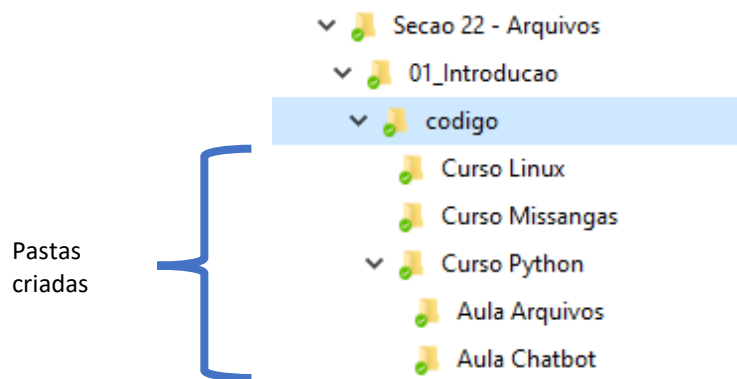


Figure 5 - Diretórios criados

Veja a árvore de diretórios no *Windows* e a criação no *Linux*.

```

Command Prompt

P:\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 22 - Arquivos\01_Introducao\codigo>tree
Folder PATH listing for volume pCloud Drive
Volume serial number is 00000041 5401:F3F6
P:..
├── Curso Python
│   ├── Aula Arquivos
│   └── Aula Chatbot
├── Curso Linux
└── Curso Missangas

P:\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 22 - Arquivos\01_Introducao\codigo>

```

Figure 6 - Árvore de diretórios exibida no Windows com o comando `tree`

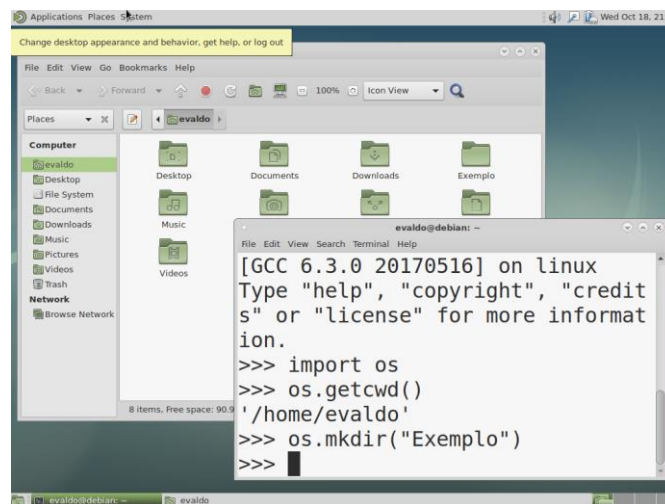


Figure 7 - Criando diretório no Linux

Criando vários diretórios

A função `mkdir()` cria apenas um diretório por vez, por exemplo, se tentássemos criar o diretório “Aula Arquivos” com o comando `os.mkdir("Curso Python/Aula Arquivos")` sem ter criado o diretório “Curso Python” antes, seria gerado um erro. Veja a seguir, onde tento criar a pasta “ponteiros” dentro de uma pasta inexistente.

```

import os

os.mkdir("Curso C/Ponteiros")

```

```

...\\python.exe "...\\criando_diretorio_erro.py"
Traceback (most recent call last):
  File "...\\criando_diretorio_erro.py", line 3, in <module>
    os.mkdir("Curso C/Ponteiros")
FileNotFoundError: [WinError 3] The system cannot find the path specified: 'Curso C/Ponteiros'
Process finished with exit code 1

```

Para criar vários diretórios, usamos a função `makedirs()`.

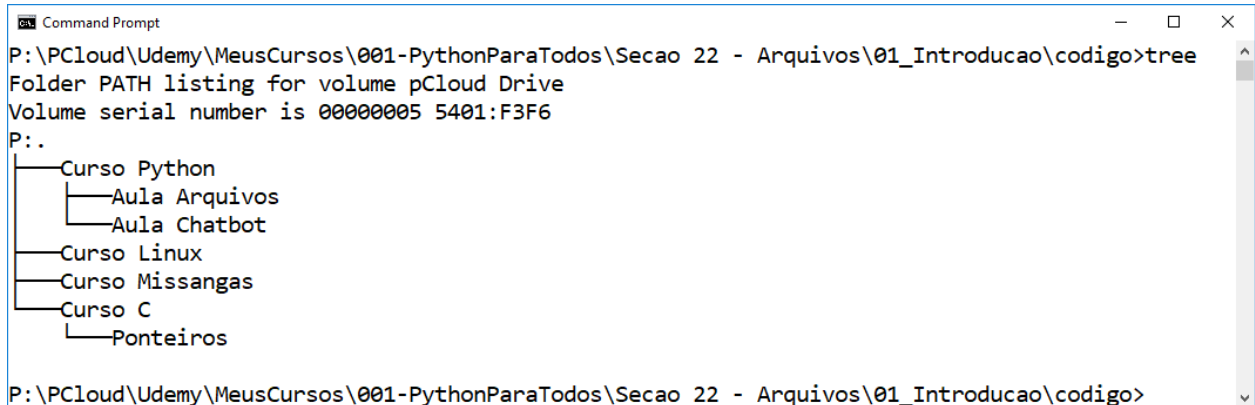
<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>
contato@pythonparatodos.com.br



A função `makedirs()` cria todos os diretórios e subdiretórios informados.

```
import os

os.makedirs("Curso C/Ponteiros")
```



```
Command Prompt
P:\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 22 - Arquivos\01_Introducao\codigo>tree
Folder PATH listing for volume pCloud Drive
Volume serial number is 00000005 5401:F3F6
P:..
|_Curso Python
|   |_Aula Arquivos
|   |_Aula Chatbot
|_Curso Linux
|_Curso Missangas
|_Curso C
|   |_Ponteiros

P:\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 22 - Arquivos\01_Introducao\codigo>
```

Figure 8 - Visualizando a árvore de diretórios no Windows com o comando `tree`

Navegando em diretórios

Para navegar entre diretórios usamos a função `chdir()` do módulo `os`.

```
import os

os.chdir("Curso Linux")
print("Pasta atual:", os.getcwd())
os.chdir("../")
print("Pasta atual:", os.getcwd())
os.chdir("Curso Python")
print("Pasta atual:", os.getcwd())
os.chdir("Aula Arquivos")
print("Pasta atual:", os.getcwd())
os.chdir("../../Curso Missangas")
print("Pasta atual:", os.getcwd())
```

```
C:\Users\Evaldo\AppData\Local\Programs\Python\Python36\python.exe "P:\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao
22 - Arquivos\01_Introducao\codigo\navegando_diretorios.py"
Pasta atual: P:\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 22 - Arquivos\01_Introducao\codigo\Curso Linux
Pasta atual: P:\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 22 - Arquivos\01_Introducao\codigo
Pasta atual: P:\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 22 - Arquivos\01_Introducao\codigo\Curso Python
Pasta atual: P:\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 22 - Arquivos\01_Introducao\codigo\Curso Python\Aula
Arquivos
Pasta atual: P:\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 22 - Arquivos\01_Introducao\codigo\Curso Missangas
Process finished with exit code 0
```



Renomeando arquivos e diretórios

Para renomear arquivos ou diretórios usamos a função `rename()`.

Veja como renomear um diretório.

```
import os

os.rename("Curso Missangas", "Curso Maquiagem")
```

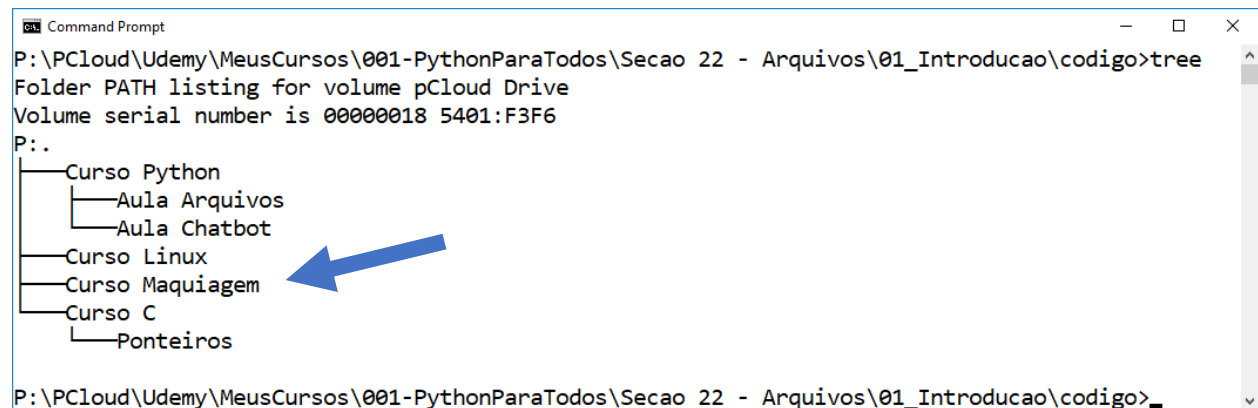


Figure 9 - Visualizando o diretório renomeado

Para renomear um diretório que não esteja no path atual, você deve informar o caminho.

```
import os

os.rename("Curso C/Ponteiros", "Curso C/Variaveis")
```

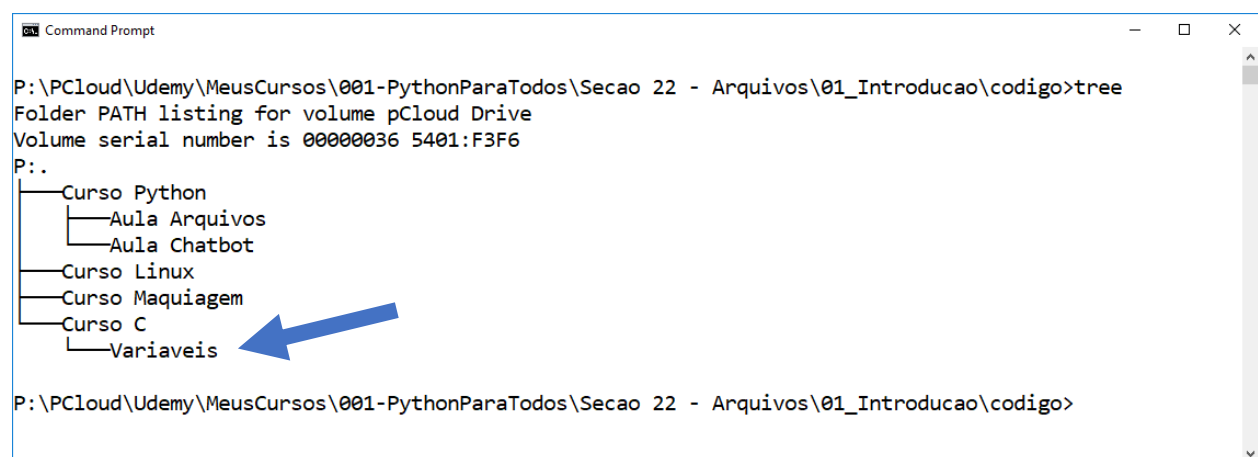


Figure 10 - Visualizando o diretório renomeado

Veja agora como renomear um arquivo.



```
import os

arquivo = open("nome_antigo.txt", "w")
arquivo.write("Vou mudar o nome de nome_antigo.txt para nome_novo.txt.")
arquivo.close()

os.rename("nome_antigo.txt", "nome_novo.txt")
```

```
Command Prompt

P:\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 22 - Arquivos\01_Introducao\codigo>dir *.txt
Volume in drive P is pCloud Drive
Volume Serial Number is 5401-F3F6

Directory of P:\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 22 - Arquivos\01_Introducao\codigo

17/10/2017  09:52 PM                181 arquivo_teste.txt
18/10/2017  12:18 AM                284 arquivo_teste_dois.txt
18/10/2017  12:25 AM             1,527 hino_nacional.txt
18/10/2017  11:00 PM                 55 nome_novo.txt
               4 File(s)              2,047 bytes
               0 Dir(s)  5,007,384,576 bytes free

P:\PCloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 22 - Arquivos\01_Introducao\codigo>
```

Figure 11 - Visualizando arquivo renomeado

Movendo um arquivo com a função `rename()`

A função `rename()` pode ser usada para mover um arquivo, caso você informe um diretório diferente. Veja como mover um arquivo.

```
import os

os.rename("nome_novo.txt", "Curso Linux/nome_novo.txt")
```

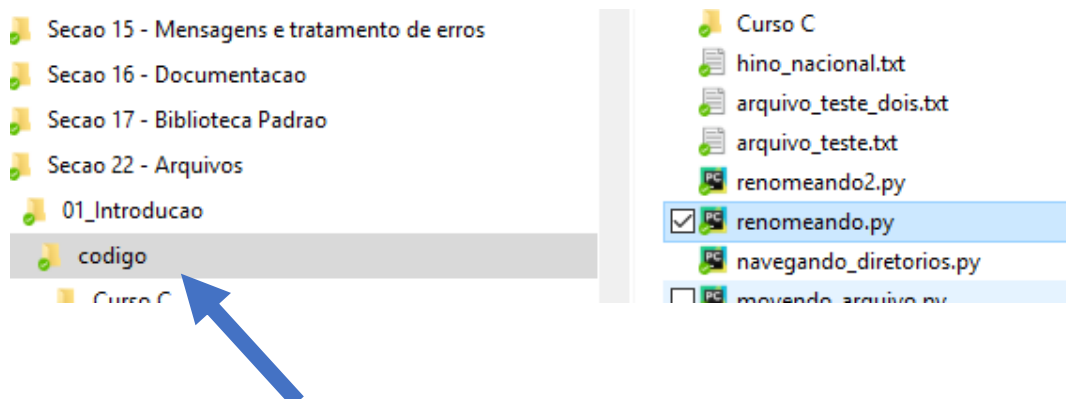


Figure 12 - Identificando que o arquivo não está mais na pasta

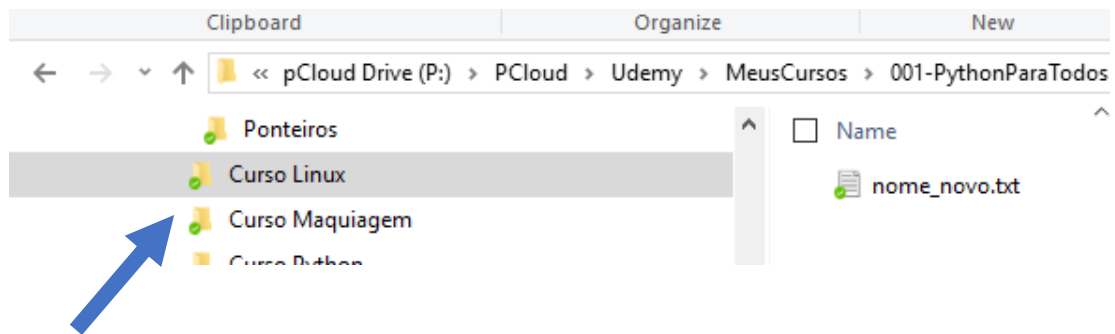


Figure 13 - Identificando que o arquivo foi movido

Listando arquivos e diretórios com a função `listdir()`

O retorno da função `listdir()` é uma lista com o conteúdo do diretório informado.

Veja a lista de arquivos da pasta Fonts que fica na pasta Windows.

```
import os
print(os.listdir("C:\\Windows\\Fonts"))
```

```
C:\Users\Evaldo\AppData\Local\Programs\Python\Python36\python.exe "P:/PCloud/Udemy/MeusCursos/001-PythonParaTodos/Secao
22 - Arquivos/01_Introducao/codigo/listando.py"
['8514fix.fon', '8514fixe.fon', '8514fixg.fon', '8514fixr.fon', '8514fixt.fon', '8514oem.fon', '8514oeme.fon', '8514oemg.fon', '8514oemr.fon',
'8514oemt.fon', '8514sys.fon', '8514syse.fon', '8514sysg.fon', '8514sysr.fon', '8514syst.fon', '85775.fon', '85855.fon', '85f1255.fon',
...CONTEÚDO REMOVIDO
'vgaf1255.fon', 'vgaf1256.fon', 'vgaf1257.fon', 'vgaf874.fon', 'vgafix.fon', 'vgafixe.fon', 'vgafixg.fon', 'vgafixr.fon', 'vgafixt.fon', 'vgaoem.fon',
'vgas1255.fon', 'vgas1256.fon', 'vgas1257.fon', 'vgas874.fon', 'vgasys.fon', 'vgasyse.fon', 'vgasysg.fon', 'vgasysr.fon', 'vgasyst.fon',
'VINERITC.TTF', 'VIVALDII.TTF', 'VLADIMIR.TTF', 'webdings.ttf', 'wingding.ttf', 'WINGDNG2.TTF', 'WINGDNG3.TTF', 'yandex.ttf',
'YuGothB.ttc', 'YuGothL.ttc', 'YuGothM.ttc', 'YuGothR.ttc']
Process finished with exit code 0
```

Veja a lista de arquivos e pastas da pasta “/home/evaldo” em uma máquina Linux.



```

Applications Places System
Terminal
File Edit View Search Terminal Tabs Help
Terminal
license" for more information.
>>> import os
>>> os.listdir("/home/evaldo")
['.python_history', '.bash_logout', '.ba
sh_history', 'Templates', 'Videos', '.gn
upg', '.local', 'Exemplo', '.profile', '.
xsession-errors', 'Music', '.xsession-e
rrors.old', 'Pictures', '.bashrc', 'Desk
top', 'Downloads', '.config', '.cache',
'Public', '.ICEauthority', '.dmrc', 'Doc
uments', '.Xauthority']
>>>

```

Figure 14 - Executando `listdir()` no Linux

Identificando diretórios e arquivos com `isdir()` e `isfile()`

Como o `listdir()` retorna uma lista, vamos fazer um loop percorrendo cada item do resultado. Usando a função `isdir()` ou `isfile()` que fazem parte do módulo `os.path` vamos validar cada um destes itens.

```

import os

for objeto in os.listdir("."):
    if os.path.isdir(objeto):
        print(objeto, "é um diretório!")
    elif os.path.isfile(objeto):
        print(objeto, "é um arquivo!")

```

```

.idea é um diretório!

Curso Python é um diretório!

Curso Linux é um diretório!

...REMOVIDO CONTEÚDO.

movendo_arquivo.py é um arquivo!

listando.py é um arquivo!

validar.py é um arquivo!

Process finished with exit code 0

```

```

import os

diretorios = []
arquivos = []
for objeto in os.listdir("."):
    if os.path.isdir(objeto):
        diretorios.append(objeto)

```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
elif os.path.isfile(objeto):
    arquivos.append(objeto)

print("Diretórios:", diretorios)
print("Arquivos:", arquivos)
```

Diretórios: ['.idea', 'Curso Python', 'Curso Linux', 'Curso Maquiagem', 'Curso C']

Arquivos: ['arquivo_teste.txt', '02_lendo_arquivo.py', '03_lendo_arquivo.py', '04_lendo_arquivo.py', '05_lendo_arquivo.py', 'arquivo_teste_dois.txt', '06_escrevendo_lista.py', '07_lendo_escrevendo_with.py', 'hino_nacional.txt', '09_hino_nacional.py', '08_hino_nacional.py', '10_hino_nacional.py', '11_hino_nacional.py', '12_caminho_atual.py', '13_criando_diretorio.py', '16_navegando_diretorios.py', '17_renameando.py', '18_renameando.py', 'movendo_arquivo.py', '19_renameando.py', '20_listando.py', '21_arquivo_diretorio.py', 'arquivo_diretorio2.py', '22_verificar_existencia.py', '01_escrevendo_arquivo.py', 'obtendo_informacoes.py', '14_criando_diretorio_erro.py', '15_criando_diretorio.py']

Verificando a existência de um arquivo ou diretório

Para verificar a existência de um arquivo ou diretório usamos a função `exists()` do módulo `path`.

```
import os.path

while True:
    nome = input("Informe um nome de arquivo ou pasta (informe 'sair' para finalizar): ")
    if nome.upper() == "SAIR":
        break
    if os.path.exists(nome):
        print(nome, "existe!!!")
        if os.path.isdir(nome):
            print(nome, "é um diretório.")
        elif os.path.isfile(nome):
            print(nome, "é um arquivo.")
    else:
        print(nome, "NÃO existe!!!")
```

```
Informe um nome de arquivo ou pasta (informe 'sair' para finalizar): teste.doc
teste.doc NÃO existe!!!

Informe um nome de arquivo ou pasta (informe 'sair' para finalizar): hino_nacional.py
hino_nacional.py existe!!!
hino_nacional.py é um arquivo.

Informe um nome de arquivo ou pasta (informe 'sair' para finalizar): Curso Linux
Curso Linux existe!!!
Curso Linux é um diretório.

Informe um nome de arquivo ou pasta (informe 'sair' para finalizar): Testando
Testando NÃO existe!!!

Informe um nome de arquivo ou pasta (informe 'sair' para finalizar): sair
Process finished with exit code 0
```



Obtendo informações do arquivo

```
import os.path
import time

while True:
    nome = input("Informe um nome de arquivo (informe 'sair' para finalizar): ")
    if nome.upper() == "SAIR":
        break
    print("Nome:", nome)
    print("Tamanho:", os.path.getsize(nome), "bytes")
    print("Data da criação:", time.ctime(os.path.getctime(nome)))
    print("Data de modificação:", time.ctime(os.path.getmtime(nome)))
    print("Data de acesso:", time.ctime(os.path.getatime(nome)))
```

Informe um nome de arquivo (informe 'sair' para finalizar): hino_nacional.py

Nome: hino_nacional.py

Tamanho: 332 bytes

Data da criação: Wed Oct 18 01:08:37 2017

Data de modificação: Wed Oct 18 01:08:37 2017

Data de acesso: Wed Oct 18 01:08:37 2017

Informe um nome de arquivo (informe 'sair' para finalizar): olá.txt

olá.txt NÃO existe!!!

Informe um nome de arquivo (informe 'sair' para finalizar):



Manipulando caminhos

Cada sistema operacional trabalha de uma forma específica quando se trata de caminhos de arquivos e pastas, no Linux e no Mac OS X o caractere separador de pastas/diretórios é o “/”, já no Windows, é o “\”.

A biblioteca padrão do Python oferece algumas funções interessantes relacionadas a caminhos de arquivos e pastas.

os.path.abspath()

Retorna o caminho absoluto do *path* informado por parâmetro. Se o caminho não começar com “/”, o diretório atual é acrescentado, retornando o caminho completo à partir da raiz. No caso do Windows, incluindo também a letra do disco (drive) “C:”, “D:”, etc.

os.path.basename()

Retorna apenas a última parte do caminho.

os.path.dirname()

Retorna o caminho à esquerda da última barra.

os.path.splitdrive()

Usado no Windows para separar a letra do drive. Retorna uma tupla, onde a letra do drive é o primeiro elemento, e o restante do caminho, o segundo elemento. No Linux,

os.path.commonpath()

Retorna o sub-path comum nos caminhos informados.

os.path.commonprefix()

Retorna o prefixo de caminho mais longo de todos caminhos da lista.

os.path.join()

Une em um path os dados informados.

```
import os

caminho="Curso Python/Aula Arquivos/Caminhos"
caminho2="Curso Python/Aula Chatbot/Facebook Messenger"

if not os.path.exists(caminho):
    os.mkdir("Curso Python/Aula Arquivos/Caminhos")
if not os.path.exists(caminho2):
    os.mkdir("Curso Python/Aula Chatbot/Facebook Messenger")

caminho3 = "C:\\Program Files\\Internet Explorer\\en-US"

print("Caminho absoluto:", os.path.abspath(caminho))
print("Nome base:", os.path.basename(caminho))
print("Nome diretório:", os.path.dirname(caminho))
print("Separando letra do drive no Windows:", os.path.splitdrive(caminho3))
print("Caminho comum:", os.path.commonpath([caminho, caminho2]))
print("Prefixo comum:", os.path.commonprefix([caminho, caminho2]))
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
juncao = os.path.join("C:\\", "Teste", "Python", "Arquivos", "junção")
print("Resultado join:", juncao)
```

Caminho absoluto: P:\Pcloud\Udemy\MeusCursos\001-PythonParaTodos\Secao 22 - Arquivos\01_Introducao\codigo\Curso Python\Aula Arquivos\Caminhos

Nome base: Caminhos

Nome diretório: Curso Python/Aula Arquivos

Separando letra do drive no Windows: ('C:', '\\Program Files\\Internet Explorer\\en-US')

Caminho comum: Curso Python

Prefixo comum: Curso Python/Aula

Resultado join: C:\Teste\Python\Arquivos\junção

Acessando subdiretórios recursivamente

A função `walk()` navega em todas os subdiretórios à partir do caminho informado, retornando o caminho, os diretórios e os arquivos. Com ela podemos listar todo conteúdo de determinado caminho.

```
import os

for raiz, diretorios, arquivos in os.walk("P:\\Pcloud\\Udemy\\MeusCursos"
                                           "\\001-PythonParaTodos"
                                           "\\Secao 22 - Arquivos\\01_Introducao"):
    print("\nCaminho completo:", raiz)
    for diret in diretorios:
        print(f"{diret}/")
    for arq in arquivos:
        print(f"{arq}")
```

Excluindo arquivos e diretórios/pastas

Para excluir arquivos usamos a função `remove()`.

```
import os

# Criando um arquivo, gravando, lendo e excluindo
arq = open("arquivo_ecluir.txt", 'w')
arq.write("Esse arquivo será excluído\n")
arq.close()
arq = open("arquivo_ecluir.txt", 'r')
print("Conteúdo do arquivo:", arq.read())
arq.close()
# Para excluir, o arquivo não pode estar aberto
os.remove("arquivo_ecluir.txt")
# Nesta linha teremos um erro, porque o arquivo foi excluído
arq = open("arquivo_ecluir.txt", 'r')
```

Traceback (most recent call last):

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Conteúdo do arquivo: Esse arquivo será excluído

File "P:/PCloud/Udemy/MeusCursos/001-PythonParaTodos/Secao 22 - Arquivos/01_Introducao/codigo/27_excluindo_arquivo.py", line 13, in <module>

```
arq = open("arquivo_ecluir.txt", 'r')
```

FileNotFoundError: [Errno 2] No such file or directory: 'arquivo_ecluir.txt'

Para excluir diretórios vazios usamos a função *rmdir()*.

```
import os

nome_diretorio = "Esta Pasta Será Excluída"
if not os.path.exists(nome_diretorio):
    os.mkdir(nome_diretorio)

if os.path.exists(nome_diretorio):
    print(f"O diretório '{nome_diretorio}' existe!")
else:
    print(f"O diretório '{nome_diretorio}' não existe!")

os.rmdir(nome_diretorio)

if os.path.exists(nome_diretorio):
    print(f"O diretório '{nome_diretorio}' existe!")
else:
    print(f"O diretório '{nome_diretorio}' não existe!")
```

O diretório 'Esta Pasta Será Excluída' existe!

O diretório 'Esta Pasta Será Excluída' não existe!

Para excluir diretórios com conteúdo usamos a função *rmtree()* do módulo *shutil*.

```
import os
import shutil

dir1 = "Diretorio1"
dir2 = "Diretorio2"
dir3 = "Diretorio3"
arq1 = "Arquivo1.txt"
arq2 = "Arquivo2.txt"

if not os.path.exists(dir1):
    os.makedirs(dir1 + "/" + dir2 + "/" + dir3)

arq1 = open(dir1 + "/texto.txt", "w")
arq1.write("Arquivo de texto para teste.\n")
arq1.close()

arq2 = open(dir1 + "/" + dir2 + "/texto2.txt", "w")
arq2.write("Arquivo de texto para teste.\n")
arq2.close()

for raiz, diretorios, arquivos in os.walk(dir1):
    print("\nCaminho completo:", raiz)
    for diret in diretorios:
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
print(f"{diret}/")
for arq in arquivos:
    print(f"{arq}")

try:
    os.rmdir(dir1)
except:
    print(f"Não pode excluir '{dir1}' porque tem conteúdo.")

shutil.rmtree(dir1)

if not os.path.exists(dir1):
    print("Os diretórios foram apagados.")
```

```
Caminho completo: Diretorio1
Diretorio2/
texto.txt
Caminho completo: Diretorio1\Diretorio2
Diretorio3/
texto2.txt
Caminho completo: Diretorio1\Diretorio2\Diretorio3
Não pode excluir 'Diretorio1' porque tem conteúdo.
Os diretórios foram apagados.
```



Banco de dados MySQL

O que é um banco de dados relacional

Em resumo um banco de dados é uma coleção de dados relacionados. Com dados, queremos dizer fatos conhecidos que podem ser registrados e possuem um significado. Por exemplo, considere os nomes, números de telefone e endereços dos seus amigos e parentes. Você pode ter registrado esses dados em uma agenda ou, talvez, os tenha armazenado em um disco rígido do computador ou até mesmo em seu smartphone utilizando um sistema para isso.

Bancos de dados relacionais já são consolidados e difundidos há bastante tempo. Usaremos este modelo na maioria de nossas aulas.

Exemplos de bancos de dados relacionais:



O que é um banco de dados NoSQL (Not Only SQL)

Bancos de dados NoSQL estão cada vez mais conquistando espaço nas organizações.

São bancos “não-relacionais”, baseados em documentos ou na teoria dos grafos, por exemplo.



Um sistema gerenciador de banco de dados (SGBD) é uma coleção de programas que permite aos usuários criar e manter um ou mais bancos de dados.

O SGBD é um sistema de software de uso geral que facilita o processo de definição, construção, manipulação e compartilhamento de bancos de dados entre diversos usuários e aplicações. Definir um banco de dados envolve especificar os tipos, estruturas e restrições dos dados a serem armazenados.

O que é um sistema gerenciador de banco de dados

Um sistema gerenciador de banco de dados (SGBD) é uma coleção de programas que permite aos usuários criar e manter um ou mais bancos de dados.

O SGBD é um sistema de software de uso geral que facilita o processo de definição, construção, manipulação e compartilhamento de bancos de dados entre diversos usuários e aplicações. Definir um banco de dados envolve especificar os tipos, estruturas e restrições dos dados a serem armazenados.

A definição ou informação descritiva do banco de dados também é armazenada pelo SGBD na forma de um catálogo ou dicionário, chamado de metadados. A construção do banco de dados é o processo de armazenar os dados em algum meio controlado pelo SGBD. A manipulação de um banco de dados inclui funções como consulta ao banco de dados para recuperar dados específicos, atualização do banco de dados para refletir mudanças no minimundo e geração de relatórios com base nos dados.

Um SGBD controla e permite acesso aos bancos de dados por vários usuários e sistemas.

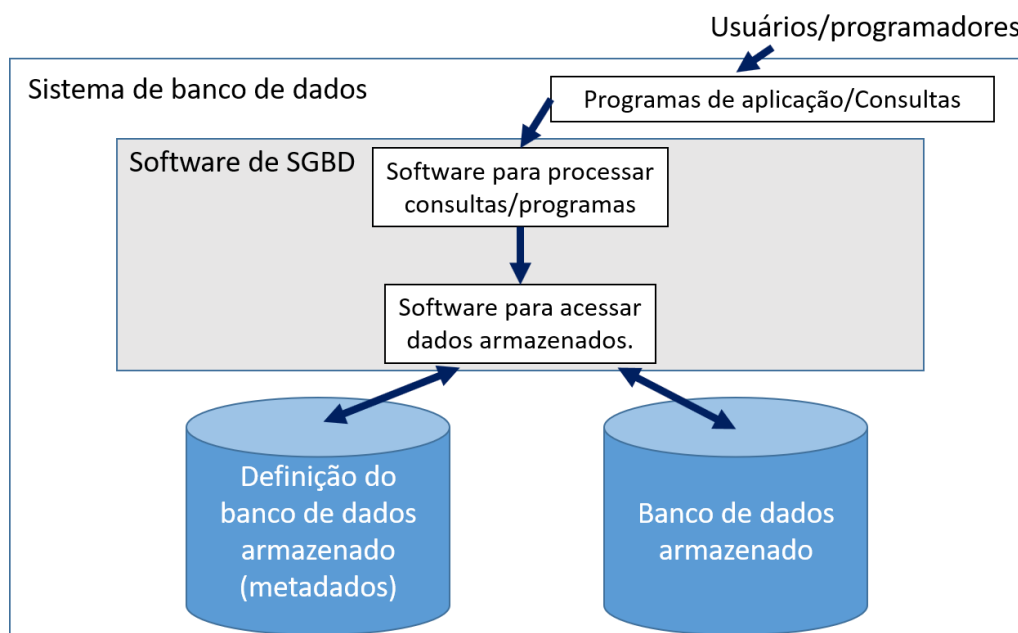


Figura 226 - Sistema gerenciador de banco de dados

Conceitos básicos de bancos relacionais



Vamos ver alguns conceitos básicos começando por tabelas.

O que é uma tabela? Em um banco de dados relacional, uma tabela é o local onde armazenamos os nossos dados. Por exemplo, podemos ter uma tabela para armazenar as informações de nossos clientes, fornecedores e vendas (no caso de um sistema de uma empresa), ou de nossos amigos (no caso de uma agenda pessoal). Uma tabela é composta por colunas (campos) e linhas (registros). As colunas são definidas no momento da criação da tabela (podendo ser removidas ou adicionadas novas colunas), antes de criar uma coluna precisamos saber o que será armazenado nela, ou seja, o tipo de dados. Já as linhas são os dados, também conhecidos como “registros” que são inseridos na tabela.

Antes de criar uma tabela precisamos saber qual sua finalidade e quais informações vamos armazenar na mesma.

Veja este exemplo:

Precisamos criar uma tabela para guardar os clientes de um pequeno estabelecimento comercial. Já sabemos o nome da tabela, que será “cliente”. Precisamos armazenar o nome (texto), o número do telefone (texto) e o e-mail (texto), precisamos também criar um campo que será o identificador único para o registro no banco de dados. Este campo será chamado id (identificador) e será um campo numérico sem casas decimais (inteiro). O campo de e-mail poderia ser utilizado como identificador único, porém, utilizar um campo de texto para identificação do registro não é aconselhado porque torna a busca da informação mais pesada, tornando o sistema mais lento.

Aqui entra o conceito de “chave primária” que é o nosso campo único (que não se repete em toda tabela), no nosso exemplo é o campo “id”.

Veja um exemplo de perguntas simples a serem feitas antes de criarmos a tabela.

NOME TABELA	CLIENTES		
QUAL O NOME DO CAMPO?	O QUE VAI ARMAZENAR?	QUAL É O TAMANHO MÁXIMO?	É A CHAVE PRIMÁRIA?
ID	NÚMERO INTEIRO		SIM
NOME	TEXTO	100	
TELEFONE	TEXTO	14	
E-MAIL	TEXTO	100	

SQL

SQL (Structured Query Language ou Linguagem Estruturada de Consulta) é uma linguagem utilizada para “conversar” com um gerenciador de banco de dados. Usamos comandos SQL para inserir, alterar, consultar e excluir registros em tabelas de bancos de dados. Também usamos comandos SQL para criar e realizar manutenção em elementos do banco de dados, como tabelas, índices, colunas, etc.

Veremos comandos SQL nas próximas aulas.

Baixando e instalando o MySQL

Para baixar o MySQL acesse: <https://dev.mysql.com/downloads/windows/installer/8.0.html>

Vamos baixar a versão Community. Que possui licença baseada na GPL sendo assim, podemos utilizar em nossos exemplos sem termos que comprar uma licença para isso.

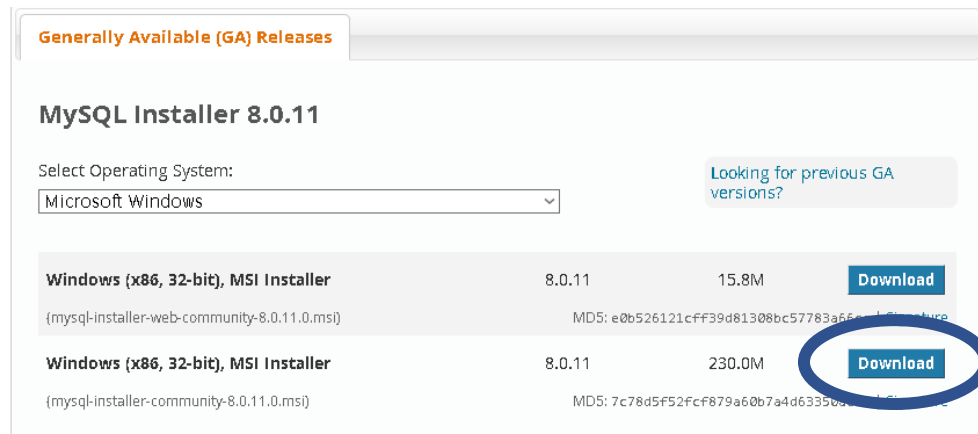


Figura 227 - Baixando o instalador do MySQL

Na página seguinte você pode efetuar login, ou se cadastrar, ou simplesmente clicar em “Não obrigado, basta iniciar meu download” e aguardar o término do download.

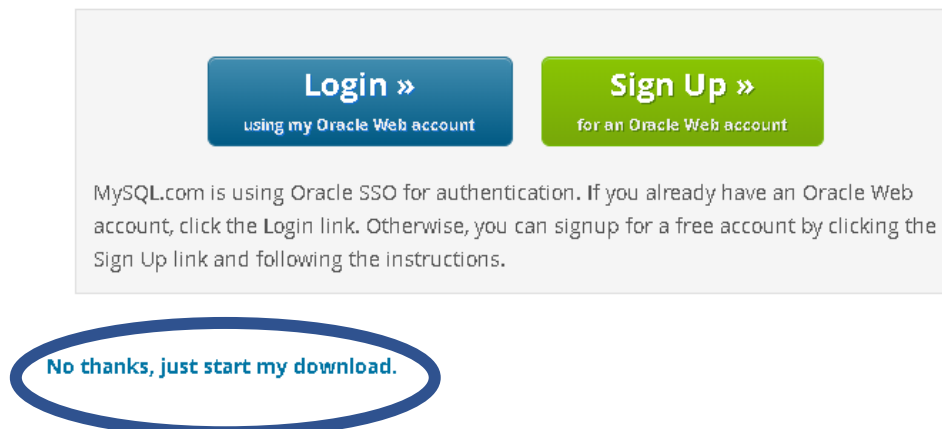


Figura 228 - Respondendo a pergunta para iniciar o download

Após o download execute o arquivo mysql-installer-community-8.0.11.0.msi (versão disponível na data de gravação desta aula), caso apareça alguma confirmação do Windows responda “Sim”.

A primeira tela exibida é a tela da licença, marque a opção informando que aceita os termos e clique em “Next”.

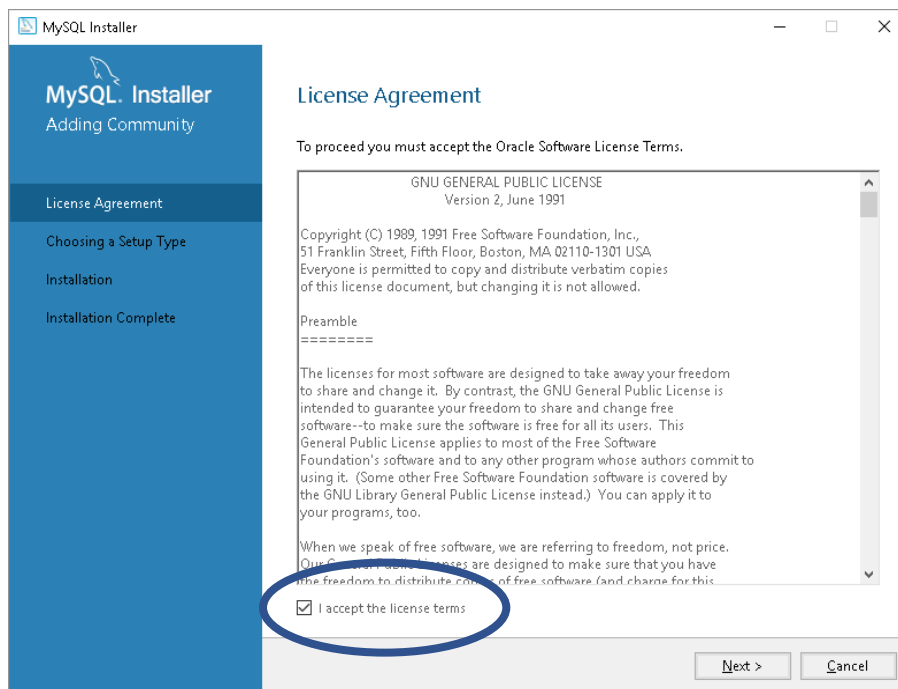


Figura 229 - Aceitando os termos da licença

Na janela seguinte escolha a opção “Custom”, para selecionar os itens a serem instalados e clique em “Next”.

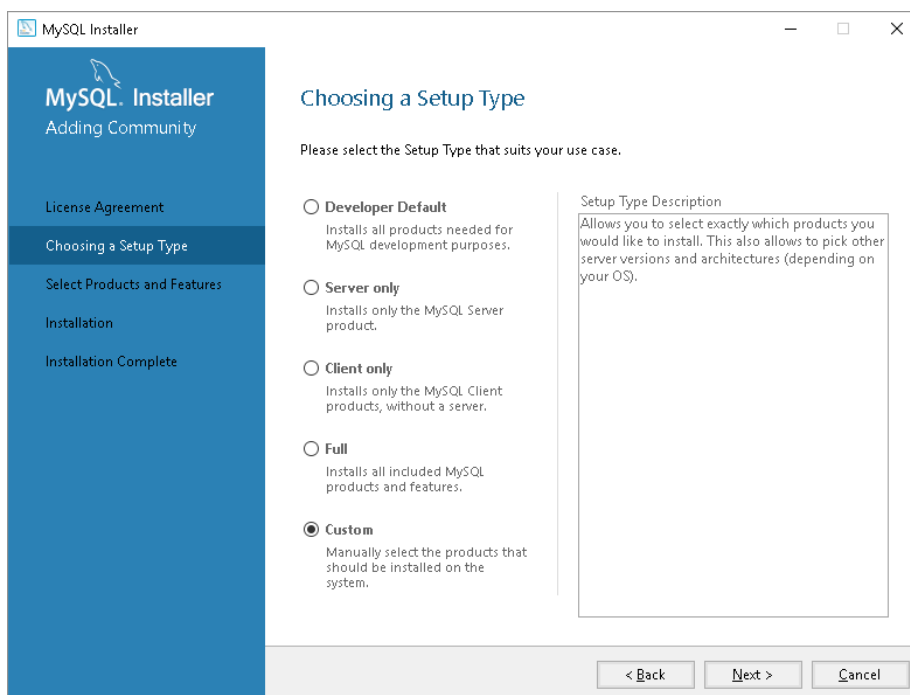


Figura 230 - Escolhendo o tipo de instalação

Temos que selecionar os produtos ou recursos à esquerda e clicar na seta jogando para a direita. Os itens que selecionei estão na imagem. Ao finalizar clique em “Next”.

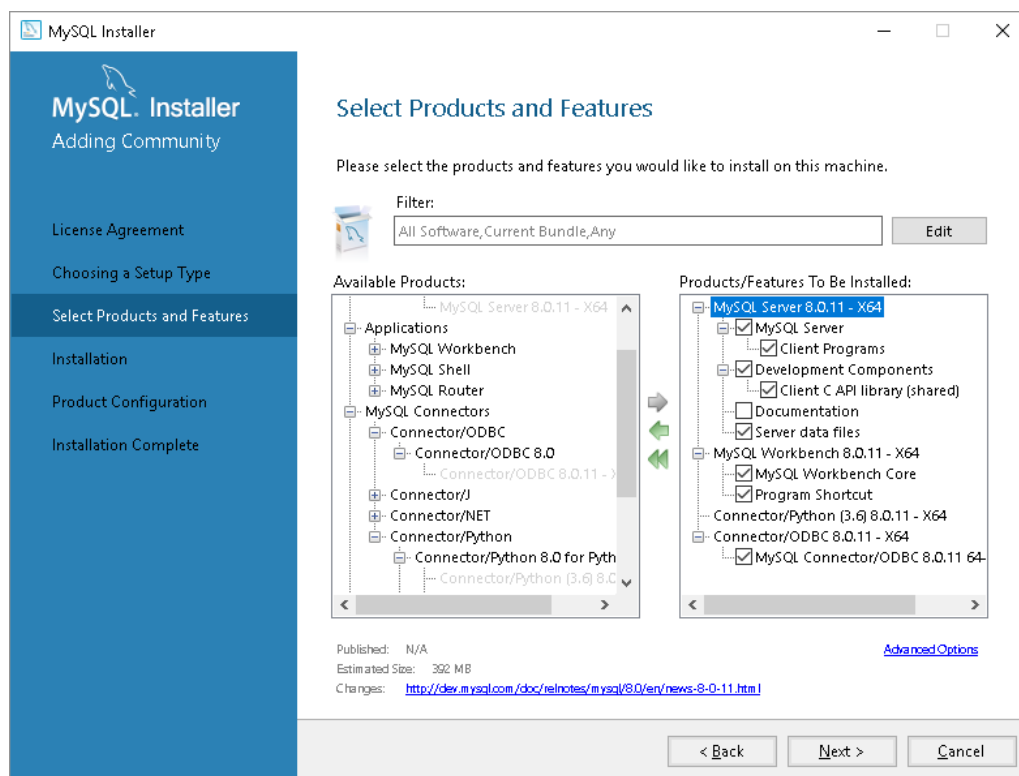


Figura 231 - Selecionando produtos e recursos

Na tela seguinte será exibido um resumo da instalação.

Clique em “Execute”. E ao término da instalação clique em “Next” novamente.

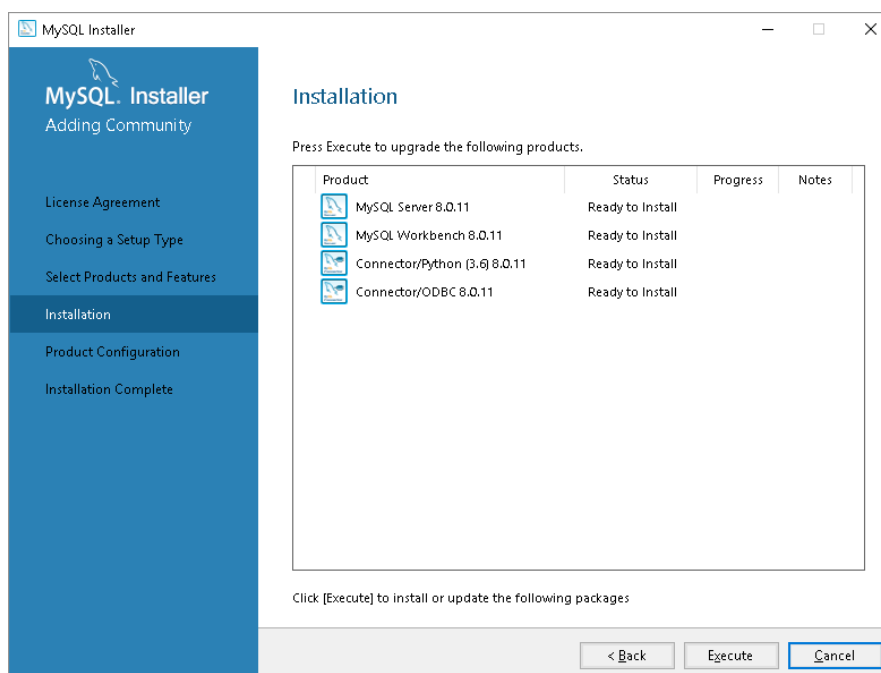


Figura 232 - Resumo da instalação

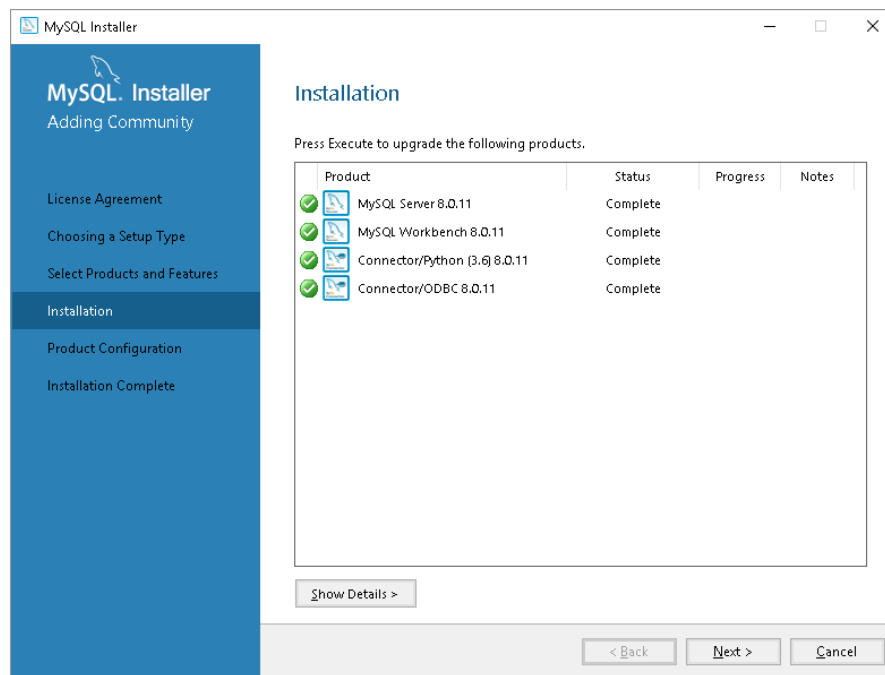


Figura 233 - Instalação concluída

Na tela seguinte clique novamente em “Next” e na próxima, “Next” também.

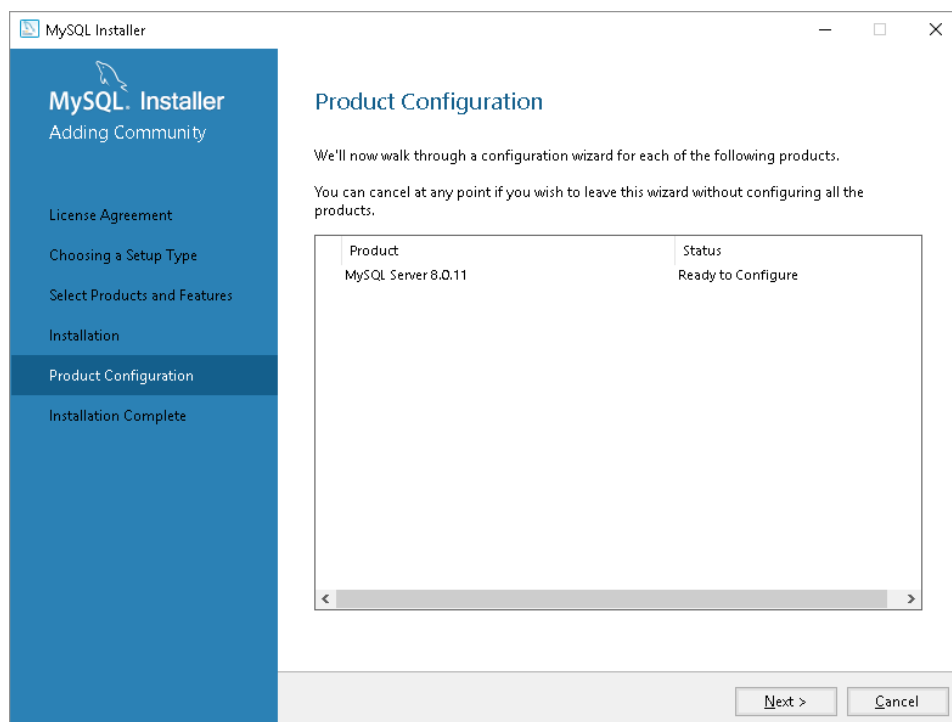


Figura 234 - Configuração do MySQL

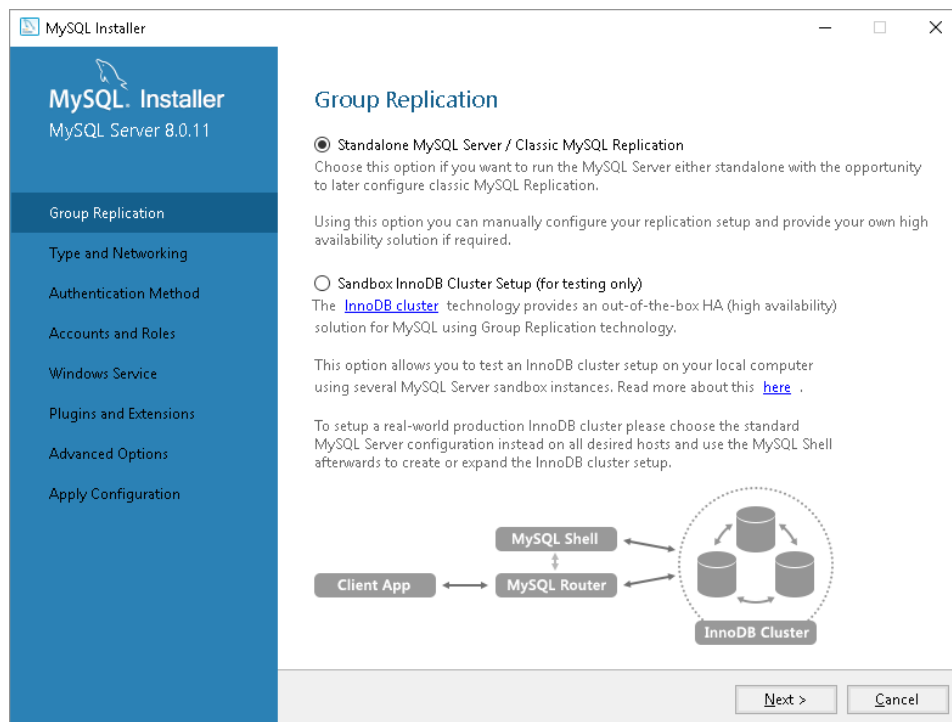


Figura 235 - Configuração de replicação

Na tela seguinte mantenha a configuração como “Development Computer” e mantenha a porta padrão TCP/IP 3306. Pode também deixar marcada a opção para abrir a porta 3306 no Firewall do Windows. Clique em “Next”.

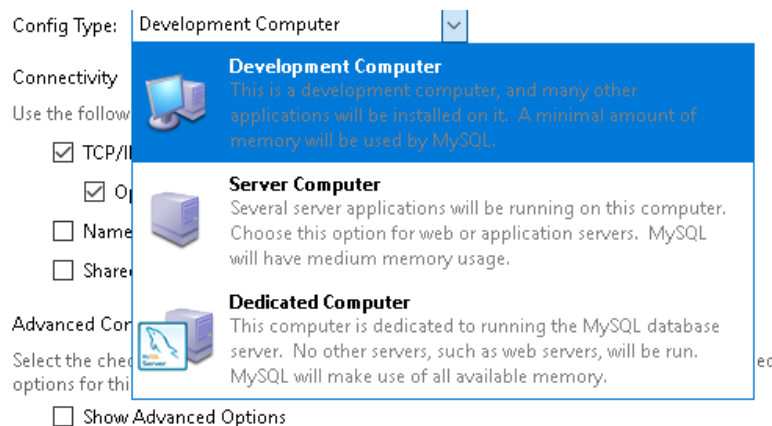


Figura 236 - Tipo de servidor

Use the following controls to select how you would like to connect to this server.

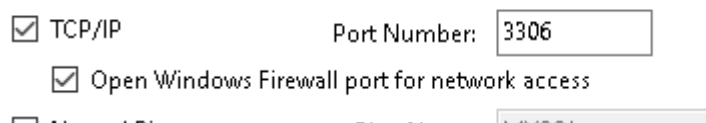


Figura 237 - Porta de conexão ao MySQL

Na tela seguinte temos que selecionar o método de autenticação. Pode deixar o método de autenticação forte. Clique em “Next”.

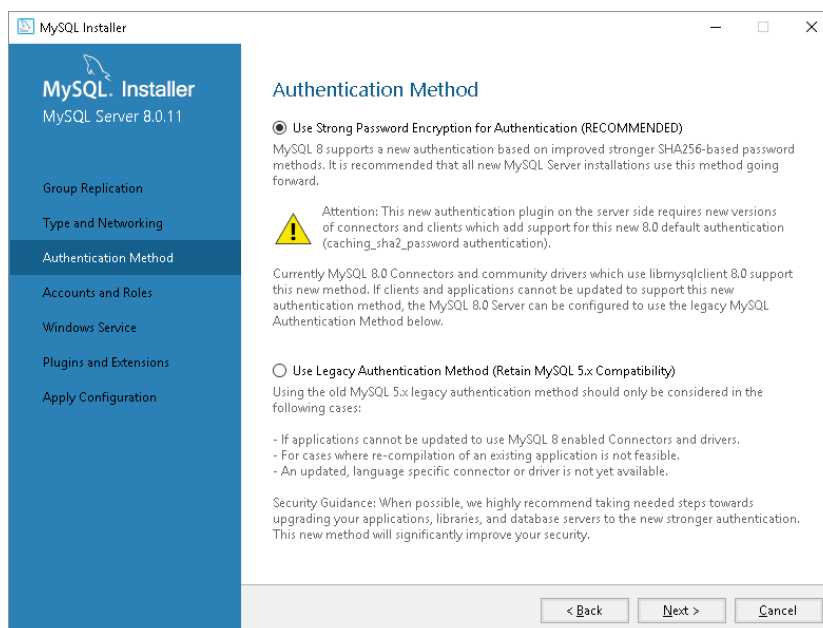


Figura 238 - Método de autenticação

Na tela seguinte informe a senha para conexão ao banco de dados e repita a senha. Coloque uma senha que não esqueça e clique em “Next”.

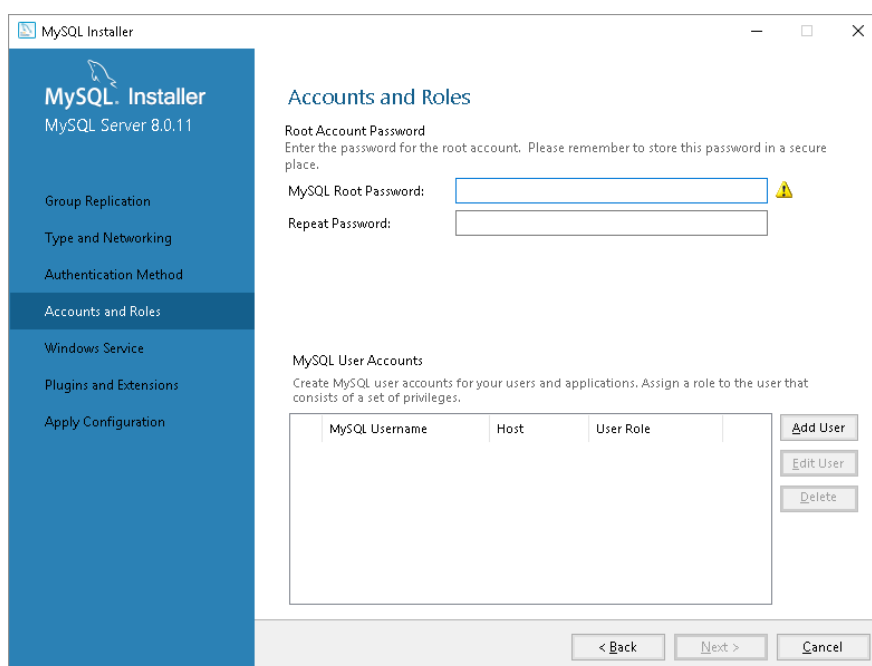


Figura 239 - Contas e papéis

Agora temos a configuração do MySQL como serviço onde são selecionados o nome do serviço, se o serviço vai iniciar com o Windows e qual usuário vai iniciar o serviço. Pode manter o padrão e clicar em “Next”.

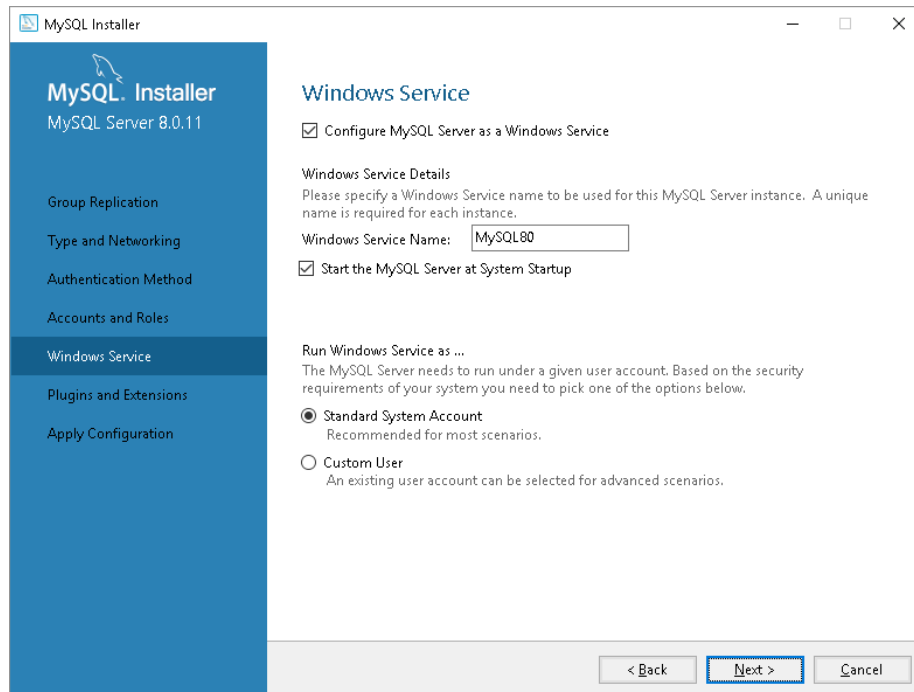


Figura 240 - Configurando como serviço

Na próxima tela podemos habilitar o MySQL como um “Document Store”, que é um recurso novo para trabalhar como um banco NoSQL. Não vamos utilizar por enquanto, então não vamos ativar. Somente clique em “Next”.

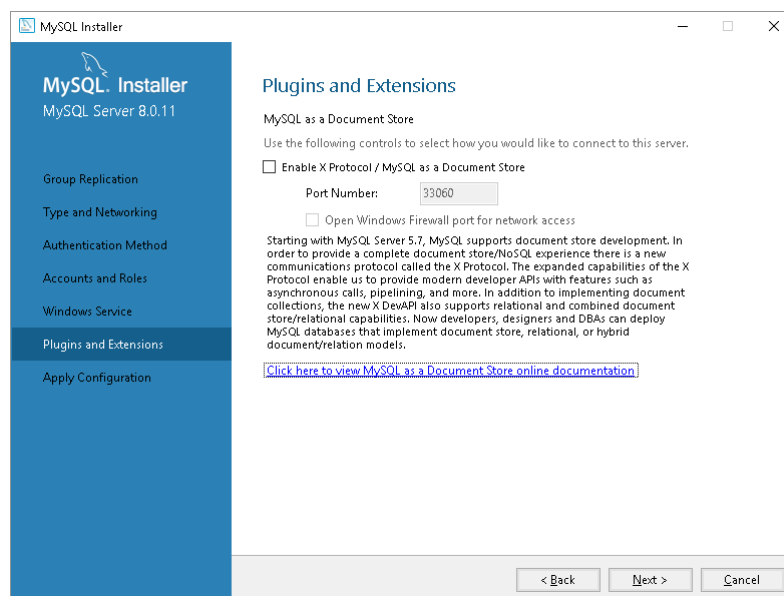


Figura 241 - Plugins e extensões

Na tela seguinte clique em “Execute” para aplicar a configuração.

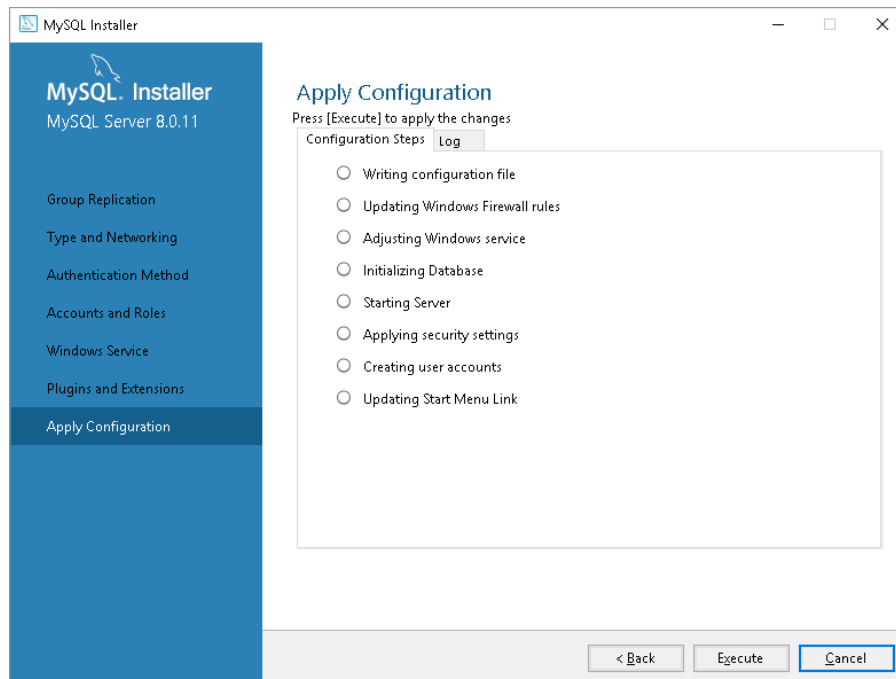


Figura 242 - Aplicando a configuração

Finalizadas as configurações, clique em “Finish” e na tela exibida a seguir, clique em “Next”.

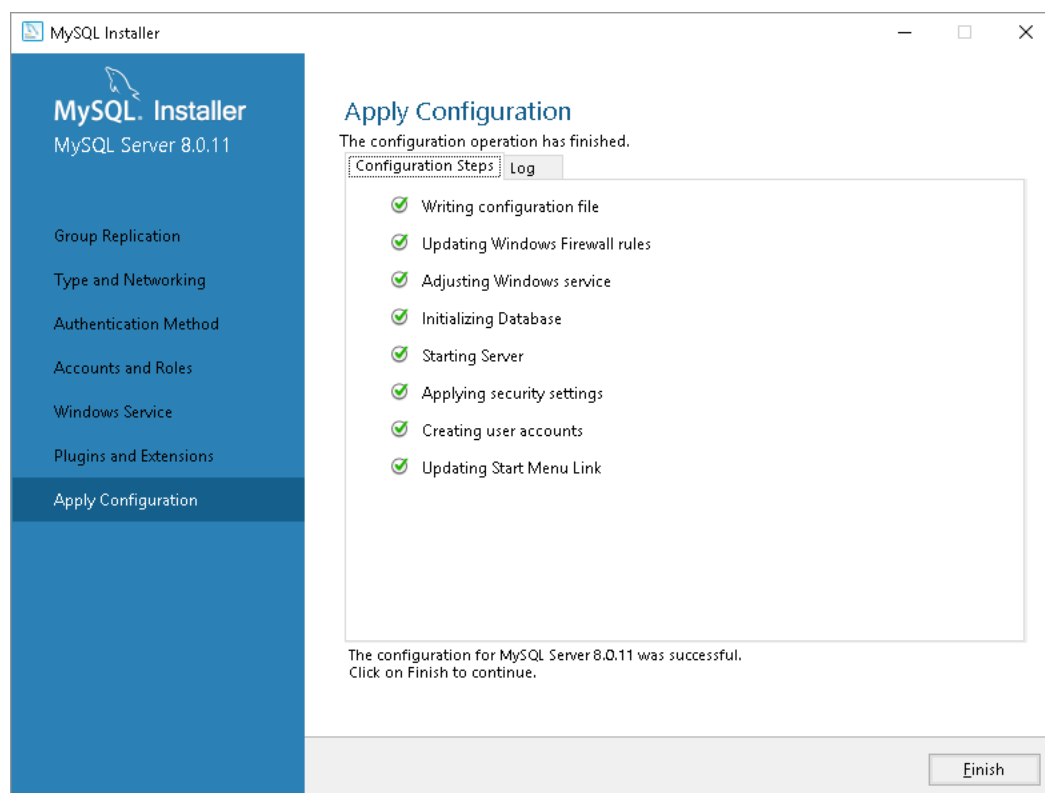


Figura 243 - Aplicando a configuração

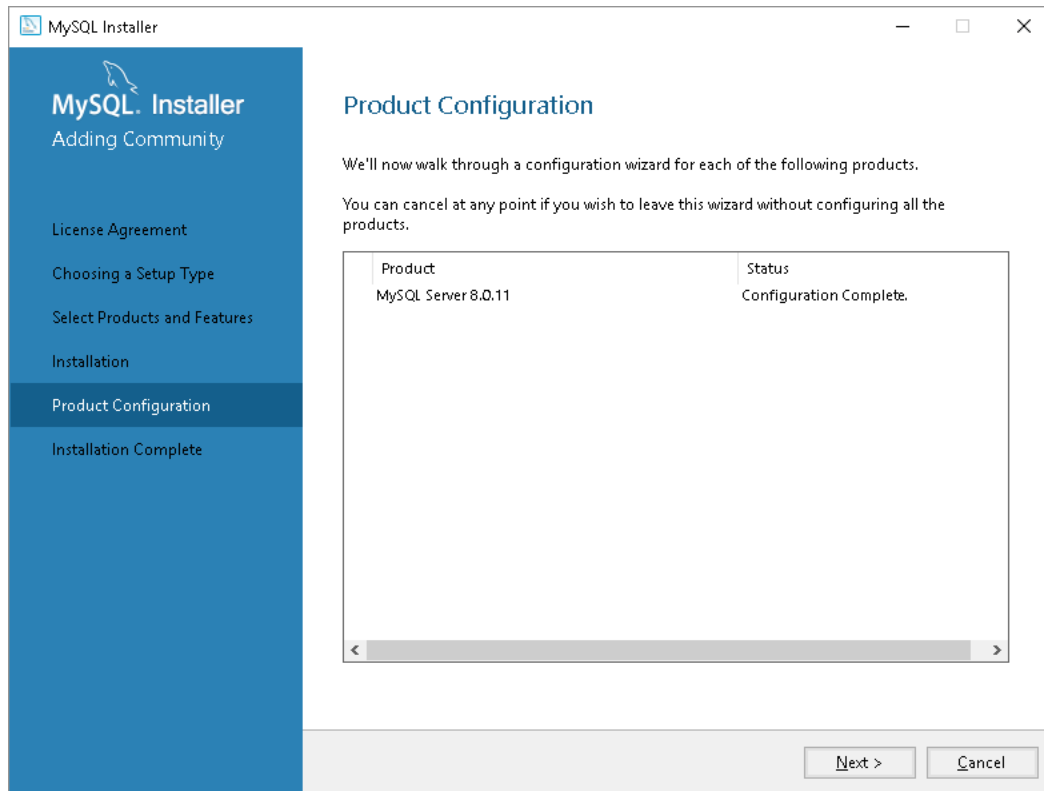


Figura 244 - Configuração do produto

Finalizada a instalação podemos escolher se o instalador deve iniciar o MySQL Workbench, que é a ferramenta que utilizamos para manipular os bancos de dados. Deixe marcado e clique em “Finish”.

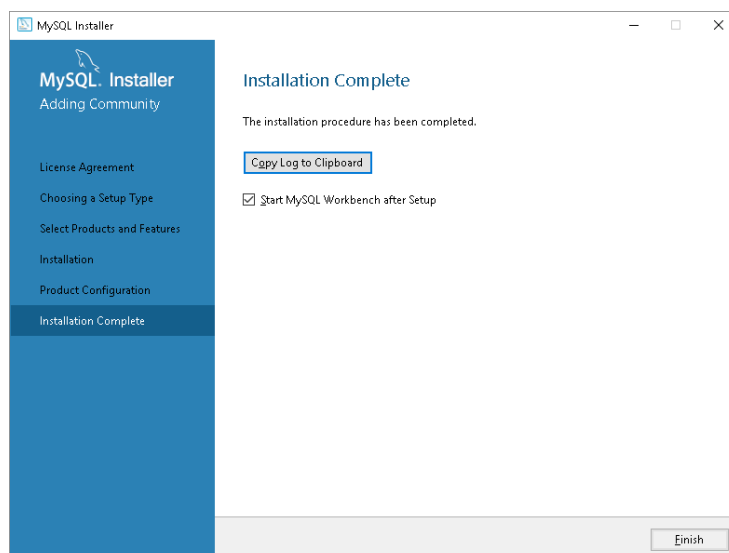


Figura 245 - Instalação concluída

Será aberto o programa MySQL Workbench. Veja na segunda imagem a opção para executar no MySQL Workbench no menu iniciar do Windows.

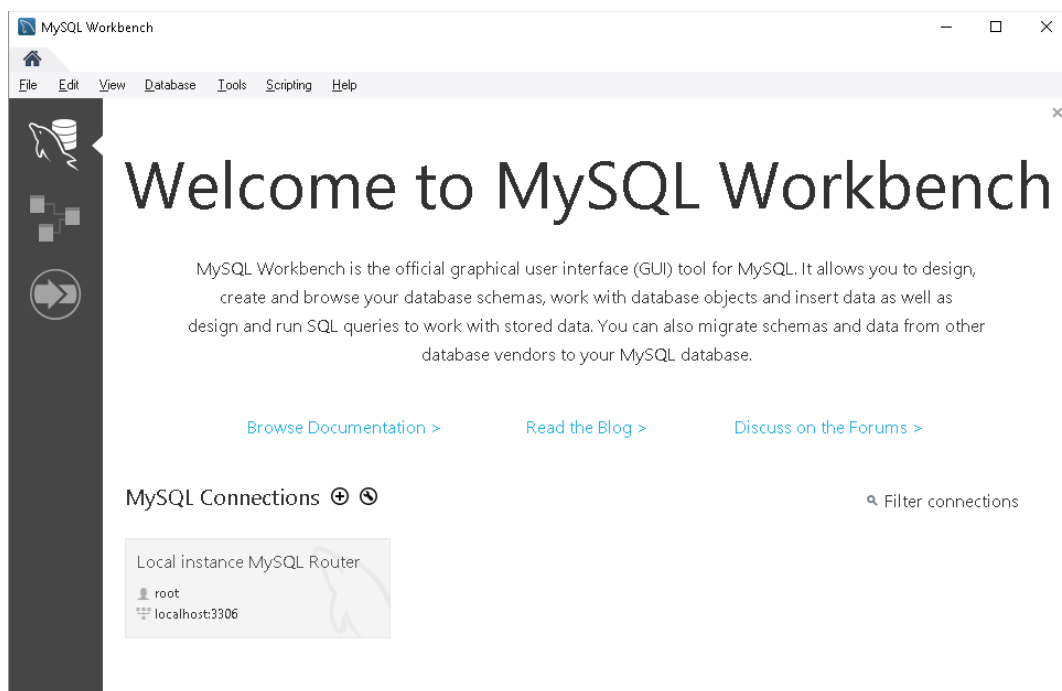


Figura 246 - MySQL Workbench

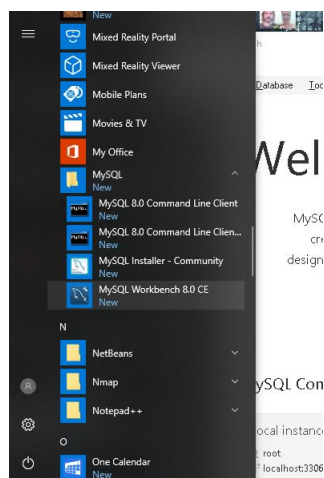


Figura 247 - MySQL no menu iniciar

Instalando o MySQL no Linux Mint 18.3

Nesta aula vamos realizar a instalação do MySQL na distribuição Linux Mint 18.3 que é baseada no Ubuntu 16.04 LTS.

O primeiro passo é acessar <https://dev.mysql.com/downloads/repo/apt/> e baixar o arquivo `mysql-apt-config_0.8.10-1_all.deb` (arquivo disponível no momento da gravação da aula). Para isso clique em “Download” e na próxima página clique em “No thanks, just start my download”.

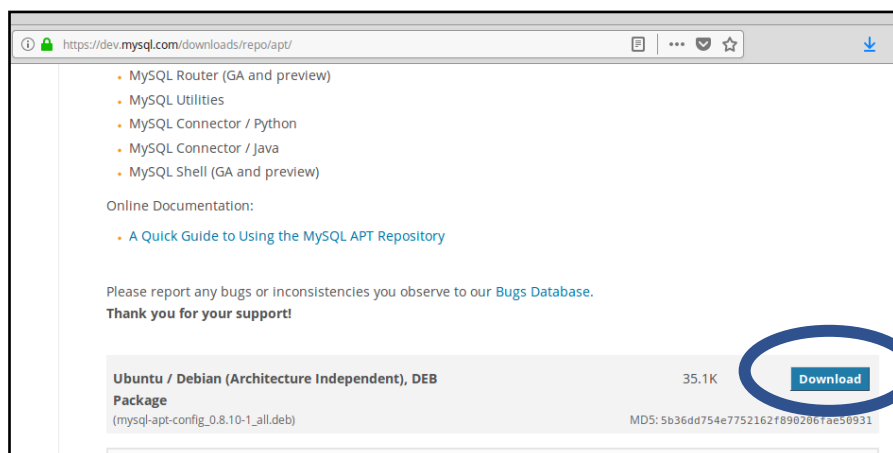


Figura 248 - Baixando MySQL para Linux

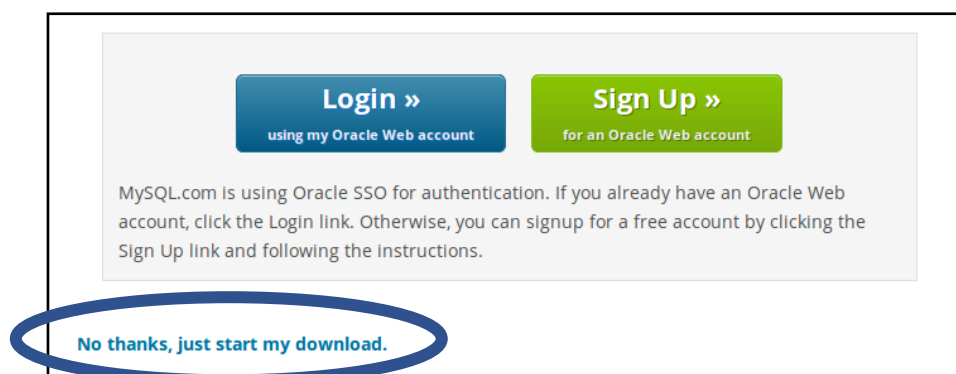


Figura 249 - Baixando MySQL para Linux

Finalizado o download, execute o arquivo baixado. Ao executar o arquivo será exibida a janela de instalação. Clique em “Install Package” e informe a senha de root para iniciar a instalação. Na janela seguinte escolha “ubuntu xenial”, clique em “Forward”. Na próxima tela deixe selecionado “MySQL Server & Cluster (Currently selected: mysql-8.0)” e clique em “Forward”.

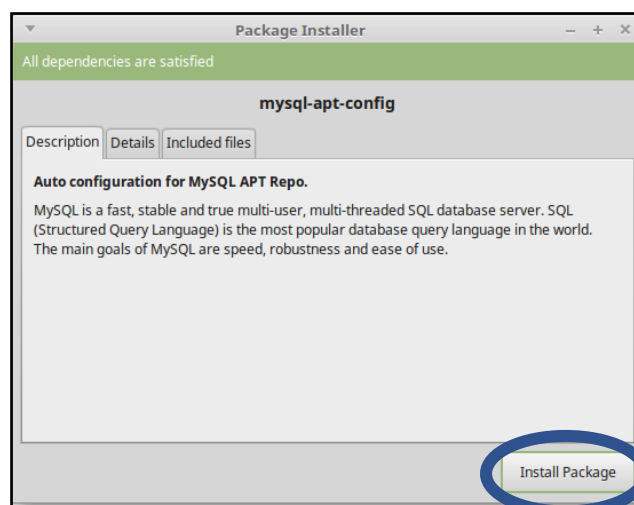


Figura 250 – Instalando o MySQL no Linux

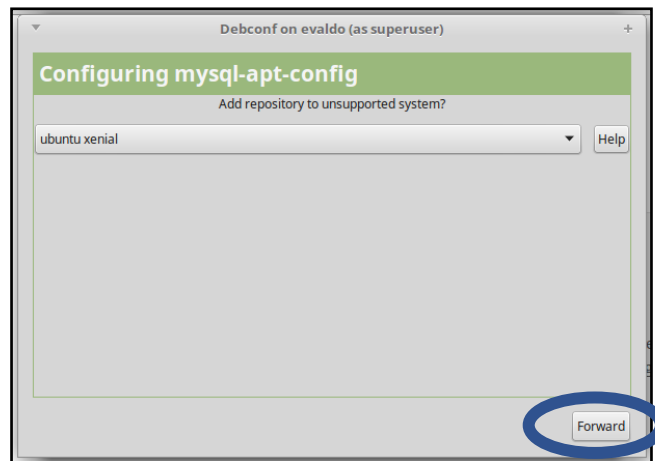


Figura 251 – Instalando o MySQL no Linux

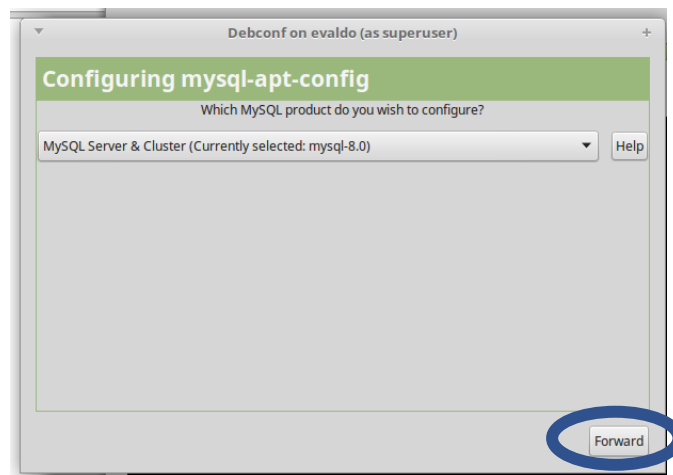


Figura 252 – Instalando o MySQL no Linux

Na tela seguinte, deixe selecionado “mysql-8.0” e clique em “Forward”. E, ao voltar para a tela de configuração, clique na lista e selecione “Ok”. Clique em “Forward” e clique em close na tela seguinte. Feche a janela do instalador.

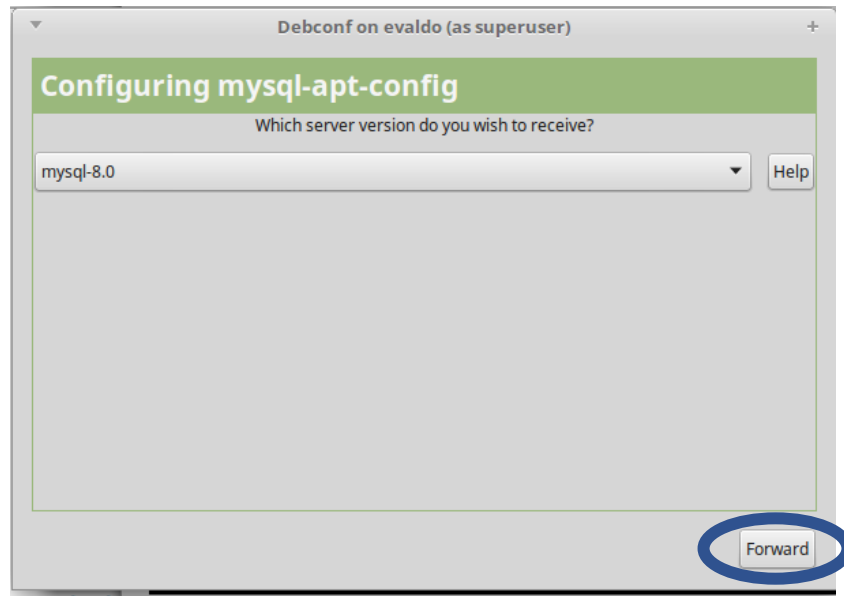


Figura 253 – Instalando o MySQL no Linux

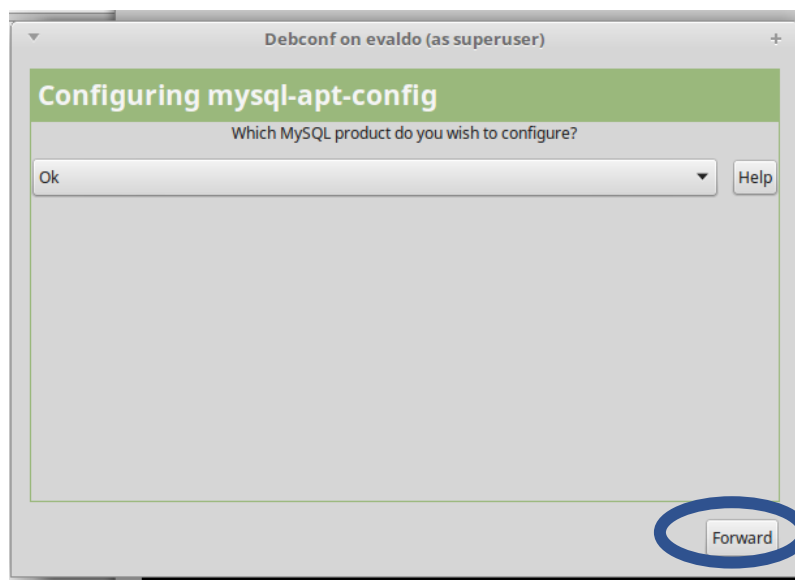


Figura 254 – Instalando o MySQL no Linux

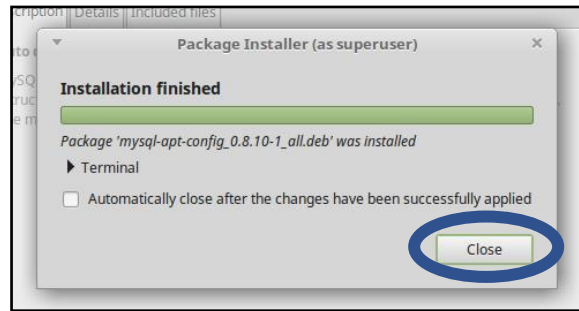


Figura 255 – Instalando o MySQL no Linux

Agora você tem que abrir o terminal e instalar o MySQL com os comandos a seguir:

\$ sudo apt-get update

\$ sudo apt-get install mysql-server

Na instalação será necessário realizar configurações como foi feito no Windows.

Na primeira tela você tem que informar a senha para acesso ao MySQL e pressionar a tecla “enter”. Deverá repetir a senha e pressionar “enter” novamente

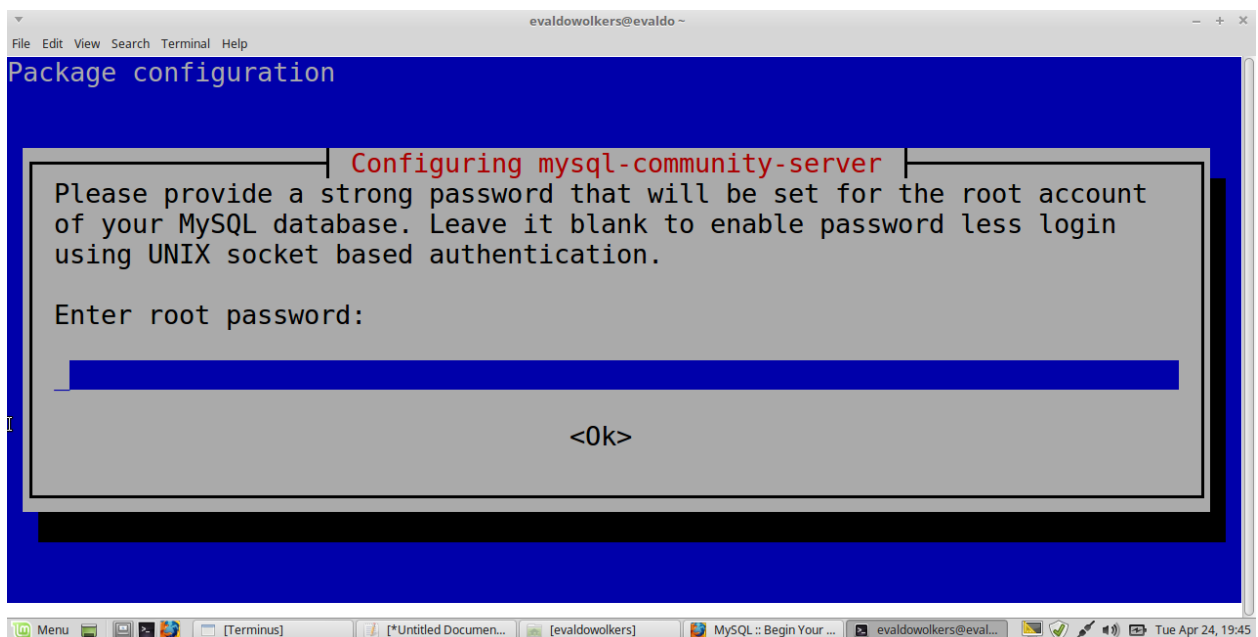


Figura 256 – Instalando o MySQL no Linux

A próxima tela é um aviso. Pressione a tecla “tab” para que o foco esteja na opção “ok” e pressione “Enter”.

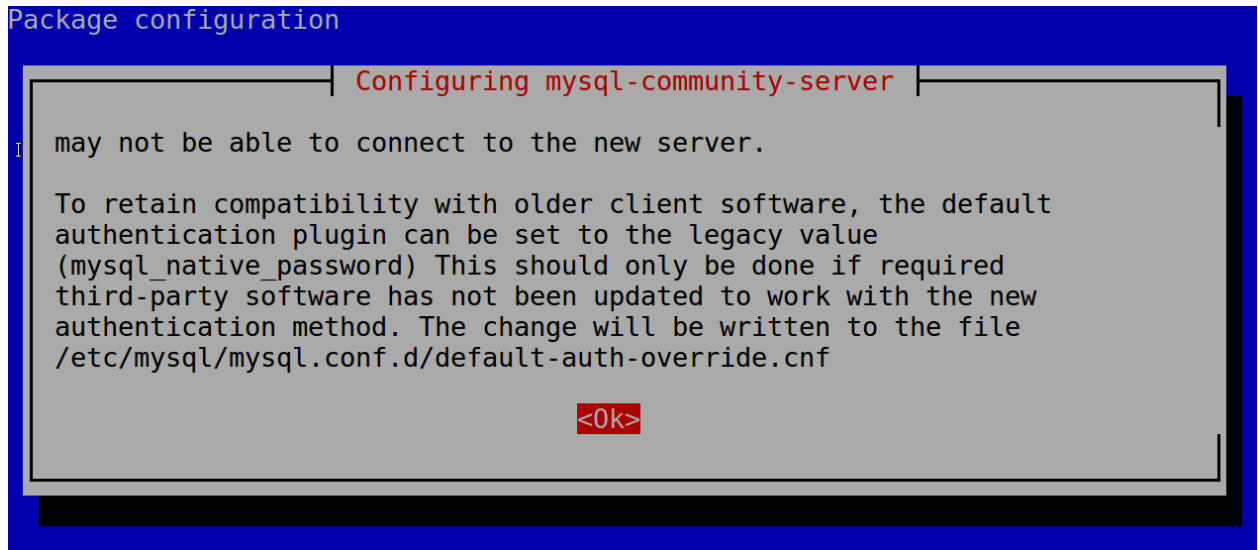


Figura 257 – Instalando o MySQL no Linux

Marque a opção “Use Legacy Authentication Method...” e pressione “Enter”.

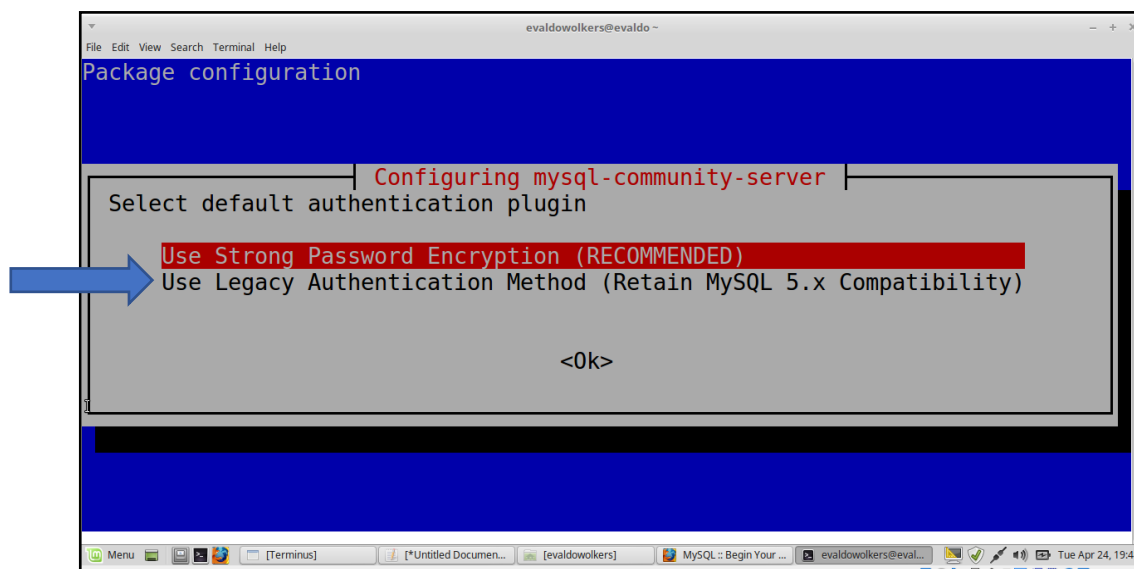


Figura 258 – Instalando o MySQL no Linux

Inicie o MySQL com o comando a seguir:

```
$ sudo service mysql start
```

Por fim, instale o Workbench.

```
$ sudo apt-get install mysql-workbench-community
```

Após a instalação, abra o Workbench acessando o menu “Programming”.

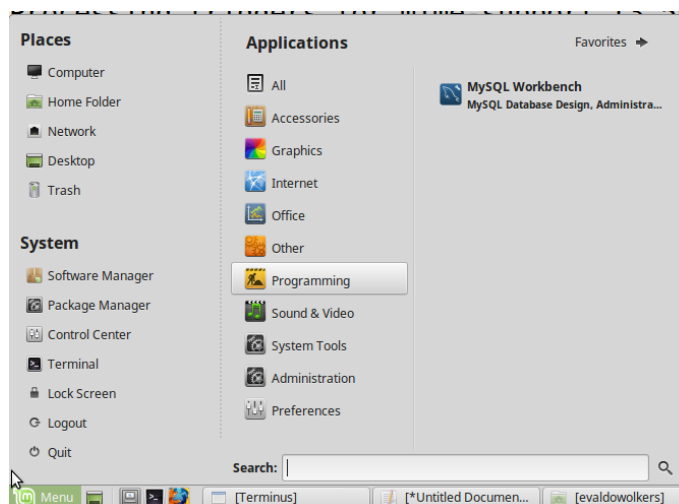


Figura 259 – Instalando o MySQL no Linux

Veja a tela do Workbench.

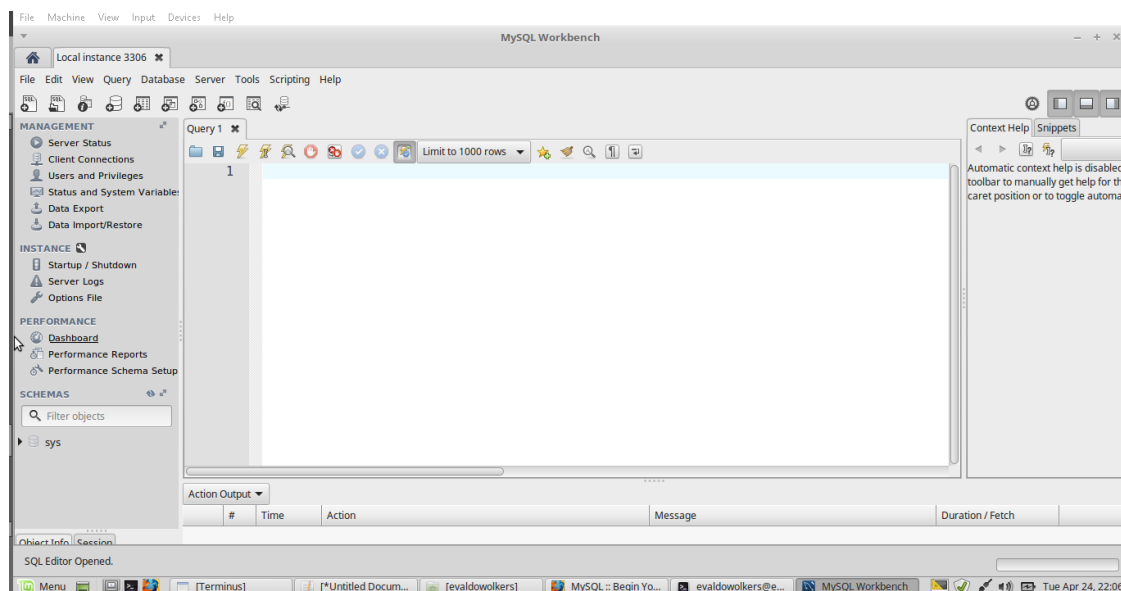


Figura 260 – Instalando o MySQL no Linux

Trabalhando com o MySQL Workbench

Sobre o MySQL Workbench

O MySQL Workbench é uma ferramenta visual unificada para arquitetos de banco de dados, desenvolvedores e administradores (DBAs). O MySQL Workbench fornece modelagem de dados, desenvolvimento SQL e ferramentas abrangentes de administração para configuração de servidores, administração de usuários, backup de banco de dados e muito mais.

Acessando o MySQL Workbench

Para acessar o MySQL Workbench no Windows, basta ir no menu iniciar, grupo MySQL e clicar em “MySQL Workbench 8.0 CE” e no Linux, no menu Programming escolha “MySQL Workbench” ou no terminal digite “mysql-workbench” sem as aspas.

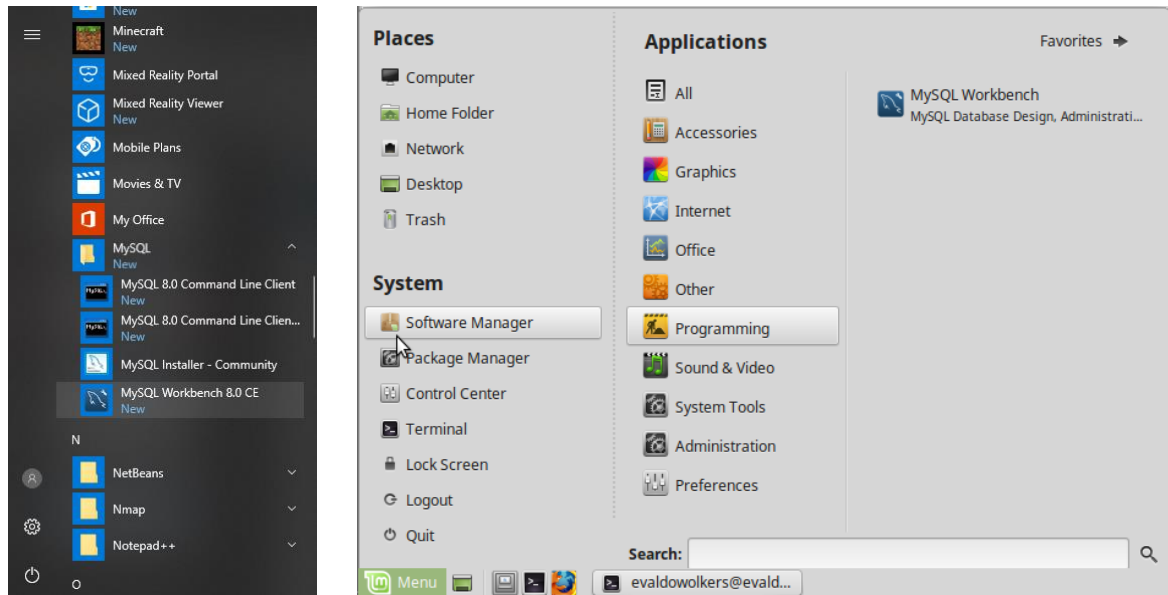


Figura 261 - Acessando o MySQL Workbench

A interface do MySQL Workbench

Janela principal do MySQL Workbench com mensagem de boas vindas.

Você pode conectar-se à instância local do MySQL dando dois cliques na conexão exibida conforme imagem ao lado. Caso seja solicitada a senha, informe a senha criada na instalação do MySQL.

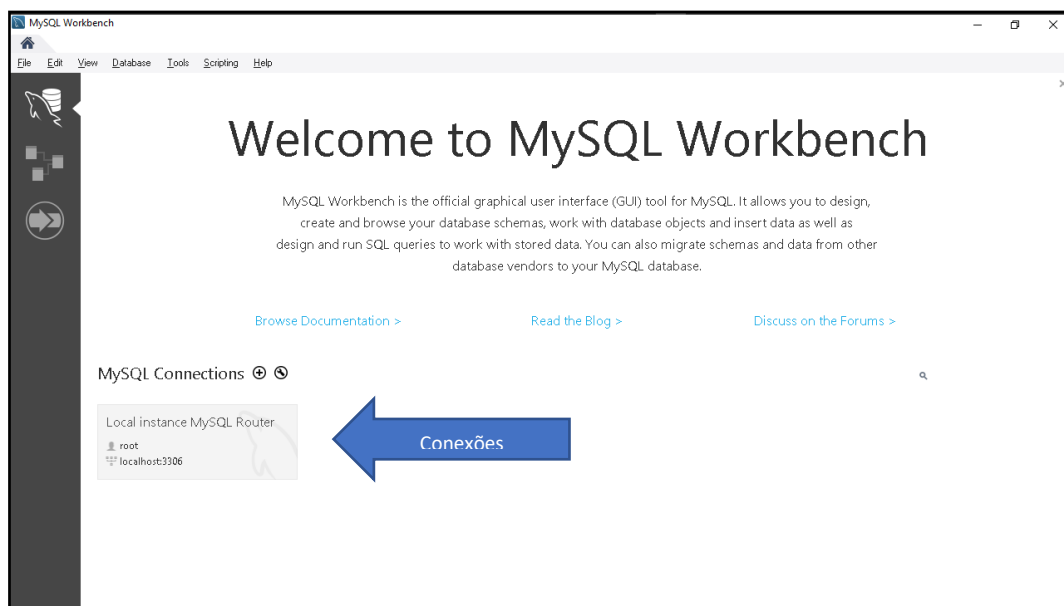


Figura 262 - Janela do MySQLWorkbench

Após se conectar à instância local, será exibida a janela a seguir.

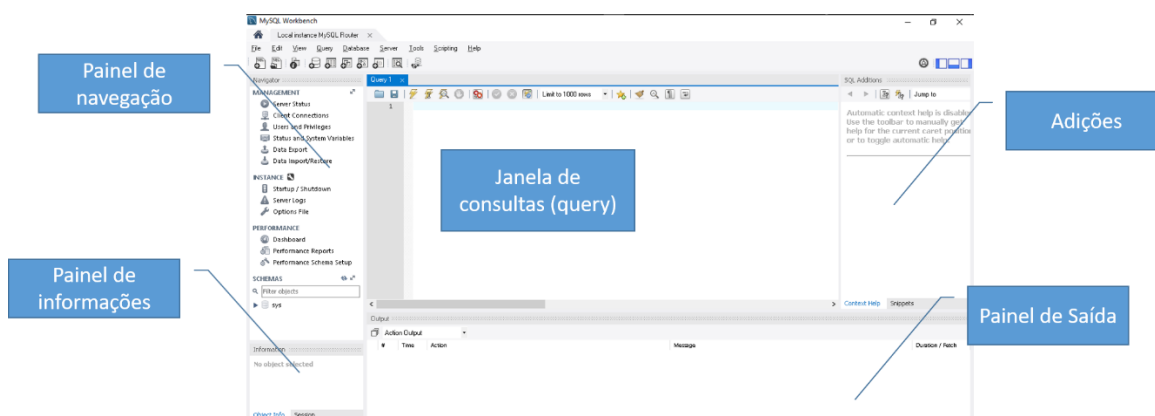


Figura 263 - MySQL Workbench

Criando um schema

Vamos criar o nosso banco de dados, ou “esquema” (schema) como é denominado o banco de dados no MySQL.

Para isso vá até o painel de navegação, onde está o Schema sys e clique com o botão direito do mouse na área vazia conforme imagem ao lado e escolha “Create Schema”.

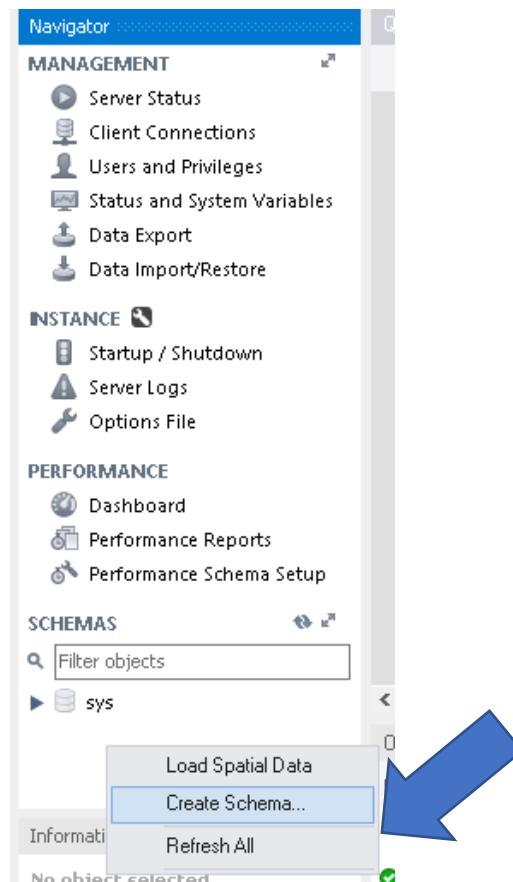


Figura 264 - Criando um schema

Será exibida uma guia com dados para criação do esquema.

No campo “Name:” informe um nome para nosso esquema, vamos chamá-lo de “agenda”.

Em seguida, clique em “Apply”. Na janela seguinte confirme clicando em “Apply” novamente e depois em “Finish”.

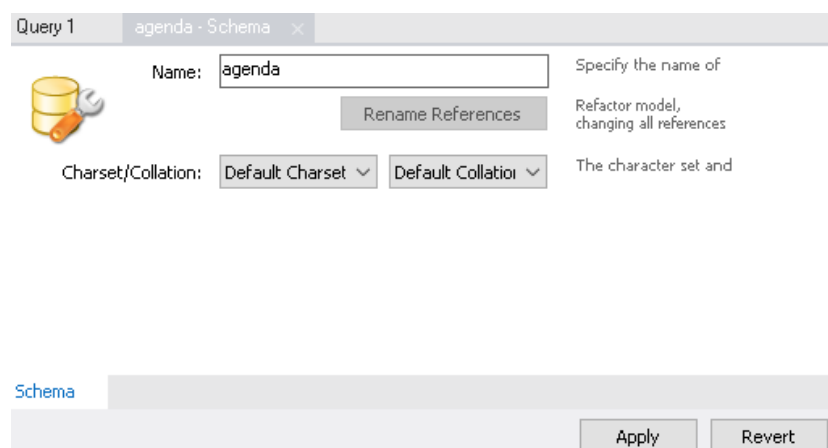


Figura 265 - Criando um schema

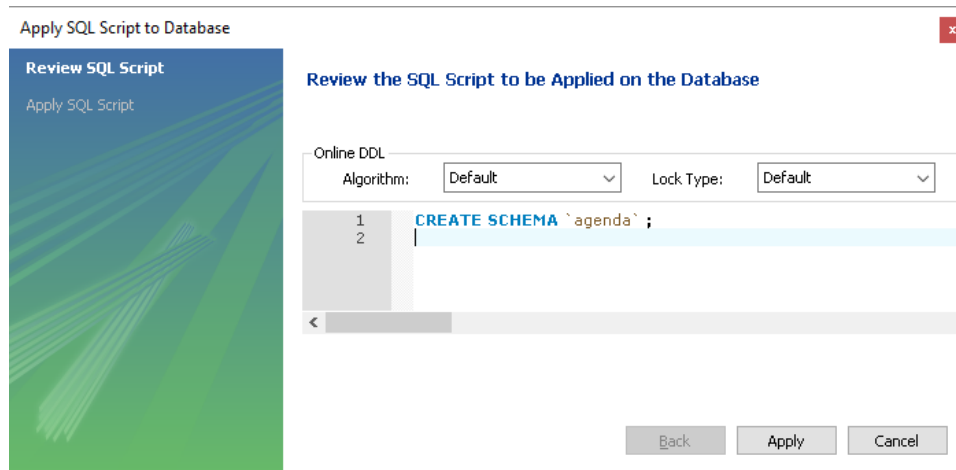


Figura 266 - Criando um schema

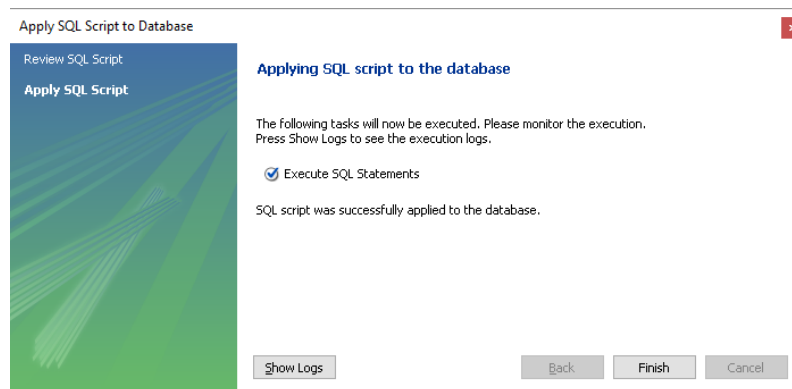


Figura 267 - Criando um schema

Observe que será exibida o esquema “agenda” no painel de navegação. Pode fechar a janela “agenda – Schema” neste momento.

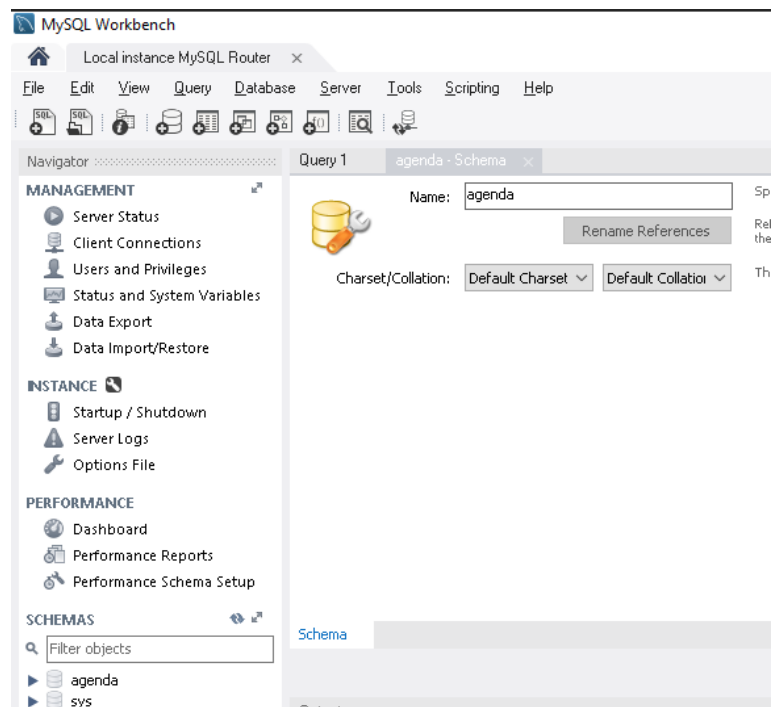


Figura 268 - Schema agenda criado

Visualizando os objetos do schema

Clique na seta ao lado esquerdo do esquema “agenda” para expandir.

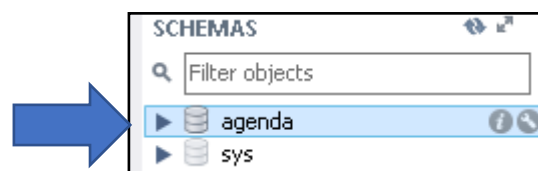


Figura 269 - Schemas

Veja que serão exibidas as seguintes opções:

Tabelas, visualizações, procedimentos armazenados e funções.

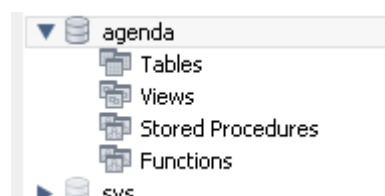


Figura 270 - Objetos do schema agenda

Criando uma tabela

Clique com o botão direito do mouse sobre “Tabelas” e escolha a opção “Criar Tabela”.

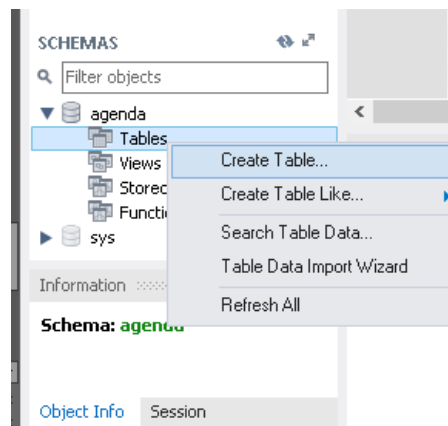


Figura 271 - Criando uma tabela

Será exibida uma guia chamada “new_table - Table” para definirmos a nossa tabela.

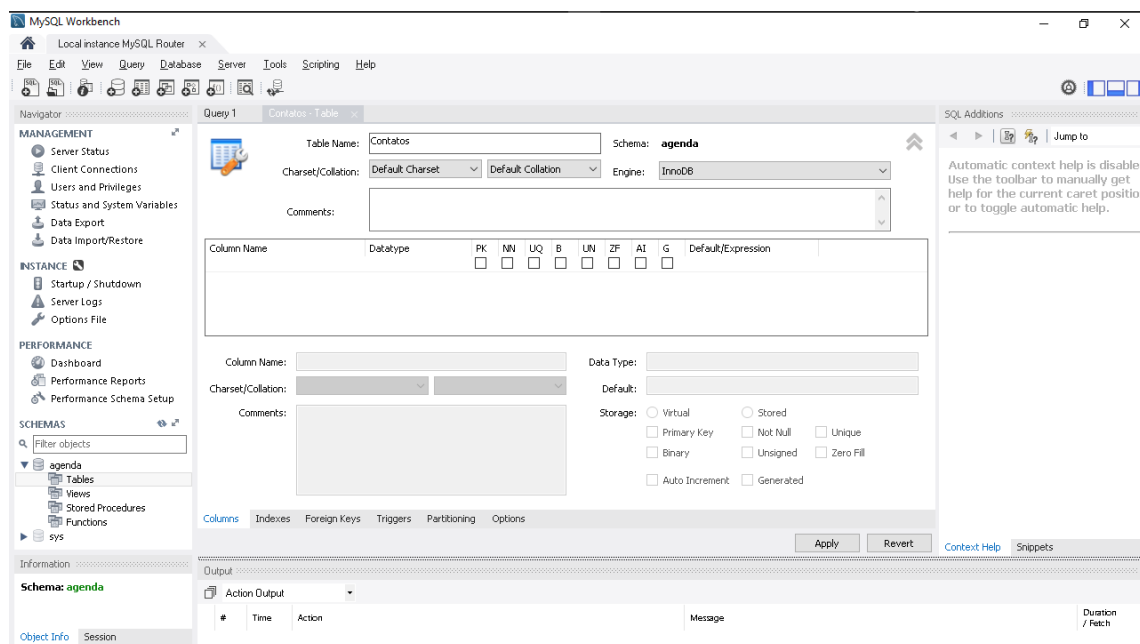


Figura 272 - Criando uma tabela

Em “Table Name:” informe “contatos” e na área para definição das colunas das tabelas defina os seguintes campos:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G
 id	INT	☒	☒	☒	☐	☐	☐	☒	☐
 nome	VARCHAR(100)	☐	☒	☐	☐	☐	☐	☐	☐
 telefone	VARCHAR(13)	☐	☐	☒	☐	☐	☐	☐	☐
 celular	VARCHAR(14)	☐	☐	☒	☐	☐	☐	☐	☐
		☐	☐	☐	☐	☐	☐	☐	☐

Figura 273 - Criando uma tabela

Tipo de dados: INT = Número Inteiro, Varchar = Texto(tamanho).

PK = Primary Key (Chave primária).

NN = Not Null (Não nulo).

UQ = Unique (Único).

AI = Auto Increment (Auto incremento – gerado automaticamente)

Após definir os campos clique em “Apply” e na janela exibida, clique em “Apply” novamente e depois em “Finish”. Pode fechar a aba “contatos – Table”.

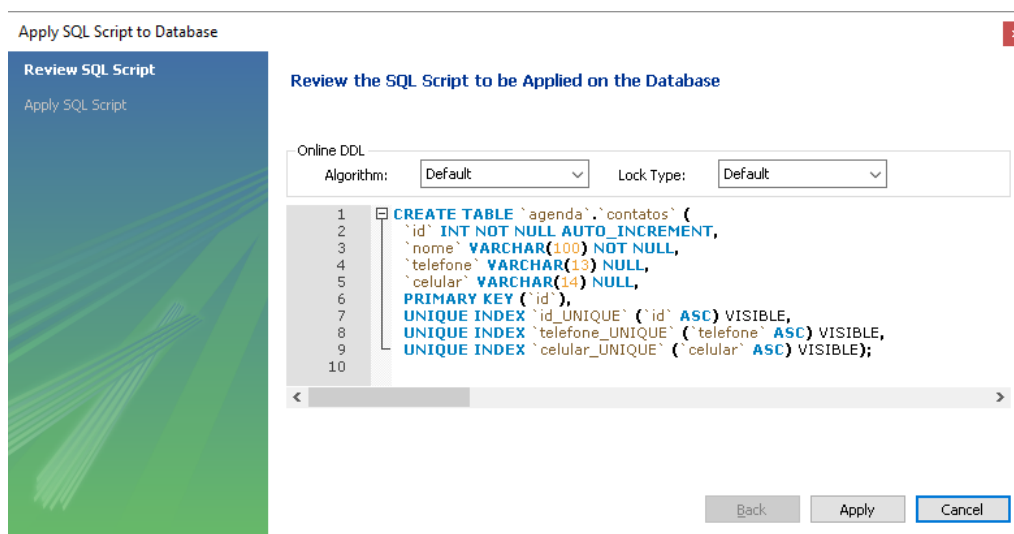
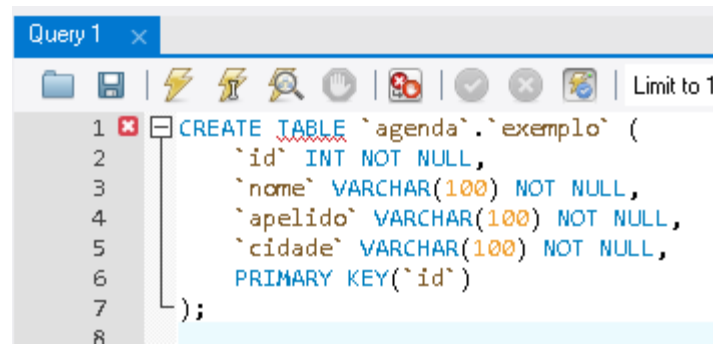


Figura 274 - Criando uma tabela

Você observou que na janela de confirmação foi exibido o comando SQL para criação da tabela “contato” no schema “agenda”.

```
CREATE TABLE `agenda`.`contatos` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `nome` VARCHAR(100) NOT NULL,
  `telefone` VARCHAR(13) NULL,
  `celular` VARCHAR(14) NULL,
  PRIMARY KEY (`id`),
  UNIQUE INDEX `id_UNIQUE` (`id` ASC) VISIBLE,
  UNIQUE INDEX `telefone_UNIQUE` (`telefone` ASC) VISIBLE,
  UNIQUE INDEX `celular_UNIQUE` (`celular` ASC) VISIBLE);
```

Figura 275 - Script da criação de tabela



```
1 CREATE TABLE `agenda`.`exemplo` (
2   `id` INT NOT NULL,
3   `nome` VARCHAR(100) NOT NULL,
4   `apelido` VARCHAR(100) NOT NULL,
5   `cidade` VARCHAR(100) NOT NULL,
6   PRIMARY KEY(`id`)
7 );
8
```

Figura 276 - Script da criação de tabela

Como exemplo, crie uma tabela denominada “exemplo” escrevendo o comando SQL da imagem a seguir na janela de consultas e execute-o clicando em .

Tipos de dados no MySQL

Antes de criar um campo de uma tabela você precisa conhecer os dados que serão armazenados para que escolha o melhor tipo de dados para o mesmo.

Vamos ver agora alguns dos tipos de dados disponibilizados pelo MySQL.

Categoria	Tipos de dados
Numeric (Numérico)	Inteiros (Valor exato): INTEGER ou INT, SMALLINT, TINYINT, MEDIUMINT e BIGINT Ponto Fixo (Valor exato): DECIMAL, NUMERIC Ponto Flutuante (Valor aproximado): FLOAT, DOUBLE
Date and Time (Data e Hora)	DATE, DATETIME e TIMESTAMP



String (Texto)

CHAR e VARCHAR

NUMÉRICO: INTEIROS (Valor exato)

Table 11.1 Required Storage and Range for Integer Types Supported by MySQL

Type	Storage (Bytes)	Minimum Value Signed	Minimum Value Unsigned	Maximum Value Signed	Maximum Value Unsigned
TINYINT	1	-128	0	127	255
SMALLINT	2	-32768	0	32767	65535
MEDIUMINT	3	-8388608	0	8388607	16777215
INT	4	-2147483648	0	2147483647	4294967295
BIGINT	8	-2 ⁶³	0	2 ⁶³ -1	2 ⁶⁴ -1

Figura 277 - Números inteiros

NUMÉRICO: PONTO FIXO (Valor exato)

Os tipos DECIMAL e NUMERIC armazenam dados numéricos exatos. Esses tipos são usados quando é importante preservar precisão exata, por exemplo com dados monetários.

No MySQL, NUMERIC é implementado como DECIMAL, então o exemplo a seguir sobre decimal, aplica-se a numeric.

Quando vamos criar uma coluna decimal ou numérica temos que definir a precisão e a escala como a seguir:

salario DECIMAL(6, 2)

Neste exemplo, 6 é a precisão e 2 a escala (casas decimais).

Podemos armazenar no campo salário a seguinte faixa de valores:

-9999.99 a 9999.99.

Se for informado zero na escala "DECIMAL(6,0)", o valor decimal não vai conter o ponto decimal ou a parte fracionária.

NUMÉRICO: PONTO FLUTUANTE (Valor aproximado)

Os tipos FLOAT e DOUBLE representam valores aproximados de dados numéricos.

Semelhante aos números de ponto fixo, definimos um campo assim:

numero FLOAT(7,4)

O campo "numero" poderá receber valores entre -999.9999 e 999.9999.

Para estes tipos de dados o MySQL faz arredondamentos. Por exemplo, se informarmos o valor 999.00009 no campo "numero" o valor aproximado será 999.0001.

Sendo assim, não podemos utilizar estes tipos de dados em se tratando de números exatos.



DATA E HORA: DATE, DATETIME e TIMESTAMP

O tipo DATE é usado para valores com uma parte de data, mas sem parte de hora. O MySQL recupera e exibe valores DATE no formato 'AAAA-MM-DD'. O intervalo suportado para este campo é entre '1000-01-01' e '9999-12-31'.

O tipo DATETIME é usado para valores que contenham partes de data e hora. O MySQL recupera e exibe valores DATETIME no formato 'AAAA-MM-DD HH:MM:SS'. O intervalo suportado é entre '1000-01-01 00:00:00' e '9999-12-31 23:59:59'.

O tipo de dados TIMESTAMP é usado para valores que contenham partes de data e hora. TIMESTAMP tem um intervalo entre '1970-01-01 00:00:01'UTC e '2038-01-19 03:14:07'UTC.

(O Tempo Universal Coordenado, abreviadamente UTC (do inglês Universal Time Coordinated), também conhecido como tempo civil, é o fuso horário de referência a partir do qual se calculam todas as outras zonas horárias do mundo.)

Um valor de DATETIME ou TIMESTAMP pode incluir uma parte fracionária de segundos à direita em até microssegundos (6 dígitos) de precisão. Em particular, qualquer parte fracionária em um valor inserido em uma coluna DATETIME ou TIMESTAMP é armazenada em vez de descartada. Com a parte fracionária incluída, o formato desses valores é 'AAAA-MM-DD HH:MM:SS[.fração]', o intervalo para valores de DATETIME é de '1000-01-01 00:00:00.000000' a '9999-12-31 23:59:59.999999', e o intervalo para valores TIMESTAMP é de '1970-01-01 00:00:01.000000' a '2038-01-19 03:14:07.999999'. A parte fracionária deve sempre ser separada do resto do tempo por um ponto decimal; nenhum outro delimitador de segundos fracionários é reconhecido.

TEXTO: CHAR E VARCHAR

Os tipos CHAR e VARCHAR são semelhantes, mas diferem na maneira como os dados são armazenados e recuperados. Ambos são declarados com um comprimento que indica o número máximo de caracteres que deseja armazenar. Por exemplo CHAR(10) pode conter até 10 caracteres.

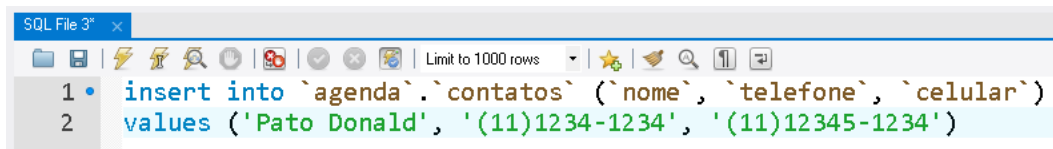
O tamanho armazenado para uma coluna CHAR será sempre o número de caracteres informado, pois esta coluna terá os valores preenchidos com espaços à direita, porém, na busca desse campo, os espaços são removidos. Ou seja, se você criar um campo CHAR de tamanho 10, mesmo que preencha o campo com o texto "Ola", o campo ocupará o espaço de 10 caracteres (10 bytes), preenchendo com espaços à direita. O comprimento de um campo CHAR pode ser definido entre 0 e 255.

Colunas do tipo VARCHAR são de tamanho variável. O comprimento pode ser especificado como um valor entre 0 e 65.535. Diferente do CHAR, os valores em campos VARCHAR são armazenados como um prefixo de 1 ou 2 bytes de comprimento mais os dados. O prefixo de comprimento indica o número de bytes no valor. Uma coluna usa um byte de comprimento se os valores (conteúdo do campo) não exigirem mais do que 255 bytes, dois bytes de comprimento se os valores exigirem mais de 255 bytes.

Inserindo dados em uma tabela

Para inserir dados em uma tabela usamos o comando "insert".

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>
contato@pythonparatodos.com.br



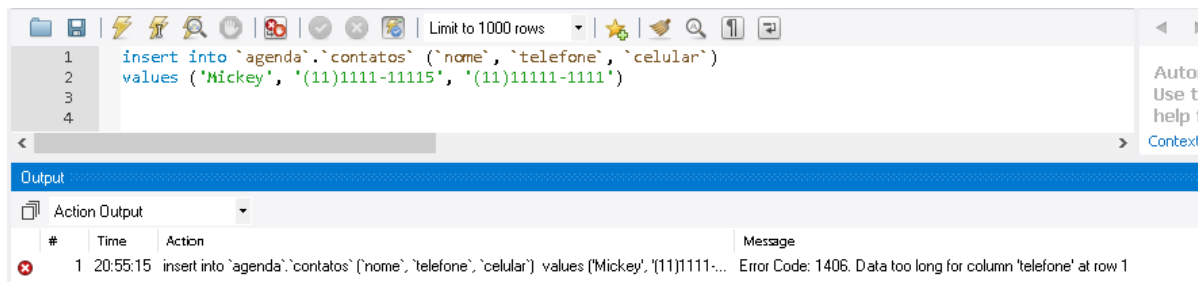
```

1 • insert into `agenda`.`contatos` (`nome`, `telefone`, `celular`)
2 values ('Pato Donald', '(11)1234-1234', '(11)12345-1234')

```

Figura 278 - Comando para inserção de dados

Gerando um erro ao tentar inserir um telefone com um tamanho maior do que o tamanho do campo (13 caracteres).



```

1 insert into `agenda`.`contatos` (`nome`, `telefone`, `celular`)
2 values ('Mickey', '(11)1111-11115', '(11)11111-1111')
3
4

```

Output

#	Time	Action	Message
1	20:55:15	insert into `agenda`.`contatos` (`nome`, `telefone`, `celular`) values ('Mickey', '(11)1111-...	Error Code: 1406. Data too long for column 'telefone' at row 1

Figura 279 - Erro ao inserir contato

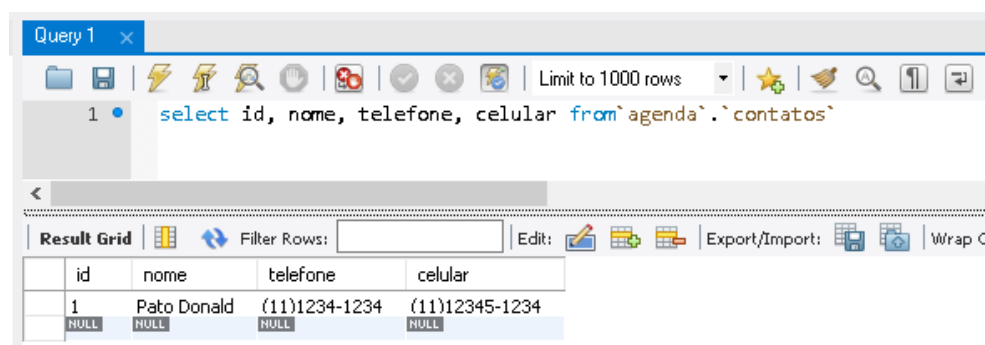
Consultando dados em uma tabela

Para visualizar os dados em uma tabela usamos o comando “select”. Você pode especificar a lista de colunas a retornar ou retornar todas usando o “*”.

Exemplos:

select nome from agenda.contatos

select * from agenda.contatos



```

1 • select id, nome, telefone, celular from `agenda`.`contatos`

```

id	nome	telefone	celular
1	Pato Donald	(11)1234-1234	(11)12345-1234
NULL	NULL	NULL	NULL

Figura 280 - Selecionando dados

Vamos agora inserir mais alguns registros na nossa tabela de contatos:

insert into agenda.contatos (nome, telefone, celular) values ('Mickey', '(11)1111-1111', '(11)11111-1111');

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
insert into agenda.contatos (nome, telefone, celular) values ('Pluto', '(11)2222-2222', '(11)22222-2222');
```

```
insert into agenda.contatos (nome, telefone) values ('Minie', '(11)3333-3333');
```

```
insert into agenda.contatos (nome, celular) values ('Tio Patinhas', '(11)44444-4444');
```

```
insert into agenda.contatos (nome, telefone, celular) values ('Margarida', '(11)5555-5555', '(11)55555-5555');
```

```
insert into agenda.contatos (nome, telefone, celular) values ('Cascão', '(11)6666-6666', '(11)66666-6666');
```

```
insert into agenda.contatos (nome, telefone, celular) values ('Cebolinha', '(11)7777-7777', '(11)77777-7777');
```

```
insert into agenda.contatos (nome, telefone, celular) values ('Mônica', '(11)8888-8888', '(11)88888-8888');
```

```
insert into agenda.contatos (nome, telefone, celular) values ('Magali', '(11)9999-9999', '(11)99999-9999');
```

	id	nome	telefone	celular
1	1	Pato Donald	(11)1234-1234	(11)12345-1234
2	2	Mickey	(11)1111-1111	(11)11111-1111
3	3	Pluto	(11)2222-2222	(11)22222-2222
4	4	Minie	(11)3333-3333	NULL
5	5	Tio Patinhas	NULL	(11)44444-4444
6	6	Margarida	(11)5555-5555	(11)55555-5555
7	7	Cascão	(11)6666-6666	(11)66666-6666
8	8	Cebolinha	(11)7777-7777	(11)77777-7777
9	9	Mônica	(11)8888-8888	(11)88888-8888
10	10	Magali	(11)9999-9999	(11)99999-9999
		NULL	NULL	NULL

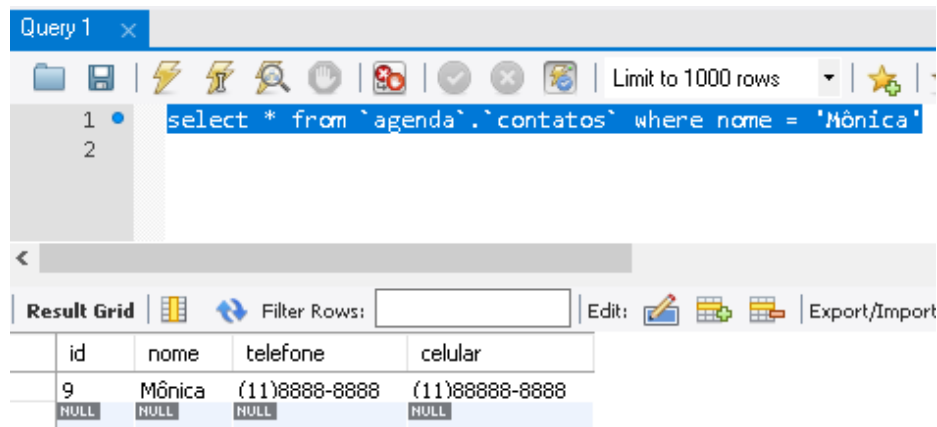
Figura 281 - Selecionando dados de contato

Podemos filtrar o resultados das nossas consultas usando a cláusula “where” do comando “select”.

Exemplo:

```
select * from agenda.contatos where nome = 'Mônica'
```

Utilizando o “=” o conteúdo do campo tem que ser idêntico ao valor informado.



Usando “Where” com “Like” para retornar os registros contendo parte do valor informado.

select * from agenda.contatos where nome like 'M%'

O percentual é um caracter “coringa” que serve para substituir caracteres na busca. No exemplo acima serão retornados todos contatos cujo nome inicie com “M” e não importa os caracteres depois do “M”.

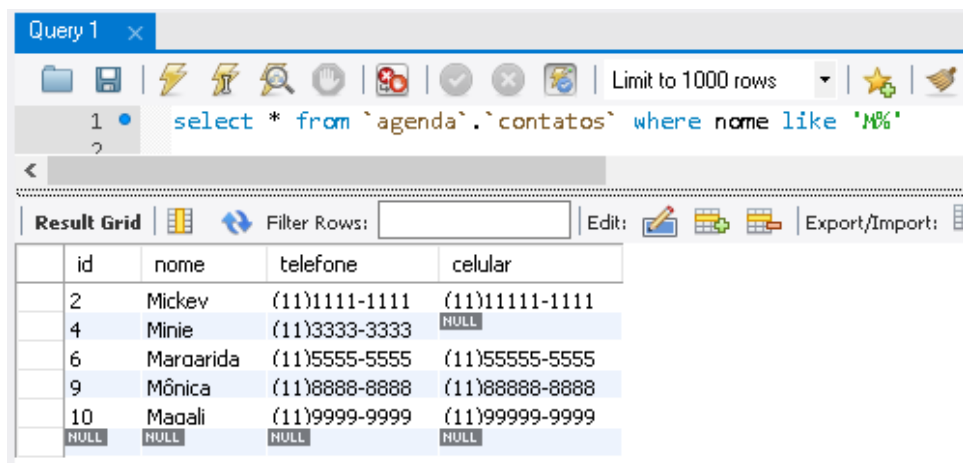


Figura 282 - Usando o like para selecionar registros

Usando o “and” para retornar o resultado com base em duas informações.

select * from agenda.contatos where celular is not null and nome like 'M%'

Serão retornados todos os contatos cujo nome inicie com “M” e que tenham celular (não seja nulo).

A “Minie” não será exibida porque não tem celular (Veja slide anterior). O campo celular está com o valor nulo “null”.

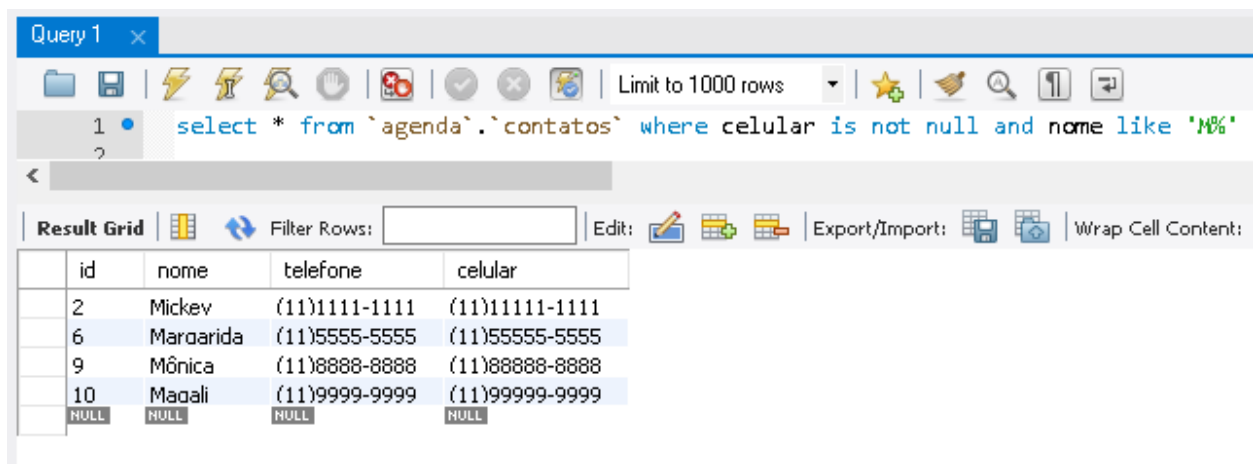


Figura 283 - Selecionando registros usando like e null

Usando o “or” para retornar o resultado com base em uma ou outra informação.

`select * from agenda.contatos where (celular is not null and nome like 'M%') or (nome like '%Donald')`

Serão retornados todos os contatos cujo nome inicie com “M” e que tenham celular (não seja nulo) ou que tenham “Donald” no nome

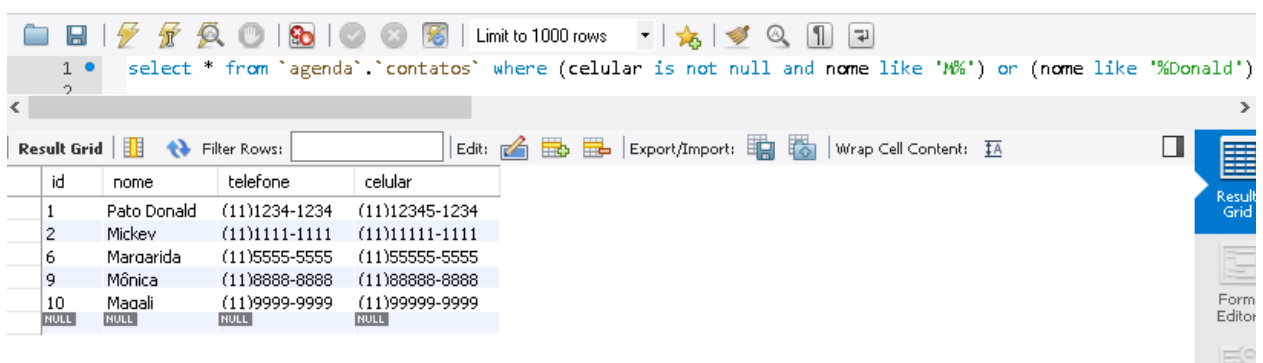


Figura 284 - Utilizando "or" e "and" para selecionar registros

Alterando dados em uma tabela

Para alterar dados em um registro de uma tabela usamos o comando “update” em conjunto com cláusulas “where” para determinar qual registro queremos atualizar.

Exemplo:

`update agenda.contatos set celular = '(21)12345-1111' where id = 9`

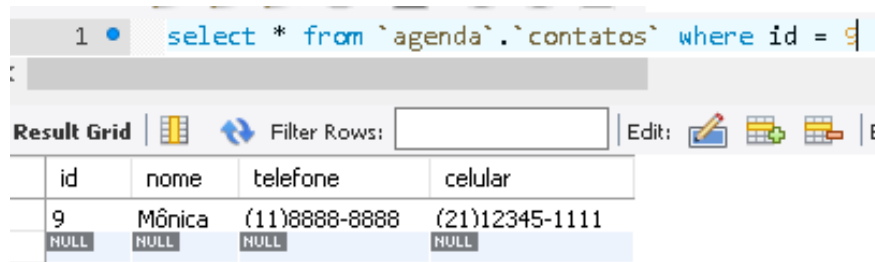
Este comando atualiza o celular do registro onde o campo “id” é igual a 9.

Veja na imagem que o celular da “Mônica” era (11)88888-8888 e agora é (21)12345-1111.

Muito cuidado na hora de fazer uma alteração especificando bem qual registro será alterado

para não alterar registros errados.

Um update sem uma cláusula where irá alterar todos registros da tabela.



1 • `select * from `agenda`.`contatos` where id = 9`

Result Grid | Filter Rows: | Edit: |

id	nome	telefone	celular
9	Mônica	(11)8888-8888	(21)12345-1111
NULL	NULL	NULL	NULL

Figura 285 - Verificando registro alterado

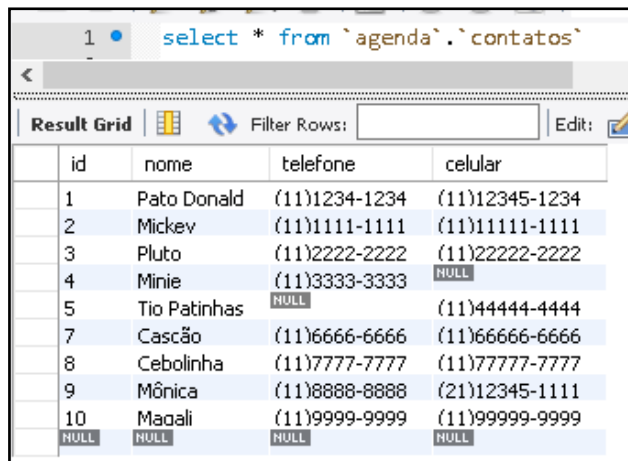
Excluindo dados em uma tabela

Para excluir um registro de uma tabela usamos o comando “delete” em conjunto com cláusulas “where” para determinar qual registro queremos excluir. CUIDADO ao usar o comando delete, seu uso indevido pode trazer graves problemas.

Exemplo:

`delete from agenda.contatos where telefone = "(11)5555-5555"`

Este comando exclui o registro que possua o telefone informado na cláusula where. No nosso caso, excluiu o registro 6 que é o registro da “Margarida”. Veja:



1 • `select * from `agenda`.`contatos``

Result Grid | Filter Rows: | Edit: |

id	nome	telefone	celular
1	Pato Donald	(11)1234-1234	(11)12345-1234
2	Mickey	(11)1111-1111	(11)11111-1111
3	Pluto	(11)2222-2222	(11)22222-2222
4	Minie	(11)3333-3333	NULL
5	Tio Patinhas	NULL	(11)44444-4444
7	Cascão	(11)6666-6666	(11)66666-6666
8	Cebolinha	(11)7777-7777	(11)77777-7777
9	Mônica	(11)8888-8888	(21)12345-1111
10	Maçali	(11)9999-9999	(11)99999-9999
NULL	NULL	NULL	NULL

O registro 6 foi excluído

Figura 286 - Verificando registro excluído

Conectando ao banco MySQL com MySQL Connector/Python



Sobre o MySQL Connector/Python

MySQL Connector/Python é um driver de banco de dados padronizado para plataformas e desenvolvimento em Python. Nós o usaremos para conectarmos ao banco de dados MySQL com Python. O MySQL Connector/Python pode ser instalado no momento da instalação do MySQL. Para conferir se o mesmo está instalado em seu sistema execute o import conforme imagem abaixo:

```

Command Prompt - python
C:\Users\evaldo>python
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 17:00:18) [MSC v.1900 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import mysql.connector
>>>

```

Figura 287 - Importando o MySQL Connector

Baixando e instalando o MySQL Connector/Python

Caso o MySQL Connector/Python não esteja instalado você pode baixar o instalador em:

<https://dev.mysql.com/downloads/connector/python/>

Nesta página você pode escolher o sistema operacional e escolher o instalador adequado para sua versão do Python.

Estão disponíveis versões para diversos sistemas operacionais e também diversas versões do Python.

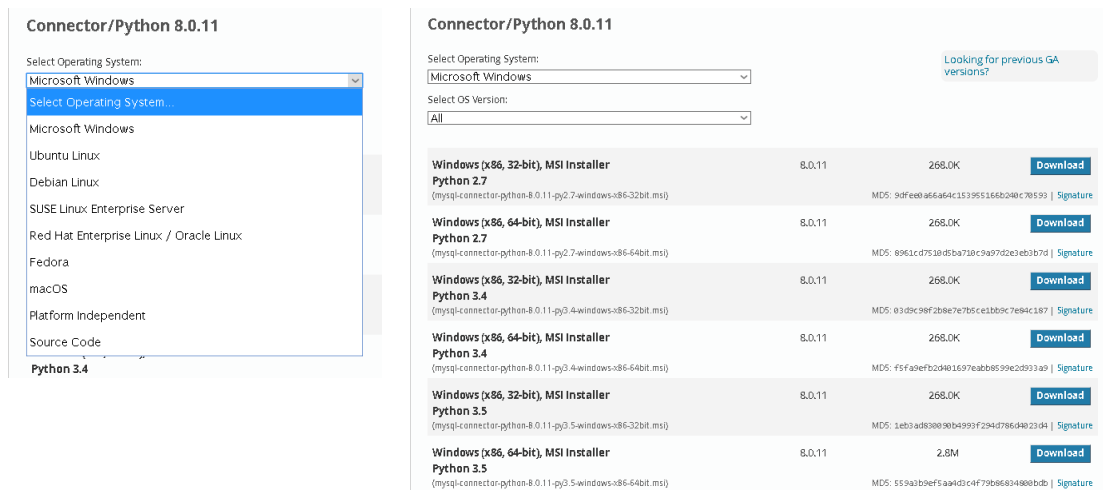


Figura 288 - Baixando o MySQL Connector

No Windows você pode instalar com o comando:

\$ pip install mysql-connector-python

Em distribuições baseadas em Debian como o Linux Mint você pode instalar através do seguinte comando (como administrador):

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



\$ sudo pip install mysql-connector-python

Ou

\$ sudo pip3 install mysql-connector-python

Usando o MySQL Connector/Python

Após instalação podemos executar nosso primeiro exemplo. Para acessar o MySQL usando o Python podemos usar o módulo “mysql”. Veja um exemplo:

```
# Importando o MySQL Connector/Python
import mysql.connector

# Criando o objeto de conexão através do connect() com os parâmetros:
# usuário, senha, endereço do servidor e nome do banco/esquema
# o connect retorna um objeto MySQLConnection
conexao = mysql.connector.connect(user='root', password='1234',
                                 host='127.0.0.1', database='agenda')

# Fechando a conexão com o banco de dados
conexao.close()
```

Ao executar este código com as informações de conexão corretas, não haverá nenhum resultado porque simplesmente conectamos ao MySQL e desconectamos em seguida. Execute o código com algum dado inválido (nome do usuário, senha, host ou nome do esquema) para ver que ocorrerá um erro.

Uma outra forma de se conectar ao banco de dados é usando a classe `connection.MySQLConnection()`.

```
# Importando o MySQL Connector/Python
from mysql.connector import connection

# Criando o objeto de conexão através do connection.MySQLConnection() com os parâmetros:
# usuário, senha, endereço do servidor e nome do banco/esquema
# o connect retorna um objeto MySQLConnection
conexao = connection.MySQLConnection(user='root', password='1234',
                                     host='127.0.0.1', database='agenda')

# Fechando a conexão com o banco de dados
conexao.close()
```

Todos os dois métodos são válidos e funcionam igualmente, porém, em nossos exemplos, usaremos `connector()`.

Para lidar com erros de conexão, use a instrução `try` e capture todos os erros usando a exceção `errors.Error` (que é uma subclasse de `StandardError`) conforme exemplo a seguir.

```
import mysql.connector
from mysql.connector import errorcode

# Tratando erros de conexao usando errorcode
try:
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```

conexao = mysql.connector.connect(user='root', password='1234',
                                   host='127.0.0.1', database='agenda')
except mysql.connector.Error as erro:
    if erro.errno == errorcode.ER_ACCESS_DENIED_ERROR:
        print("Usuário ou senha inválidos")
    elif erro.errno == errorcode.ER_BAD_DB_ERROR:
        print("Banco de dados não existe")
    else:
        print(erro)
else:
    conexao.close()

```

Você pode substituir os parâmetros de conexão por um dicionário e desempacotar na chamada da função.

Relembrando:

*args -> é chamado de “argument unpacking” e é utilizado para “desempacotar” argumentos de um dicionário.

**kwargs -> é chamado de “keyword argument unpacking” e é utilizado para “desempacotar” baseado em palavra-chave (keyword).

Veja um exemplo:

```

>>> def teste(nome, sobrenome):
...     print(f"O nome é: {nome}")
...     print(f"O sobrenome é: {sobrenome}")
...
>>> dic = {'nome': 'evaldo', 'sobrenome': 'Wolkers'}
>>> print(*dic)
nome sobrenome
>>> teste(**dic)
O nome é: evaldo
O sobrenome é: Wolkers
>>> dic = {'nomez': 'evaldo', 'sobrenome': 'Wolkers'}
>>> teste(**dic)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: teste() got an unexpected keyword argument 'nomez'

```

Figura 289 - Erro quando existe uma keyword no dicionário que não existe na chamada da função.

Veja o exemplo utilizando um dicionário:

```

import mysql.connector

# Criando um dicionário com dados de conexão
configuracao = {
    'user': 'root',
    'password': '1234',
    'host': '127.0.0.1',
    'database': 'agenda'
}

# Informando o dicionario criado usando **kwargs
conexao = mysql.connector.connect(**configuracao)
conexao.close()

```



Executando instruções SQL com Python

Inserindo registros

Executando o comando SQL insert para inserir registros na tabela de contatos.

```
# Importando o MySQL Connector/Python
import mysql.connector

# Criando o objeto de conexão através do connect com os parâmetros:
# usuário, senha, endereço do servidor e nome do banco/esquema
# o connect retorna um objeto MySQLConnection
conexao = mysql.connector.connect(user='root', password='1234',
                                 host='127.0.0.1', database='agenda')

# Criando um cursor
# Cursor é uma estrutura de controle que permite percorrer
# registros, além de facilitar a adição e remoção de registros
# do banco de dados
cursor = conexao.cursor()

# Montando uma String com o comando de insert
inserir_contato = ("insert into contatos (nome, telefone, celular) "
                  "values ('Pateta', '(27)1111-1111', '(27)12345-3333')")

# Executando o comando insert
cursor.execute(inserir_contato)
# Gravando a informação permanentemente
conexao.commit()
# Fechando o cursor
cursor.close()
# Fechando a conexão
conexao.close()
```

Inserindo registros utilizando uma tupla com os dados.

```
import mysql.connector

conexao = mysql.connector.connect(user='root', password='1234',
                                 host='127.0.0.1', database='agenda')
cursor = conexao.cursor()

inserir_contato = ("insert into contatos (nome, telefone, celular) "
                  "values (%s, %s, %s)")

# Criando uma tupla com os dados a serem inseridos
contato = ('Rapunzel', '(61)2525-5858', '(61)98745-5555')
cursor.execute(inserir_contato, contato)
conexao.commit()
cursor.close()
conexao.close()
```

Inserindo registros utilizando um dicionário com os dados.

```
import mysql.connector

conexao = mysql.connector.connect(user='root', password='1234',
                                 host='127.0.0.1', database='agenda')
cursor = conexao.cursor()

# Montando o comando insert com as chaves do dicionário
inserir_contato = ("insert into contatos (nome, telefone, celular) "
                  "values (%(nome_contato)s, %(telefone)s, %(celular)s)")
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
# Criando um dicionário com os dados a serem inseridos
contato = {'nome_contato': 'Bela Adormecida',
           'telefone': '(61)2222-5657',
           'celular': '(61)95874-5854'}

cursor.execute(inserir_contato, contato)
conexao.commit()
cursor.close()
conexao.close()
```

Alterando registros

Executando o comando update para alterar um registro da tabela contatos.

```
import mysql.connector

conexao = mysql.connector.connect(user='root', password='1234',
                                 host='127.0.0.1', database='agenda')
cursor = conexao.cursor()

atualizar_contato = ("update contatos set nome = %s, telefone = %s, celular = %s where id = 1")
contato = ('Pocahontas', '(28)8888-8888', '(28)88888-8888')

cursor.execute(atualizar_contato, contato)
conexao.commit()
cursor.close()
conexao.close()
```

Apagando registros

Executando o comando delete para apagar um registro da tabela contatos.

```
import mysql.connector

conexao = mysql.connector.connect(user='root', password='1234',
                                 host='127.0.0.1', database='agenda')
cursor = conexao.cursor()

remover_contato = ("delete from contatos where id = 1")

cursor.execute(remover_contato)
conexao.commit()
cursor.close()
conexao.close()
```

Consultando registros

Consultando dados da tabela contatos com o comando select.

```
import mysql.connector

conexao = mysql.connector.connect(user='root', password='1234',
                                 host='127.0.0.1', database='agenda')
cursor = conexao.cursor()
consulta = ("select nome, telefone, celular from contatos where nome like 'M%")
cursor.execute(consulta)

for (nome, telefone, celular) in cursor:
    print(f"Nome: {nome}, telefone: {telefone}, celular: {celular}")
```



```
cursor.close()
conexao.close()
```

Criando um cadastro simples sem interface gráfica

Introdução

Nesta aula vamos criar um programa exemplo para realizar cadastros, alterações e consultas de contatos em nossa tabela.

Será um programa bem simples (sem conceitos muito avançados), sem interface gráfica, sem orientação a objetos, utilizando apenas alguns conceitos simples aprendidos até o momento.

O que vamos usar?

Para desenhar as telas vamos usar print, para perguntar/capturar a opção desejada vamos usar input, para interagir com o banco de dados vamos utilizar mysql.connector, para inserir um contato na base de dados vamos utilizar lista, para listar na tela os contatos vamos utilizar um loop for, para exportar o cadastro vamos usar comandos de criação e escrita em arquivos, para as opções de menu vamos usar um dicionário com a opção e a função que será executada, para validar a opção selecionada vamos usar if/elif/else e para limpar a tela vamos validar se o sistema está sendo executado no Windows para usarmos o comando "cls" ou senão usaremos o comando "clear" (para isso importaremos "os" e "platform").

Execução do sistema

```
Command Prompt - python temp2.py
*****
*      BEM-VINDO AO SISTEMA DE CADASTRO      *
*-----*
*      ESCOLHA UMA DAS OPÇÕES DISPONÍVEIS    *
*      1 - CADASTRAR UM CONTATO                *
*      2 - ALTERAR UM CONTATO                  *
*      3 - LISTAR CONTATOS                     *
*      4 - EXPORTAR CONTATOS                   *
*      0 - SAIR DO SISTEMA                     *
*****
Informe a opção desejada: 1

Command Prompt - python temp2.py
*****
*      CADASTRANDO UM NOVO CONTATO            *
*****
Informe o nome do contato (obrigatório): teste
Informe o telefone do contato (obrigatório): 123456
Informe o celular do contato (obrigatório): 12212
Confirma cadastro do contato: teste? (S/N): s
Contato cadastrado com sucesso!
Pressione uma tecla para voltar...

```



```

Command Prompt - python temp2.py
Id: 3
Nome: Pluto
Telefone: (11)2222-2222
Celular: (11)2222-2222
*****
Id: 2
Nome: Mickey
Telefone: (11)1111-1111
Celular: (11)1111-1111
*****
Pressione uma tecla para voltar ao menu principal...

Command Prompt - python temp2.py
*          ESCOLHA UMA DAS OPÇÕES DISPONÍVEIS          *
*          1 - CADASTRAR UM CONTATO                     *
*          2 - ALTERAR UM CONTATO                       *
*          3 - LISTAR CONTATOS                          *
*          4 - EXPORTAR CONTATOS                        *
*          0 - SAIR DO SISTEMA                          *
*****
Informe a opção desejada: 4
Informe o nome do arquivo: a
Exportação realizada.
Pressione uma tecla para voltar ao menu principal...

```

Implementando o exemplo

Importando as classes que serão utilizadas:

```

import os
import platform
import mysql.connector

```

Criando variável com as informações de conexão ao banco de dados:

```

dados_conexao = {"user": "root", "password": "1234",
                  "host": "127.0.0.1", "database": "agenda"}

```

Implementando a função para limpar a tela e sair do sistema:

```

def limpar_tela():
    if platform.system() == "Windows":
        os.system("cls")
    else:
        os.system("clear")

def sair():
    print("Obrigado por utilizar o sistema!")
    os._exit(1)

```

Definindo as mensagens dos menus e títulos:

```

def mensagem_menu_principal():
    limpar_tela()
    print("*****")
    print("*          BEM-VINDO AO SISTEMA DE CADASTRO          *")
    print("*-----*")
    print("*          ESCOLHA UMA DAS OPÇÕES DISPONÍVEIS          *")
    print("*          1 - CADASTRAR UM CONTATO                     *")
    print("*          2 - ALTERAR UM CONTATO                       *")
    print("*          3 - LISTAR CONTATOS                          *")
    print("*          4 - EXPORTAR CONTATOS                        *")
    print("*          0 - SAIR DO SISTEMA                          *")
    print("*****")

```



```
def mensagem_menu_alterar():
    limpar_tela()
    print("*****")
    print("*          ALTERANDO UM CONTATO EXISTENTE          *")
    print("*-----*")
    print("*      ESCOLHA A OPÇÃO PARA LOCALIZAR O CONTATO      *")
    print("*          1 - ID                                     *")
    print("*          2 - NOME                                    *")
    print("*          3 - TELEFONE                                 *")
    print("*          4 - CELULAR                                  *")
    print("*          0 - MENU PRINCIPAL                          *")
    print("*****")

def mensagem_cadastrar():
    limpar_tela()
    print("*****")
    print("*          CADASTRANDO UM NOVO CONTATO                *")
    print("*****")
```

Implementando a função do menu principal somente imprimindo a opção informada:

```
def menu_principal():
    mensagem_menu_principal()
    opcao = input("Informe a opção desejada: ")
    print(f"Opção: {opcao}")
```

Definindo a chamada ao menu principal:

```
if __name__ == "__main__":
    menu_principal()
```

Execute o sistema para testar seu funcionamento.

Implementando o dicionário com as funções que serão executadas conforme escolha das opções do menu principal. Por enquanto só temos implementada a função sair:

```
acoes_menu_principal = {
    # '1': cadastrar,
    # '2': alterar,
    # '3': listar,
    # '4': exportar,
    '0': sair
}
```

Vamos mudar o menu principal para chamar uma função de acordo com a opção informada (verificando a opção no dicionário).

```
def menu_principal():
    try:
        mensagem_menu_principal()
        opcao = input("Informe a opção desejada: ")
        acoes_menu_principal[opcao]()
    except:
        input("Opção inválida.\nPressione uma tecla para voltar ao menu principal.")
        menu_principal()
```



Execute e teste a opção de sair do sistema.

Implementando a função cadastrar.

```
def cadastrar():
    mensagem_cadastrar()
    nome = input("Informe o nome do contato (obrigatório): ")
    telefone = input("Informe o telefone do contato (obrigatório): ")
    celular = input("Informe o celular do contato (obrigatório): ")

    if nome and telefone and celular:
        confirma = input(f"Confirma cadastro do contato: {nome}? (S/N): ")
        if confirma.upper() == "S":
            try:
                contato = []
                contato.append(nome)
                contato.append(telefone)
                contato.append(celular)
                conexao = mysql.connector.connect(**dados_conexao)
                cursor = conexao.cursor()
                comando_sql = "insert into contatos (nome, telefone, celular) " \
                               "values (%s, %s, %s) "
                cursor.execute(comando_sql, contato)
                conexao.commit()
                cursor.close()
                conexao.close()
                input("Contato cadastrado com sucesso!\n"
                     "Pressione uma tecla para voltar ao menu principal...")
            except:
                input("Ocorreu um erro no cadastro.\n"
                     "Pressione uma tecla para voltar ao menu principal...")
        else:
            input("Cadastro cancelado.\n"
                 "Pressione uma tecla para voltar ao menu principal...")
    else:
        input("Todos os campos são obrigatórios.\n"
             "Pressione uma tecla para voltar ao menu principal...")

    menu_principal()
```

Altere o dicionário de ações, disponibilizando a chamada à função cadastrar.

```
acoes_menu_principal = {
    '1': cadastrar,
    # '2': alterar,
    # '3': listar,
    # '4': exportar,
    '0': sair
}
```

Execute o sistema e teste o cadastro.

Implementando a função alterar:

```
def alterar():
    mensagem_menu_alterar()
    opcao = input("Informe a opção desejada: ")
    try:
        acoes_menu_alterar[opcao]()
    except:
        input("Opção inválida.\nPressione uma tecla para voltar...")
        alterar()
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



Crie o dicionário `acoes_menu_alterar`.

```
acoes_menu_alterar = {
    # '1': pesquisar_id,
    # '2': pesquisar_nome,
    # '3': pesquisar_telefone,
    # '4': pesquisar_celular,
    '0': menu_principal
}
```

Mude o dicionário `acoes_menu_principal` tirando o comentário da chamada à função `alterar`. A única opção funcionando será a opção de retorno ao menu principal. Execute e teste o sistema.

```
acoes_menu_principal = {
    '1': cadastrar,
    '2': alterar,
    # '3': listar,
    # '4': exportar,
    '0': sair
}
```

Implementando a busca por id para alteração.

Vamos mudar o dicionário `acoes_menu_alterar` como segue:

```
acoes_menu_alterar = {
    '1': pesquisar_id,
    # '2': pesquisar_nome,
    # '3': pesquisar_telefone,
    # '4': pesquisar_celular,
    '0': menu_principal
}
```

Agora vamos implementar a função `pesquisar_id`:

```
def pesquisar_id(id=""):
    try:
        if not id:
            id = input("Informe o id do contato: ")
        comando_sql = f"select id, nome, telefone, celular from contatos" \
            f"where id = {id}"
        conexao = mysql.connector.connect(**dados_conexao)
        cursor = conexao.cursor()
        cursor.execute(comando_sql)
        contato = []
        for id, nome, telefone, celular in cursor:
            contato = {"id": id, "nome": nome, "telefone": telefone,
                "celular": celular}
        cursor.close()
        conexao.close()

        alterar_contato(contato)
    except Exception as err:
        print("Ocorreu erro na busca por id! ", err)
        input("Pressione uma tecla para voltar...")
        alterar()
```



Agora vamos implementar a função `alterar_contato`:

```
def alterar_contato(contato):
    nome = input(f"Informe o nome do contato ({contato['nome']}): ") or \
        contato['nome']
    telefone = input(f"Informe o telefone do contato ({contato['telefone']}): ") or \
        contato['telefone']
    celular = input(f"Informe o celular do contato ({contato['celular']}): ") or \
        contato['celular']

    confirma = input(f"Confirma alteração do contato: {nome}? (S/N): ")
    if confirma.upper() == "S":
        try:
            comando_sql = f"update contatos set nome = '{nome}', " \
                f"telefone = '{telefone}', celular = '{celular}' " \
                f"where id = {contato['id']}"
            conexao = mysql.connector.connect(**dados_conexao)
            cursor = conexao.cursor()
            cursor.execute(comando_sql)
            conexao.commit()
            cursor.close()
            conexao.close()
            input("Contato alterado com sucesso!\n"
                "Pressione uma tecla para voltar...")
        except:
            input("Ocorreu um erro na alteração.\n"
                "Pressione uma tecla para voltar...")
    else:
        input("Alteração cancelada.\n"
            "Pressione uma tecla para voltar...")
    alterar()
```

Execute o programa e faça algumas alterações em alguns dos contatos cadastrados.

Agora vamos implementar a localização por nome, telefone e celular.

Altere o menu de ações da função `alterar` e implemente a função de pesquisa por nome.

```
acoes_menu_alterar = {
    '1': pesquisar_id,
    '2': pesquisar_nome,
    '3': pesquisar_telefone,
    '4': pesquisar_celular,
    '0': menu_principal
}
```

```
def pesquisar_nome():
    try:
        nome = input("Informe o nome do contato: ")
        comando_sql = f"select id, nome, telefone, celular from contatos " \
            f"where nome like '%{nome}%'"
        conexao = mysql.connector.connect(**dados_conexao)
        cursor = conexao.cursor()
        cursor.execute(comando_sql)
        contato = []
        for id, nome, telefone, celular in cursor:
            contato = {"id": id, "nome": nome, "telefone": telefone,
                "celular": celular}
        cursor.close()
        conexao.close()
```



```

    if contato:
        print(f"Contato localizado: id:{contato['id']},nome:{contato['nome']}, "
              f"telefone:{contato['telefone']},celular:{contato['celular']}")
        pesquisar_id(contato['id'])
except Exception as err:
    print("Ocorreu erro na busca por nome! ", err)
    input("Pressione uma tecla para voltar...")
    alterar()

```

Implementando a função de pesquisa por telefone.

```

def pesquisar_telefone():
    try:
        telefone = input("Informe o telefone do contato: ")
        comando_sql = f"select id, nome, telefone, celular from " \
                      f"contatos where telefone = '{telefone}'"
        conexao = mysql.connector.connect(**dados_conexao)
        cursor = conexao.cursor()
        cursor.execute(comando_sql)
        contato = []
        for id, nome, telefone, celular in cursor:
            contato = {"id": id, "nome": nome, "telefone": telefone,
                      "celular": celular}
        cursor.close()
        conexao.close()

        if contato:
            print(f"Contato localizado: id: {contato['id']}, "
                  f"nome: {contato['nome']}, "
                  f"telefone: {contato['telefone']}, "
                  f"celular: {contato['celular']}")
            pesquisar_id(contato['id'])
    except Exception as err:
        print("Ocorreu erro na busca por telefone! ", err)
        input("Pressione uma tecla para voltar...")
        alterar()

```

Implementando a função de pesquisa por celular. Teste todas opções de busca.

```

def pesquisar_celular():
    try:
        celular = input("Informe o celular do contato: ")
        comando_sql = f"select id, nome, telefone, celular from " \
                      f"contatos where celular = '{celular}'"
        conexao = mysql.connector.connect(**dados_conexao)
        cursor = conexao.cursor()
        cursor.execute(comando_sql)
        contato = []
        for id, nome, telefone, celular in cursor:
            contato = {"id": id, "nome": nome, "telefone": telefone,
                      "celular": celular}
        cursor.close()
        conexao.close()

        if contato:
            print(f"Contato localizado: id: {contato['id']}, "
                  f"nome: {contato['nome']}, "
                  f"telefone: {contato['telefone']}, "
                  f"celular: {contato['celular']}")
            pesquisar_id(contato['id'])
    except Exception as err:
        print("Ocorreu erro na busca por celular! ", err)

```



```
input("Pressione uma tecla para voltar...")
alterar()
```

Implementando a função que vai listar os contatos:

```
def listar():
    try:
        comando = "select id, nome, telefone, celular from contatos order by id desc"
        conexao = mysql.connector.connect(**dados_conexao)
        cursor = conexao.cursor()
        cursor.execute(comando)
        print("*****")
        for id, nome, telefone, celular in cursor:
            print(f"Id: {id}")
            print(f"Nome: {nome}")
            print(f"Telefone: {telefone}")
            print(f"Celular: {celular}")
            print("*****")

        cursor.close()
        conexao.close()
        input("Pressione uma tecla para voltar ao menu principal...")
    except:
        print("Ocorreu erro no listar.")
    menu_principal()
```

Retire o comentário da opção listar em acoes_menu_principal e teste o sistema.

Implementando a função que vai exportar os contatos para um arquivo de texto:

```
def exportar():
    try:
        nome_arquivo = input("Informe o nome do arquivo: ")

        comando = "select id, nome, telefone, celular from" \
            "contatos order by id desc"
        contatos = []
        conexao = mysql.connector.connect(**dados_conexao)
        cursor = conexao.cursor()

        cursor.execute(comando)
        for id, nome, telefone, celular in cursor:
            contatos.append({"id": id, "nome": nome,
                            "telefone": telefone, "celular": celular})

        cursor.close()
        conexao.close()

        arquivo = open(nome_arquivo, "w", encoding="utf-8")

        for contato in contatos:
            registro = []
            registro.append(f"{contato['id']};{contato['nome']};"
                           f"{contato['telefone']};{contato['celular']}\n")
            arquivo.writelines(registro)

        arquivo.close()
        print("Exportação realizada.")
        input("Pressione uma tecla para voltar ao menu principal...")
        menu_principal()
    except Exception as err:
```



```
print("Correu erro na função exportar! ", err)
input("Pressione uma tecla para voltar ao menu principal...")
```

Retire o comentário da opção exportar em `acoes_menu_principal` e teste o sistema.

Exercício:

Como exercício implemente a funcionalidade de exclusão.

O comando sql a ser usado é o:

`delete from contatos where id = x`

Onde: x é o id informado pelo usuário.

Código completo do sistema com algumas alterações para usar um novo método chamado `executar_comando`:

```
import os
import platform
import mysql.connector

dados_conexao = {"user": "root", "password": "1234",
                  "host": "127.0.0.1", "database": "agenda"}

def limpar_tela():
    if platform.system() == "Windows":
        os.system("cls")
    else:
        os.system("clear")

def sair():
    print("Obrigado por utilizar o sistema!")
    os._exit(1)

def mensagem_menu_principal():
    limpar_tela()
    print("*****")
    print("BEM-VINDO AO SISTEMA DE CADASTRO")
    print("*****")
    print("ESCOLHA UMA DAS OPÇÕES DISPONÍVEIS")
    print("1 - CADASTRAR UM CONTATO")
    print("2 - ALTERAR UM CONTATO")
    print("3 - LISTAR CONTATOS")
    print("4 - EXPORTAR CONTATOS")
    print("0 - SAIR DO SISTEMA")
    print("*****")

def mensagem_menu_alterar():
    limpar_tela()
    print("*****")
    print("ALTERANDO UM CONTATO EXISTENTE")
    print("*****")
    print("ESCOLHA A OPÇÃO PARA LOCALIZAR O CONTATO")
    print("1 - ID")
    print("2 - NOME")
    print("3 - TELEFONE")
    print("4 - CELULAR")
    print("0 - MENU PRINCIPAL")
    print("*****")
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```

def mensagem_cadastrar():
    limpar_tela()
    print("*****")
    print("*          CADASTRANDO UM NOVO CONTATO          *")
    print("*****")

def menu_principal():
    try:
        mensagem_menu_principal()
        opcao = input("Informe a opção desejada: ")
        acoes_menu_principal[opcao]()
    except:
        input("Opção inválida.\nPressione uma tecla para voltar"
              " ao menu principal.")
        menu_principal()

def executar_acoes_menu(menu, opcao):
    try:
        if menu and opcao:
            if menu == "menu_principal":
                acoes_menu_principal[opcao]()
            elif menu == "menu_alterar":
                acoes_menu_alterar[opcao]()
            else:
                print("Opção inválida")
                input("Pressione uma tecla para continuar...")
        else:
            print("Opção inválida")
            input("Pressione uma tecla para continuar...")
    except:
        print("Opção inválida")
        input("Pressione uma tecla para continuar...")

    menu_principal()

def cadastrar():
    mensagem_cadastrar()
    nome = input("Informe o nome do contato (obrigatório): ")
    telefone = input("Informe o telefone do contato (obrigatório): ")
    celular = input("Informe o celular do contato (obrigatório): ")

    if nome and telefone and celular:
        confirma = input(f"Confirma cadastro do contato: {nome}? (S/N): ")
        if (confirma.upper() == "S"):
            contato = []
            contato.append(nome)
            contato.append(telefone)
            contato.append(celular)
            executar_comando("cadastrar", "", contato)
        else:
            print("Cadastro cancelado.")
            input("Pressione uma tecla para voltar ao menu principal...")
    else:
        print("Todos os campos são obrigatórios.")
        input("Pressione uma tecla para voltar ao menu principal...")

    menu_principal()

def alterar():
    mensagem_menu_alterar()
    opcao = input("Informe a opção desejada: ")
    executar_acoes_menu("menu_alterar", opcao)

```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```
def pesquisar_id(id=""):
    try:
        if not id:
            id = input("Informe o id do contato: ")
            comando_sql = f"select id, nome, telefone, celular from " \
                f"contatos where id = {id}"
            contato = executar_comando("localizar", comando_sql, "")
            alterar_contato(contato)
        except Exception as err:
            print("Ocorreu erro na busca por id! ", err)
            input("Pressione uma tecla para voltar...")
            alterar()

def pesquisar_nome():
    try:
        nome = input("Informe o nome do contato: ")
        comando_sql = f"select id, nome, telefone, celular from " \
            f"contatos where nome like '%{nome}%"
        contato = executar_comando("localizar", comando_sql, "")

        if contato:
            print(f"Contato localizado: id: {contato['id']}, "
                f"nome: {contato['nome']}, "
                f"telefone: {contato['telefone']}, "
                f"celular: {contato['celular']}")
            pesquisar_id(contato['id'])
        except Exception as err:
            print("Ocorreu erro na busca por nome! ", err)
            input("Pressione uma tecla para voltar...")
            alterar()

def pesquisar_telefone():
    try:
        telefone = input("Informe o telefone do contato: ")
        comando_sql = f"select id, nome, telefone, celular from " \
            f"contatos where telefone = '{telefone}'"
        contato = executar_comando("localizar", comando_sql, "")

        if contato:
            print(f"Contato localizado: id: {contato['id']}, "
                f"nome: {contato['nome']}, "
                f"telefone: {contato['telefone']}, "
                f"celular: {contato['celular']}")
            pesquisar_id(contato['id'])
        except Exception as err:
            print("Ocorreu erro na busca por telefone! ", err)
            input("Pressione uma tecla para voltar...")
            alterar()

def pesquisar_celular():
    try:
        celular = input("Informe o celular do contato: ")
        comando_sql = f"select id, nome, telefone, celular from " \
            f"contatos where celular = '{celular}'"
        contato = executar_comando("localizar", comando_sql, "")

        if contato:
            print(f"Contato localizado: id: {contato['id']}, "
                f"nome: {contato['nome']}, "
                f"telefone: {contato['telefone']}, "
                f"celular: {contato['celular']}")
            pesquisar_id(contato['id'])
        except Exception as err:
```

<https://www.udemy.com/python-3-para-todos> - <https://www.pythonparatodos.com.br>

contato@pythonparatodos.com.br



```

        print("Ocorreu erro na busca por celular! ", err)
        input("Pressione uma tecla para voltar...")
        alterar()

def alterar_contato(contato):
    nome = input(f"Informe o nome do contato ({contato['nome']}): ") or \
        contato['nome']
    telefone = input(f"Informe o telefone do contato ({contato['telefone']}): ") \
        or contato['telefone']
    celular = input(f"Informe o celular do contato ({contato['celular']}): ") \
        or contato['celular']

    confirma = input(f"Confirma alteração do contato: {nome}? (S/N): ")
    if (confirma.upper() == "S"):
        comando_sql = f"update contatos set nome = '{nome}', telefone = '{telefone}'," \
            f"celular = '{celular}' where id = {contato['id']}"
        executar_comando("alterar", comando_sql, "")
    else:
        print("Alteração cancelada.")
        input("Pressione uma tecla para voltar...")
        alterar()

def listar():
    contatos = executar_comando("listar", "", "")
    print("*****")
    for contato in contatos:
        print(f"Id: {contato['id']}")
        print(f"Nome: {contato['nome']}")
        print(f"Telefone: {contato['telefone']}")
        print(f"Celular: {contato['celular']}")
        print("*****")
    input("Pressione uma tecla para voltar ao menu principal...")
    menu_principal()

def exportar():
    try:
        nome_arquivo = input("Informe o nome do arquivo: ")
        contatos = executar_comando("listar", "", "")
        arquivo = open(nome_arquivo, "w", encoding="utf-8")

        for contato in contatos:
            registro = []
            registro.append(f"{contato['id']}:{contato['nome']};"
                f"{contato['telefone']}:{contato['celular']}\n")
            arquivo.writelines(registro)

        arquivo.close()
        print("Exportação realizada.")
        input("Pressione uma tecla para voltar ao menu principal...")
        menu_principal()
    except Exception as err:
        print("Correu erro na função exportar! ", err)
        input("Pressione uma tecla para voltar ao menu principal...")

def executar_comando(acao="", comando="", dados=""):
    conexao = mysql.connector.connect(**dados_conexao)
    cursor = conexao.cursor()

    if acao == "cadastrar":
        try:
            comando_sql = "insert into contatos (nome, telefone, " \
                "celular) values (%s, %s, %s) "

```



```

        cursor.execute(comando_sql, dados)
        conexao.commit()
        cursor.close()
        conexao.close()
        print("Contato cadastrado com sucesso!")
    except Exception as err:
        print("Ocorreu um erro no cadastro! ", err)

    input("Pressione uma tecla para voltar...")
    menu_principal()
elif acao == "alterar":
    try:
        cursor.execute(comando)
        conexao.commit()
        cursor.close()
        conexao.close()
        print("Contato alterado com sucesso!")
    except Exception as err:
        print("Ocorreu um erro na alteração! ", err)

    input("Pressione uma tecla para voltar...")
    alterar()
elif acao == "localizar":
    registro = ""
    cursor.execute(comando)
    for id, nome, telefone, celular in cursor:
        registro = {"id":id, "nome":nome, "telefone":telefone,
                    "celular":celular}

    cursor.close()
    conexao.close()
    return registro
elif acao == "listar":
    comando = "select id, nome, telefone, celular from " \
              "contatos order by id desc"

    registros = []
    cursor.execute(comando)
    for id, nome, telefone, celular in cursor:
        registros.append({"id":id, "nome":nome,
                          "telefone":telefone, "celular":celular})

    cursor.close()
    conexao.close()
    return registros
else:
    print("Comando inválido.")
    input("Pressione uma tecla para sair...")
    cursor.close()
    conexao.close()

acoes_menu_principal = {
    '1': cadastrar,
    '2': alterar,
    '3': listar,
    '4': exportar,
    '0': sair
}

acoes_menu_alterar = {
    '1': pesquisar_id,
    '2': pesquisar_nome,
    '3': pesquisar_telefone,
    '4': pesquisar_celular,
    '0': menu_principal
}

```



```
if __name__ == "__main__":  
    menu_principal()
```



Referências Bibliográficas do curso

LIVROS

Phillips, Dusty, Python 3 Object-oriented Programming, Second Edition, Birmingham, Packt Publishing Ltd, 2015, ISBN 978-1-78439-878-1

Hattem, Rick V, Mastering Python, Birmingham, Packt Publishing Ltd, 2016, ISBN 978-1-78528-972-9

Romano, Fabrizio, Learning Python, Packt Publishing Ltd, 2015, ISBN 978-1-78355-171-2

Aggarwal, Shalabh, Flask Framework Cookbook, Birmingham, Packt Publishing Ltd, 2015, ISBN 978-1-78398-340-7

Slatkin, Brett, Python Eficaz: 59 maneiras de programar melhor em Python, São Paulo, Novatec, 2016, ISBN 978-85-7522-510-3

Giridhar, Chetan, Aprendendo Padrões de Projeto em Python, São Paulo, Novatec, 2016, ISBN 978-85-7522-523-3

Reitz, Kenneth; Schlusser, Tanya, O Guia do Mochileiro Python: Melhores práticas para desenvolvimento, São Paulo, Novatec, 2017, ISBN 978-85-7522-541-7

Menezes, Nilo C M, Introdução à Programação com Python: Algoritmos e lógica de programação para iniciantes, 2ª edição, São Paulo, Novatec, 2014, ISBN 978-85-7522-408-3

Borges, Luiz E, Python para desenvolvedores, São Paulo, Novatec, 2014, ISBN 978-85-7522-405-2

Grinberg, Miguel, Desenvolvimento web com Flask, São Paulo, Novatec, 2018, ISBN 978-85-7522-681-0

SITES

Documentação oficial do Python, Disponível em <<https://docs.python.org/3/>>

Algoritmos e Python, Disponível em <<http://neci-python.blogspot.com.br/>>

Documentação PyQt5, Disponível em <<http://pyqt.sourceforge.net/Docs/PyQt5/>>

Documentação Qt, Disponível em <<http://doc.qt.io/qt-5/classes.html>>

Python Tutorials, Disponível em <<https://pythonspot.com>>

BogoToBogo, Disponível em <<http://www.bogotobogo.com/python/pytut.php>>

Secnetix, Disponível em <<http://www.secnetix.de/olli/Python/>>

Python Brasil, Disponível em <<https://wiki.python.org.br>>